

Oxy

AI Wearable Personal Assistant Complete Codebase Reference

June 2026 · Branch: [claude/quirky-franklin-GI1Jj](#)

Layer	Technology	Purpose
Backend API	Node.js / Express	REST + WebSocket server, Gemini, Supabase
Gemini Live	live.js (WebSocket)	Streaming STT + TTS + tool execution
iOS App	SwiftUI / Swift 5.9	Chat UI, voice input, native integrations
Pendant BLE	PendantLiveService	Streams raw PCM → Gemini Live (16 kHz L16)
Firmware	Arduino / nRF52840	PDM mic capture, BLE NUS, LED feedback
Database	Supabase (PostgreSQL)	Conversations, memory, connectors, users
Auth	HMAC-SHA256 tokens	Signed session payloads, 7-day TTL

Table of Contents

Project Overview

- README
- Comprehensive Overview

Backend — API Server

- api/index.js
- live.js (Gemini Live WebSocket)
- companion-live.js
- connectors/index.js
- auth.js
- runtime.js
- package.json

iOS App — Entry & Models

- OxyApp.swift
- AppState.swift
- Message.swift

iOS App — Services

- APIClient.swift
- AuthService.swift
- ChatService.swift
- VoiceInputManager.swift
- KeychainHelper.swift
- LocationManager.swift
- AppDelegate.swift
- NativeIntegrationManager.swift
- PendantLiveService.swift (NEW)
- PendantAudioBridge.swift

iOS App — Views

- MainTabView.swift
- ChatView.swift
- ChatViewModel.swift
- MessageBubble.swift
- SettingsView.swift
- LoginView.swift
- HistoryView.swift
- MemoryView.swift
- ConnectorsView.swift
- ProactiveView.swift
- Color+Oxy.swift
- ErrorBanner.swift
- LoadingIndicator.swift

Firmware

- firmware/README.md
- oxy-pendant.ino

Documentation

- Onboarding Permission Matrix

- Hardening Runbook
- Privacy Support Checklist

Project Overview

README — README.md

```
# Oxy

Oxy is an AI-powered personal assistant that you talk to like a friend. It connects to the real services you use every day – Gmail, Google Calendar, etc.

## What It Does

- **Conversational AI** – Chat via text or voice. Oxy responds naturally, remembers personal context across conversations, and adapts its tone to you.
- **Voice I/O** – Record audio from your browser, get it transcribed (Gemini), processed by the AI, and hear a spoken reply (Gemini TTS) – all in the browser.
- **Real Actions** – Oxy doesn't just talk. When you say "text Sarah I'm running late" or "book an Uber to the station", it actually does it through the connectors.
- **Memory** – Oxy automatically extracts and stores personal facts from conversations ("Works at KPMG", "Has a dog named Biscuit") and uses them to personalize responses.
- **Connectors** – A pluggable connector system lets Oxy interface with external services. Each connector handles auth, token refresh, and API calls.
- **Apple Shortcuts Bridge** – An included `.shortcut` file and generator script let Oxy trigger native iOS actions (iMessage, Reminders, HomeKit, etc.)

## Architecture

...



```

graph TD
 subgraph Frontend_PWA [Frontend PWA]
 React[React 18 · Babel in-browser · Single index.html]
 Voice[Voice recording · SSE streaming · Service Worker]
 end

 Frontend_PWA --- HTTPS[HTTPS]

 subgraph API_Server [API Server Express 5]
 api_index[api/index.js - chat, audio, memory, connectors, auth]
 api_proxy[api/proxy.js - action dispatch helper]
 end

 Frontend_PWA --- API_Server
 API_Server --- HTTPS

 subgraph Supabase [Supabase]
 postgres[(Postgres)]
 tables[Tables: memories, conversations, action_log, connectors, preferences]
 end

 subgraph Connector_System [Connector System]
 google[connectors/google.js - Gmail + Cal]
 telegram[connectors/telegram.js - Messaging]
 uber[connectors/uber.js - Ride booking]
 trainline[connectors/trainline.js - Train search]
 index[connectors/index.js - Registry]
 end

 Supabase --- Connector_System

 subgraph External_APIS [External APIs]
 gemini_llm[Gemini LLM + TTS]
 gemini_stt[Gemini STT]
 google[Gmail, Cal]
 telegram[Telegram User API]
 spotify[Spotify Web API]
 twilio[Twilio SMS/calls]
 transport[TransportAPI trains]
 maps[Google Maps Geocoding]
 home[Home Assistant]
 end

 Connector_System --- External_APIS

```



...

### Frontend

The entire UI lives in a single `index.html` file – React 18 loaded from CDN, compiled in-browser by Babel. No build step required. It's a Progressive Web App.

**Pages:**
| Page | Description |
|-----|-----|
| Chat | Main conversation view with text input and voice recording (hold-to-talk) |
| Connectors | Toggle connected services on/off, OAuth flows for Google & Telegram |
| Memory | View and manage what Oxy remembers about you |
| Action History | Log of every action Oxy has executed |
| Settings | Voice toggle, autonomy level, data management |

### Backend

An Express 5 server deployed as a standard Node.js process on Cloud Run.

**Core flow for a message:**
1. User sends text or audio
2. Audio is transcribed via Gemini
3. Conversation history, memories, preferences, and connected-app context are loaded from Supabase
4. Gemini generates a response (with Google Search grounding enabled)
5. If the response contains an `<action>` block, actions are dispatched to the connector system
6. For data-fetching actions (train times, emails, calendar), results are fed back to Gemini for natural summarisation
7. TTS audio is generated via Gemini and streamed back alongside the text response
8. Memory facts are extracted and saved in the background

### Connector System

Connectors are modular service integrations in `connectors/`. Each exports:
```

- `SUPPORTED\_ACTIONS` — array of action type strings it handles
- `execute(userId, action, params)` — performs the action and returns `{ success, text, ... }`

Currently implemented:

```
| Connector | Actions | Auth |
|-----|-----|-----|
| **Google** | `send_email`, `get_emails`, `search_emails`, `create_calendar_event`, `get_calendar_events` | OAuth 2.0 (per-user tokens stored in db) |
| **Telegram** | `send_telegram`, `get_telegram_contacts` | Telegram User API with phone verification + optional 2FA |
| **Uber** | `book_uber` | Deep links (no auth required) |
| **Trainline** | `search_trains` | TransportAPI keys + Trainline booking URLs |
```

Planned (UI-visible but not yet implemented): iMessage, WhatsApp, Spotify, Apple Reminders, Deliveroo, Monzo, HomeKit, Google Maps, Notion, etc.

MCP Server

`mcp-server.js` is a standalone Express server (port 3100) exposing Oxy's tools via a simple `POST /tools` JSON-RPC interface. This enables external tools to interact with Oxy.

Database (Supabase)

```
| Table | Purpose |
|-----|-----|
| `conversations` | Chat history (role, content, timestamps per user) |
| `memories` | Extracted personal facts about each user |
| `action_log` | Audit trail of every action executed |
| `connectors` | Per-user connector state and OAuth tokens |
| `preferences` | Learned user preferences (response length, tone, format) |
```

Schema is in `supabase-migration.sql`.

Getting Started

Prerequisites

- Node.js 18+
- A Supabase project (free tier works)
- API keys for the services you want to use

Setup

1. **Clone the repo**

```
```bash
git clone https://github.com/commonsfounder/Oxy.git
cd Oxy
```
```

2. **Install dependencies**

```
```bash
npm install
```
```

3. **Configure environment variables**

```
```bash
cp .env.example .env
```
```

Fill in the values — at minimum you need:

- `SUPABASE\_URL` and `SUPABASE\_KEY` — for the database
- `GEMINI\_API\_KEY` — for the AI (conversation + TTS + speech-to-text)
- `OXY\_SESSION\_SECRET` — signs per-user login sessions

Optional (enable more connectors):

- `GMAIL\_CLIENT\_ID`, `GMAIL\_CLIENT\_SECRET`, `GMAIL\_REFRESH\_TOKEN` — Gmail + Calendar
- `TELEGRAM\_API\_ID`, `TELEGRAM\_API\_HASH` — Telegram messaging
- `TWILIO\_ACCOUNT\_SID`, `TWILIO\_AUTH\_TOKEN`, `TWILIO\_PHONE\_NUMBER` — SMS/calls
- `SPOTIFY\_CLIENT\_ID`, `SPOTIFY\_CLIENT\_SECRET`, `SPOTIFY\_REFRESH\_TOKEN` — Music
- `TRANSPORT\_API\_APP\_ID`, `TRANSPORT\_API\_APP\_KEY` — Live train times
- `GOOGLE\_MAPS\_API\_KEY` — Geocoding + Google Places lookup for Uber/local destinations
- `APNS\_KEY\_ID`, `APNS\_TEAM\_ID`, `APNS\_BUNDLE\_ID`, `APNS\_PRIVATE\_KEY` — iOS proactive push notifications
- `APNS\_USE\_SANDBOX=true` — use APNs sandbox while testing development builds
- `HOME\_ASSISTANT\_URL`, `HOME\_ASSISTANT\_TOKEN` — Smart home

4. **Run the database migration**

Execute the SQL in `supabase-migration.sql` against your Supabase project (via the SQL editor in the Supabase dashboard).

5. **Start the server**

```
```bash
npm start # production
npm run dev # development (with nodemon)
```
```

The app is available at `http://localhost:3000` (or whichever port Express binds to).

6. **Create your first user**

Open the app and register with a user ID and password. Oxy now uses per-user accounts and signed sessions instead of a shared API secret.

Deploying to Cloud Run

Cloud Run is the primary deploy target for Oxy. This repo now runs as a standard Node service and includes:

- `server.js` listening on `0.0.0.0:\$PORT`
- `Dockerfile` for Cloud Run source or container deploys
- `.dockerignore` to keep deploys lean
- `cloudrun.env.example.yaml` for runtime env vars
- `/realtime-voice` WebSocket handling for Gemini Live sessions

1. ****Install and auth `gcloud`****

```
```bash
gcloud auth login
gcloud config set project YOUR_PROJECT_ID
```
```

2. ****Enable the required services****

```
```bash
gcloud services enable run.googleapis.com cloudbuild.googleapis.com artifactregistry.googleapis.com
```
```

3. ****Create your Cloud Run env file****

```
```bash
cp cloudrun.env.example.yaml cloudrun.env.yaml
```
```

Fill in the real values. For production, Secret Manager is better than plain env files, but this is the fastest path to a working deploy. If you want Gemini Live over Vertex AI instead of the Developer API, also fill in:

- `GOOGLE\_GENAI\_USE\_ENTERPRISE=true`
- `GOOGLE\_CLOUD\_PROJECT`
- `GOOGLE\_CLOUD\_LOCATION`

4. ****Deploy from source****

```
```bash
gcloud run deploy oxy \
 --source . \
 --region europe-west2 \
 --allow-unauthenticated \
 --env-vars-file cloudrun.env.yaml
```
```

Cloud Run will use the checked-in `Dockerfile` automatically.

5. ****Set `APP\_URL` after the first deploy****

Once the service is up, get its public URL:

```
```bash
gcloud run services describe oxy \
 --region europe-west2 \
 --format="value(status.url)"
```
```

Then update the service so OAuth and browser postMessage checks use the real Cloud Run URL:

```
```bash
gcloud run services update oxy \
 --region europe-west2 \
 --update-env-vars APP_URL=https://YOUR-SERVICE-URL.a.run.app
```
```

6. ****Smoke-test****

Check:

- `GET /health`
- register/login
- typed chat
- hold-to-talk voice
- Google connector OAuth callback

7. ****Run proactive briefings as a separate Cloud Run Job****

The app now includes a standalone proactive runner at `npm run proactive:job`. Deploy that as a Cloud Run Job and schedule it every 15 minutes.

```
```bash

gcloud run jobs create oxy-proactive \
 --source . \
 --region europe-west2 \
 --command npm \
 --args run,proactive:job \
 --tasks 1 \
 --max-retries 1
```
```

Give the job the same runtime env vars/secrets as the main `oxy` service.

Then schedule it:

```
```bash
```

```
gcloud scheduler jobs create http oxy-proactive-every-15m \
 --location europe-west2 \
 --schedule "*/15 * * * *" \
 --uri "https://run.googleapis.com/apis/run.googleapis.com/v1/namespaces/YOUR_PROJECT_ID/jobs/oxy-proactive:run" \
 --http-method POST \
 --oauth-service-account-email YOUR_SERVICE_ACCOUNT@YOUR_PROJECT_ID.iam.gserviceaccount.com
...
```

#### Notes:

- `mcp-server.js` is a separate process. If you still want the MCP server in Cloud Run, deploy it as a second service instead of trying to run b

#### ## Project Structure

```
...
Oxy/
 index.html # Full frontend (React PWA, CSS, all pages)
 server.js # Express entry point
 sw.js # Service worker (offline + push notifications)
 manifest.json # PWA manifest
 package.json # Dependencies and scripts
 .env.example # Environment variable template
 supabase-migration.sql # Database schema
 mcp-server.js # Standalone MCP tool server
 create-shortcut.js # Apple Shortcuts generator
 Oxy.shortcut # Pre-built Apple Shortcut file
 api/
 index.js # Main API (chat, audio, memory, connectors, auth)
 proxy.js # Action dispatch helper
 geocoding.js # Google Maps geocoding helper
 connectors/
 index.js # Connector registry and dispatcher
 google.js # Gmail + Google Calendar connector
 telegram.js # Telegram User API connector
 uber.js # Uber deep-link connector
 trainline.js # UK train search connector
 icons/ # PWA icons (192px, 512px, SVG)
...
```

#### ## How It Works – End to End

1. **\*\*You speak or type\*\*** → The frontend captures audio via `MediaRecorder` or takes text input
2. **\*\*Audio is transcribed\*\*** → Sent to `/process-audio`, transcribed by Gemini
3. **\*\*Context is assembled\*\*** → Your memories, conversation history, preferences, connected apps, and messaging patterns are loaded from Supabase
4. **\*\*Gemini thinks\*\*** → The AI generates a response using all available context, with Google Search grounding for real-world facts
5. **\*\*Actions are executed\*\*** → If the response includes an `<action>` block, each action is dispatched to the relevant connector
6. **\*\*Results are spoken back\*\*** → Gemini TTS converts the response to audio, streamed back as base64 WAV via SSE
7. **\*\*Memory is updated\*\*** → Personal facts are extracted and saved for future conversations
8. **\*\*Preferences evolve\*\*** → If you say things like "be more concise" or "use bullet points", Oxy adapts

#### ## Gemini Live Prototype Branch

The repo also includes a separate companion-focused Gemini Live prototype over WebSocket. It is meant for evaluating a future native companion a

#### ### Prototype path

- WebSocket endpoint: `/companion-live`
- Intended client: companion/mobile app, not the current web UI
- Flow:
  1. client authenticates with a session token as the first socket message
  2. client sends `session.start`
  3. client streams PCM audio chunks with `audio.append`
  4. backend opens one Gemini Live session and forwards audio directly
  5. Gemini handles STT, reasoning, and TTS natively
  6. tool calls are executed through Oxy connectors and returned into the same Live session
  7. live audio and transcripts stream back in real time

#### ### Prototype socket events

##### Client to server:

- `auth`
- `session.start`
- `audio.append`
- `audio.end`
- `session.stop`

##### Server to client:

- `session.connecting`
- `session.authenticated`
- `session.ready`
- `status`
- `telemetry`
- `transcript.user`
- `transcript.assistant`
- `audio`
- `actions`

```
- `turn.complete`
- `error`

Benchmarking the prototype against the current pipeline

Use the included script to compare the current `/process-audio` pipeline with the prototype `/companion-live` path:

```bash
npm run benchmark:voice -- --url https://YOUR-SERVICE.a.run.app --user YOUR_USER --password YOUR_PASSWORD --audio C:\path\sample.wav --voice Aoe
```

The benchmark reports:

- first transcription latency
- first assistant text latency
- first assistant audio latency
- turn complete latency

This is the intended decision tool before committing to a full migration.

License

This project is proprietary. All rights reserved.
```

Comprehensive Overview — OXY\_COMPREHENSIVE\_OVERVIEW\_MAY\_2026.md

```
Oxy Comprehensive Overview - May 2026

Generated from:
- `/Users/chizigamonyewuchi/Desktop/oxy-state-may-2026.md`
- The live codebase in `/Users/chizigamonyewuchi/Documents/Oxy`

Principle used in this document: where the May state doc and the codebase differ, the codebase is treated as the current implementation truth. T

1. What Changed From The May State Doc

This is the most important section because the May doc is slightly behind the repo in several areas.

1.1 Pricing Changed

The May doc says:
- Pendant: £349
- Kickstarter early bird: £299
- Subscription: £24.99/month or £179/year
- First month free

Updated pricing direction:
- Pendant remains positioned around £349 unless deliberately changed later.
- Kickstarter early bird remains sensible at £299.
- Subscription must now be treated as maximum £15/month.
- Annual pricing should be recalculated from the new ceiling. A clean option is £149/year or lower.
- Any subscription margin, milestone, and investor narrative based on £24.99/month is now outdated.

Implications:
- Oxy is more accessible and easier to justify as a daily life assistant.
- The subscription story becomes "premium but not painful" instead of high SaaS pricing.
- API-cost discipline matters more. Gemini Live, caching, shorter speech, deterministic routing, and future on-device intent classification beco
- At £15/month, blended net revenue after App Store/web mix is materially lower than the May model. The product can still work, but usage caps,

Suggested new subscription model:

Plan	Price	Notes
Oxy Core	Included / free trial	Basic setup, limited assistant use, onboarding, connector demos
Oxy Plus	£14.99/month max	Main subscription; voice, memory, proactive briefings, connectors
Annual	£129-£149/year	Keeps monthly equivalent below £12.50
Founder / Kickstarter	3-12 months included	Useful for conversion and launch momentum

1.2 Proactiveness Is No Longer Just "Broken"

The May doc says proactive triggers are written but broken because serverless kills the interval.

The repo now includes a proper direction:
- `proactive-job.js` runs `runProactiveSweep`.
- `npm run proactive:job` exists in `package.json`.
- README documents deploying a separate Cloud Run Job scheduled every 15 minutes.
- API endpoints now exist for:
 - `POST /proactive/:userId/run`
 - `ALL /proactive/sweep`
 - `GET /briefings/:userId`
 - `POST /briefings/:id/read`

Current state:
- Proactive system is implemented as backend logic and has a Cloud Run Job path.
- It still depends on deployment setup, env vars, device registration, native context, and scheduler configuration.
```



- It is better described as "implemented, needs deployment verification" rather than "broken."

### ### 1.3 The iOS App Is No Longer Purely Future-Tense

The May doc says the iOS companion app is blocked until the Mac arrives.

The repo now contains a SwiftUI native app under `OxyApp/OxyApp` with:

- Auth/login/register
- Chat UI
- Streaming SSE chat
- Image chat
- Connectors view
- Memory view
- Action history
- Proactive briefings tab
- Settings
- Native integrations for location, contacts, HealthKit, Reminders, Calendar, MusicKit/Apple Music, MessageUI, FaceTime, and local place search
- Keychain-backed session storage
- APNs/device registration plumbing

Current state:

- The native app exists as code.
- It appears to be a serious companion-app implementation, not merely a placeholder.
- Hardware BLE bridging is not present in the inspected files.
- Dynamic Island / Live Activities are not visibly implemented in the inspected code.
- Accessibility API control is not visibly implemented in the inspected code.

### ### 1.4 Gemini Live Is More Than A Plan

The May doc describes Gemini Live migration as in progress.

The repo now has two WebSocket voice paths:

- `/realtime-voice` in `live.js`
- `/companion-live` in `companion-live.js`

There is also a benchmark script:

- `scripts/benchmark-voice-pipelines.js`

Current state:

- The current PWA path still supports upload/SSE voice through `/process-audio`.
- The chat path supports streaming text and sentence-level TTS chunks.
- Gemini Live is present as a realtime WebSocket implementation and companion prototype.
- Full product migration still needs practical verification across devices.

### ### 1.5 Connector Set Expanded

The May doc lists live connectors as Gmail, Google Calendar, Uber, Telegram, Trainline, with Trainline unreliable.

The repo's implemented connector registry now includes:

- Google: Gmail + Calendar
- Uber
- Uber Eats
- Deliveroo
- Netflix
- Telegram
- Trainline
- Maps

The UI-visible connector list in the current backend is:

- Google
- Netflix
- Deliveroo
- Uber Eats
- Uber
- Maps
- Telegram
- Trainline

Important distinction:

- Some are true API integrations with stored tokens.
- Some are app-opening/deep-link connectors.
- Some degrade gracefully to web links when APIs are unavailable.

### ### 1.6 Action Safety Is More Mature

The May doc talks about action execution generally.

The repo now has explicit action contracts:

- Required fields
- Risk levels
- Confirmation/review mode
- Execution mode
- Recovery metadata
- Connector health diagnosis
- Pending review flow

High-risk actions such as sending email, Telegram messages, calls, and food orders are review-gated rather than blindly executed.

### ### 1.7 Product Is Now Multimodal And Creative

The May doc focuses on voice assistant, memory, and phone control.

The repo also supports:

- Image uploads for chat
- Image generation
- Diagram artifact generation
- Presentation outline/deck artifact generation

These are implemented as internal/native creative tools in `api/index.js` and exposed to the model through the system prompt.

### ### 1.8 Auth Is Now Per-User

The README and code show that Oxy now uses:

- User registration
- User login
- Password hashing with `crypto.scryptSync`
- Signed session tokens
- Per-user Supabase records
- Auth checks on most API routes

This is meaningfully more production-shaped than a shared API secret.

### ### 1.9 MCP Exists, But Is Separate

The repo has `mcp-server.js`, a standalone Express service on port 3100 with a simple `POST /tools` endpoint.

It supports tools like:

- SMS/Telegram messaging
- Reminders
- Calls
- Spotify playback
- Calendar event creation
- Home Assistant control

This is not the same as the main app connector architecture. It should be described as adjacent infrastructure / external-agent bridge, not the

---

## ## 2. Product Definition

Oxy is a wearable AI assistant designed to let a person press a pendant, speak naturally, and have things happen without opening their phone.

Core idea:

- A pendant around the neck.
- Press to talk.
- Oxy listens, understands, remembers, and executes.
- It uses the phone and cloud as the heavy compute/network layer.
- It is positioned as a life operating system, not a chatbot.

What Oxy is:

- A wearable personal assistant.
- A hands-free and screen-light interface for daily life.
- A connector/action system for real services.
- A memory system that learns user context over time.
- A fashion-tech object with swappable shell identity.
- A companion app plus backend today, with pendant hardware as the full product expression.

What Oxy is not:

- Not a phone replacement.
- Not only a note taker.
- Not a coding assistant.
- Not a generic ChatGPT wrapper.
- Not a passive surveillance pendant.
- Not a camera-first product.

Positioning:

- "Where fashion meets intelligence."
- "One intelligence. Infinite expression."

Strategic thesis:

Everyday AI becomes truly useful when it can do three things at once:

- Be present in the moment through voice.
- Know the user through memory and native context.
- Act through real connected services.

---

## ## 3. The Moat

The May doc identifies six moat layers. They still hold, with some updated implementation evidence.

### ### 3.1 Hardware

The hardware vision is a titanium pendant with:

- 38mm circular form factor.
- Press-to-talk interaction.
- Bone conduction output.
- BLE phone bridge.
- Swappable shell system.
- Jewelry-box charger.

Current codebase status:

- Hardware firmware is not present in this repo.
- BLE bridge code is not visible in the Swift app yet.
- Hardware remains a product/roadmap area rather than a current code implementation.

### ### 3.2 Media And Brand

The product depends heavily on visual desirability:

- Fashion object first impression.
- Editorial renders.
- Shell drops.
- Designer collaborations.
- Jewelry-box unboxing.

Current codebase status:

- App visual language exists in PWA and SwiftUI.
- Product renders/assets described in the May doc are not part of this repo except app icons and connector icons.

### ### 3.3 Deep Tech

Implemented or partially implemented:

- Gemini chat.
- Gemini STT.
- Gemini TTS.
- Gemini Live WebSockets.
- Memory extraction.
- Prompt caching.
- Context caching.
- Deterministic intent routing.
- Action contracts.
- Connector health recovery.
- Native context ingestion.

Future:

- On-device intent classification.
- Offline commands.
- Local action triage on the pendant/phone.

### ### 3.4 UX

Implemented:

- PWA chat, memory, settings, connectors, history.
- SwiftUI native chat and tabs.
- Streaming responses.
- Voice playback.
- Action cards.
- Review sheets.
- Deep-link opening.
- Briefing feed.

Planned / not visibly implemented:

- Dynamic Island / Live Activities.
- BLE pendant control.
- Accessibility API phone control.
- Finished jewelry-box UX.

### ### 3.5 Trust

Trust posture:

- Press-to-talk is central to the brand.
- No camera in hardware plan.
- User memory is visible/editable.
- High-risk actions are review-gated.
- Per-user auth exists.
- Connector tokens are stored per user in Supabase.

Still needs:

- Public privacy policy.
- Data deletion/export UX beyond current memory deletion.
- Token encryption strategy beyond database storage.
- Security review before real users.

### ### 3.6 High-Agency AI

Implemented:

- Gmail send/read/search.
- Calendar read/create.
- Telegram send/contacts.

- Uber links.
- Uber Eats links.
- Deliveroo links.
- Netflix links.
- Maps place/directions/trip planning.
- Trainline search/station board.
- Native iOS actions for messages, calls, reminders, calendar, music, health answers, contacts, places.

Planned / partial:

- Accessibility API app control.
- More reliable commerce booking.
- Richer native app automations.
- Broader connector ecosystem.

---

## ## 4. Hardware Overview

The hardware vision is taken from the May state doc because the repo does not contain firmware or CAD.

### ### 4.1 Form Factor

- 38mm circular pendant.
- Worn around the neck.
- Integrated bail.
- Chain included.
- Core + shell architecture.

The core contains:

- MCU.
- Microphone.
- Battery.
- Haptics.
- Bone conduction driver.
- Touch/press input.
- Charging.
- IMU/health sensors depending on version.

The shell provides:

- Fashion identity.
- Material finish.
- Swappable expression.
- Drop/collab revenue.

### ### 4.2 Materials

Target shell:

- Grade 5 titanium.

Reasons:

- Lighter than stainless steel.
- Premium feel.
- Hypoallergenic.
- Strong scratch resistance.
- Better vibration conduction for bone conduction.
- Supports premium pricing.

Face:

- Frosted Gorilla Glass 6.

### ### 4.3 Battery And Connectivity

Battery:

- May doc target: 400-500mAh.

Connectivity:

- BLE-only pendant.
- Phone handles internet.
- Backend handles AI and services.

Intended path:

Pendant -> BLE -> iOS companion app -> WiFi/5G -> Oxy backend -> connectors/Gemini -> response -> phone -> BLE -> per

### ### 4.4 Chips

Prototype:

- ESP32-S3.

Production target:

- nRF52840 via Raytac MDBT50Q-512K.

Reasoning:

- ESP32-S3 is easier for prototyping and possible TensorFlow Lite Micro experiments.
- nRF52840 is more suitable for BLE wearable power use.

### ### 4.5 Bone Conduction

V1:

- Transducer on back of pendant.
- Vibrates through titanium shell into collarbone.

V2:

- Transducer integrated into chain at neck.
- Better acoustic path.
- Requires more complex mechanical/electrical design.

Key blocker:

- Needs physical testing. The May doc is clear that V1 must be tested against the collarbone before committing.

### ### 4.6 Charging

Vision:

- Jewelry box is the charger.
- Qi wireless charging.
- Pendant sits in custom recess.
- Portable enough for travel.

This is strategically important because it makes the object feel like jewelry, not a gadget.

### ### 4.7 Locked BOM From May Doc

| Component                  | Part Number / Target           |
|----------------------------|--------------------------------|
| Production MCU             | Raytac MDBT50Q-512K / nRF52840 |
| Dev MCU                    | ESP32-S3                       |
| Microphone                 | Knowles SPH0645LM4H            |
| Bone conduction transducer | AAC Technologies AT23B46       |
| Bone conduction amp        | TI TPA6205                     |
| Charging IC                | TI BQ25185                     |
| Touch IC                   | Infineon CY8CMBR3102           |
| IMU                        | Bosch BMI270                   |
| Heart rate                 | Maxim MAX30102                 |
| LED                        | Worldsemi WS2812B SMD 5050     |
| Haptic motor               | AAC Technologies NFP-ELV1040   |
| Qi module                  | TDK WR202                      |
| Battery                    | ATL 503040 500mAh              |

### ### 4.8 Manufacturing

PCB:

- 4-layer FR4.
- 38mm circular.
- 1.6mm thickness.
- Black soldermask.
- ENIG finish.
- JLCPCB target.

Shell:

- CNC titanium.
- Xometry or Shenzhen supplier.
- One sample before batch.

Missing:

- Gerbers.
- PCBA quote.
- CNC quote.
- Acoustic test data.
- Thermal/power measurements.
- Waterproofing plan.
- DFM review.

---

## ## 5. Software Architecture

Oxy is currently a hybrid product:

- Node/Express backend.
- PWA frontend.
- Native SwiftUI iOS app.
- Supabase database.
- Gemini model layer.
- Connector/action system.
- Realtime voice prototypes.
- Standalone MCP server.

### ### 5.1 Runtime Stack

Backend:

- Node.js.
- Express 5.1.
- WebSocket via `ws`.
- Multer for file uploads.
- Axios for external APIs.

- Supabase JS client.
- Google Gemini SDKs:
  - `@google/generative-ai`
  - `@google/genai`

#### Frontend PWA:

- Single `index.html`.
- React 18 from CDN.
- Babel in browser.
- Service worker.
- PWA manifest.

#### iOS:

- SwiftUI.
- Observation.
- URLSession.
- SSE parsing.
- Keychain.
- CoreLocation.
- HealthKit.
- Contacts.
- EventKit/Reminders.
- MusicKit/MediaPlayer.
- MessageUI.
- AVFoundation.

#### Deploy:

- Cloud Run primary target.
- Dockerfile included.
- Cloud Run Job intended for proactive sweeps.

### ### 5.2 Main Server Entry

#### `server.js`:

- Imports the Express app from `api/index.js`.
- Creates an HTTP server.
- Attaches `/realtime-voice`.
- Attaches `/companion-live`.
- Listens on `0.0.0.0:\$PORT`, default 3000.

### ### 5.3 Required Environment Variables

#### Required by `runtime.js`:

- `SUPABASE\_URL`
- `SUPABASE\_KEY`
- `GEMINI\_API\_KEY`
- `OXY\_SESSION\_SECRET`

#### Optional:

- `TIMEZONE`
- `APP\_URL`
- Google OAuth env vars.
- Telegram env vars.
- TransportAPI env vars.
- Google Maps / Places env vars.
- APNs env vars.
- Twilio env vars.
- Spotify env vars.
- Home Assistant env vars.
- Vertex/enterprise Gemini env vars.

---

## ## 6. Backend API Surface

Most routes require session auth. Public routes include:

- `/`
- `/health`
- `/install-shortcut`
- `/auth/register`
- `/auth/login`
- `/auth/google/callback`

### ### 6.1 Auth

#### Routes:

- `POST /auth/register`
- `POST /auth/login`

#### Implementation:

- User IDs must match `/^[a-zA-Z0-9\_-]{1,128}\$/`.
- Password must be 8-1024 characters on register.
- Passwords are hashed using Node crypto scrypt with a random salt.
- Session token is signed with HMAC SHA-256.
- Session TTL is 7 days.
- Auth accepted through `Authorization: Bearer ...` or `X-Session-Token`.

### ### 6.2 Chat And Voice

#### Routes:

- `POST /chat`
- `POST /process-audio`
- `POST /chat-with-image`
- `POST /tts-preview`

#### `/chat`:

- Supports JSON chat.
- Supports streaming via SSE when `stream=true`.
- Supports TTS when `tts=true`.
- Loads memory, preferences, history, connector context, location/native hints.
- Uses deterministic quick replies and deterministic routing for some local/travel requests.
- Uses Gemini for general reasoning.
- Parses `<action>` blocks.
- Executes actions through the action runner.
- Streams status events, text chunks, replacement text, actions, audio chunks, and done events.

#### `/process-audio`:

- Accepts uploaded audio.
- Rate-limited to 10 requests per minute per user.
- Transcribes with Gemini.
- Runs the main chat pipeline.
- Generates TTS.
- Streams SSE events.

#### `/chat-with-image`:

- Accepts image upload.
- Sends image to Gemini with message context.
- Can execute resulting actions.
- Can return TTS.

#### `/tts-preview`:

- Generates a short voice preview.

### ### 6.3 Images And Creative Artifacts

#### Routes:

- `POST /images/generate`
- `POST /chat-with-image`

#### Internal actions:

- `generate\_visual`
- `create\_diagram`
- `create\_presentation`

#### Current creative capabilities:

- Generate a visual image from prompt and optional reference image.
- Analyze image context in chat.
- Create a diagram artifact with Mermaid plus generated preview image.
- Create a presentation outline/deck artifact with slide structure and generated cover image.

### ### 6.4 Memory

#### Routes:

- `POST /memory`
- `GET /memory/:userId`
- `DELETE /memory/:userId`

#### Memory behavior:

- User can manually add memory.
- Oxy extracts a short personal fact after interactions.
- Memory is injected into the system prompt.
- Memory is treated as background context, not something to surface randomly.
- `forget\_memory` exists as an action.

#### Memory sources:

- `fact`
- `manual\_profile`
- Other action-derived sources as inserted by code.

### ### 6.5 Preferences

#### Routes:

- `GET /preferences/:userId`
- `DELETE /preferences/:userId`

#### Preference behavior:

- Oxy can learn style/preferences from user instructions.
- Preferences are stored in Supabase.
- Preferences are injected into prompt context.
- Some proactive state is also stored as preferences.

### ### 6.6 Connectors

## Routes:

- `GET /connectors/:userId`
- `POST /connectors`
- Google OAuth:
  - `GET /auth/google/redirect-uri`
  - `GET /auth/google/start`
  - `GET /auth/google/callback`
- Telegram auth:
  - `POST /auth/telegram/start`
  - `POST /auth/telegram/verify`
  - `POST /auth/telegram/2fa`

## Connector UI states:

- `available`
- `connected`
- `needs\_reconnect`
- `needs\_setup`
- `degraded`

## ### 6.7 Actions

## Routes:

- `POST /action-log`
- `GET /action-log/:userId`
- `GET /action-contracts`

## Action system:

- Model returns structured action blocks.
- Actions are validated against contracts.
- Missing fields create recoverable errors.
- High-risk actions can enter pending review.
- User can confirm/cancel/revise pending actions.
- Action results are logged.
- Connector problems are diagnosed into recovery metadata.

## ### 6.8 Native And Devices

## Routes:

- `POST /devices/register`
- `POST /native/context`
- `POST /native/local-action`

## Native context includes:

- Location.
- Health.
- Capabilities.
- Settings.

## Native local actions:

- iOS can execute some requests locally.
- The result is logged to the backend so Oxy memory/history stays coherent.

## ### 6.9 Proactive

## Routes:

- `GET /briefings/:userId`
- `POST /briefings/:id/read`
- `POST /proactive/:userId/run`
- `ALL /proactive/sweep`
- `GET /briefing/:userId`
- `GET /briefing-legacy/:userId`

## Proactive system can create:

- Wake briefing.
- Midday briefing.
- Evening briefing.
- Health alert for unusually low heart rate.
- Location food reminder when near home in evening.
- Follow-up for failed actions that need reconnection or permission.

## Proactive windows:

- Wake: 6-10.
- Midday: 12-14.
- Evening: 17-20.

## Autonomy gating:

- Low/Quiet settings suppress interval briefings.
- Food reminder needs Active/Bold/High style autonomy.
- Health alerts require `healthAlerts`.
- Location reminders require `locationReminders`.

## ### 6.10 History

## Routes:

- `GET /history/:userId`
- `GET /history/:userId/search`
- `GET /history/:userId/around`



```
- `GET /history/:userId/date`
```

History enables:

- Loading latest conversation.
- Searching previous messages.
- Jumping around a point in time.
- Loading a specific date.

```
6.11 Debug/Health/Install
```

Routes:

- `GET /debug/:userId`
- `GET /health`
- `GET /install-shortcut`
- `GET /`

`/debug/:userId` exposes useful implementation flags such as connector state and Gemini key presence, but should be treated carefully in product

```

```

```
7. AI Model Layer
```

```
7.1 Main Models
```

Defaults in code:

- Primary chat model: `gemini-3-flash-preview`
- Fast model: `gemini-3-flash-preview`
- Streaming chat model: fast model unless overridden.
- Gemini Live model configurable via env.
- Image model: `gemini-2.5-flash-image`.

Env overrides:

- `OXY\_REASONING\_MODEL`
- `GEMINI\_MODEL`
- `OXY\_FAST\_MODEL`
- `GEMINI\_FAST\_MODEL`
- `OXY\_STREAM\_MODEL`
- `OXY\_LIVE\_MODEL`

```
7.2 Search Grounding
```

The code has `needsSearch` logic that enables search for:

- Latest/current questions.
- Public figures.
- News.
- Companies.
- Prices.
- Weather/forecast.
- Launch/release/current availability.
- Entity questions likely to have changed.

It avoids search for:

- Messaging.
- Email.
- Calendar.
- Reminders.
- Booking/order commands.
- Music commands.
- Memory operations.
- Personal-context questions.

This matters because Oxy should not hallucinate changing facts.

```
7.3 Prompt Philosophy
```

The system prompt tells Oxy:

- The user leads the conversation.
- Memory is background context only.
- Do not randomly surface stored facts.
- Do not repeat already-stated context.
- Do not mention time/date unless needed.
- Use search/tool/context evidence for changing public facts.
- Stop after one confirmation sentence when an action succeeds.

This is good product behavior. It prevents the assistant from sounding like a creepy memory dump.

```
7.4 Prompt Caching And Context Caching
```

Implemented:

- Prompt cache warming.
- Prompt cache state map.
- User context cache with 5-minute TTL.
- Cache invalidation after action execution or native context changes.

Reason:

- Reduce latency.
- Reduce repeated Supabase reads.

- Improve voice responsiveness.

### ### 7.5 Voice

Current voice paths:

- Upload/SSE path through `/process-audio`.
- Chat SSE path with optional TTS.
- `/realtime-voice` WebSocket path.
- `/companion-live` WebSocket prototype.

TTS:

- Gemini streaming TTS endpoint.
- Multiple prebuilt Gemini voices.
- Default voice: `Aoede`.
- Sentence-level TTS chunking.
- PCM converted to WAV.

Voice settings in clients:

- Voice on/off.
- Voice selection.
- Voice engine selection in PWA between current path and Gemini Live prototype.

### ### 7.6 Gemini Live

Implemented paths:

- `/realtime-voice` in `live.js`
- `/companion-live` in `companion-live.js`

Client events include auth, session start, audio append, audio end, and stop.

Server events include:

- Auth/session status.
- Telemetry.
- User transcript.
- Assistant transcript.
- Audio.
- Actions.
- Turn complete.
- Errors.

Current interpretation:

- Gemini Live is real code now.
- It should be benchmarked and tested on device before being declared the primary product path.

---

## ## 8. Action And Connector System

### ### 8.1 Action Contracts

Actions are defined in `api/action-contracts.js`.

Each contract can include:

- Risk level.
- Required fields.
- Optional fields.
- Aliases.
- Example input.
- Success summary.
- Failure summary.
- Confirmation policy.
- Execution mode.
- Guidance.

### ### 8.2 Implemented Action Types

Messaging / communication:

- `send\_message`
- `make\_call`
- `send\_email`
- `send\_telegram`
- `get\_telegram\_contacts`

Calendar / reminders:

- `create\_reminder`
- `create\_calendar\_event`
- `get\_calendar\_events`

Email:

- `get\_emails`
- `search\_emails`

Travel / location:

- `book\_uber`
- `find\_place`
- `get\_directions`
- `plan\_trip`

```

- `search_trains`
- `station_board`

Food:
- `order_uber_eats`
- `order_deliveroo`

Entertainment:
- `play_music`
- `add_to_music_playlist`
- `search_netflix_title`
- `add_to_netflix_list`

Memory:
- `forget_memory`

Creative:
- `generate_visual`
- `create_diagram`
- `create_presentation`

```

### ### 8.3 Safety Model

Examples:

- Low-risk data lookups execute directly.
- Food orders are review-required.
- Email send is review-required.
- Telegram send is review-required.
- Calls are review-required.
- Some actions open external apps rather than complete purchase/order fully.

This is the right posture for early Oxy:

- Fast where safe.

- Friction where consequences are real.

### ### 8.4 Connector Registry

Current registered connectors:

```

- `google`
- `uber`
- `ubereats`
- `deliveroo`
- `netflix`
- `telegram`
- `trainline`
- `maps`

```

The registry maps action names to connector modules.

### ### 8.5 Google Connector

Actions:

```

- `send_email`
- `get_emails`
- `search_emails`
- `create_calendar_event`
- `get_calendar_events`

```

Auth:

- OAuth2.
- Tokens stored per user in Supabase.
- Refresh token support.
- If refresh fails due to expired/revoked token, connector is marked disconnected.

Notes:

- Gmail send uses raw RFC-ish email payload.
- Gmail read/search fetches message snippets/metadata.
- Calendar creates/reads events.

### ### 8.6 Telegram Connector

Actions:

```

- `send_telegram`
- `get_telegram_contacts`

```

Auth:

- Telegram User API.
- Phone auth.
- Code verification.
- Optional 2FA.
- Session stored per user in Supabase.

Notes:

- Sends through user's Telegram session, not bot-only flow.
- Contact matching is search-based and can fail if the contact is not found.

### ### 8.7 Uber Connector

**Action:**

- `book\_uber`

**Implementation:**

- Builds Uber deep links.
- Uses destination search/geocoding.
- Can use current location for pickup.
- Fallback opens Uber with query rather than direct booking.

**Important:**

- This is not a full Uber REST booking flow in current code.
- The May doc's "Uber REST API, not deep links" is outdated for current repo behavior.

### ### 8.8 Uber Eats Connector

**Action:**

- `order\_uber\_eats`

**Implementation:**

- Builds Uber Eats deep link/search URL.
- Review-gated by action contract.

### ### 8.9 Deliveroo Connector

**Action:**

- `order\_deliveroo`

**Implementation:**

- Builds Deliveroo deep link/search URL.
- Review-gated by action contract.

### ### 8.10 Netflix Connector

**Actions:**

- `search\_netflix\_title`
- `add\_to\_netflix\_list`

**Implementation:**

- Builds Netflix app/web links.
- Does not appear to use private Netflix account API.

### ### 8.11 Trainline Connector

**Actions:**

- `search\_trains`
- `station\_board`

**Implementation:**

- Uses TransportAPI where available.
- Builds Trainline booking URLs.
- Has fallback behavior if live rail data is unavailable.

**Current status:**

- More robust than the May doc implies, but still dependent on TransportAPI permissions and keys.
- The UI marks Trainline degraded if TransportAPI env vars are missing.

### ### 8.12 Maps Connector

**Actions:**

- `find\_place`
- `get\_directions`
- `plan\_trip`

**Implementation:**

- Uses Google Places/Directions keys when configured.
- Falls back to Apple Maps links when Google setup is missing or fails.
- Supports nearby place ranking using location.
- Supports directions modes.
- Supports rail-first trip planning with Trainline booking URLs.

This is one of the biggest codebase additions beyond the May doc.

---

## ## 9. Frontend PWA

The PWA lives almost entirely in `index.html`.

### ### 9.1 Pages

**Pages:**

- Chat
- History
- Connectors

- Memory
- Settings

### ### 9.2 Chat

#### Features:

- Text chat.
- Hold-to-talk recording.
- Realtime voice WebSocket attempt with fallback to upload flow.
- SSE streaming.
- TTS playback.
- Image attachment/chat.
- Action cards.
- Briefing injection.
- Status labels like Listening, Processing, Online.
- Recent action loading.
- Chat history loading.
- History search/date navigation.

### ### 9.3 Connectors

#### Features:

- Shows connector list from backend.
- Google OAuth popup flow.
- Telegram auth flow.
- Toggle connectors on/off.
- Connector status display.

### ### 9.4 Memory

#### Features:

- View memory summary.
- Add manual memory/profile anchor.
- Clear memory.

### ### 9.5 History

#### Features:

- Action history.
- Action statuses.
- Action results.

### ### 9.6 Settings

#### Features:

- Voice on/off.
- Voice engine selection.
- Voice selection.
- Voice preview.
- Autonomy level.
- Local settings persistence.

### ### 9.7 PWA Assets

#### `manifest.json`:

- Name: Oxy.
- Display: standalone.
- Portrait orientation.
- Categories: social, lifestyle.
- Icons included.

#### `sw.js`:

- Caches shell assets.
- Network-first for HTML.
- Cache-first for static assets.
- Handles push notifications.
- Avoids caching Supabase/Google API calls.

---

## ## 10. Native iOS App

The native app is in `OxyApp/OxyApp`.

### ### 10.1 App Structure

#### Main tabs:

- Chat
- Proactive
- History
- Connectors
- Memory
- Settings

#### Core services:

- `APIClient`
- `AuthService`

- `ChatService`
- `LocationManager`
- `NativeIntegrationManager`
- `VoiceInputManager`
- `KeychainHelper`

### ### 10.2 Auth And Session

App state:

- Stores user ID and token.
- Restores session.
- Logs out and clears Keychain.

API client:

- Uses `Authorization: Bearer <token>`.
- Clears session on 401.
- Default backend URL currently points to Cloud Run:
  - `https://oxy-151340634966.europe-west2.run.app`

### ### 10.3 Chat

Features:

- Sends messages through `/chat`.
- Streams SSE events.
- Plays base64 WAV audio.
- Opens deep links from action results.
- Sends image messages through `/chat-with-image`.
- Loads history.
- Searches chat history.
- Supports action review and message composer handoff.

### ### 10.4 Native Local Actions

The native app can intercept some requests and complete them locally before calling the cloud model.

Local/native areas:

- Messages through MessageUI.
- FaceTime/calls.
- Apple Music playback.
- Apple Music add-to-library/playlist.
- Health questions.
- Contact hints.
- Native place search.
- Reminders.
- Calendar events.

The result is logged back to `/native/local-action`.

### ### 10.5 Native Context

The app syncs:

- Location.
- Health snapshot.
- Capabilities.
- Settings.

Endpoint:

- `POST /native/context`

This powers proactive briefings and contextual actions.

### ### 10.6 Permissions

Declared in `Info.plist`:

- Microphone.
- Speech recognition.
- Location when-in-use.
- Location always.
- Contacts.
- Health.
- Apple Music.
- Calendar.
- Reminders.
- Photo library.

Entitlement:

- HealthKit enabled.

### ### 10.7 Proactive View

Features:

- Loads briefings.
- Shows only useful visible briefings.
- Mark as read.
- Manual "check now" path.

### ### 10.8 Missing Or Not Visible In Inspected iOS Code

Not found in inspected files:

- BLE pendant connection.
- CoreBluetooth bridge.
- Dynamic Island / Live Activities.
- Accessibility API automation.
- Watch app.
- Production APNs delegate registration beyond app delegate hook and token registration plumbing.

These should remain roadmap items until implemented.

---

## ## 11. Database Schema

Defined in `supabase-migration.sql`.

### ### 11.1 Tables

`users`:

- `id`
- `user\_id`
- `password\_hash`
- timestamps

`conversations`:

- user messages and assistant messages.
- role/content/timestamp.

`memories`:

- extracted/manual personal facts.
- source.

`action\_log`:

- action JSON.
- status.
- error.
- timestamp.

`connectors`:

- connector ID.
- enabled flag.
- tokens JSON.
- per-user uniqueness.

`preferences`:

- key/value user preferences.

- also used for some proactive sent-state keys.

`devices`:

- platform.
- push token.
- timezone.

`native\_context`:

- latest location.
- health.
- capabilities.
- settings.

`briefings`:

- proactive briefing feed.
- kind/title/body/source/metadata/read fields.

### ### 11.2 Database Strengths

- Simple schema.
- Per-user rows.
- Good indexes on user/time.
- JSONB gives flexibility for tokens, action payloads, native context, metadata.

### ### 11.3 Database Risks

- Token storage strategy needs hardening before production.
- No row-level security policy shown in migration.
- Service key likely used server-side, which is normal, but production data controls need review.
- Preferences as generic key/value can grow messy without conventions.
- Conversations table may grow quickly and needs retention strategy.

---

## ## 12. Security And Privacy

Implemented:

- Per-user auth.

- Signed session tokens.
- Password hashing.
- Most routes require auth.
- User ID matching prevents accessing another user's data through path params/body.
- CORS can be restricted with `APP\_URL`.
- CSP and HSTS headers are set.
- High-risk action review exists.

Needs attention:

- Token encryption at rest.
- Supabase RLS and service-role boundary.
- Debug endpoint exposure.
- OAuth state/session hardening review.
- Data retention/deletion policy.
- Privacy policy.
- Production logging review, especially around messages and action traces.
- Rate limits beyond audio endpoint.
- Abuse controls for image generation/TTS.
- APNs secret handling.

---

## ## 13. Deployment

Primary target:

- Google Cloud Run.

Main service:

- `server.js`
- port from `PORT`.
- Dockerfile included.

Proactive:

- Separate Cloud Run Job.
- `npm run proactive:job`.
- Schedule every 15 minutes with Cloud Scheduler.

Environment:

- `cloudrun.env.example.yaml` exists.
- README documents deployment steps.

Important:

- `npm` was not available in the current Codex shell path during this audit, so automated tests were not run here. The repo does define `npm test`.

---

## ## 14. MCP Server

`mcp-server.js` is a standalone Express app on port 3100.

Tools:

- `send\_message`
- `create\_reminder`
- `get\_reminders`
- `complete\_reminder`
- `make\_call`
- `play\_music`
- `create\_calendar\_event`
- `smart\_home`

Integrations:

- Twilio.
- Telegram Bot API.
- Local JSON reminders file.
- Spotify.
- Google Calendar OAuth refresh credentials.
- Home Assistant.

Interpretation:

- Useful bridge for external agents.
- Not the main app's connector/action system.
- Some actions overlap with main contracts but are implemented separately.
- If kept, it should be aligned with the main connector architecture or clearly documented as experimental/legacy.

---

## ## 15. Business Model

### ### 15.1 Hardware Revenue

Current product pricing assumption:

- Pendant: £349.
- Early bird: £299.
- Collector tier can remain around £449 if it includes extra shell(s) and free subscription period.

### ### 15.2 Subscription Revenue



Updated cap:

- Maximum £15/month.

Recommended:

- £14.99/month.
- £129-£149/year.
- First month free.
- Kickstarter includes 3 months or more depending on tier.

### ### 15.3 Shell Revenue

Launch shells:

- Brushed titanium.
- PVD Noir.
- PVD Gold.

Drop shells:

- Rose gold.
- Seasonal.
- Designer collabs.

Price:

- £79-£89 still plausible.

Strategic role:

- Shells turn hardware into fashion and repeat revenue.
- Drops create cultural moments without requiring new electronics.

### ### 15.4 Unit Economics From May Doc

May estimates:

| Volume      | Unit cost | Margin at £349 |
|-------------|-----------|----------------|
| 100 units   | ~£240-276 | £73-109        |
| 1,000 units | ~£188     | £161           |
| 5,000 units | ~£145     | £204           |

Status:

- Still estimates.
- Need real PCBA and CNC quotes.
- Subscription price change does not affect hardware margin but affects blended company margin.

### ### 15.5 Subscription Economics Need Rework

Old May doc:

- Blended subscription revenue: ~£21.99.
- API cost: ~£5.85.
- Net: ~£16/user/month.

With £14.99/month:

- Web users: near £14.99 before payment fees.
- App Store users: about £10.49 after 30% Apple cut before other costs.
- A 60% web / 40% app blended gross after Apple cut is about £13.19 before payment fees.
- If API costs stayed at £5.85, gross contribution becomes roughly £7.34 before infra/support/payment costs.

Conclusion:

- The subscription can still work, but API cost must be held down.
- Annual web checkout should be pushed strongly.
- Gemini Live latency and cost should be measured, not assumed.
- On-device/local intent routing is financially valuable.

### ### 15.6 Updated Revenue Milestones At £14.99

Very rough contribution model using ~£7-£9/month contribution after Apple/API costs:

| Subscribers | Rough monthly contribution |
|-------------|----------------------------|
| 500         | £3.5k-£4.5k                |
| 1,000       | £7k-£9k                    |
| 5,000       | £35k-£45k                  |
| 10,000      | £70k-£90k                  |

This is lower than the May doc. Hardware margin and shell drops therefore matter more.

---

## ## 16. Go-To-Market

The May plan is still coherent:

- Build useful TestFlight app before hardware exists.
- Use proactive iOS assistant as the waitlist hook.
- Convert early software users into Kickstarter backers.
- Launch Kickstarter around September target.
- Demo video must show real-life value with phone untouched.

### ### 16.1 Strongest Launch Narrative

"A 20-year-old from Solihull built the wearable assistant Humane tried to make, but as a private, camera-free pendant that actually controls you."

Important:

- Avoid sounding like "another AI chatbot."
- Demonstrate action, not conversation.
- Show phone staying in pocket.
- Show fashion object clearly.
- Show privacy: press-to-talk, no camera.

### ### 16.2 Best Demo Scenarios

High-conversion demos:

- "Text Mum I am on my way."
- "Find the nearest gym and open directions."
- "Book me an Uber home."
- "What train can I take to London by 9?"
- "Remind me to call Sarah tomorrow."
- "What did I say about my exams?"
- "Summarise my latest emails."
- "Add this song to my playlist."
- "I am near home, remind me to sort food."

### ### 16.3 Channels

From May doc:

- Pinterest.
- Reddit.
- Product Hunt.
- Press emails.
- Kickstarter.
- Creator affiliates.
- Shell drops.
- UK jewelry designer collabs.

Additional suggestion from codebase reality:

- TestFlight waitlist should lead with the iOS app capabilities now visible in code: chat, native actions, proactive briefings, connectors, memo

---

## ## 17. Visual Identity

May doc:

- Warm precision.
- Deep space black.
- Teal accent.
- Off-white text.
- Premium dark mode.

Current codebase:

- PWA uses dark UI with stone/warm neutral accents.
- SwiftUI appears to support themes and accent colors.
- Connector icons are included as asset sets.

Potential mismatch:

- The May doc emphasizes teal; the current app styling leans warm stone/taupe in many places.
- Decide whether teal remains the brand accent or whether the app has evolved toward a warmer jewelry-like palette.

Visual assets still needed:

- Jewelry box charger render.
- Core + shell separated render.
- Shell swap animation/diagram.
- Realistic titanium material studies.
- On-neck scale shots.
- App screenshots.
- Hardware-to-app connection visual.

---

## ## 18. Competitive Landscape

The May competitor table remains useful:

- Taya.
- Nirva.
- Poppy.
- Omi.
- Meta Ray-Ban.
- Clicky.
- Gemini Spark.
- Possible Apple pendant.

Oxy's differentiated combination:

- Pendant form factor.
- No camera.
- Press-to-talk trust model.
- Bone conduction.
- Phone/action control.

- Memory.
- Proactive context.
- Fashion shell system.
- Native companion app.
- Real connector execution.

## Risk:

- Big companies can copy software assistants.
- Hardware/fashion/community/brand is the harder-to-copy part.
- Execution speed matters because the product category is moving quickly.

---

## ## 19. Current Working State

Based on the codebase, the following are present:

## Backend:

- Express API.
- Auth.
- Chat.
- SSE streaming.
- Gemini integration.
- TTS.
- STT.
- Image chat.
- Image generation.
- Memory.
- Preferences.
- Action contracts.
- Connector registry.
- Proactive logic.
- Cloud Run deployment documentation.

## PWA:

- Chat.
- Voice recording.
- Realtime voice attempt/fallback.
- Connectors.
- Memory.
- History.
- Settings.
- Service worker.
- Push notification handling.

## iOS:

- SwiftUI app.
- Auth.
- Chat.
- Streaming.
- Audio playback.
- Image messages.
- Native context.
- Native local actions.
- Proactive tab.
- Connectors.
- Memory.
- History.
- Settings.

## Connectors:

- Google.
- Telegram.
- Uber.
- Uber Eats.
- Deliveroo.
- Netflix.
- Trainline.
- Maps.

## Not visibly present:

- Pendant firmware.
- BLE bridge.
- Hardware CAD.
- PCB design.
- Dynamic Island.
- Accessibility API.
- Production billing/subscription.
- Payment flow.
- User-facing privacy/legal docs.
- Full production observability.

---

## ## 20. Main Risks

### ### 20.1 Latency

Voice assistant quality depends on latency.

Current mitigations:

- Gemini Live prototypes.
- Streaming text.
- Sentence-level TTS.
- Prompt/context caching.
- Deterministic routing.

Still needed:

- Real benchmark numbers.
- Device testing.
- Bad-network behavior.
- Audio interruption/cancel behavior.

### ### 20.2 Cost

The new £15/month cap increases pressure.

Need:

- Per-turn cost tracking.
- Model routing.
- TTS duration limits.
- Rate limits.
- Local/native execution first where possible.
- Annual web subscription push.

### ### 20.3 Connector Reliability

Risks:

- OAuth expiry.
- API permission changes.
- Deep links changing.
- TransportAPI limitations.
- Telegram session issues.
- Google verification requirements.

Mitigations already present:

- Connector health diagnostics.
- Reconnect states.
- Fallback links.
- Review gating.

### ### 20.4 Hardware Feasibility

Risks:

- Bone conduction audibility from pendant body.
- Battery life.
- BLE reliability.
- Acoustic privacy/leakage.
- Waterproofing/sweat.
- Titanium CNC cost.
- Charging alignment.

Must test early:

- Cheap bone conduction transducer on collarbone.
- ESP32-S3 BLE/audio prototype.
- Battery draw.
- Mic quality from pendant position.

### ### 20.5 Trust

Risks:

- Always-listening perception.
- Token/data concerns.
- Accidental sends/orders.

Mitigations:

- Press-to-talk.
- No camera.
- Review-gated high-risk actions.
- Visible memory.
- Clear privacy story.

### ### 20.6 App Store / Platform

Risks:

- Accessibility automation restrictions.
- Background execution limits.
- Push notification approval/behavior.
- OAuth app verification.
- Deep-link fragility.

Mitigations:

- Native local actions where allowed.

- TestFlight first.
- Clear user-triggered action model.

---

## ## 21. Roadmap

### ### 21.1 Immediate

Product/code:

- Verify current backend deploy.
- Run smoke tests in an environment with npm available.
- Confirm `/chat`, `/process-audio`, `/companion-live`, and `/realtime-voice` work against Cloud Run.
- Benchmark current voice path vs Gemini Live.
- Verify Google OAuth end-to-end.
- Verify Telegram auth end-to-end.
- Verify Maps/Trainline fallbacks.
- Verify proactive Cloud Run Job schedule.
- Update pricing everywhere to max £15/month.

iOS:

- Build and run SwiftUI app.
- Confirm auth.
- Confirm SSE chat.
- Confirm image chat.
- Confirm local actions.
- Confirm HealthKit/location context sync.
- Confirm briefings.

Business:

- Rewrite economics around £14.99.
- Decide annual price.
- Decide Kickstarter subscription inclusion.

### ### 21.2 June-July

Hardware:

- ESP32-S3 prototype.
- Mic test.
- Bone conduction test.
- BLE phone bridge prototype.
- Resin shell/physical dummy.

Software:

- Add BLE bridge to iOS.
- Add live voice path to native app if not already done.
- Add basic pendant event simulation.
- Add production logging and metrics.

### ### 21.3 August

TestFlight:

- 50-100 users.
- Daily-use loop.
- Proactive briefings.
- Native actions.
- Memory.
- Connector setup.
- Feedback collection.

Marketing:

- Demo script.
- Landing/Kickstarter page.
- Product renders.
- Press story.

### ### 21.4 September-December

Kickstarter:

- Launch.
- Convert TestFlight users.
- Hardware sample.
- PCBA quotes.
- CNC sample.
- Manufacturing plan.

Production:

- nRF52840 PCB.
- KiCad design.
- JLCPCB PCBA.
- Titanium supplier.
- Charger prototype.

### ### 21.5 2027

Scale:

- Fulfil Kickstarter.

- Subscription conversion.
- Shell drops.
- Creator affiliates.
- Angel round only from strength.
- On-device intelligence.
- Deeper native automation.

---

## ## 22. Key Decisions To Make

1. Final subscription price:
  - Recommended: £14.99/month and £129-£149/year.
2. Teal vs warm stone accent:
  - Current app leans warm. May doc says teal. Pick one system.
3. Gemini Live primary path:
  - Decide only after benchmark and device testing.
4. Uber strategy:
  - Current code uses deep links/fallbacks. If REST booking is still desired, that is future work.
5. MCP strategy:
  - Keep separate as external-agent bridge, or merge useful tools into main connector system.
6. Native-first vs cloud-first:
  - iOS code already has strong native local action capability. Leaning into this reduces cost and latency.
7. Privacy/token hardening:
  - Must be addressed before real users.
8. Hardware claim discipline:
  - Do not overclaim bone conduction or battery life until tested.

---

## ## 23. One-Sentence Current State

Oxy has evolved from a product concept with a backend into a real multi-surface assistant stack: authenticated Cloud Run backend, PWA, SwiftUI i

## Backend — API Server

### api/index.js — api/index.js

```
require('dotenv').config();

// Error monitoring – set SENTRY_DSN environment variable to enable
if (process.env.SENTRY_DSN) {
 try {
 const Sentry = require('@sentry/node');
 Sentry.init({
 dsn: process.env.SENTRY_DSN,
 environment: process.env.NODE_ENV || 'production',
 tracesSampleRate: 0.1,
 });
 console.log(JSON.stringify({ severity: 'INFO', event: 'sentry.initialized' }));
 } catch (e) {
 console.error(JSON.stringify({ severity: 'WARN', event: 'sentry.init.failed', error: e.message }));
 }
}

const express = require('express');
const path = require('path');
const crypto = require('crypto');
const cors = require('cors');
const multer = require('multer');
const axios = require('axios');
const { GoogleGenAI: ModernGoogleGenAI } = require('@google/genai');
const { dispatch, IMPLEMENTED_CONNECTORS } = require('../connectors');
const googleConnector = require('../connectors/google');
const telegram = require('../connectors/telegram');
const { inferDeterministicAction } = require('../intent-router');
const { createActionRunner } = require('../services/action-runner');
const {
 isPendingCancelMessage,
 isPendingConfirmMessage,
 isPendingRevisionMessage,
 reviewTitleForAction
} = require('../services/pending-review');
const {
 ACTION_CONTRACTS,
 actionPromptBlock,
 validateActionWithContract
} = require('../action-contracts');
const {
 createGeminiServiceClient,
 createSupabaseServiceClient,
 getMissingRuntimeEnv,
 logMissingRuntimeEnvOnce
} = require('../runtime');
const { getSearchReason, needsSearch } = require('../services/search-intent');
const {
 buildResolvedContext,
 isContextualReference,
 resolveContextualTurn
} = require('../services/context-brain');
const {
 createSessionToken,
 getAuthenticatedUserId,
 hashPassword,
 requireSessionAuth,
 signPayload,
 verifyPassword,
 verifySignedPayload
} = require('../auth');
const { connectorForAction } = require('../services/connector-health');
const { getRuntimeVersion } = require('../services/runtime-version');
const { shouldClarifyPreviousPlace } = require('../services/contextual-routing');

const app = express();
const upload = multer({ storage: multer.memoryStorage(), limits: { fileSize: 25 * 1024 * 1024 } });
const APP_URL = process.env.APP_URL || '';
const ALLOWED_ORIGINS = [APP_URL].filter(Boolean);

// Structured JSON logging
function log(level, event, extra = {}) {
 const entry = { timestamp: new Date().toISOString(), severity: level.toUpperCase(), event, ...extra };
 if (level === 'error') {
 console.error(JSON.stringify(entry));
 } else {
 console.log(JSON.stringify(entry));
 }
}

function escapeHtml(str) {
```

```

 return String(str)
 .replace(/&/g, '&')
 .replace(/</g, '<')
 .replace(/>/g, '>')
 .replace(/"/g, '"')
 .replace(/'/g, ''');
 }

 function safeParseJSON(val) {
 if (typeof val !== 'string') return val;
 try { return JSON.parse(val); } catch { return val; }
 }

 function parseLooseJson(text) {
 if (!text || typeof text !== 'string') return null;
 const cleaned = text.trim().replace(/^(?:json)?\s*/i, '').replace(/\s```$/, '').trim();
 try { return JSON.parse(cleaned); } catch { return null; }
 }

 function escapeLikePattern(value) {
 return String(value || '').replace(/[\%_]/g, match => `\\${match}`);
 }

 const USER_ID_RE = /^[a-zA-Z0-9_-]{1,28}$/;
 const MAX_PASSWORD_LENGTH = 1024;
 function isValidUserId(id) {
 return typeof id === 'string' && USER_ID_RE.test(id);
 }

 function requireValidUserIdValue(userId, res) {
 if (!isValidUserId(userId)) {
 res.status(400).json({ error: 'Valid userId is required.' });
 return false;
 }
 return true;
 }

 function requireMatchingUser(req, res, candidateUserId) {
 if (!requireValidUserIdValue(candidateUserId, res)) return false;

 const authenticatedUserId = getAuthenticatedUserId(req);
 if (!authenticatedUserId || authenticatedUserId !== candidateUserId) {
 res.status(403).json({ error: 'Forbidden' });
 return false;
 }
 return true;
 }

 function humanizeActionType(type) {
 if (!type) return 'Action';
 return String(type)
 .replace(/_/g, ' ')
 .replace(/\b\w/g, ch => ch.toUpperCase());
 }

 function signOAuthState(userId) {
 return signPayload({ type: 'google_oauth', userId }, 15 * 60 * 1000);
 }

 function verifyOAuthState(state) {
 const payload = verifySignedPayload(state);
 if (!payload || payload.type !== 'google_oauth') return null;
 return isValidUserId(payload.userId) ? payload.userId : null;
 }

 app.use(cors({
 origin(origin, cb) {
 if (!origin) return cb(null, true);
 if (!ALLOWED_ORIGINS.length || ALLOWED_ORIGINS.includes(origin)) return cb(null, true);
 return cb(new Error('Not allowed by CORS'));
 },
 credentials: true
 }));
 app.use(express.json());
 app.use((req, res, next) => {
 res.setHeader('X-Oxy-Commit', getRuntimeVersion().gitCommit);
 res.setHeader('Vary', 'Origin');
 res.setHeader(
 'Content-Security-Policy',
 [
 "default-src 'self'",
 "script-src 'self' 'unsafe-inline' https://cdnjs.cloudflare.com",
 "style-src 'self' 'unsafe-inline'",
 "img-src 'self' data: https:",
 "connect-src 'self' https://generativelanguage.googleapis.com https://*.googleapis.com https://api.telegram.org ws: wss:",
 "font-src 'self' data: https:",
 "media-src 'self' blob: data:",
]
);
 });

```



```

 "object-src 'none'",
 "base-uri 'self'",
 "frame-ancestors 'none'"
].join('; ');
});
res.setHeader('Strict-Transport-Security', 'max-age=31536000; includeSubDomains; preload');
res.setHeader('X-Content-Type-Options', 'nosniff');
res.setHeader('X-Frame-Options', 'DENY');
res.setHeader('Referrer-Policy', 'strict-origin-when-cross-origin');
res.setHeader('Permissions-Policy', 'camera=(), microphone=(self), geolocation=(self)');
next();
});

app.use((req, res, next) => {
 const publicPaths = new Set([
 '/',
 '/health',
 '/version',
 '/privacy',
 '/terms',
 '/support',
 '/robots.txt',
 '/humans.txt',
 '/changelog',
 '/install-shortcut',
 '/auth/google/callback',
 '/auth/register',
 '/auth/login',
 '/auth/forgot-password',
 '/auth/reset-password'
]);

 if (publicPaths.has(req.path)) return next();

 // requireSessionAuth verifies signature + expiry, then we check token_version for revocation
 return requireSessionAuth(req, res, async () => {
 const { userId, tokenVersion } = req.auth;
 // Only check token_version if it's present in the token (backwards compat)
 if (tokenVersion !== undefined && tokenVersion !== null) {
 try {
 const { data: userRow } = await supabase
 .from('users')
 .select('token_version')
 .eq('user_id', userId)
 .maybeSingle();
 if (userRow && userRow.token_version !== tokenVersion) {
 log('warn', 'auth.middleware.rejected', { reason: 'token_version_mismatch', userId });
 return res.status(401).json({ error: 'Session expired' });
 }
 } catch (e) {
 log('warn', 'auth.middleware.token_version_check_failed', { error: e.message });
 }
 }
 next();
 });
});

const rateLimitStores = [];
const audioRateLimit = new Map();

function requestIp(req) {
 return String(req.headers['x-forwarded-for'] || '')
 .split(',')[0]
 .trim()
 || req.ip
 || req.socket?.remoteAddress
 || 'unknown';
}

function createRateLimiter(maxHits, windowMs, keyFn = requestIp) {
 const store = new Map();
 rateLimitStores.push({ store, windowMs });
 return (req, res, next) => {
 const key = keyFn(req) || requestIp(req);
 const now = Date.now();
 const recentHits = (store.get(key) || []).filter(t => now - t < windowMs);
 if (recentHits.length >= maxHits) {
 log('warn', 'rate_limit.exceeded', { key, endpoint: req.path });
 return res.status(429).json({ error: 'Too many requests. Try again in a moment.' });
 }
 store.set(key, [...recentHits, now]);
 return next();
 };
}

function userOrIpRateKey(req) {

```

```

const bodyUserId = req.body?.userId;
const authenticatedUserId = getAuthenticatedUserId(req);
return authenticatedUserId || bodyUserId || requestIp(req);
}

const registerRateLimiter = createRateLimiter(5, 60 * 1000);
const loginRateLimiter = createRateLimiter(10, 60 * 1000);
const chatRateLimiter = createRateLimiter(30, 60 * 1000, userOrIpRateKey);
const imageRateLimiter = createRateLimiter(10, 60 * 1000, userOrIpRateKey);
const forgotPasswordRateLimiter = createRateLimiter(3, 60 * 60 * 1000);
const GEMINI_TTS_VOICES = new Set([
 'Zephyr', 'Puck', 'Charon', 'Kore', 'Fenrir', 'Leda', 'Orus', 'Aoede',
 'Callirrhoe', 'Autonoe', 'Enceladus', 'Iapetus', 'Umbriel', 'Algieba',
 'Despina', 'Erinome', 'Algenib', 'Rasalgethi', 'Laomedea', 'Achernar',
 'Alnilam', 'Schedar', 'Gacrux', 'Pulcherrima', 'Achird', 'Zubeneigenubi',
 'Vindematrix', 'Sadachbia', 'Sadaltager', 'Sulafat'
]);

// Prune stale rate-limit entries (skip in serverless — Maps are ephemeral per invocation)
setInterval(() => {
 const now = Date.now();
 for (const { store, windowMs } of rateLimitStores) {
 for (const [key, timestamps] of store) {
 const recent = timestamps.filter(t => now - t < windowMs);
 if (recent.length === 0) store.delete(key);
 else store.set(key, recent);
 }
 }
 for (const [uid, timestamps] of audioRateLimit) {
 const recent = timestamps.filter(t => now - t < 60000);
 if (recent.length === 0) audioRateLimit.delete(uid);
 else audioRateLimit.set(uid, recent);
 }
}, 5 * 60 * 1000).unref();

const supabase = createSupabaseServiceClient();
const genAI = createGeminiServiceClient();
const modernGenAI = new ModernGoogleGenAI({ apiKey: process.env.GEMINI_API_KEY || process.env.GOOGLE_API_KEY });
logMissingRuntimeEnvOnce('api bootstrap');

const CONTEXT_CACHE_TTL = 5 * 60 * 1000;
const CONTEXT_CACHE_MAX = 500;
const contextCache = new Map();

// Prune expired context cache entries (skip in serverless)
setInterval(() => {
 const now = Date.now();
 for (const [uid, entry] of contextCache) {
 if (now - entry.ts > CONTEXT_CACHE_TTL) contextCache.delete(uid);
 }
}, 10 * 60 * 1000).unref();

const TIMEZONE = process.env.TIMEZONE || 'Europe/London';
const PRIMARY_CHAT_MODEL = process.env.OXY_REASONING_MODEL || process.env.GEMINI_MODEL || 'gemini-3-flash-preview';
const FAST_MODEL = process.env.OXY_FAST_MODEL || process.env.GEMINI_FAST_MODEL || 'gemini-3-flash-preview';
const STREAMING_CHAT_MODEL = process.env.OXY_STREAM_MODEL || FAST_MODEL;
const PROMPT_CACHE_TTL = process.env.OXY_PROMPT_CACHE_TTL || '3600s';
const promptCacheStates = new Map();
const PROACTIVE_MORNING_PREF = 'proactive.morning_briefing.date';
const PROACTIVE_FAILURE_PREF = 'proactive.failed_action.id';
const PROACTIVE_WINDOWS = [
 { id: 'wake', label: 'Wake briefing', start: 6, end: 10 },
 { id: 'midday', label: 'Midday briefing', start: 12, end: 14 },
 { id: 'evening', label: 'Evening briefing', start: 17, end: 20 }
];

const DEVICE_PLATFORM_ALLOWLIST = new Set(['ios', 'web']);

setTimeout(() => {
 ensurePromptCacheWarm(null, STREAMING_CHAT_MODEL).catch(() => {});
 if (PRIMARY_CHAT_MODEL !== STREAMING_CHAT_MODEL) {
 ensurePromptCacheWarm(null, PRIMARY_CHAT_MODEL).catch(() => {});
 }
}, 0);

function createRequestTrace(label) {
 const startedAt = Date.now();
 const prefix = `[trace:${label}]`;
 return {
 log(step, extra = '') {
 const suffix = extra ? `${extra}` : '';
 console.log(`${prefix} +${Date.now() - startedAt}ms ${step}${suffix}`);
 },
 async run(step, fn) {
 const opStart = Date.now();
 console.log(`${prefix} +${opStart - startedAt}ms BEGIN ${step}`);
 try {
 const result = await fn();
 }
 }
 };
}

```

```

 console.log(`${prefix} +${Date.now() - startedAt}ms END ${step} (${Date.now() - opStart}ms)`);
 return result;
 } catch (error) {
 console.log(`${prefix} +${Date.now() - startedAt}ms FAIL ${step} (${Date.now() - opStart}ms) ${error.message}`);
 throw error;
 }
}
};
}

function getLocalDateKey(date = new Date()) {
 return new Intl.DateTimeFormat('en-CA', {
 timeZone: TIMEZONE,
 year: 'numeric',
 month: '2-digit',
 day: '2-digit'
 }).format(date);
}

function getLocalHour(date = new Date()) {
 return Number(new Intl.DateTimeFormat('en-GB', {
 timeZone: TIMEZONE,
 hour: '2-digit',
 hour12: false
 }).format(date));
}

function getLocalMinute(date = new Date()) {
 return Number(new Intl.DateTimeFormat('en-GB', {
 timeZone: TIMEZONE,
 minute: '2-digit'
 }).format(date));
}

function getBriefingWindow(now = new Date()) {
 const hour = getLocalHour(now);
 return PROACTIVE_WINDOWS.find(window => hour >= window.start && hour <= window.end) || null;
}

function proactiveSweepAuthorized(req) {
 const configuredSecret = process.env.PROACTIVE_SWEEP_SECRET;
 if (!configuredSecret) return true;
 const provided = req.get('x-proactive-secret') || req.query.secret || req.body?.secret;
 return provided === configuredSecret;
}

function parseJsonObject(value) {
 const parsed = safeParseJSON(value);
 return parsed && typeof parsed === 'object' && !Array.isArray(parsed) ? parsed : {};
}

function base64UrlJson(value) {
 return Buffer.from(JSON.stringify(value)).toString('base64url');
}

function apnsAuthToken() {
 const keyId = process.env.APNS_KEY_ID;
 const teamId = process.env.APNS_TEAM_ID;
 const privateKey = (process.env.APNS_PRIVATE_KEY || '').replace(/\n/g, '\n');
 if (!keyId || !teamId || !privateKey) return '';
 const header = base64UrlJson({ alg: 'ES256', kid: keyId });
 const payload = base64UrlJson({ iss: teamId, iat: Math.floor(Date.now() / 1000) });
 const signingInput = `${header}.${payload}`;
 const signature = crypto.sign('sha256', Buffer.from(signingInput), privateKey).toString('base64url');
 return `${signingInput}.${signature}`;
}

async function sendPushToUser(userId, briefing) {
 const bundleId = process.env.APNS_BUNDLE_ID;
 const token = apnsAuthToken();
 if (!bundleId || !token) return { sent: 0, skipped: true };

 const { data: devices, error } = await supabase
 .from('devices')
 .select('push_token, platform')
 .eq('user_id', userId);
 if (error || !Array.isArray(devices)) return { sent: 0, error: error?.message || 'No devices' };

 const host = process.env.APNS_USE_SANDBOX === 'true' ? 'https://api.sandbox.push.apple.com' : 'https://api.push.apple.com';
 let sent = 0;
 await Promise.all(devices
 .filter(device => device.platform === 'ios' && device.push_token)
 .map(async device => {
 try {
 await axios.post(
 `${host}/3/device/${device.push_token}`,

```

```

 {
 aps: {
 alert: {
 title: briefing.title || 'Oxy',
 body: briefing.body || briefing.text || ''
 },
 sound: 'default',
 'mutable-content': 1
 },
 briefingId: briefing.id,
 kind: briefing.kind
 },
 {
 headers: {
 authorization: `bearer ${token}`,
 'apns-topic': bundleId,
 'apns-push-type': 'alert',
 'content-type': 'application/json'
 },
 timeout: 10000
 }
);
 sent += 1;
} catch (err) {
 console.warn('[push] APNs send failed:', err?.response?.data || err.message);
}
}));
return { sent };
}

```

```

async function createBriefing(userId, { kind, title, body, source = 'proactive', metadata = {}, push = true }) {
 const insert = {
 user_id: userId,
 kind,
 title,
 body,
 source,
 metadata,
 read: false,
 created_at: new Date().toISOString()
 };
 const { data, error } = await supabase
 .from('briefings')
 .insert(insert)
 .select('id, kind, title, body, source, metadata, read, created_at')
 .single();
 if (error) throw error;

 await saveMessage(userId, 'assistant', { text: body, kind: 'briefing' });
 if (push) await sendPushToUser(userId, data).catch(err => console.warn('[push] failed:', err.message));
 return data;
}

```

```
const OXY_SYSTEM_PROMPT = `You are Oxy. Your friend. Actually helpful.`
```

#### CORE ETHOS:

- You're genuinely here to help. Be direct, grounded, and useful.
- You are not a thin command router. Use broad general knowledge, conversation context, memory, native hints, and tools together like a capable
- For timeless knowledge, explain confidently from what you know. For current, personal, local, live, or account-specific facts, ground the answer
- Feel alive without being performative: curious, sharp, warm, and honest. If the user is stressed or annoyed, fix the problem first.
- You are trusted because you are careful with real-world actions, not because you rush them.
- Execute clear requests. Ask for the missing detail when the request is underspecified.
- Never treat casual chat as task completion.
- Understand the user's meaning, not just keywords. If a phrase could be conversation, correction, factual question, or action, use the surrounding
- Back-and-forth matters. Treat short follow-ups like "that one", "is this right", "look it up", "bruh", "not that", and "I mean..." as referring
- Vague references are real intent. "it", "that", "there", "the one", "same", "again", "what about tomorrow", and "no I mean..." should resolve
- You're a person they trust, not a corporate chatbot.
- Talk like a real friend – casual, natural, direct. Avoid performative corporate-speak in chat, but use professional language when the context
- For simple conversational questions, keep replies to a maximum of 2 sentences. Voice replies must be concise.

#### FACTUALITY:

- Say what you know. Don't fill gaps with guesses or confident bullshit.
- You sometimes have Google Search available for current/real-time questions. When search results are present, use them.
- For factual questions about current events, recent news, company information, public figures, prices, schedules, or anything that could have changed
- Do not rely on training data alone for changeable real-world facts when search grounding is available.
- Never invent dates. If a date is not grounded by search/tool/context evidence, omit the date or say you need to check.
- Never invent names. If asked who, which person, or why names were missing, use grounded evidence or say you don't have the names.
- If the user challenges a factual answer ("huh", "what", "why", "that seems wrong", "source?"), re-check the facts instead of switching to personal
- Admit uncertainty plainly: "I don't know" beats making stuff up.
- Don't hallucinate dates, events, details, or claim confidence on things outside your knowledge.
- If the user asks for a factual answer and you are missing evidence, say that clearly and ask one short follow-up or say you don't know.
- If the user asks you to do something but key details are missing, ask only for the missing detail instead of inventing it.
- Never imply you saw, checked, sent, booked, verified, or found something unless tool results or conversation context explicitly show that.
- When using memory, treat it as possibly stale personal context, not as proof of current real-world facts.

#### ACTIONS YOU CAN TAKE:

Return an action block only when the user clearly asks for one of these actions and the required details are present.

If key details are missing, ask for the smallest missing detail instead of inventing it.

```

${actionPromptBlock()}

```

#### ABSOLUTE RULES:

1. Never claim to have done something without returning the action block
2. Use action blocks only for clear action requests, not greetings, reactions, drafts, explanations, or vague intent.
3. Never refuse an action unless it's actively harmful
4. Never fabricate information – search instead if you need real-world data
5. Never say you "can't" do something that's in the actions list above
6. Always include a spoken sentence alongside every action block – never return the action block alone
7. For train/rail questions, prefer a grounded text answer from search over the old transport connector. Do not use `plan_trip`, `search_trains`, or `find_train`
- 7a. Only use `get_directions/plan_trip` for travel when the user explicitly asks you to open a route, navigation, Maps, or a ride handoff. Otherwise use `find_place` or `find_transport`
- 7b. Use `get_directions` for generic local directions, walking, driving, and bus questions when a route summary is useful. Never pretend a route exists if it doesn't
- 7c. If train or route data is unavailable, say why plainly and give the best grounded alternative. Do not paraphrase failures into "there are no results"
- 7d. For follow-ups like "yeah but what train is it", "what platform", or "what about tomorrow", use the recent route/action context instead of the full history
8. If you are unsure, ask a brief clarifying question instead of guessing
9. Separate observed facts from suggestions: suggestions are fine, fabricated facts are not
10. When a workflow would benefit from a visual, deck, preview, diagram, or study aid, use the visual actions above instead of only describing them
11. Recent action results are real state. Don't repeat successful actions unless the user clearly asks you to repeat them.
- 11a. If the user asks a question about a previous action result ("is this right?", "is this the most popular?", "why did you choose this?", "bruh", "what about that?", "what about that?", "what about that?"), refer to the most recent action result
- 11b. If the user asks to act on a recent answer ("play it", "book that", "send it", "open the nearest one"), act on the most recent conversation action
12. If a recent action failed and the user asks to retry, fix, redo, or "do the failed one", retry only the failed action unless they explicitly ask for something else
13. Pay close attention to which previous actions succeeded versus failed before deciding what to do next.
14. When executing communication actions, use the right register for the medium and relationship automatically.
15. Email quality matters. If the user says "email X saying Y", turn Y into a complete, useful email draft instead of copying a terse fragment.
16. Default email tone is warm, clear, and human. Most email is professional or corporate, so use polished business language when the thread calls for it
17. Match requested tone. If the user says casual, friendly, firm, apologetic, confident, less desperate, short, or professional, make the draft match
18. Emails to unknown or professional contacts should be polished, structured, and appropriate to the business context, but not padded. Emails to friends/family can be more casual
19. Messages on conversational channels like iMessage, WhatsApp, or Telegram should be brief, natural, and text-like.
20. Do not send placeholder emails. If the user only says "say hello", "introduce myself", "make it professional", or otherwise gives no real message, do not send an email
- 20a. Never send an email body that is just a generic template. The body must contain specific content from the user, current conversation, memory, or search results
- 20b. If the user asks you to rewrite, improve, make more professional, or lengthen a just-sent email, do not send another email unless they explicitly ask for it
21. If the user asks you to send "a link", the outgoing message must contain an actual URL from the user's message, tool results, or explicit context
- 21a. Calendar beats music. If the user says "calendar", "schedule", or "event", do not use Apple Music just because the phrase contains "add". Use `add_to_calendar` instead
- 21b. If the user corrects you with "I mean..." or "not that", preserve the original task details and only change the misunderstood part.
22. For plain local place requests like "nearest gym", "closest McDonald's", or "coffee near me", use `find_place` with the user's natural phrase
- 22a. For ride/taxi/Uber requests like "get me an Uber to the nearest gym", use `book_uber` and pass the user's natural destination phrase. Do not use `find_place`
- 22b. Missing-info policy: infer low-risk context from device location, memory, or the user's phrase when available; ask only for genuinely blocking information
- 22c. Action risk policy: searches, place lookup, train lookup, directions, and opening Uber/Maps to a destination are low risk. Drafting is medium risk. Do not draft actions that require user input
- 22d. For Apple Music requests: use `play_music` for "play/listen to X"; use `add_to_music_playlist` when the user asks to add a song/album to their playlist
- 22e. For music requests that depend on current facts, charts, rankings, popularity, trends, or words like "right now", first use search grounding
23. Infer the appropriate format from context. The user should not need to specify formatting.
24. If the user asks you to forget, delete, wipe, or remove something from memory, use `forget_memory` instead of just saying you will do it.
25. For "forget that" or "delete that from memory", use scope "recent" unless they clearly mean all memory.

```

function normalizeGeminiHistory(history) {
 const mapped = history.map(m => ({
 role: m.role === 'user' ? 'user' : 'model',
 parts: [{ text: toGeminiHistoryText(m) }]
 }));
 // Drop leading model turns – Gemini requires starting with user
 while (mapped.length > 0 && mapped[0].role !== 'user') mapped.shift();
 // Collapse consecutive same-role turns by keeping only the last
 const out = [];
 for (const msg of mapped) {
 if (out.length > 0 && out[out.length - 1].role === msg.role) {
 out[out.length - 1] = msg;
 } else {
 out.push(msg);
 }
 }
 return out;
}

function parseActions(fullResponse) {
 const match = fullResponse.match(/<action>([\s\S]*)</action>/);
 const spoken = fullResponse.replace(/<action>[\s\S]*</action>/g, '').trim();
 let actions = [];

 if (match) {
 try {
 // Strip markdown code fences Gemini sometimes wraps around JSON
 const raw = match[1].trim().replace(/^```(?:json)?\s*/i, '').replace(/\s*```$/, '').trim();
 const parsed = JSON.parse(raw);
 actions = parsed.actions || [];
 } catch (e) {
 console.warn('[parseActions] failed:', e.message, '| raw:', match[1].trim().slice(0, 200));
 }
 }

 return { spoken, actions };
}

function containsUrl(text) {
 return /\bhttps?:\\\/\\\/S+\/i.test(String(text || ''));
}

```

```

function isLinkSendRequest(message) {
 return /\b(send|text|message|telegram|whatsapp|message|email)\b/i.test(String(message || '')) &&
 /\blink\b/i.test(String(message || ''));
}

function pcmToWav(pcmBuffer, sampleRate = 24000) {
 const numChannels = 1, bitsPerSample = 16;
 const byteRate = sampleRate * numChannels * bitsPerSample / 8;
 const blockAlign = numChannels * bitsPerSample / 8;
 const header = Buffer.alloc(44);
 header.write('RIFF', 0);
 header.writeUInt32LE(36 + pcmBuffer.length, 4);
 header.write('WAVE', 8);
 header.write('fmt ', 12);
 header.writeUInt32LE(16, 16);
 header.writeUInt16LE(1, 20);
 header.writeUInt16LE(numChannels, 22);
 header.writeUInt32LE(sampleRate, 24);
 header.writeUInt32LE(byteRate, 28);
 header.writeUInt16LE(blockAlign, 32);
 header.writeUInt16LE(bitsPerSample, 34);
 header.write('data', 36);
 header.writeUInt32LE(pcmBuffer.length, 40);
 return Buffer.concat([header, pcmBuffer]);
}

function invalidateUserContextCache(userId) {
 if (userId) contextCache.delete(userId);
}

function summarizeActionInput(input) {
 if (!input || typeof input !== 'object') return '';
 const preferredKeys = ['contact', 'to', 'title', 'destination', 'query', 'restaurant', 'item', 'origin', 'topic', 'brief'];
 const values = preferredKeys
 .map(key => input[key])
 .filter(Boolean)
 .slice(0, 3);
 return values.length ? ` (${values.join(' · ')})` : '';
}

function summarizeActionOutcome(entry) {
 const type = entry?.action || entry?.type || 'action';
 const result = entry?.result || {};
 const status = result.success === false ? 'failed' : 'succeeded';
 const detail = (result.error || result.text || '').trim();
 let emailContext = '';
 if (['get_emails', 'search_emails'].includes(type) && Array.isArray(result.emails) && result.emails.length) {
 emailContext = result.emails.slice(0, 3).map((email, index) => {
 const body = String(email.body || email.snippet || '').slice(0, 1200);
 return `Email ${index + 1}: From ${email.from || 'Unknown'} | Subject ${email.subject || '(No subject)'} | Thread ${email.threadId || ''}`.join(' ');
 }).join(' ');
 }
 return ` ${humanizeActionType(type)} ${summarizeActionInput(entry?.input || result?.input)}: ${status}${detail ? ` - ${detail}` : ''}${emailContext}`;
}

function toGeminiHistoryText(message) {
 const content = message?.content || '';
 const actionLines = Array.isArray(message?.actions) ? message.actions.map(summarizeActionOutcome).filter(Boolean) : [];
 if (!actionLines.length) return content || conversationFallbackText(message);
 return [content, 'Action results:', ...actionLines].filter(Boolean).join('\n');
}

function serializeLoggedAction(action, result) {
 return JSON.stringify({
 type: action?.type || '',
 input: action?.input || {},
 status: result?.success ? 'executed' : 'failed',
 resultText: typeof result?.text === 'string' ? result.text.slice(0, 280) : '',
 error: result?.success ? null : (result?.error || null)
 });
}

function getWavDurationMs(buffer) {
 try {
 if (!buffer || buffer.length < 44) return null;
 const sampleRate = buffer.readUInt32LE(24);
 const byteRate = buffer.readUInt32LE(28);
 const dataSize = buffer.readUInt32LE(40);
 if (!sampleRate || !byteRate || !dataSize) return null;
 return Math.round((dataSize / byteRate) * 1000);
 } catch {
 return null;
 }
}

function normalizeTranscript(text) {
 return String(text || '').trim().replace(/^[\"'\s]+|["'\s]+$/g, '');
}

```

```

}

function isImplausibleTranscript(text, durationMs) {
 const normalized = normalizeTranscript(text);
 if (!normalized) return false;
 const words = normalized.split(/\s+/).filter(Boolean);
 if (!durationMs || durationMs < 400) return words.length > 4;
 const wordsPerSecond = words.length / Math.max(durationMs / 1000, 0.5);
 if (durationMs < 1500 && words.length > 7) return true;
 if (durationMs < 2500 && words.length > 12) return true;
 return wordsPerSecond > 4.8;
}

async function transcribeAudio(buffer) {
 const audioBase64Input = buffer.toString('base64');
 const audioPart = { inlineData: { mimeType: 'audio/wav', data: audioBase64Input } };
 const transcribeModel = genAI.getGenerativeModel({ model: FAST_MODEL });
 const durationMs = getWavDurationMs(buffer);

 const prompts = [
 'Transcribe this audio exactly. Return only the spoken words. If any part is unclear, omit it rather than guessing. If there is no clear speech, return an empty string.',
 'Verbatim transcription only. Do not answer the user. Do not infer intent. Do not add any words that are not clearly audible. If unclear, return an empty string.'
];

 let lastTranscript = '';
 for (const prompt of prompts) {
 const response = await transcribeModel.generateContent({
 contents: [{ role: 'user', parts: [{ text: prompt }, audioPart] }],
 generationConfig: { temperature: 0, topP: 0.1, topK: 1 }
 });
 const transcript = normalizeTranscript(response.response.text());
 lastTranscript = transcript;

 if (transcript && !isImplausibleTranscript(transcript, durationMs)) {
 return transcript;
 }
 }

 return isImplausibleTranscript(lastTranscript, durationMs) ? '' : lastTranscript;
}

function firstSentences(text, max = 2) {
 const sentences = text.match(/^[^!?]+[.!?]+/g) || [];
 return (sentences.slice(0, max).join(' ').trim() || text.slice(0, 200)).trim();
}

async function generateStructuredObject(prompt, fallback = null, imageFile = null) {
 const model = genAI.getGenerativeModel({ model: FAST_MODEL });
 const parts = [{ text: `${prompt}\n\nReturn JSON only. No markdown fences.` }];
 if (imageFile?.buffer && imageFile?.mimeType?.startsWith('image/')) {
 parts.push({ inlineData: { mimeType: imageFile.mimeType, data: imageFile.buffer.toString('base64') } });
 }
 const result = await model.generateContent({ contents: [{ role: 'user', parts }] });
 return parseLooseJson(result.response.text()) || fallback;
}

function stripActionMarkupForDisplay(text) {
 if (!text) return '';
 return text
 .replace(/<action>[\s\S]*?</action>/g, '')
 .replace(/<action>[\s\S]*$/g, '');
}

function splitCompleteSentences(text) {
 const cleaned = stripActionMarkupForDisplay(text || '');
 const matches = cleaned.match(/^[^!?]+[.!?]+(?:["']\s*)+?/g) || [];
 return matches.map(sentence => sentence.trim()).filter(Boolean);
}

function extractAlreadyStatedContext(history = []) {
 const seen = new Set();
 const lines = [];
 const recentAssistantTurns = history
 .filter(entry => entry.role === 'assistant')
 .slice(-8);

 for (const turn of recentAssistantTurns) {
 const content = stripActionMarkupForDisplay(turn.content || '').trim();
 if (!content) continue;
 const snippets = (content.match(/^[^!?]+[.!?]+(?:["']\s*)+?/g) || [content])
 .map(s => s.trim())
 .filter(Boolean)
 .slice(0, 2);
 for (const snippet of snippets) {
 const normalized = snippet
 .toLowerCase()
 .replace(/\s+/g, ' ');
 seen.add(normalized);
 }
 }
}

```

```

 .replace(/[\^p{L}\p{N}\s:]/gu, '')
 .trim();
 if (!normalized || normalized.length < 12 || seen.has(normalized)) continue;
 seen.add(normalized);
 lines.push(snippet);
 if (lines.length >= 6) return lines;
 }
}

return lines;
}

function buildDynamicSystemPrompt(memory, preferences, availableActions, userContext, statedContext = []) {
 const timeStr = new Date().toLocaleString('en-GB', { timeZone: TIMEZONE });
 const dateStr = getLocalDateKey();
 return `WHAT YOU KNOW ABOUT THIS PERSON:
 ${memory || 'Nothing yet.'}

 HOW THE USER LIKES THINGS (learned over time):
 ${preferences || 'Still learning.'}

 CONNECTED APPS:
 ${availableActions}

 NATIVE CREATIVE TOOLS:
 - generate_visual for contextual images, mockups, study aids, previews, and supporting visuals
 - create_diagram for explaining systems, concepts, and workflows
 - create_presentation for slide structures and decks

 CONTEXT YOU ALREADY STATED IN THIS CONVERSATION:
 ${statedContext.length ? statedContext.map(line => `~ ${line}`).join('\n') : 'Nothing important has been stated yet.'}

 Current date: ${dateStr}
 Current time for internal reasoning only: ${timeStr}

 RESPONSE RULES:
 - The user leads the conversation. Follow their topic instead of steering into unrelated stored memory.
 - Treat stored memory as background context for understanding, not as content to surface by default.
 - Only mention stored memory when it is directly relevant to what the user just said, asked, or asked you to do.
 - Treat personal fact statements like "my usual station is Birmingham New Street" as memory to acknowledge, not as a place, web, or app search.
 - If a greetings or simple check-ins like "hi", "hey", or "ok", just respond naturally to that message. Do not surface legal cases, health goals,
 - Do not repeat context you already stated earlier in this conversation.
 - Especially avoid repeating time/date, current plans, study topics, or personal brief details unless the user directly asks again.
 - Do not mention the current time or date unless the user asked for it or it is necessary for the action/result.
 - If a factual answer involves public figures, news, violence, legal events, prices, schedules, or recent/current facts, do not provide names,
 - If the user questions or challenges your previous factual answer, correct only the factual issue. Do not answer with meta/persona language.
 - If an action is completed successfully, stop after one confirmation sentence. No follow-up question, no summary, no check-in.

 ${userContext}`;
}

function isEmailReplyDraftRequest(message = '') {
 return /\b(reply|respond|email back|write back|get back to|send (him|her|them) back)\b/i.test(String(message || ''));
}

function senderMemoryContext(memory = '', sender = {}) {
 const needles = [
 sender.name,
 sender.address,
 String(sender.address || '').split('@')[0]
].filter(Boolean).map(value => String(value).toLowerCase());
 if (!needles.length) return '';
 return String(memory || '')
 .split(/\n|;/)
 .map(line => line.trim())

 .filter(line => {
 const lower = line.toLowerCase();
 return needles.some(needle => needle.length >= 3 && lower.includes(needle));
 })
 .slice(0, 8)
 .join('\n');
}

function scoreEmailCandidate(email = {}, message = '') {
 const haystack = [
 email.from,
 email.senderName,
 email.senderAddress,
 email.subject
].filter(Boolean).join(' ').toLowerCase();
 const terms = String(message || '')
 .toLowerCase()
 .split(/^[a-z0-9@._-]+/i)
 .filter(term => term.length >= 3);
 return terms.reduce((score, term) => score + (haystack.includes(term) ? 1 : 0), 0);
}

```



```

}

function findRecentEmailTarget(history = [], message = '') {
 const emails = [];
 for (const turn of [...history].reverse()) {
 for (const entry of [...(turn.actions || [])].reverse()) {
 const action = entry?.action || entry?.type;
 if (!['get_emails', 'search_emails'].includes(action)) continue;
 const resultEmails = entry?.result?.emails;
 if (Array.isArray(resultEmails)) emails.push(...resultEmails);
 }
 if (emails.length) break;
 }
 if (!emails.length) return null;
 const ranked = emails
 .map((email, index) => ({ email, index, score: scoreEmailCandidate(email, message) }))
 .sort((a, b) => b.score - a.score || a.index - b.index);
 return ranked[0]?.email || null;
}

async function buildEmailReplyDraftContext(userId, message, history, memory, preferences, trace = null) {
 if (!isEmailReplyDraftRequest(message)) return '';
 const target = findRecentEmailTarget(history, message);
 if (!target?.threadId) return '';
 try {
 const thread = trace
 ? await trace.run('gmail.thread_context.fetch', () => googleConnector.getThreadContext(userId, target.threadId))
 : await googleConnector.getThreadContext(userId, target.threadId);
 const latestFromThem = [...(thread?.messages || [])]
 .reverse()
 .find(email => (email.senderAddress || email.from) & & (email.senderAddress || email.from) !== target.to) || target;
 const sender = {
 name: latestFromThem.senderName || target.senderName || '',
 address: latestFromThem.senderAddress || target.senderAddress || '',
 raw: latestFromThem.from || target.from || ''
 };
 const senderMemory = senderMemoryContext(memory, sender) || 'No sender-specific memory found.';
 const threadText = String(thread?.text || target.body || target.snippet || '').slice(0, 14000);
 if (!threadText) return '';
 return `GMAIL REPLY DRAFTING CONTEXT:

The user is replying to an existing Gmail thread.
Thread ID: ${target.threadId}
Sender name: ${sender.name || 'Unknown'}
Sender address: ${sender.address || sender.raw || 'Unknown'}
Memory about this sender:
${senderMemory}

User communication style/preferences:
${preferences || 'No explicit communication preferences yet.'}

Full thread text:
${threadText}

Reply drafting instruction:
- If you produce a send_email action for this reply, include thread_id "${target.threadId}", to "${sender.address || sender.raw}", subject "${target.subject}"
- Draft from the full thread, not only the latest snippet.
- Match the user's normal tone, the relationship shown in memory/thread context, and the thread's existing formality.
- If this is a business/corporate thread, be professional, complete, and polished.
- Do not add fake warmth, generic padding, or pleasantries the user would not use.
- Stop when the point is made. No filler.`;
 } catch (err) {
 if (trace) trace.log('gmail.thread_context.fetch_failed', err.message);
 return `GMAIL REPLY DRAFTING CONTEXT:

The user appears to be replying to a Gmail thread, but the full thread could not be fetched: ${err.message}
Use the recent email result only if enough context is visible; otherwise ask one short clarification.`;
 }
}

function buildLocationContext(location) {
 const lat = Number(location?.latitude ?? location?.lat);
 const lng = Number(location?.longitude ?? location?.lng);
 if (!Number.isFinite(lat) || !Number.isFinite(lng)) return '';
 return `CURRENT DEVICE LOCATION:
Latitude: ${lat}
Longitude: ${lng}
Use these coordinates for "near me", "nearest", "closest", pickup, traffic, and local place requests. Do not invent a place; use a tool/action to find it.`;
}

function buildNativeHintsContext(nativeHints) {
 if (!nativeHints || typeof nativeHints !== 'object') return '';
 const lines = [];
 const contacts = Array.isArray(nativeHints.contacts) ? nativeHints.contacts.slice(0, 5) : [];
 if (contacts.length) {
 lines.push('NATIVE CONTACT MATCHES:');
 for (const contact of contacts) {
 const bits = [
 contact.displayName || contact.name,

```

```

 contact.phone &&& `phone ${contact.phone}`,
 contact.email &&& `email ${contact.email}`
].filter(Boolean);
 if (bits.length) lines.push(`- ${bits.join(' ')}`);
 }
}
if (nativeHints.place?.name || nativeHints.place?.address) {
 lines.push('NATIVE PLACE MATCH:');
 lines.push(`- ${[nativeHints.place.name, nativeHints.place.address].filter(Boolean).join(' ')}`);
}
if (!lines.length) return '';
return `${lines.join('\n')}\nUse these native hints to resolve casual references, but still follow action rules and reviews.`;
}

```

```

function buildPendingActionContext(pendingAction) {
 if (!pendingAction?.action) return '';
 return `PENDING ACTION AWAITING REVIEW:
${JSON.stringify(pendingAction.action, null, 2)}

```

If the user is revising it, return the full revised action block and keep it in review. Do not execute, send, book, call, or order until they confirm.

```

function buildResolvedContextBlock(resolvedContext) {
 if (!resolvedContext || !resolvedContext.label) return '';
 const safe = {
 kind: resolvedContext.kind || 'unknown',
 label: String(resolvedContext.label || '').slice(0, 1200),
 source: resolvedContext.source || 'assistant_answer',
 confidence: resolvedContext.confidence || 'low',
 suggestedAction: resolvedContext.suggestedAction || undefined
 };
 return `RESOLVED SHORT-TERM CONTEXT:
${JSON.stringify(safe, null, 2)}

```

Use this to resolve vague follow-ups like "it", "that", "there", "same", "again", "what about tomorrow", and "the other one". If confidence is low, return empty.

```

function buildQuickTurnContext(preferences, statedContext = []) {
 return `FAST TURN MODE:
For tiny greetings or acknowledgements, reply in no more than two very short sentences.
Make the first sentence a tiny acknowledgement of 1-3 words when possible.
Keep the total reply under 10 words unless the user explicitly asks for more.
If the user says "huh", "what", "what do you mean", or similar, briefly clarify the previous answer or admit the confusion. Do not mention personal details.
Do not recap the user's saved memories, plans, recent actions, or personal brief unless they directly asked for that context.
The user leads the conversation. Reply to what they just said instead of surfacing unrelated memory.
Treat memory as background context only. If the user just says hi, say hi back.
Keep it warm, effortless, and concise.
Do not repeat context you already mentioned earlier in this conversation.
Already stated context:
${statedContext.length ? statedContext.map(line => `- ${line}`).join('\n') : '- none'}

```

```

USER STYLE PREFERENCES:
${preferences || 'Still learning.'};

```

```

function isQuickTurnMessage(message) {
 const text = String(message || '').trim();
 if (!text || text.length > 32) return false;
 if (/[/?]/.test(text)) return false;
 const normalized = text.toLowerCase().replace(/[\p{L}\p{N}]/gu, '').trim();
 const wordCount = normalized.split(/\s+/).filter(Boolean).length;
 if (wordCount > 5) return false;
 return /^(hi|hey|hello|yo|sup|hiya|haha|lol|huh|what|wait|ok|okay|kk|cool|nice|great|sure|yep|yes|nah|no|thanks|thank you|morning|good morning)/.test(normalized);
}

```

```

function getDeterministicQuickReply(message) {
 const normalized = String(message || '').toLowerCase().replace(/[\p{L}\p{N}]/gu, '').trim();
 if (/^(hi|hey|hello|yo|sup|hiya)$/.test(normalized)) return 'Hey.';
 if (/^(morning|good morning)$/.test(normalized)) return 'Morning.';
 if (/^(afternoon|good afternoon)$/.test(normalized)) return 'Afternoon.';
 if (/^(evening|good evening)$/.test(normalized)) return 'Evening.';
 if (/^(thanks|thank you)$/.test(normalized)) return 'Anytime.';
 if (/^(haha|lol)$/.test(normalized)) return 'Yeah.';
 if (/^(ok|okay|kk|cool|nice|great|sure|yep|yes)$/.test(normalized)) return 'Got it.';
 if (/^(nah|no)$/.test(normalized)) return 'Got you.';
 return '';
}

```

```

function formatLondonYMD(date = new Date()) {
 const parts = new Intl.DateTimeFormat('en-CA', {
 timeZone: 'TIMEZONE',
 year: 'numeric',
 month: '2-digit',
 day: '2-digit'
 }).formatToParts(date).reduce((acc, part) => {
 if (part.type !== 'literal') acc[part.type] = part.value;
 return acc;
 }, {});
}

```

```

 }, {});
 return `${parts.year}-${parts.month}-${parts.day}`;
}

function addDaysToYMD(ymd, days) {
 const [year, month, day] = String(ymd).split('-').map(Number);
 const date = new Date(Date.UTC(year, month - 1, day + days));
 return date.toISOString().slice(0, 10);
}

function extractRelativeDateYMD(text) {
 const lower = String(text || '').toLowerCase();
 const today = formatLondonYMD();
 if (/^btomorrow\b/.test(lower)) return addDaysToYMD(today, 1);
 if (/^btoday\b/.test(lower)) return today;
 return null;
}

function cleanCalendarTitle(text) {
 return String(text || '')
 .replace(/^(okay|ok|please|pls|can you|could you)\s+/i, '')
 .replace(/\b(i\s+mean\s+)?add\s+(it|that)?\s*to\s+my\s+calendar\b/i, '')
 .replace(/\b(add|create|put|schedule)\b/i, '')
 .replace(/\b(to|in|on)\s+my\s+calendar\b/i, '')
 .replace(/\bfor\s+(today|tomorrow)\b/i, '')
 .replace(/\b(today|tomorrow)\b/i, '')
 .replace(/\ball\s+day\b/i, '')
 .replace(/\bat\s+d{1,2}(?:\d{2})?\s*(am|pm)?\b/i, '')
 .replace(/\s+/g, ' ')
 .trim()
 .replace(/^["]|["]$ /g, '');
}

function isCalendarCorrectionOnly(text) {
 const cleaned = String(text || '')
 .toLowerCase()
 .replace(/[\?!\.]+$/g, '')
 .replace(/\b(today|tomorrow)\b/g, ' ')
 .replace(/\bat\s+d{1,2}(?:\d{2})?\s*(am|pm)?\b/g, ' ')
 .replace(/\s+/g, ' ')
 .trim();
 return /^(no|nah|actually|wait|sorry)?\s*(i\s+mean\s+)?(the\s+)?(my\s+)?calendar$/i.test(cleaned) ||
 /^(no|nah|actually|wait|sorry)?\s*(add|put|make)\s+(it|that|this)\s+(to|in|on)\s+(my\s+)?calendar$/i.test(cleaned);
}

function extractCalendarEventInput(message, fallbackMessage = '') {
 const source = String(message || '');
 const fallback = String(fallbackMessage || '');
 const combined = `${source} ${fallback}`.trim();
 const hasCalendarIntent = /\b(calendar|schedule|event)\b/i.test(combined) ||
 /\b(add|put|create)\b.+ \b(tomorrow|today|all day)\b/i.test(combined);
 if (!hasCalendarIntent) return null;

 const dateYMD = extractRelativeDateYMD(combined);
 if (!dateYMD) return null;

 const allDay = /\ball\s+day\b/i.test(combined);
 const correctionOnly = isCalendarCorrectionOnly(source);
 let title = correctionOnly ? '' : cleanCalendarTitle(source);
 if (!title || /^(it|that|this|calendar)$/i.test(title) || /\bi\s+mean\s+calendar\b/i.test(title)) {
 title = cleanCalendarTitle(fallback);
 }
 if (!title) return null;

 if (allDay) {
 return {
 title,
 start_date: `${dateYMD}T00:00:00`,
 end_date: `${addDaysToYMD(dateYMD, 1)}T00:00:00`,
 timezone: TIMEZONE
 };
 }

 const timeMatch = combined.match(/\bat\s+(\d{1,2})(?:\d{2})?\s*(am|pm)?\b/i);
 let hour = 9;
 let minute = 0;
 if (timeMatch) {
 hour = Number(timeMatch[1]);
 minute = Number(timeMatch[2] || 0);
 const suffix = (timeMatch[3] || '').toLowerCase();
 if (suffix === 'pm' && hour < 12) hour += 12;
 if (suffix === 'am' && hour === 12) hour = 0;
 }
 const start = `${dateYMD}T${String(hour).padStart(2, '0')}:${String(minute).padStart(2, '0')}:00`;
 const endHour = Math.min(hour + 1, 23);
 const end = `${dateYMD}T${String(endHour).padStart(2, '0')}:${String(minute).padStart(2, '0')}:00`;

```

```

 return { title, start_date: start, end_date: end, timezone: TIMEZONE };
}

async function getRecentLoggedActions(userId, trace = null, limit = 8, options = {}) {
 const since = parseClientTimestamp(options.since);
 const fetchActions = () => {
 let query = supabase
 .from('action_log')
 .select('action, status, error, created_at')
 .eq('user_id', userId)
 .order('created_at', { ascending: false })
 .limit(Math.min(Math.max(Number(limit) || 8, 1), 20));
 if (since) query = query.gte('created_at', since.toISOString());
 return query;
 };
 const { data, error } = trace
 ? await trace.run('supabase.action_log.contextual_fetch', fetchActions)
 : await fetchActions();
 if (error || !data) return [];
 return data.map(row => {
 const parsed = safeParseJSON(row.action) || row.action || {};
 return {
 type: parsed.type || parsed.action || '',
 input: parsed.input || {},
 status: parsed.status || row.status || '',
 resultText: parsed.resultText || '',
 error: parsed.error || row.error || '',
 created_at: row.created_at
 };
 });
}

function lastActionOfType(actions, types) {
 const wanted = Array.isArray(types) ? types : [types];
 return actions.find(action => wanted.includes(action.type));
}

function isClarificationRequest(message) {
 return /^(huh|what|what?|wdym|what do you mean|what does that mean|i don't get it)$/i.test(String(message || '').trim());
}

async function inferContextualDeterministicTurn(userId, message, settings, trace = null, options = {}) {
 const text = String(message || '').trim();
 const normalized = text.toLowerCase();
 const historyOptions = { since: options.since };

 if (isContextualReference(text)) {
 const [history, recentActions, memory] = await Promise.all([
 getHistory(userId, trace, 12, historyOptions),
 getRecentLoggedActions(userId, trace, 10, historyOptions),
 getMemory(userId, trace)
]);
 const resolvedTurn = resolveContextualTurn({
 message: text,
 history,
 recentActions,
 memory,
 settings
 });
 if (resolvedTurn?.spokenOnly || resolvedTurn?.actions?.length) {
 if (trace) {
 trace.log('context_brain.resolved', JSON.stringify({
 reason: resolvedTurn.reason,
 kind: resolvedTurn.resolvedContext?.kind,
 label: String(resolvedTurn.resolvedContext?.label || '').slice(0, 140),
 action: resolvedTurn.actions?.[0]?.type || null,
 confidence: resolvedTurn.resolvedContext?.confidence
 }));
 }
 return resolvedTurn;
 }
 }

 if (isClarificationRequest(text)) {
 const history = await getHistory(userId, trace, 10, historyOptions);
 const lastAssistant = [...history].reverse().find(row => row.role === 'assistant' && String(row.content || '').trim());
 if (lastAssistant?.content) {
 return {
 reason: 'clarify_previous_turn',
 spokenOnly: true,

 spoken: `I meant: ${String(lastAssistant.content).trim()}`
 };
 }
 }
}

const isCalendarCorrection = /\b\s+mean\b/i.test(text) && /\bcalendar\b/i.test(text);

```

```

const isDatedCalendarAdd = /\b(add|put|create)\b.+ \b(today|tomorrow|all day|at\s+d{1,2}(\?:\d{2})?\s*(am|pm)?)\b/i.test(text) &&
!/\b(song|album|playlist|music|apple music)\b/i.test(text);
if (/\b(calendar|schedule|event)\b/i.test(text) || isCalendarCorrection || isDatedCalendarAdd) {
 const history = await getHistory(userId, trace, 8, historyOptions);
 const previousUser = [...history].reverse()
 .find((row => row.role === 'user' && row.content !== message && (
 isCalendarCorrection
 ? !isCalendarCorrectionOnly(row.content || '')
 : /\b(calendar|schedule|event|tomorrow|today|all day)\b/i.test(row.content || ''))));
 const input = extractCalendarEventInput(text, previousUser?.content || '');
 if (input) {
 return {
 reason: isCalendarCorrection ? 'calendar_correction' : 'calendar_direct',
 spoken: "I'll add that to your calendar.",
 actions: [{ type: 'create_calendar_event', input }]
 };
 }
}

if (/\b(i'?m|i'm)\s+taking\s+the\s+(bus|train|tube|tram|transit)\b/i.test(normalized) || /^by\s+(bus|train|tube|tram|transit)$/i.test(normalized)) {
 const actions = await getRecentLoggedActions(userId, trace, 8, historyOptions);
 const lastTravel = lastActionOfType(actions, ['get_directions', 'plan_trip']);
 const destination = lastTravel?.input?.destination;
 if (destination) {
 const mode = /\b(bus|train|tube|tram|transit)\b/i.test(normalized) ? 'transit' : (settings?.preferredTransportMode || 'transit');
 const input = { destination, mode };
 if (lastTravel.input?.origin) input.origin = lastTravel.input.origin;
 if (lastTravel.input?.arrival_time) input.arrival_time = lastTravel.input.arrival_time;
 if (lastTravel.input?.departure_time) input.departure_time = lastTravel.input.departure_time;
 return {
 reason: 'travel_mode_correction',
 spoken: "I'll redo that for transit.",
 actions: [{ type: 'get_directions', input }]
 };
 }
}

if (shouldClarifyPreviousPlace(normalized)) {
 const actions = await getRecentLoggedActions(userId, trace, 8, historyOptions);
 const lastPlace = lastActionOfType(actions, 'find_place');
 if (lastPlace?.input?.query) {
 return {
 reason: 'place_result_clarification',
 spokenOnly: true,
 spoken: `That was the nearest result Places returned for "${lastPlace.input.query}". If it looks wrong, ask me to re-check nearby and I'll find a better one.`
 };
 }
}

return null;
}

function getPromptCacheState(modelName = STREAMING_CHAT_MODEL) {
 const cacheKey = `${modelName}:${OXY_SYSTEM_PROMPT}`;
 let cacheState = promptCacheStates.get(cacheKey);
 if (!cacheState) {
 cacheState = { key: cacheKey, name: '', expireAt: 0, pending: null };
 promptCacheStates.set(cacheKey, cacheState);
 }
 return cacheState;
}

async function ensurePromptCacheWarm(trace = null, modelName = STREAMING_CHAT_MODEL) {
 const cacheState = getPromptCacheState(modelName);
 if (cacheState.name && Date.now() < cacheState.expireAt) {
 if (trace) trace.log('prompt_cache.hit', cacheState.name);
 return cacheState.name;
 }
 if (cacheState.pending) {
 if (trace) trace.log('prompt_cache.pending');
 return cacheState.pending;
 }
 cacheState.pending = (async () => {
 try {
 const cached = trace
 ? await trace.run('gemini.caches.create', () => modernGenAI.caches.create({
 model: modelName,
 config: {
 displayName: `oxy-base-system-prompt-${modelName.replace(/^[a-z0-9-]+/gi, '-')}`,
 systemInstruction: OXY_SYSTEM_PROMPT,
 ttl: PROMPT_CACHE_TTL
 }
 }))
 : await modernGenAI.caches.create({
 model: modelName,
 config: {

```

```

 displayName: `oxy-base-system-prompt-${modelName.replace(/^[a-z0-9-]+/gi, '-')}`,
 systemInstruction: OXY_SYSTEM_PROMPT,
 ttl: PROMPT_CACHE_TTL
 }
 });
 cacheState.name = cached?.name || '';
 cacheState.expireAt = Date.now() + 55 * 60 * 1000;
 if (trace) trace.log('prompt_cache.created', cacheState.name || 'no-name');
 return cacheState.name;
 } catch (error) {
 if (trace) trace.log('prompt_cache.unavailable', error.message);
 return '';
 } finally {
 cacheState.pending = null;
 }
})();
return cacheState.pending;
}

function getPromptCacheName(trace = null, modelName = STREAMING_CHAT_MODEL) {
 const cacheState = getPromptCacheState(modelName);
 if (cacheState.name && Date.now() < cacheState.expireAt) {
 if (trace) trace.log('prompt_cache.hit', cacheState.name);
 return cacheState.name;
 }
 if (cacheState.pending) {
 if (trace) trace.log('prompt_cache.pending');

 return cacheState.name || '';
 }
 if (trace) trace.log('prompt_cache.warm_start');
 ensurePromptCacheWarm(null, modelName).catch(() => {});
 return cacheState.name || '';
}

function buildModernGenerateRequest({ dynamicSystemPrompt, useSearch, cachedContentName, baseHistory, userContent }) {
 // Keep control instructions authoritative. Cached prompts force dynamic rules into
 // conversation content, which is too weak for tool use and factuality.
 const canUseCachedPrompt = false;
 const config = {
 systemInstruction: `${OXY_SYSTEM_PROMPT}\n\n${dynamicSystemPrompt}`.trim(),
 temperature: useSearch ? 0.1 : 0.2,
 topP: 0.8,
 topK: 20
 };
 if (useSearch) config.tools = [{ googleSearch: {} }];
 const firstUserText = typeof userContent?.parts?.[0]?.text === 'string' ? userContent.parts[0].text : '';
 if (isQuickTurnMessage(firstUserText)) {
 config.maxOutputTokens = 32;
 config.temperature = 0.1;
 }

 const dynamicContextParts = canUseCachedPrompt
 ? [{ text: `Persistent user context for this conversation:\n\n${dynamicSystemPrompt}` }]
 : [];

 return {
 config,
 contents: [
 ...baseHistory,
 ...(dynamicContextParts.length ? [{ role: 'user', parts: dynamicContextParts }] : []),
 userContent
]
 };
}

async function recoverEmptyModelResponse({ model, initialRequest, message, trace = null }) {
 const recoveryRequest = {
 config: {
 ...initialRequest.config,
 temperature: 0.2,
 maxOutputTokens: Math.max(initialRequest.config.maxOutputTokens || 0, 512)
 },
 contents: [
 ...initialRequest.contents,
 { role: 'model', parts: [{ text: '[empty response]' }] },
 {
 role: 'user',
 parts: [{
 text: [
 'Your previous response was empty. Recover the turn now.',
 'Answer the user directly, or return a valid action block if an action is clearly needed.',
 'Do not apologize for the empty response unless the user asked about it.',
 'Use search grounding if it is available in this request.',
 '',
 `User message: ${message}`
].join('\n')
]
 }
]
 };
}

```

```

 }}
 }
}
];
};
try {
 const response = trace
 ? await trace.run('gemini.generateContent.empty_recovery', () => modernGenAI.models.generateContent({
 model,
 contents: recoveryRequest.contents,
 config: recoveryRequest.config
 }))
 : await modernGenAI.models.generateContent({
 model,
 contents: recoveryRequest.contents,
 config: recoveryRequest.config
 });
 return (response.text || '').trim();
} catch (error) {
 if (trace) trace.log('gemini.empty_recovery_fail', error.message);
 return '';
}
}

async function runActions(userId, actions) {
 const results = [];
 for (const action of actions) {
 console.log('[action] executing:', action.type, action.input);
 const result = await dispatch(userId, action.type, action.input || {});
 console.log('[action] result:', action.type, JSON.stringify(result));
 results.push({ action: action.type, result });
 await supabase.from('action_log').insert({
 user_id: userId,
 action: serializeLoggedAction(action, result),
 status: result.success ? 'executed' : 'failed',
 error: result.success ? null : (result.error || null),
 created_at: new Date().toISOString()
 });
 }
 invalidateUserContextCache(userId);
 return results;
}

const GEMINI_TTS_MODELS = [
 'gemini-3.1-flash-tts-preview',
 'gemini-2.5-flash-preview-tts'
];
const GEMINI_IMAGE_MODEL = 'gemini-2.5-flash-image';
let preferredTtsModel = null;

function buildVoiceExcerpt(text) {
 const trimmed = String(text || '').trim();
 if (!trimmed) return '';
 const sentences = trimmed.match(/^[^!?]+[.!?]+/g) || [];
 let excerpt = '';
 for (const sentence of sentences.slice(0, 2)) {
 const candidate = `${excerpt} ${sentence}`.trim();
 if (candidate.length > 180) break;
 excerpt = candidate;
 }
 return (excerpt || trimmed.slice(0, 180)).trim();
}

async function generateSpeech(text, voiceName = 'Aoede') {
 if (!text || !text.trim()) return null;
 const safeVoiceName = GEMINI_TTS_VOICES.has(voiceName) ? voiceName : 'Aoede';
 console.log(`[tts] generateSpeech start voice=${safeVoiceName} chars=${text.trim().length}`);
 const failures = [];
 const orderedModels = preferredTtsModel
 ? [preferredTtsModel, ...GEMINI_TTS_MODELS.filter(name => name !== preferredTtsModel)]
 : GEMINI_TTS_MODELS;

 for (const modelName of orderedModels) {
 const controller = new AbortController();
 const timeoutId = setTimeout(() => controller.abort(), 9000);
 try {
 const resp = await axios.post(
 `https://generativelanguage.googleapis.com/v1beta/models/${modelName}:generateContent`,
 {
 contents: [{ parts: [{ text }] }],
 generationConfig: {
 responseModalities: ['AUDIO'],
 speechConfig: { voiceConfig: { prebuiltVoiceConfig: { voiceName: safeVoiceName } } }
 }
 },
 { signal: controller.signal, headers: { 'x-goog-api-key': process.env.GEMINI_API_KEY } }
);
 const base64Audio = resp.data?.candidates?.[0]?.content?.parts?.[0]?.inlineData?.data;
 }
 }
}

```

```

 if (!base64Audio) {
 throw new Error(`Gemini TTS returned empty audio for voice ${safeVoiceName}.`);
 }
 preferredTtsModel = modelName;
 console.log(`[tts] using model ${modelName} with voice ${safeVoiceName}`);
 console.log(`[tts] generateSpeech ready voice=${safeVoiceName} bytes=${Buffer.from(base64Audio, 'base64').length}`);
 return pcmToWav(Buffer.from(base64Audio, 'base64')).toString('base64');
 } catch (err) {
 const detail = err?.response?.data?.error?.message || err?.response?.data || err.message;
 console.error(`[tts] generateSpeech fail voice=${safeVoiceName} model=${modelName}`, detail);
 failures.push(`${modelName}: ${typeof detail === 'string' ? detail : JSON.stringify(detail)}`);
 } finally {
 clearTimeout(timeoutId);
 }
}

throw new Error(`TTS failed (${safeVoiceName}): ${failures.join(' | ')}`);
}

const ACTION_STATUS_LABELS = {
 send_email: 'Sending email',
 get_emails: 'Checking emails',
 search_emails: 'Searching emails',
 create_calendar_event: 'Creating calendar event',
 get_calendar_events: 'Checking calendar',
 book_uber: 'Booking Uber',
 find_place: 'Finding place',
 get_directions: 'Checking directions',
 plan_trip: 'Planning trip',
 send_telegram: 'Sending Telegram message',
 get_telegram_contacts: 'Checking Telegram contacts',
 search_trains: 'Checking train times',
 order_uber_eats: 'Opening Uber Eats',
 order_deliveroo: 'Opening Deliveroo',
 search_netflix_title: 'Searching Netflix',
 add_to_netflix_list: 'Opening Netflix list',
 forget_memory: 'Updating memory',
 generate_visual: 'Generating visual',
 create_diagram: 'Creating diagram',
 create_presentation: 'Building presentation'
};

function getActionStatusLabel(actionType, phase = 'start') {
 const base = ACTION_STATUS_LABELS[actionType] || humanizeActionType(actionType);
 if (phase === 'complete') return `${base} complete`;
 if (phase === 'failed') return `${base} failed`;
 return base;
}

function actionCompletionPhase(result) {
 return result?.success === false ? 'failed' : 'complete';
}

async function* generateSpeechStream(text, voiceName = 'Aoede') {
 if (!text || !text.trim()) return;
 const safeVoiceName = GEMINI_TTS_VOICES.has(voiceName) ? voiceName : 'Aoede';
 console.log(`[tts] generateSpeechStream start voice=${safeVoiceName} chars=${text.trim().length}`);
 const failures = [];
 const orderedModels = preferredTtsModel
 ? [preferredTtsModel, ...GEMINI_TTS_MODELS.filter(name => name !== preferredTtsModel)]
 : GEMINI_TTS_MODELS;

 for (const modelName of orderedModels) {
 const controller = new AbortController();
 const timeoutId = setTimeout(() => controller.abort(), 9000);
 let sawAudio = false;
 try {
 const resp = await axios.post(
 `https://generativelanguage.googleapis.com/v1beta/models/${modelName}:streamGenerateContent?alt=sse`,
 {
 contents: [{ parts: [{ text }] }],
 generationConfig: {
 responseModalities: ['AUDIO'],
 speechConfig: { voiceConfig: { prebuiltVoiceConfig: { voiceName: safeVoiceName } } }
 }
 },
 {
 signal: controller.signal,
 responseType: 'stream',
 headers: {
 Accept: 'text/event-stream',
 'x-goog-api-key': process.env.GEMINI_API_KEY
 }
 }
);
 } catch {
 failures.push(modelName);
 }
 }

 let buffer = '';

```



```

 for await (const rawChunk of resp.data) {
 buffer += rawChunk.toString('utf8');
 const events = buffer.split(/\r?\n\r?\n/);
 buffer = events.pop() || '';

 for (const event of events) {
 const lines = event.split(/\r?\n/).filter(line => line.startsWith('data: '));

 for (const line of lines) {
 const payload = line.slice(6).trim();
 if (!payload || payload === '[DONE]') continue;
 const parsed = JSON.parse(payload);
 const parts = parsed?.candidates?.[0]?.content?.parts || [];
 for (const part of parts) {
 const base64Audio = part?.inlineData?.data || part?.inline_data?.data;
 if (!base64Audio) continue;
 sawAudio = true;
 if (preferredTtsModel !== modelName) {
 preferredTtsModel = modelName;
 console.log(`[tts] using model ${modelName} with voice ${safeVoiceName}`);
 }
 console.log(`[tts] stream chunk ready voice=${safeVoiceName} bytes=${Buffer.from(base64Audio, 'base64').length}`);
 yield pcmToWav(Buffer.from(base64Audio, 'base64')).toString('base64');
 }
 }
 }
 }

 if (!sawAudio) {
 throw new Error(`Gemini TTS returned empty audio for voice ${safeVoiceName}.`);
 }
 return;
 } catch (err) {
 const detail = err?.response?.data?.error?.message || err?.response?.data || err.message;
 console.error(`[tts] generateSpeechStream fail voice=${safeVoiceName} model=${modelName}`, detail);
 failures.push(`${modelName}: ${typeof detail === 'string' ? detail : JSON.stringify(detail)}`);
 } finally {
 clearTimeout(timeoutId);
 }
}

throw new Error(`TTS failed (${safeVoiceName}): ${failures.join(' | ')}`);
}

function createSentenceTtsStreamer({ voiceName, sse, trace = null, onSpeakingStart = null }) {
 let lastSentenceCount = 0;
 let nextEmitIndex = 0;
 const readyAudio = new Map();
 const tasks = [];
 let speakingNotified = false;

 const flushReadyAudio = () => {
 while (readyAudio.has(nextEmitIndex)) {
 const state = readyAudio.get(nextEmitIndex);
 while (state.chunks.length) {
 const audio = state.chunks.shift();
 const chunkIndex = state.sentCount++;
 if (!speakingNotified) {
 speakingNotified = true;
 if (onSpeakingStart) onSpeakingStart();
 }
 sse({ type: 'audio', data: audio, format: 'wav', mimeType: 'audio/wav', seq: nextEmitIndex, chunk: chunkIndex });
 if (trace) trace.log(`tts.chunk_sent.${nextEmitIndex}.${chunkIndex}`);
 }
 if (!state.done) break;
 readyAudio.delete(nextEmitIndex);
 nextEmitIndex += 1;
 }
 };

 const schedule = sentence => {
 const trimmed = sentence.trim();
 if (!trimmed) return;
 const seq = tasks.length;
 if (trace) trace.log(`tts.chunk_schedule.${seq}`, JSON.stringify(trimmed.slice(0, 80)));
 if (seq === 0) {
 const earlyClause = trimmed.match(/^([^\s;:,]{1,24}[^\s;:,])\s+(.+)$/);
 if (earlyClause) {
 schedule(earlyClause[1].trim());
 schedule(earlyClause[2].trim());
 return;
 }
 }
 };

 if (trace) trace.log(`tts.chunk_start.${seq}`);
 const state = { chunks: [], done: false, sentCount: 0 };
 readyAudio.set(seq, state);
 const task = (async () => {

```

```

 try {
 for await (const audio of generateSpeechStream(trimmed, voiceName)) {
 state.chunks.push(audio);
 flushReadyAudio();
 }
 state.done = true;
 flushReadyAudio();
 } catch (error) {
 if (trace) trace.log(`tts.chunk_fail.${seq}`, error.message);
 throw error;
 }
 })();
 tasks.push(task);
};

return {
 ingest(text) {
 const sentences = splitCompleteSentences(text);
 for (let i = lastSentenceCount; i < sentences.length; i += 1) {
 schedule(sentences[i]);
 }
 lastSentenceCount = sentences.length;
 },
 async flushRemainder(text) {
 const cleaned = stripActionMarkupForDisplay(text || '').trim();
 if (!cleaned) return;
 const matches = [...cleaned.matchAll(/[\^.!?]+[.!?]+(?:["'\`\\])+(?/g)];
 let consumedLength = 0;
 for (let i = 0; i < Math.min(lastSentenceCount, matches.length); i += 1) {
 consumedLength = (matches[i].index || 0) + matches[i][0].length;
 }
 const remainder = cleaned.slice(consumedLength).trim();
 if (remainder) schedule(remainder);
 },
 async waitForAll() {
 await Promise.all(tasks);
 flushReadyAudio();
 }
};
}

async function generateImage(prompt, imageFile) {
 if (!prompt || !prompt.trim()) {
 throw new Error('Image prompt is required.');
```

```

async function analyzeImage(prompt, imageFile) {
 if (!imageFile?.buffer) throw new Error('An image attachment is required.');
```

```

 if (!imageFile.mimetype || !imageFile.mimetype.startsWith('image/')) {
 throw new Error('Only image uploads are supported.');
```

```

 }

 const model = genAI.getGenerativeModel({ model: FAST_MODEL });
 const result = await model.generateContent({
 contents: [{
 role: 'user',
 parts: [
 { text: prompt?.trim() || 'Describe this image clearly and practically.' },
 { inlineData: { mimeType: imageFile.mimetype, data: imageFile.buffer.toString('base64') } }
]
 }]
 });

 return {
 success: true,
 text: result.response.text()?.trim() || 'I looked through the image.',
 artifact: {
 type: 'image_analysis',
 image: imageFile.buffer.toString('base64'),
 mimeType: imageFile.mimetype,
 title: 'Attached image'
 }
 };
}

async function createDiagramArtifact(input, imageFile) {
 const topic = input?.topic || input?.brief || 'the topic';
 const goal = input?.goal || input?.usage || 'make the idea easy to understand';
 const attachmentNote = imageFile ? 'An image or screenshot is attached. Incorporate what is visible if relevant.' : '';
 const spec = await generateStructuredObject(
 `Create a clean teaching diagram plan for "${topic}".
Goal: ${goal}
${attachmentNote}

Return strict JSON with:
{
 "title": "short title",
 "summary": "one sentence",
 "mermaid": "valid mermaid flowchart or mindmap syntax",
 "visual_prompt": "prompt for an elegant flat educational diagram preview image"
}`,
 {
 title: topic,
 summary: `A simple diagram for ${topic}.`,
 mermaid: `flowchart TD\n  A[${topic}] --> B[Key idea]\n  B --> C[Outcome]`,
 visual_prompt: `A refined educational diagram about ${topic}, minimal, elegant, dark background, warm neutral accents`
 },
 imageFile || null
);

 const preview = await generateImage(spec.visual_prompt || `An elegant educational diagram about ${topic}.`, imageFile || null);
 return {
 success: true,
 text: spec.summary || `I made a diagram for ${topic}.`,
 artifact: {
 type: 'diagram',
 title: spec.title || topic,
 summary: spec.summary || '',
 mermaid: spec.mermaid || '',
 image: preview.image,
 mimeType: preview.mimeType,
 caption: preview.text
 }
 };
}

async function createPresentationArtifact(input, imageFile) {
 const topic = input?.topic || 'the topic';
 const audience = input?.audience || 'the intended audience';
 const objective = input?.objective || 'explain the topic clearly';

 const slideCount = Math.min(Math.max(Number(input?.slide_count) || 6, 3), 10);
 const attachmentNote = imageFile ? 'A reference image or screenshot is attached. Use it as source context where relevant.' : '';

 const deck = await generateStructuredObject(
 `Create a concise premium presentation outline.
Topic: ${topic}
Audience: ${audience}
Objective: ${objective}
Slides: ${slideCount}
${attachmentNote}

Return strict JSON:

```

```

{
 "title": "deck title",
 "subtitle": "deck subtitle",
 "theme": "short visual direction",
 "slides": [
 {
 "title": "slide title",
 "bullets": ["bullet", "bullet"],
 "speaker_notes": "one or two lines",
 "visual_prompt": "what image or visual should appear on this slide"
 }
]
},
{
 title: topic,
 subtitle: objective,
 theme: 'Clean editorial study deck',
 slides: Array.from({ length: slideCount }, (_, i) => ({
 title: i === 0 ? topic : `Slide ${i + 1}`,
 bullets: ['Key point', 'Supporting detail'],
 speaker_notes: 'Talk through the main point simply.',
 visual_prompt: `An elegant visual supporting ${topic}`
 })))
},
imageFile || null
);

const coverPrompt = deck.slides?.[0]?.visual_prompt || `A premium presentation cover visual for ${topic}, minimalist, intelligent, editorial`;
const cover = await generateImage(coverPrompt, imageFile || null);
return {
 success: true,
 text: `I built a ${slideCount}-slide presentation structure for ${topic}.`,
 artifact: {
 type: 'slide_deck',
 title: deck.title || topic,
 subtitle: deck.subtitle || objective,
 theme: deck.theme || '',
 image: cover.image,
 mimeType: cover.mimeType,
 slides: (deck.slides || []).slice(0, slideCount)
 }
};
}

function normalizeContactLookup(value) {
 return String(value || '').
 .toLowerCase()
 .replace(/^[^a-z0-9+@.]+/g, ' ')
 .replace(/\s+/g, ' ')
 .trim();
}

function looksLikeMessageAddress(value) {
 const text = String(value || '').trim();
 if (!text) return false;
 return /^[^+?@-9][0-9\s()-.]{5,$}/.test(text) || /^[^@\s]+@[^@\s]+\.[^@\s]+$/.test(text);
}

function resolveNativeMessageContact(contact, nativeHints) {
 if (looksLikeMessageAddress(contact)) {
 return { label: contact, value: contact };
 }
 const normalizedContact = normalizeContactLookup(contact);
 const contacts = Array.isArray(nativeHints?.contacts) ? nativeHints.contacts : [];
 const match = contacts.find(candidate => {
 const names = [
 candidate.displayName,
 candidate.phone,
 candidate.email
].map(normalizeContactLookup).filter(Boolean);
 return names.some(name => {
 name === normalizedContact ||
 name.includes(normalizedContact) ||
 normalizedContact.includes(name)
 });
 });
 const value = match?.phone || match?.email || '';
 return {
 label: match?.displayName || contact,
 value: looksLikeMessageAddress(value) ? value : ''
 };
}

async function executeAction(userId, action, params, context = {}) {
 const connectorId = connectorForAction(action);
 if (connectorId & & connectorId !== 'maps') {
 const enabledConnectors = await getEnabledConnectors(userId, context.trace || null);

```

```

 if (!enabledConnectors.includes(connectorId)) {
 return {
 success: false,
 error: `${connectorId} is disabled. Re-enable it in Connectors before confirming this action.`
 };
 }
 }
}

const enrichedParams = {
 ...(params || {}),
 ...(context.location ? { location: context.location } : {})
};

switch (action) {
 case 'send_message': {
 const contact = String(params?.contact || '').trim();
 const message = String(params?.message || '').trim();
 if (!contact || !message) return { success: false, error: 'send_message requires contact and message' };
 const resolvedContact = resolveNativeMessageContact(contact, context.nativeHints);
 if (!resolvedContact.value) {
 return {
 success: false,
 error: `I need a phone number for ${contact}. Turn on Contacts access for Oxy or include the number.`
 };
 }
 return {
 success: true,
 text: `Message ready for ${resolvedContact.label}. Review and tap Send.`,
 cardText: `To ${resolvedContact.label} · ${message}`,
 actionSummary: 'Message ready',
 deepLink: `sms:${encodeURIComponent(resolvedContact.value)}?&body=${encodeURIComponent(message)}`
 };
 }
 case 'make_call': {
 const contact = String(params?.contact || '').trim();
 if (!contact) return { success: false, error: 'make_call requires a contact' };
 return {
 success: true,
 text: `Opening FaceTime for ${contact}.`,
 deepLink: `facetime://${encodeURIComponent(contact)}`
 };
 }
 case 'play_music': {
 const query = String(params?.query || params?.song || params?.title || '').trim();
 if (!query) return { success: false, error: 'play_music requires a query' };
 return {
 success: true,
 text: `Starting playback for ${query}.`,
 cardText: query,
 actionSummary: 'Music requested',
 deepLink: `music://music.apple.com/search?term=${encodeURIComponent(query)}`,
 webLink: `https://music.apple.com/search?term=${encodeURIComponent(query)}`,
 nativeExecution: 'music'
 };
 }
 case 'add_to_music_playlist': {
 const query = String(params?.query || params?.song || params?.title || '').trim();
 const playlist = String(params?.playlist || params?.playlistName || '').trim();
 if (!query) return { success: false, error: 'add_to_music_playlist requires a query' };
 return {
 success: true,
 text: playlist
 ? `Opening Apple Music for ${query}. Add it to ${playlist} there.`
 : `Opening Apple Music for ${query}.`,
 cardText: playlist ? `${query} · ${playlist}` : query,
 actionSummary: playlist ? 'Music ready' : 'Music opened',
 deepLink: `music://music.apple.com/search?term=${encodeURIComponent(query)}`,
 webLink: `https://music.apple.com/search?term=${encodeURIComponent(query)}`
 };
 }
 case 'forget_memory': {
 return forgetMemory(userId, params || {});
 }
 case 'generate_visual': {
 const brief = params?.brief || params?.prompt || params?.topic;
 if (!brief) return { success: false, error: 'generate_visual needs a brief.' };
 const prompt = [
 brief,
 params?.style ? `Style: ${params.style}` : '',
 params?.usage ? `Usage: ${params.usage}` : ''
].filter(Boolean).join('\n');
 const visual = await generateImage(prompt, context.imageFile || null);
 return {
 success: true,
 text: visual.text || 'I made a visual for this.',
 artifact: {
 type: 'image',
 title: params?.usage || 'Generated visual',
 }
 };
 }
}

```

```

 image: visual.image,
 mimeType: visual.mimeType
 }
 };
}
case 'create_diagram':
 return createDiagramArtifact(params || {}, context.imageFile || null);
case 'create_presentation':
 return createPresentationArtifact(params || {}, context.imageFile || null);
default:
 return dispatch(userId, action, enrichedParams);
}
}

const executeActions = createActionRunner({
 executeAction,
 invalidateUserContextCache,
 setPendingAction,
 validateAction: validateActionWithContract,
 logAction: (userId, action, result) => supabase.from('action_log').insert({
 user_id: userId,
 action: serializeLoggedAction(action, result),
 status: result.pending ? 'pending' : result.success ? 'executed' : 'failed',
 error: result.success ? null : (result.error || null),
 created_at: new Date().toISOString()
 })
});

async function getMemory(userId, trace = null) {
 const fetchMemory = () => supabase
 .from('memories')
 .select('content, source')
 .eq('user_id', userId)
 .order('created_at', { ascending: false })
 .limit(50);
 const { data, error } = trace
 ? await trace.run('supabase.memories.fetch', fetchMemory)
 : await fetchMemory();

 if (error || !data) return '';
 const manualProfile = data.find(m => m.source === 'manual_profile')?.content?.trim();
 const facts = data
 .filter(m => m.source !== 'manual_profile')
 .map(m => m.content)
 .filter(Boolean);
 return [manualProfile, ...facts].filter(Boolean).join('\n');
}

async function saveMemory(userId, content, source = 'fact') {
 if (source === 'manual_profile') {
 const { data: inserted } = await supabase
 .from('memories')
 .insert({ user_id: userId, content, source, created_at: new Date().toISOString() })
 .select('id');

 if (inserted?.[0]?.id) {
 await supabase
 .from('memories')
 .delete()
 .eq('user_id', userId)
 .eq('source', 'manual_profile')
 .neq('id', inserted[0].id);
 }
 return;
 }

 await supabase
 .from('memories')
 .insert({ user_id: userId, content, source, created_at: new Date().toISOString() });
}

async function forgetMemory(userId, { scope = '', query = '' } = {}) {
 const normalizedScope = String(scope || '').toLowerCase();
 const normalizedQuery = String(query || '').trim();

 if (normalizedScope === 'all') {
 const { error } = await supabase.from('memories').delete().eq('user_id', userId);
 if (error) throw error;
 return { success: true, text: 'I cleared what I had in memory.' };
 }

 if (normalizedScope === 'recent') {
 const { data, error } = await supabase
 .from('memories')
 .select('id, content')
 .eq('user_id', userId)
 .order('created_at', { ascending: false })

```

```

 .limit(1);
 if (error) throw error;
 if (!data?.length) return { success: true, text: 'There was nothing stored to forget.' };
 const { error: deleteError } = await supabase.from('memories').delete().eq('id', data[0].id);
 if (deleteError) throw deleteError;
 return { success: true, text: 'I forgot the most recent memory.' };
 }

 if (normalizedQuery) {
 const { data, error } = await supabase
 .from('memories')
 .select('id, content')
 .eq('user_id', userId)
 .ilike('content', `%${escapeILikePattern(normalizedQuery)}%`);
 if (error) throw error;
 if (!data?.length) {
 return { success: true, text: `I couldn't find anything stored about "${normalizedQuery}"` };
 }
 const ids = data.map(row => row.id);
 const { error: deleteError } = await supabase.from('memories').delete().in('id', ids);
 if (deleteError) throw deleteError;
 return {
 success: true,
 text: ids.length === 1
 ? `I forgot what I had stored about "${normalizedQuery}"`
 : `I removed ${ids.length} memories about "${normalizedQuery}"`
 };
 }

 return { success: false, error: 'forget_memory needs scope "recent" or "all", or a query.' };
}

async function getMemorySummary(userId) {
 const { data, error } = await supabase
 .from('memories')
 .select('content, source, created_at')
 .eq('user_id', userId)
 .order('created_at', { ascending: false })
 .limit(50);

 if (error || !data) {
 return { total: 0, profile: false, learned: 0, lastUpdated: null };
 }

 const manualProfile = data.find(m => m.source === 'manual_profile');
 const learned = data.filter(m => m.source !== 'manual_profile');
 return {
 total: data.length,
 profile: !!manualProfile,
 learned: learned.length,
 lastUpdated: data[0]?.created_at || null
 };
}

async function extractMemoryFact(userId, text) {
 try {
 const model = genAI.getGenerativeModel({ model: FAST_MODEL });
 const result = await model.generateContent(
 `Extract one short personal fact worth remembering from this message. Write it as a concise note (e.g. "Works at KPMG", "Has a dog named B...`
);
 const fact = result.response.text().trim().replace(/^["]|["]$g, '');
 if (!fact) return null;

 // Skip if we already know this
 const { data: existing } = await supabase
 .from('memories').select('content').eq('user_id', userId);
 const alreadyKnown = (existing || []).some(m =>
 m.content.toLowerCase().includes(fact.toLowerCase()) ||
 fact.toLowerCase().includes(m.content.toLowerCase())
);
 return alreadyKnown ? null : fact;
 } catch {
 return null;
 }
}

function shouldSaveMemory(text) {
 if (isMemoryDeletionRequest(text)) return false;
 const triggers = [
 'remember', 'my ', 'i'm ', 'i am ', 'i work', 'i live',
 'i hate', 'i love', 'i need', 'i want', 'i've got', 'i have',
 'my name', 'my job', 'my partner', 'my wife', 'my husband',
 'my kids', 'my boss', 'my flat', 'my car', "don't tell"
];
 const lower = text.toLowerCase();
 return triggers.some(t => lower.includes(t));
}

```

```

function isMemoryDeletionRequest(text) {
 return /\b(forget|delete|remove|wipe|clear)\b.*\b(memory|remembered|know)\b/i.test(String(text || ''))
 || /\bforget that\b/i.test(String(text || ''));
}

function parseClientTimestamp(value) {
 if (!value) return null;
 const parsed = new Date(String(value));
 if (Number.isNaN(parsed.getTime())) return null;
 const now = Date.now();
 if (parsed.getTime() > now + 5 * 60 * 1000) return null;
 return parsed;
}

async function getHistory(userId, trace = null, limit = 12, options = {}) {
 const since = parseClientTimestamp(options.since);
 const fetchHistory = () => {
 let query = supabase
 .from('conversations')
 .select('id, role, content, created_at')
 .eq('user_id', userId)
 .neq('role', 'system')
 .order('created_at', { ascending: false })
 .limit(Math.min(Math.max(Number(limit) || 12, 1), 200));
 if (since) query = query.gte('created_at', since.toISOString());
 return query;
 };
 const { data, error } = trace
 ? await trace.run('supabase.conversations.fetch_history', fetchHistory)
 : await fetchHistory();

 if (error || !data) return [];
 return data.reverse().map(normalizeConversationRow);
}

function serializeConversationContent(payload) {
 if (payload == null) return '';
 if (typeof payload === 'string') return payload;
 if (typeof payload !== 'object') return String(payload);

 const next = {};
 if (typeof payload.text === 'string') next.text = payload.text;
 if (typeof payload.image === 'string') next.image = payload.image;
 if (Array.isArray(payload.actions) && payload.actions.length) next.actions = payload.actions;
 if (typeof payload.audio === 'string') next.audio = payload.audio;
 if (typeof payload.kind === 'string') next.kind = payload.kind;

 return Object.keys(next).length === 1 && typeof next.text === 'string'
 ? next.text
 : JSON.stringify(next);
}

function conversationFallbackText(entry) {
 if (entry?.content) return entry.content;
 if (entry?.image) return 'Generated image';
 if (entry?.actions?.length) {
 const firstAction = entry.actions[0]?.action || entry.actions[0]?.type || 'action';
 return humanizeActionType(firstAction);
 }
 return '';
}

function normalizeConversationRow(row) {
 const parsed = safeParseJSON(row?.content);
 if (parsed && typeof parsed === 'object' && !Array.isArray(parsed)) {
 return {
 ...row,
 content: typeof parsed.text === 'string' ? parsed.text : '',
 image: typeof parsed.image === 'string' ? parsed.image : null,
 actions: Array.isArray(parsed.actions) ? parsed.actions : [],
 audio: typeof parsed.audio === 'string' ? parsed.audio : null,
 kind: typeof parsed.kind === 'string' ? parsed.kind : null
 };
 }
 return {
 ...row,
 content: typeof row?.content === 'string' ? row.content : String(row?.content || ''),
 image: null,
 actions: [],
 audio: null,
 kind: null
 };
}

function buildConversationSessions(rows = []) {
 const sorted = rows
 .map(normalizeConversationRow)

```



```

 .filter(row => row.created_at)
 .sort((a, b) => new Date(a.created_at) - new Date(b.created_at));
const sessions = [];
const gapMs = 45 * 60 * 1000;

for (const row of sorted) {
 const createdAt = new Date(row.created_at);
 const lastSession = sessions[sessions.length - 1];
 const lastAt = lastSession ? new Date(lastSession.last_at) : null;
 const dayChanged = lastAt && createdAt.toISOString().slice(0, 10) !== lastAt.toISOString().slice(0, 10);
 const gapChanged = lastAt && createdAt.getTime() - lastAt.getTime() > gapMs;
 if (!lastSession || dayChanged || gapChanged) {
 sessions.push({
 id: row.id || row.created_at,
 title: '',
 preview: '',
 started_at: row.created_at,
 last_at: row.created_at,
 message_count: 0
 });
 }

 const session = sessions[sessions.length - 1];
 const text = conversationFallbackText(row).trim();
 session.last_at = row.created_at;
 session.message_count += 1;
 if (!session.title && row.role === 'user' && text) session.title = text.slice(0, 80);
 if (text) session.preview = text.slice(0, 140);
}

return sessions
 .map(session => ({
 ...session,

 title: session.title || session.preview || 'Untitled chat'
 }))
 .reverse()
 .slice(0, 30);
}

async function saveMessage(userId, role, content, trace = null) {
 const insertMessage = () => supabase
 .from('conversations')
 .insert({
 user_id: userId,
 role,
 content: serializeConversationContent(content),
 created_at: new Date().toISOString()
 });
 if (trace) {
 await trace.run(`supabase.conversations.insert_${role}`, insertMessage);
 } else {
 await insertMessage();
 }
 invalidateUserContextCache(userId);
}

async function getPreferences(userId, trace = null) {
 const fetchPreferences = () => supabase
 .from('preferences')
 .select('key, value')
 .eq('user_id', userId);
 const { data, error } = trace
 ? await trace.run('supabase.preferences.fetch', fetchPreferences)
 : await fetchPreferences();
 if (error || !data) return '';
 return data.map(p => `${p.key}: ${p.value}`).join('\n');
}

async function getPreferenceEntries(userId) {
 const { data, error } = await supabase
 .from('preferences')
 .select('key, value')
 .eq('user_id', userId);
 if (error || !data) return [];
 return data;
}

async function getPreferenceMap(userId) {
 const entries = await getPreferenceEntries(userId);
 return Object.fromEntries(entries.map(entry => [entry.key, entry.value]));
}

async function setPreferenceValue(userId, key, value) {
 await supabase
 .from('preferences')
 .upsert({

```

```

 user_id: userId,
 key,
 value,
 updated_at: new Date().toISOString()
 }, { onConflict: 'user_id,key' });
}

async function getPendingAction(userId) {
 const { data, error } = await supabase
 .from('preferences')
 .select('value')
 .eq('user_id', userId)
 .eq('key', PENDING_ACTION_PREF)
 .maybeSingle();
 if (error || !data?.value) return null;
 try {
 const parsed = JSON.parse(data.value);
 if (!parsed?.action?.type) return null;
 return parsed;
 } catch {
 return null;
 }
}

async function setPendingAction(userId, action, context = {}) {
 const payload = {
 action,
 createdAt: new Date().toISOString(),
 userMessage: context.userMessage || '',
 nativeHints: context.nativeHints || null
 };
 await setPreferenceValue(userId, PENDING_ACTION_PREF, JSON.stringify(payload));
 return payload;
}

async function clearPendingAction(userId) {
 await supabase
 .from('preferences')
 .delete()
 .eq('user_id', userId)
 .eq('key', PENDING_ACTION_PREF);
}

async function getEnabledConnectors(userId, trace = null) {
 const fetchConnectors = () => supabase
 .from('connectors')
 .select('connector_id')
 .eq('user_id', userId)
 .eq('enabled', true);
 const { data, error } = trace
 ? await trace.run('supabase.connectors.fetch_enabled', fetchConnectors)
 : await fetchConnectors();
 if (error || !data) return [];
 return data.map(c => c.connector_id);
}

function buildAvailableActions(enabled) {
 const actionMap = {
 google: ['send_email', 'get_emails', 'search_emails', 'create_calendar_event', 'get_calendar_events'],
 imessage: ['send_message'],
 whatsapp: ['send_message'],
 reminders: ['create_reminder'],
 spotify: ['play_music'],
 homekit: ['homekit_control'],
 maps: ['find_place', 'get_directions', 'plan_trip'],
 uber: ['book_uber'],
 ubereats: ['order_uber_eats'],

 netflix: ['search_netflix_title', 'add_to_netflix_list'],
 telegram: ['send_telegram', 'get_telegram_contacts'],
 deliveroo: ['order_deliveroo'],
 monzo: ['check_balance'],
 betfair: ['place_bet'],
 notion: ['create_note'],
 trainline: ['search_trains', 'station_board']
 };
 const live = enabled.filter(id => IMPLEMENTED_CONNECTORS.has(id));
 if (live.length === 0) return 'No connectors enabled. Internal actions still available: forget_memory, find_place, play_music, add_to_music-pl';
 const active = live.flatMap(id => actionMap[id] || []);
 return `Available connector actions: ${active.join(', ')}. Internal actions always available: forget_memory, find_place, play_music, add_to_mu`;
}

async function savePreference(userId, key, value) {
 await supabase
 .from('preferences')
 .upsert({ user_id: userId, key, value }, { onConflict: 'user_id,key' });
}

```

```

async function getUserAccount(userId) {
 const { data, error } = await supabase
 .from('users')
 .select('user_id, password_hash, token_version, email')
 .eq('user_id', userId)
 .maybeSingle();

 if (error) throw error;
 return data;
}

async function getUserAccountByEmail(email) {
 const { data, error } = await supabase
 .from('users')
 .select('user_id, password_hash, token_version, email')
 .eq('email', email)
 .maybeSingle();

 if (error) throw error;
 return data;
}

const USER_DATA_TABLES = [
 'briefings',
 'native_context',
 'devices',
 'preferences',
 'connectors',
 'action_log',
 'memories',
 'conversations',
 'users'
];

async function fetchUserDataTable(table, userId) {
 const { data, error } = await supabase
 .from(table)
 .select('*')
 .eq('user_id', userId);
 if (error) throw error;
 return data || [];
}

function sanitizeExportRows(table, rows) {
 if (table === 'connectors') {
 return rows.map(({ tokens, ...row }) => ({
 ...row,
 hasTokens: Boolean(tokens)
 }));
 }
 if (table === 'users') {
 return rows.map(({ password_hash, ...row }) => row);
 }
 return rows;
}

async function buildUserExport(userId) {
 const entries = await Promise.all(
 USER_DATA_TABLES.map(async table => [table, sanitizeExportRows(table, await fetchUserDataTable(table, userId))]
);
 const data = Object.fromEntries(entries);
 return {
 exportedAt: new Date().toISOString(),
 userId,
 user: data.users?.[0] || null,
 conversations: data.conversations || [],
 memories: data.memories || [],
 actionLog: (data.action_log || []).map(row => ({ ...row, action: safeParseJSON(row.action) })),
 connectors: data.connectors || [],
 preferences: data.preferences || [],
 devices: data.devices || [],
 nativeContext: data.native_context || [],
 briefings: data.briefings || []
 };
}

async function getUserContext(userId, trace = null) {
 const cached = contextCache.get(userId);
 if (cached && Date.now() - cached.ts < CONTEXT_CACHE_TTL) {
 if (trace) trace.log('user_context.cache_hit');
 return cached.context;
 }

 const [connectors, memories, actionLog] = trace
 ? await Promise.all([
 trace.run('supabase.user_context.connectors', () => supabase.from('connectors').select('connector_id').eq('user_id', userId).eq('enabled', true)),
 trace.run('supabase.user_context.memories', () => supabase.from('memories').select('content').eq('user_id', userId).order('created_at', 'desc')),
 trace.run('supabase.user_context.action_log', () => supabase.from('action_log').select('*').eq('user_id', userId).order('created_at', 'desc'))
])
 : [connectors, memories, actionLog];
 return {
 connectors,
 memories,
 actionLog
 };
}

```

```

 trace.run('supabase.user_context.action_log', () => supabase.from('action_log').select('action, status, error, created_at').eq('user_id', user_id).limit(1));
 }
 await Promise.all([
 supabase.from('connectors').select('connector_id').eq('user_id', user_id).eq('enabled', true),
 supabase.from('memories').select('content').eq('user_id', user_id).order('created_at', { ascending: false }).limit(5),
 supabase.from('action_log').select('action, status, error, created_at').eq('user_id', user_id).order('created_at', { ascending: false }).limit(10);
]);

const active = (connectors.data || []).map(c => c.connector_id).join(', ') || 'none';

const contactCounts = {};
const recentActionLines = [];
for (const row of (actionLog.data || [])) {
 try {
 const a = typeof row.action === 'string' ? JSON.parse(row.action) : row.action;
 const contact = a.input?.contact;
 if (!contact || !['send_message', 'send_email', 'send_telegram'].includes(a.type)) continue;
 const channel = a.type === 'send_telegram' ? 'Telegram' : a.type === 'send_email' ? 'Email' : 'iMessage';
 const key = `${contact}||${channel}`;
 contactCounts[key] = (contactCounts[key] || 0) + 1;
 } catch {}
}
for (const row of (actionLog.data || []).slice(0, 5)) {
 try {
 const a = typeof row.action === 'string' ? JSON.parse(row.action) : row.action;
 const status = row.status === 'failed' ? 'failed' : 'succeeded';
 const detail = (row.error || a.resultText || '').trim();
 recentActionLines.push(
 `${new Date(row.created_at).toLocaleTimeString('en-GB', { timeZone: TIMEZONE, hour: '2-digit', minute: '2-digit' })}: ` +
 `${humanizeActionType(a.type)}${summarizeActionInput(a.input)} - ${status}${detail ? ` (${detail})` : ''}`
);
 } catch {}
}
const patterns = Object.entries(contactCounts)
 .sort((a, b) => b[1] - a[1])
 .slice(0, 6)
 .map(([, n]) => { const [name, ch] = k.split('||'); return `${name}: ${ch} (${n}x)`; })
 .join(', ') || 'none yet';

const memoryLines = (memories.data || []).map(m => m.content).join('; ') || 'none';
const recentActions = recentActionLines.join(' | ') || 'none yet';

const context = `LIVE USER CONTEXT:
Active connectors: ${active}
Messaging patterns: ${patterns}
Key facts: ${memoryLines}
Recent action outcomes: ${recentActions}`.slice(0, 2200);

if (contextCache.size >= CONTEXT_CACHE_MAX) {
 const oldest = contextCache.keys().next().value;
 contextCache.delete(oldest);
}
contextCache.set(user_id, { context, ts: Date.now() });
return context;
}

app.post('/auth/register', registerRateLimiter, async (req, res) => {
 try {
 const { user_id, password, email } = req.body || {};
 if (!requireValidUserIdValue(user_id, res)) return;
 if (typeof password !== 'string' || password.length < 8 || password.length > MAX_PASSWORD_LENGTH) {
 return res.status(400).json({ error: `Password must be between 8 and ${MAX_PASSWORD_LENGTH} characters.` });
 }

 const EMAIL_RE = /^[^@]+@[^\s@]+\.[^@]+\$/;
 if (email !== undefined && email !== null && email !== '') {
 if (typeof email !== 'string' || !EMAIL_RE.test(email)) {
 return res.status(400).json({ error: 'Invalid email address.' });
 }
 }

 const existing = await getUserAccount(user_id);
 if (existing) {
 return res.status(409).json({ error: 'That user ID is already taken.' });
 }

 const passwordHash = hashPassword(password);
 const insertData = {
 user_id: user_id,
 password_hash: passwordHash,
 token_version: 1,
 created_at: new Date().toISOString(),
 updated_at: new Date().toISOString()
 };
 if (email) insertData.email = email;
 }

```

```

const { error } = await supabase.from('users').insert(insertData);
if (error) throw error;

log('info', 'auth.register', { userId });

if (email) {
 try {
 const { sendWelcomeEmail } = require('./services/email');
 await sendWelcomeEmail(email, userId);
 } catch (e) {
 log('warn', 'email.welcome.failed', { error: e.message });
 }
}

res.json({ success: true, token: createSessionToken(userId, 1), userId });
} catch (err) {
 log('error', 'auth.register.error', { error: err.message });
 res.status(500).json({ error: err.message });
}
});

app.post('/auth/login', loginRateLimiter, async (req, res) => {
 try {
 const { userId, email, password } = req.body || {};
 if (typeof password !== 'string' || !password || password.length > MAX_PASSWORD_LENGTH) {
 return res.status(400).json({ error: 'Password is required and must be a reasonable length.' });
 }

 let account;
 let resolvedUserId;

 if (email) {
 const trimmedEmail = String(email).trim();
 account = await getUserAccountByEmail(trimmedEmail);
 resolvedUserId = account?.user_id;
 } else {
 const trimmedUserId = String(userId || '').trim();
 if (!requireValidUserIdValue(trimmedUserId, res)) return;
 account = await getUserAccount(trimmedUserId);
 resolvedUserId = trimmedUserId;
 }

 if (!account || !verifyPassword(password, account.password_hash)) {
 log('warn', 'auth.login.failed', { userId: resolvedUserId || 'unknown' });
 return res.status(401).json({ error: 'Invalid credentials.' });
 }

 const tokenVersion = account.token_version || 1;
 log('info', 'auth.login', { userId: account.user_id });
 res.json({ success: true, token: createSessionToken(account.user_id, tokenVersion), userId: account.user_id });
 } catch (err) {
 log('error', 'auth.login.error', { error: err.message });
 res.status(500).json({ error: err.message });
 }
});

app.post('/auth/logout', (req, res) => {
 res.json({ success: true });
});

app.post('/auth/logout-all', async (req, res) => {
 try {
 const userId = getAuthenticatedUserId(req);
 if (!userId) return res.status(401).json({ error: 'Unauthorized' });
 const { data: userRow } = await supabase.from('users').select('token_version').eq('user_id', userId).single();
 if (!userRow) return res.status(404).json({ error: 'User not found' });
 const { error } = await supabase.from('users').update({ token_version: (userRow.token_version || 1) + 1 }).eq('user_id', userId);
 if (error) throw error;
 log('info', 'auth.logout_all', { userId });
 res.json({ success: true });
 } catch (err) {
 log('error', 'auth.logout_all.error', { error: err.message });
 res.status(500).json({ error: err.message });
 }
});

app.get('/auth/sessions', async (req, res) => {
 try {
 const userId = getAuthenticatedUserId(req);
 const { data: userRow } = await supabase.from('users').select('token_version, created_at').eq('user_id', userId).maybeSingle();
 res.json({ tokenVersion: userRow?.token_version || 1, note: 'Use POST /auth/logout-all to revoke all sessions' });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.post('/auth/change-password', async (req, res) => {

```

```

try {
 const userId = getAuthenticatedUserId(req);
 const { currentPassword, newPassword } = req.body || {};
 if (typeof currentPassword !== 'string' || !currentPassword) {
 return res.status(400).json({ error: 'currentPassword is required.' });
 }
 if (typeof newPassword !== 'string' || newPassword.length < 8) {
 return res.status(400).json({ error: 'newPassword must be at least 8 characters.' });
 }
 if (currentPassword === newPassword) {
 return res.status(400).json({ error: 'New password must be different from current password.' });
 }
 const account = await getUserAccount(userId);
 if (!account || !verifyPassword(currentPassword, account.password_hash)) {
 return res.status(401).json({ error: 'Current password is incorrect.' });
 }
 const newHash = hashPassword(newPassword);
 const newVersion = (account.token_version || 1) + 1;
 await supabase.from('users').update({ password_hash: newHash, token_version: newVersion }).eq('user_id', userId);
 log('info', 'auth.change_password', { userId });
 res.json({ success: true });
} catch (err) {
 log('error', 'auth.change_password.error', { error: err.message });
 res.status(500).json({ error: err.message });
}
});

app.post('/auth/forgot-password', forgotPasswordRateLimiter, async (req, res) => {
 try {
 const { email } = req.body || {};
 const RESPONSE = { message: "If this email is registered, you'll receive a reset link shortly." };
 if (!email || typeof email !== 'string') return res.json(RESPONSE);

 const account = await getUserAccountByEmail(String(email).trim());
 if (!account) return res.json(RESPONSE);

 const token = crypto.randomBytes(32).toString('hex');
 const expiresAt = new Date(Date.now() + 60 * 60 * 1000).toISOString();
 await supabase.from('password_reset_tokens').insert({ user_id: account.user_id, token, expires_at: expiresAt });

 const resetUrl = `${process.env.APP_URL || ''}/auth/reset-password?token=${token}`;
 log('info', 'password_reset.token_created', { event: '[password-reset]', url: resetUrl });

 try {
 const { sendPasswordResetEmail } = require('./services/email');
 await sendPasswordResetEmail(account.email, resetUrl);
 } catch (e) {
 log('warn', 'email.password_reset.failed', { error: e.message });
 }

 res.json(RESPONSE);
 } catch (err) {
 log('error', 'auth.forgot_password.error', { error: err.message });
 res.json({ message: "If this email is registered, you'll receive a reset link shortly." });
 }
});

app.get('/auth/reset-password', (req, res) => {
 const { token } = req.query;
 res.setHeader('Content-Type', 'text/html; charset=utf-8');
 res.send(`<!DOCTYPE html><html><head><title>Reset Password · Oxy</title>
<style>body{font-family:sans-serif;max-width:420px;margin:60px auto;padding:0 24px;color:#1a1a1a}
h2{margin-bottom:8px}input{width:100%;padding:10px;margin:8px 0 16px 0;box-sizing:border-box;border:1px solid #ccc;border-radius:6px;font-size:1
button{width:100%;padding:12px;background:#2563eb;color:white;border:none;border-radius:6px;font-size:15px;cursor:pointer}</style>
</head><body>
<h2>Reset your password</h2>
<p>Enter a new password (minimum 8 characters).</p>
<form method="POST" action="/auth/reset-password">
 <input type="hidden" name="token" value="${escapeHtml(String(token) || '')}">
 <label>New Password<input type="password" name="newPassword" minlength="8" required></label>
 <button type="submit">Reset Password</button>
</form>
</body></html>`);
});

app.post('/auth/reset-password', express.urlencoded({ extended: false }), async (req, res) => {
 try {
 const { token, newPassword } = req.body || {};
 // Browser form submissions get an HTML page; API clients (JSON) get JSON.
 const wantsHtml = (req.get('Accept') || '').includes('text/html');

 const fail = (status, message) => {
 if (wantsHtml) {
 res.status(status).setHeader('Content-Type', 'text/html; charset=utf-8');
 return res.send(`<!DOCTYPE html><html><head><title>Reset Password · Oxy</title><style>body{font-fam
 }
 return res.status(status).json({ error: message });
 };

```

```

 };
 if (!token || !newPassword) return fail(400, 'Token and newPassword are required.');
```

```

 if (typeof newPassword !== 'string' || newPassword.length < 8) {
 return fail(400, 'Password must be at least 8 characters.');
```

```

 }
 const { data: tokenRow } = await supabase.from('password_reset_tokens').select('id, user_id, expires_at, used').eq('token', token).maybeSingle();
 if (!tokenRow || tokenRow.used || new Date(tokenRow.expires_at) < new Date()) {
 return fail(400, 'Invalid or expired reset token.');
```

```

 }
 const newHash = hashPassword(newPassword);
 const account = await getUserAccount(tokenRow.user_id);
 const newVersion = (account?.token_version || 1) + 1;
 await Promise.all([
 supabase.from('users').update({ password_hash: newHash, token_version: newVersion }).eq('user_id', tokenRow.user_id),
 supabase.from('password_reset_tokens').update({ used: true }).eq('id', tokenRow.id)
]);
 log('info', 'auth.password_reset.completed', { userId: tokenRow.user_id });
 if (wantsHtml) {
 res.setHeader('Content-Type', 'text/html; charset=utf-8');
```

```

 return res.send('<!DOCTYPE html><html><head><title>Password Reset · Oxy</title><style>body{font-family:
 }
 res.json({ success: true });
 } catch (err) {
 log('error', 'auth.reset_password.error', { error: err.message });
 res.status(500).json({ error: err.message });
 }
});

app.post('/auth/verify-email', async (req, res) => {
 try {
 const userId = getAuthenticatedUserId(req);
 const account = await getUserAccount(userId);
 if (!account || !account.email) return res.status(400).json({ error: 'No email address on this account.' });
 const token = crypto.randomBytes(32).toString('hex');
 const expiresAt = new Date(Date.now() + 24 * 60 * 60 * 1000).toISOString();
 await supabase.from('password_reset_tokens').insert({ user_id: userId, token, expires_at: expiresAt });
 const verifyUrl = `${process.env.APP_URL || ''}/auth/verify-email/confirm?token=${token}`;
 try {
 const { sendVerificationEmail } = require('./services/email');
```

```

 await sendVerificationEmail(account.email, verifyUrl);
 } catch (e) {
 log('warn', 'email.verify.failed', { error: e.message });
 }
 res.json({ success: true, message: 'Verification email sent.' });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.post('/process-audio', upload.single('audio'), async (req, res) => {
 if (!req.file) {
 return res.status(400).json({ error: 'No audio file received.' });
 }

 const userId = req.body.userId;
 if (!requireMatchingUser(req, res, userId)) return;
 const now = Date.now();
 const recentHits = (audioRateLimit.get(userId) || []).filter(t => now - t < 60000);
 if (recentHits.length >= 10) {
 return res.status(429).json({ error: 'Too many requests. Try again in a minute.' });
 }
 audioRateLimit.set(userId, [...recentHits, now]);

 res.setHeader('Content-Type', 'text/event-stream');
 res.setHeader('Cache-Control', 'no-cache');
 res.setHeader('Connection', 'keep-alive');
 res.flushHeaders();
 const audioTraceId = `process-audio:${userId}:${Date.now()}`;
 const sse = obj => {
 if (obj?.type === 'audio') {
 console.log(`[audio][backend:${audioTraceId}] sending audio event bytes=${Buffer.from(obj.data || '', 'base64').length} mime=${obj.mimeType}`);
 res.write(`data: ${JSON.stringify(obj)}\n\n`);
 }
 };

 try {
 const [userText, context] = await Promise.all([
 transcribeAudio(req.file.buffer),
 buildChatContext(userId, '', null, STREAMING_CHAT_MODEL) // message unknown yet — no search for audio transcription step
]);

 if (!userText) {
 sse({ type: 'transcription-error', error: "I couldn't clearly make out what you said." });
 sse({ type: 'done' });
 return res.end();
 }
 }
}

```

```

sse({ type: 'transcription', text: userText });
saveMessage(userId, 'user', userText).catch(() => {});

// Step 2: Send transcribed text to the main model (with full system prompt + history)
// Rebuild model with search if the transcribed text needs it
let { history, useSearch, dynamicSystemPrompt, cachedContentName } = context;
if (needsSearch(userText)) {
 const refreshed = await buildChatContext(userId, userText, null, STREAMING_CHAT_MODEL);
 useSearch = refreshed.useSearch;
 dynamicSystemPrompt = refreshed.dynamicSystemPrompt;
 cachedContentName = refreshed.cachedContentName;
}
const baseHistory = normalizeGeminiHistory(history);
const initialRequest = buildModernGenerateRequest({
 dynamicSystemPrompt,
 useSearch,
 cachedContentName,
 baseHistory,
 userContent: { role: 'user', parts: [{ text: userText }] }
});

const stream = await modernGenAI.models.generateContentStream({
 model: STREAMING_CHAT_MODEL,
 contents: initialRequest.contents,
 config: initialRequest.config
});
let fullText = '';

for await (const chunk of stream) {
 const text = chunk.text || '';
 if (text) fullText += text;
}

const { spoken, actions } = parseActions(fullText);

let actionResults = [];
let audioBase64 = null;
let ttsError = '';
if (actions.length > 0) {
 actionResults = await executeActions(userId, actions, { userMessage: userText });
 actionResults = normalizeActionResultsForClient(actionResults);
}
const dataResults = getStructuredDataResults(actionResults);
let finalSpoken = canUseDirectActionSummary(actionResults) ? summarizeActionResults(actionResults) : spoken;
if (!canUseDirectActionSummary(actionResults) && dataResults.length > 0) {
 const context = dataResults.map(a => a.text).join('\n\n');
 const followUpRequest = buildModernGenerateRequest({
 dynamicSystemPrompt,
 useSearch,
 cachedContentName,
 baseHistory,
 userContent: { role: 'user', parts: [{ text: userText }] }
 });
 followUpRequest.contents.push(
 { role: 'model', parts: [{ text: spoken || '...' }] },
 { role: 'user', parts: [{ text: `Here are the results:\n\n${context}\n\nSpeak these back naturally and conversationally. Be concise. Only` }
]);
 const followUp = await modernGenAI.models.generateContent({
 model: PRIMARY_CHAT_MODEL,
 contents: followUpRequest.contents,
 config: followUpRequest.config
 });
 finalSpoken = parseActions(followUp.text || '').spoken || context;
}
const actionConfirmation = summarizeFinishedActionsForUser(actionResults);
if (actionConfirmation) finalSpoken = actionConfirmation;
audioBase64 = await generateSpeech(buildVoiceExcerpt(finalSpoken), req.body.voice).catch(err => {
 ttsError = err.message;
 console.error(['tts error', err.message]);
 return null;
});
saveMessage(userId, 'assistant', { text: finalSpoken, actions: actionResults }).catch(() => {});

sse({ type: 'response', text: finalSpoken, actions: actionResults });
if (audioBase64) sse({ type: 'audio', data: audioBase64, format: 'wav', mimeType: 'audio/wav' });
if (ttsError) sse({ type: 'tts-error', error: ttsError });
sse({ type: 'done' });
res.end();

postResponseTasks(userId, userText);
} catch (err) {
 console.error('/process-audio error:', err.message);
 try { sse({ type: 'error', error: err.message }); res.end(); } catch {}
}
});

app.post('/images/generate', imageRateLimiter, upload.single('image'), async (req, res) => {

```



```

try {
 const { userId, prompt } = req.body;
 if (!requireMatchingUser(req, res, userId)) return;
 if (!prompt?.trim()) return res.status(400).json({ error: 'prompt is required.' });

 const result = await generateImage(prompt, req.file || null);
 const imageUrl = `data:${result.mimeType} || 'image/png';base64,${result.image}`;
 saveMessage(userId, 'user', prompt.trim()).catch(() => {});
 saveMessage(userId, 'assistant', { text: '', image: imageUrl, kind: 'image' }).catch(() => {});

 res.json({ success: true, ...result, text: '' });
} catch (err) {
 console.error('/images/generate error:', err.response?.data || err.message);
 res.status(500).json({ error: err.response?.data?.error?.message || err.message });
}
});

app.post('/chat-with-image', imageRateLimiter, upload.single('image'), async (req, res) => {
 try {
 const userId = req.body.userId;
 const message = (req.body.message || '').trim();
 const settings = safeParseJSON(req.body.settings) || {};
 const chatStartedAt = req.body.chatStartedAt || null;
 const wantsTTS = req.query.tts === 'true';

 if (!requireMatchingUser(req, res, userId)) return;
 if (!req.file) return res.status(400).json({ error: 'file is required.' });
 if (!message) return res.status(400).json({ error: 'message is required.' });

 const isImage = (req.file.mimetype || '').startsWith('image/');
 const fileLabel = isImage ? 'image' : 'file';
 const fileContextHint = isImage
 ? `The user attached an image or screenshot. Use it as context when helpful.\n\n${message}`
 : `The user attached a file (${req.file.originalname || 'document'}, type: ${req.file.mimetype}). Use its content to answer their question`;

 const [{ history, useSearch, dynamicSystemPrompt, cachedContentName }] = await Promise.all([
 buildChatContext(userId, message, null, PRIMARY_CHAT_MODEL, { chatStartedAt }),
 saveMessage(userId, 'user', `${message}\n\n[Attached ${fileLabel}: ${req.file.originalname || fileLabel}]`)
]);
 const baseHistory = normalizeGeminiHistory(history);
 const initialRequest = buildModernGenerateRequest({
 dynamicSystemPrompt,
 useSearch,
 cachedContentName,
 baseHistory,
 userContent: {
 role: 'user',
 parts: [
 { text: fileContextHint },
 { inlineData: { mimeType: req.file.mimetype, data: req.file.buffer.toString('base64') } }
]
 }
 });
 });

 const geminiRes = await modernGenAI.models.generateContent({
 model: PRIMARY_CHAT_MODEL,
 contents: initialRequest.contents,
 config: initialRequest.config
 });
 let { spoken, actions } = parseActions(geminiRes.text || '');
 let actionResults = [];

 if (actions.length > 0) {
 actionResults = await executeActions(userId, actions, { imageFile: req.file, userMessage: message });
 actionResults = normalizeActionResultForClient(actionResults);
 }

 const dataResults = getStructuredDataResults(actionResults);
 if (canUseDirectActionSummary(actionResults)) {
 spoken = summarizeActionResult(actionResults);
 } else if (dataResults.length > 0) {
 const context = dataResults.map(a => a.text).join('\n\n');
 const followUpRequest = buildModernGenerateRequest({
 dynamicSystemPrompt,
 useSearch,
 cachedContentName,
 baseHistory,
 userContent: {
 role: 'user',
 parts: [
 { text: fileContextHint },
 { inlineData: { mimeType: req.file.mimetype, data: req.file.buffer.toString('base64') } }
]
 }
 });
 });
 followUpRequest.contents.push(
 { role: 'model', parts: [{ text: spoken || '...' }] },

```

```

 { role: 'user', parts: [{ text: `Here are the action results:\n\n${context}\n\nRespond naturally and use only the results shown here please` }
]
};
const followUp = await modernGenAI.models.generateContent({
 model: PRIMARY_CHAT_MODEL,
 contents: followUpRequest.contents,
 config: followUpRequest.config
});
spoken = parseActions(followUp.text || '').spoken || spoken || context;
}
const actionConfirmation = summarizeFinishedActionsForUser(actionResults);
if (actionConfirmation) spoken = actionConfirmation;

if (!spoken) {
 spoken = actionResults.map(a => a.result?.text).filter(Boolean).join(' ') || 'I looked through it.';
}

saveMessage(userId, 'assistant', { text: spoken, actions: actionResults }).catch(() => {});
const result = { text: spoken, actions: actionResults };

if (wantsTTS) {
 try {
 const audio = await generateSpeech(buildVoiceExcerpt(spoken), settings.voice);
 if (audio) {
 console.log(`[audio][backend:chat-image] returning tts audio bytes=${Buffer.from(audio, 'base64').length} mime=audio/wav`);
 result.audio = audio;
 result.audioFormat = 'wav';
 result.audioMimeType = 'audio/wav';
 }
 } catch (ttsErr) {
 console.error('[tts error]', ttsErr.message);
 result.ttsError = ttsErr.message;
 }
}

res.json(result);
} catch (err) {
 console.error('/chat-with-image error:', err.response?.data || err.message);
 res.status(500).json({ error: err.response?.data?.error?.message || err.message });
}
});

app.post('/memory', async (req, res) => {
 try {
 const { userId, content } = req.body;
 if (!requireMatchingUser(req, res, userId)) return;
 if (!content?.trim()) return res.status(400).json({ error: 'content is required.' });
 await saveMemory(userId, content.trim(), 'manual_profile');

 res.json({ success: true });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.get('/memory/:userId', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 try {
 res.json({ summary: await getMemorySummary(req.params.userId) });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.delete('/memory/:userId', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 try {
 const result = await forgetMemory(req.params.userId, req.body || { scope: 'all' });
 res.json(result);
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.get('/user/:userId/export', async (req, res) => {
 const { userId } = req.params;
 if (!requireMatchingUser(req, res, userId)) return;
 try {
 const data = await buildUserExport(userId);
 res.setHeader('Content-Disposition', 'attachment; filename="oxy-data-export.json"');
 res.json(data);
 } catch (err) {
 console.error('/user/export error:', err.message);
 res.status(500).json({ error: 'Could not export your data right now.' });
 }
});

app.delete('/user/:userId', async (req, res) => {

```

```

const { userId } = req.params;
if (!requireMatchingUser(req, res, userId)) return;
try {
 for (const table of USER_DATA_TABLES) {
 const { error } = await supabase.from(table).delete().eq('user_id', userId);
 if (error) throw error;
 }
 contextCache.delete(userId);

 res.json({ success: true, deleted: true });
} catch (err) {
 console.error('/user/delete error:', err.message);
 res.status(500).json({ error: 'Could not delete your account right now.' });
}
});

app.post('/action-log', async (req, res) => {
 try {
 const { userId, action, status = 'executed' } = req.body;
 if (!requireMatchingUser(req, res, userId)) return;
 if (!action) return res.status(400).json({ error: 'action is required.' });
 if (!ACTION_LOG_STATUSES.has(status)) return res.status(400).json({ error: 'Invalid status.' });

 await supabase.from('action_log').insert({
 user_id: userId,
 action: JSON.stringify(action),
 status,
 created_at: new Date().toISOString()
 });
 res.json({ success: true });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.get('/action-log/:userId', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 try {
 const { data, error } = await supabase
 .from('action_log')
 .select('*')
 .eq('user_id', req.params.userId)
 .order('created_at', { ascending: false })
 .limit(50);

 if (error || !data) return res.json({ actions: [] });
 const parsed = data.map(a => ({
 ...a,
 action: safeParseJSON(a.action)
 }));
 res.json({ actions: parsed });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.get('/action-contracts', requireSessionAuth, (req, res) => {
 res.json({ actions: ACTION_CONTRACTS });
});

const CONNECTORS = [
 { id: 'google', name: 'Google (Gmail + Calendar)', icon: 'google', category: 'Google', implemented: true },
 { id: 'netflix', name: 'Netflix', icon: 'netflix', category: 'Entertainment', implemented: true },
 { id: 'deliveroo', name: 'Deliveroo', icon: 'deliveroo', category: 'Food', implemented: true },
 { id: 'ubereats', name: 'Uber Eats', icon: 'ubereats', category: 'Food', implemented: true },
 { id: 'uber', name: 'Uber', icon: 'uber', category: 'Transport', implemented: true },
 { id: 'maps', name: 'Maps', icon: 'maps', category: 'Travel', implemented: true },
 { id: 'telegram', name: 'Telegram', icon: 'telegram', category: 'Messages', implemented: true },
 { id: 'trainline', name: 'Trainline', icon: 'trainline', category: 'Transport', implemented: true },
];

const KNOWN_CONNECTOR_IDS = new Set(CONNECTORS.map(c => c.id));
const ACTION_LOG_STATUSES = new Set(['executed', 'failed', 'pending']);
const PENDING_ACTION_PREF = 'pending.action';

app.get('/connectors/:userId', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 try {
 const { data, error } = await supabase
 .from('connectors')
 .select('connector_id, enabled, tokens')
 .eq('user_id', req.params.userId);
 if (error) throw error;

 const rowsById = new Map();
 if (data) {
 data.forEach(c => rowsById.set(c.connector_id, c));
 }
 }
});

```

```

const result = CONNECTORS.map(c => {
 const row = rowsById.get(c.id);
 const enabled = row?.enabled === true;
 const hasRefreshToken = Boolean(row?.tokens?.refresh_token || row?.tokens?.session || row?.tokens?.encrypted);
 const needsReconnect = c.id === 'google' && enabled && !hasRefreshToken;
 const needsSetup = c.id === 'maps' && !(process.env.GOOGLE_MAPS_API_KEY || process.env.GOOGLE_PLACES_API_KEY);
 const degraded = c.id === 'trainline' && !(process.env.TRANSPORT_API_APP_ID || !process.env.TRANSPORT_API_APP_KEY);
 const connectionState = needsReconnect
 ? 'needs_reconnect'
 : needsSetup
 ? 'needs_setup'
 : degraded
 ? 'degraded'
 : enabled
 ? 'connected'
 : 'available';
 return {
 ...c,
 enabled,
 connectionState,
 statusText: connectionState === 'needs_reconnect'
 ? 'Reconnect needed'
 : connectionState === 'needs_setup'
 ? 'Setup needed'
 : connectionState === 'degraded'
 ? 'Fallback only'
 : connectionState === 'connected'
 ? 'Connected'
 : 'Available'
 };
});

res.json({ connectors: result });
} catch (err) {
 res.status(500).json({ error: err.message });
}
});

app.post('/connectors', async (req, res) => {
 try {
 const { userId, connectorId, enabled } = req.body;

 if (!requireMatchingUser(req, res, userId)) return;
 if (!KNOWN_CONNECTOR_IDS.has(connectorId)) {
 return res.status(400).json({ error: 'Unknown connector.' });
 }
 if (typeof enabled !== 'boolean') {
 return res.status(400).json({ error: 'enabled must be a boolean.' });
 }

 const { error } = await supabase
 .from('connectors')
 .upsert({
 user_id: userId,
 connector_id: connectorId,
 enabled,
 updated_at: new Date().toISOString()
 }, { onConflict: 'user_id,connector_id' });
 if (error) throw error;

 res.json({ success: true });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.post('/devices/register', async (req, res) => {
 try {
 const { userId, platform = 'ios', pushToken, timezone = TIMEZONE } = req.body || {};
 if (!requireMatchingUser(req, res, userId)) return;
 if (!DEVICE_PLATFORM_ALLOWLIST.has(platform)) {
 return res.status(400).json({ error: 'Unsupported device platform.' });
 }
 if (!pushToken || typeof pushToken !== 'string') {
 return res.status(400).json({ error: 'pushToken is required.' });
 }

 const { error } = await supabase.from('devices').upsert({
 user_id: userId,
 platform,
 push_token: pushToken,
 timezone,
 updated_at: new Date().toISOString()
 }, { onConflict: 'user_id,push_token' });
 if (error) throw error;
 res.json({ ok: true });
 } catch (err) {

```

```

 res.status(500).json({ error: err.message });
 }
});

app.post('/native/context', async (req, res) => {
 try {
 const { userId, location = null, health = {}, capabilities = {}, settings = {} } = req.body || {};
 if (!requireMatchingUser(req, res, userId)) return;
 const { error } = await supabase.from('native_context').upsert({
 user_id: userId,
 location,
 health,
 capabilities,
 settings,
 updated_at: new Date().toISOString()
 }, { onConflict: 'user_id' });
 if (error) throw error;
 invalidateUserContextCache(userId);
 res.json({ ok: true });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.post('/native/local-action', async (req, res) => {
 try {
 const { userId, message, result } = req.body || {};
 if (!requireMatchingUser(req, res, userId)) return;
 if (!message || !result?.action) {
 return res.status(400).json({ error: 'message and result.action are required.' });
 }

 await saveMessage(userId, 'user', String(message));
 await saveMessage(userId, 'assistant', { text: result.text || '', actions: [result] });
 await supabase.from('action_log').insert({
 user_id: userId,
 action: JSON.stringify({ type: result.action, input: { source: 'ios-native' } }),
 status: result.success === false ? 'failed' : 'executed',
 error: result.success === false ? (result.error || null) : null,
 created_at: new Date().toISOString()
 });

 invalidateUserContextCache(userId);
 res.json({ success: true });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.get('/briefings/:userId', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 try {
 const limit = Math.min(Number(req.query.limit) || 30, 100);
 const { data, error } = await supabase
 .from('briefings')
 .select('id, kind, title, body, source, metadata, read, created_at')
 .eq('user_id', req.params.userId)
 .order('created_at', { ascending: false })
 .limit(limit);
 if (error) throw error;
 const visible = (data || []).filter(briefing => briefing.kind !== 'failed_action_followup');
 res.json({ briefings: visible });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.post('/briefings/:id/read', async (req, res) => {
 try {
 const { userId } = req.body || {};
 if (!requireMatchingUser(req, res, userId)) return;
 const { error } = await supabase
 .from('briefings')
 .update({ read: true, read_at: new Date().toISOString() })
 .eq('id', req.params.id)
 .eq('user_id', userId);
 if (error) throw error;
 res.json({ ok: true });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.post('/proactive/:userId/run', async (req, res) => {
 try {
 const { userId } = req.params;

```

```

 if (!requireMatchingUser(req, res, userId)) return;
 const summary = await runProactiveForUser(userId);
 res.json({ ok: true, summary });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.all('/proactive/sweep', async (req, res) => {
 try {
 if (!proactiveSweepAuthorized(req)) {
 return res.status(401).json({ error: 'Invalid proactive sweep secret.' });
 }
 const summary = await runProactiveSweep(console);
 res.json({ ok: true, summary });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.post('/tts-preview', async (req, res) => {
 try {
 const { voice = 'Aoede', text = 'Hi, it is lovely to meet you. This is how I sound.' } = req.body || {};
 console.log(`[audio][backend:tts-preview] request voice=${voice} chars=${String(text || '').trim().length}`);
 const audio = await generateSpeech(String(text || '').trim().slice(0, 180), voice);
 if (!audio) {
 return res.status(500).json({ error: 'No preview audio was generated.' });
 }
 console.log(`[audio][backend:tts-preview] returning bytes=${Buffer.from(audio, 'base64').length} mime=audio/wav`);
 res.json({ audio, audioFormat: 'wav', audioMimeType: 'audio/wav' });
 } catch (err) {
 console.error('[audio][backend:tts-preview] error', err.message);
 res.status(500).json({ error: err.message });
 }
});

app.get('/briefing-legacy/:userId', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 try {
 const userId = req.params.userId;
 const history = await getHistory(userId);
 const latestQueuedBriefing = [...history]
 .reverse()
 .find(entry => {
 if (entry.role !== 'assistant') return false;
 if (entry.kind !== 'briefing' && entry.kind !== 'proactive') return false;
 return entry.created_at && (Date.now() - new Date(entry.created_at).getTime()) < 36 * 60 * 60 * 1000;
 });

 if (latestQueuedBriefing) {
 return res.json({ text: latestQueuedBriefing.content, actions: latestQueuedBriefing.actions || [] });
 }

 const { spoken, actions } = await buildMorningBriefing(userId, new Date());
 res.json({ text: spoken, actions });
 } catch (err) {
 console.error('/briefing error:', err.message);
 res.status(500).json({ error: err.message });
 }
});

app.get('/briefing/:userId', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 try {
 const userId = req.params.userId;
 const queuedHistory = await getHistory(userId);
 const latestQueuedBriefing = [...queuedHistory]
 .reverse()
 .find(entry => {
 if (entry.role !== 'assistant') return false;
 if (entry.kind !== 'briefing' && entry.kind !== 'proactive') return false;
 return entry.created_at && (Date.now() - new Date(entry.created_at).getTime()) < 36 * 60 * 60 * 1000;
 });

 if (!latestQueuedBriefing) {
 return res.json({});
 }

 return res.json({
 text: latestQueuedBriefing.content,
 actions: latestQueuedBriefing.actions || [],
 created_at: latestQueuedBriefing.created_at,
 kind: latestQueuedBriefing.kind || 'briefing'
 });
 } catch (err) {
 console.error('/briefing error:', err.message);
 res.status(500).json({ error: err.message });
 }
});

```

```

 }
 });

app.get('/history/:userId', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 try {
 const history = await getHistory(req.params.userId, null, req.query.limit || 50, {
 since: req.query.since
 });
 res.json({ history });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.get('/history/:userId/sessions', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 try {
 const { data, error } = await supabase
 .from('conversations')
 .select('id, role, content, created_at')
 .eq('user_id', req.params.userId)

 .neq('role', 'system')
 .order('created_at', { ascending: false })
 .limit(250);
 if (error) throw error;
 res.json({ sessions: buildConversationSessions(data || []) });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.get('/history/:userId/search', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 const q = (req.query.q || '').trim();
 if (!q) return res.json({ results: [] });
 const escapedQuery = escapeILikePattern(q);
 try {
 const { data, error } = await supabase
 .from('conversations')
 .select('id, role, content, created_at')
 .eq('user_id', req.params.userId)
 .neq('role', 'system')
 .ilike('content', `_${escapedQuery}%`)
 .order('created_at', { ascending: false })
 .limit(20);
 if (error) throw error;
 const results = (data || []).
 .map(normalizeConversationRow)
 .map(entry => ({ ...entry, content: conversationFallbackText(entry) }));
 res.json({ results });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.get('/history/:userId/around', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 const messageId = String(req.query.messageId || '').trim();
 let anchor = new Date(String(req.query.createdAt || ''));
 const beforeLimit = Math.min(Math.max(Number(req.query.before) || 40, 1), 160);
 const afterLimit = Math.min(Math.max(Number(req.query.after) || 40, 1), 160);
 try {
 if (messageId) {
 const { data: anchorRow, error: anchorError } = await supabase
 .from('conversations')
 .select('created_at')
 .eq('user_id', req.params.userId)
 .eq('id', messageId)
 .maybeSingle();
 if (anchorError) throw anchorError;
 if (!anchorRow?.created_at) return res.status(404).json({ error: 'Message not found' });
 anchor = new Date(anchorRow.created_at);
 }
 if (Number.isNaN(anchor.getTime())) return res.status(400).json({ error: 'Invalid createdAt' });

 const base = supabase
 .from('conversations')
 .select('id, role, content, created_at')
 .eq('user_id', req.params.userId)
 .neq('role', 'system');
 const [before, after] = await Promise.all([
 base.lte('created_at', anchor.toISOString()).order('created_at', { ascending: false }).limit(beforeLimit),
 supabase
 .from('conversations')
 .select('id, role, content, created_at')

```

```

 .eq('user_id', req.params.userId)
 .neq('role', 'system')
 .gt('created_at', anchor.toISOString())
 .order('created_at', { ascending: true })
 .limit(afterLimit)
 });
 if (before.error) throw before.error;
 if (after.error) throw after.error;
 const rowsByKey = new Map();
 [...(before.data || []).reverse(), ...(after.data || [])].forEach(row => {
 rowsByKey.set(`${row.created_at}:${row.role}:${row.content}`, row);
 });
 res.json({ history: [...rowsByKey.values()].map(normalizeConversationRow) });
} catch (err) {
 res.status(500).json({ error: err.message });
}
});

app.get('/history/:userId/date', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 const date = req.query.date;
 if (!date || !/^d{4}-d{2}-d{2}$/.test(date)) return res.status(400).json({ error: 'Invalid date' });
 try {
 const start = new Date(date + 'T00:00:00.000Z').toISOString();
 const end = new Date(date + 'T23:59:59.999Z').toISOString();
 const { data, error } = await supabase
 .from('conversations')
 .select('id, role, content, created_at')
 .eq('user_id', req.params.userId)
 .neq('role', 'system')
 .gte('created_at', start)
 .lte('created_at', end)
 .order('created_at', { ascending: true })
 .limit(50);
 if (error) throw error;
 res.json({ history: (data || []).map(normalizeConversationRow) });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

// Shared logic for building the Gemini model + system prompt
async function buildChatContext(userId, message, trace = null, modelName = STREAMING_CHAT_MODEL, requestContext = {}) {
 const quickTurn = !requestContext.pendingAction && isQuickTurnMessage(message);
 const historyOptions = { since: requestContext.chatStartedAt };
 const [memory, history, preferences, enabledConnectors, userContext, cachedContentName, recentActions] = await Promise.all([
 quickTurn ? Promise.resolve('') : getMemory(userId, trace),
 getHistory(userId, trace, 12, historyOptions),
 getPreferences(userId, trace),
 quickTurn ? Promise.resolve([]) : getEnabledConnectors(userId, trace),
 quickTurn ? Promise.resolve('') : getUserContext(userId, trace),
 getPromptCacheName(trace, modelName),
 quickTurn ? Promise.resolve([]) : getRecentLoggedActions(userId, trace, 8, historyOptions)
]);
 const availableActions = quickTurn ? '' : buildAvailableActions(enabledConnectors);
 const statedContext = extractAlreadyStatedContext(history);

 const resolvedContext = requestContext.resolvedContext || (!quickTurn && isContextualReference(message)
 ? buildResolvedContext(history, recentActions)
 : null);
 const emailReplyContext = quickTurn
 ? ''
 : await buildEmailReplyDraftContext(userId, message, history, memory, preferences, trace);
 const dynamicSystemPrompt = quickTurn
 ? buildQuickTurnContext(preferences, statedContext)
 : buildDynamicSystemPrompt(
 memory,
 preferences,
 availableActions,
 [
 userContext,
 buildLocationContext(requestContext.location),
 buildNativeHintsContext(requestContext.nativeHints),
 buildPendingActionContext(requestContext.pendingAction),
 emailReplyContext,
 buildResolvedContextBlock(resolvedContext)
]
 .filter(Boolean).join('\n\n'),
 statedContext
);
 const searchReason = getSearchReason(message);
 const useSearch = Boolean(searchReason);
 if (useSearch) console.log(`[search] enabled (${searchReason} for:`, message.slice(0, 80));
 if (trace && resolvedContext?.label) {
 trace.log('context_brain.prompt_context', JSON.stringify({
 kind: resolvedContext.kind,
 label: String(resolvedContext.label || '').slice(0, 140),
 source: resolvedContext.source,

```



```

 confidence: resolvedContext.confidence,
 suggestedAction: resolvedContext.suggestedAction || null
 }));
}
return {
 history: quickTurn ? history.slice(-2) : history,
 availableActions,
 useSearch,
 searchReason,
 dynamicSystemPrompt,
 cachedContentName,
 quickTurn,
 statedContext,
 resolvedContext
};
}

const DATA_ACTIONS = new Set(['search_trains', 'station_board', 'get_emails', 'get_calendar_events', 'search_emails', 'get_telegram_contacts']);
const DIRECT_SUMMARY_ACTIONS = new Set(['search_trains', 'station_board']);

async function buildMorningBriefing(userId, now = new Date()) {
 const [memory, history] = await Promise.all([
 getMemory(userId),
 getHistory(userId)
]);

 const hour = getLocalHour(now);
 const greeting = hour < 12 ? 'morning' : hour < 18 ? 'afternoon' : 'evening';
 const systemPrompt = `You are Oxy. It's ${greeting} and you're checking in with your friend.

Here's what you know about them:
${memory || 'Not much yet – learn as you go.'}

Recent conversation:
${history.slice(-5).map(m => `${m.role}: ${m.content}`).join('\n')} || 'No recent messages.'}

Give a brief morning-style update. Keep it natural and friendly – not a corporate briefing. If there's nothing interesting, just say hi and check in.
- Only mention things that are directly supported by memory or recent conversation shown above.
- Do not invent plans, meetings, news, weather, or tasks.
- If there is no concrete update, just greet them and say it's a quiet start.

The current time is: ${now.toLocaleString('en-GB', { timeZone: TIMEZONE })}`;

 const model = genAI.getGenerativeModel({
 model: PRIMARY_CHAT_MODEL,
 systemInstruction: systemPrompt
 });
 const geminiRes = await model.generateContent('whats going on today?');
 return parseActions(geminiRes.response.text());
}

async function maybeCreateMorningBriefing(userId, now = new Date()) {
 const localHour = getLocalHour(now);
 if (localHour < 6 || localHour > 11) return null;

 const prefs = await getPreferenceMap(userId);
 const todayKey = getLocalDateKey(now);
 if (prefs[PROACTIVE_MORNING_PREF] === todayKey) return null;

 const { data: latestConversation } = await supabase
 .from('conversations')
 .select('created_at')
 .eq('user_id', userId)
 .order('created_at', { ascending: false })
 .limit(1)
 .maybeSingle();

 if (!latestConversation?.created_at) return null;
 const lastConversationAt = new Date(latestConversation.created_at).getTime();
 if (Number.isNaN(lastConversationAt) || (Date.now() - lastConversationAt) > 14 * 24 * 60 * 60 * 1000) {
 return null;
 }

 const { spoken, actions } = await buildMorningBriefing(userId, now);
 const text = stripActionMarkupForDisplay(spoken || '').trim();
 if (!text) return null;

 await saveMessage(userId, 'assistant', { text, actions, kind: 'briefing' });
 await setPreferenceValue(userId, PROACTIVE_MORNING_PREF, todayKey);
 return { type: 'morning_briefing', text };
}

async function getLatestNativeContext(userId) {
 const { data } = await supabase
 .from('native_context')
 .select('location, health, capabilities, settings, updated_at')
 .eq('user_id', userId)

```

```

 .maybeSingle();
 return data || null;
}

```

```

async function buildIntervalBriefing(userId, window, nativeContext, now = new Date()) {
 const [memory, history, preferences] = await Promise.all([
 getMemory(userId),
 getHistory(userId),
 getPreferences(userId)
]);

 const health = parseJsonObject(nativeContext?.health);
 const location = parseJsonObject(nativeContext?.location);
 const systemPrompt = `You are Oxy writing a useful, concise ${window.label.toLowerCase()} for the user.

```

Use only the information shown here. Do not invent weather, traffic, calendar events, health facts, or plans. If there is nothing useful, write one warm quiet-start sentence.

```

Memory:
${memory || 'none'}

```

```

Preferences:
${preferences || 'none'}

```

```

Recent conversation:
${history.slice(-6).map(m => ` ${m.role}: ${m.content}`).join('\n') || 'none'}

```

```

Native context:
Location: ${location.latitude & & location.longitude ? `${location.latitude}, ${location.longitude}` : 'not available'}
Health: ${Object.keys(health).length ? JSON.stringify(health).slice(0, 800) : 'not available'}
Current time: ${now.toLocaleString('en-GB', { timeZone: TIMEZONE })}

```

Keep it under 70 words. Include only useful items.`;

```

const model = genAI.getGenerativeModel({
 model: PRIMARY_CHAT_MODEL,
 systemInstruction: systemPrompt
});
const geminiRes = await model.generateContent(`${window.label} now`);
return stripActionMarkupForDisplay(geminiRes.response.text() || '').trim();
}

```

```

async function maybeCreateIntervalBriefing(userId, now = new Date()) {
 const window = getBriefingWindow(now);
 if (!window) return null;

 const nativeContext = await getLatestNativeContext(userId);
 const settings = parseJsonObject(nativeContext?.settings);
 if (settings.proactiveBriefings === false) return null;
 if (!['Quiet', 'Low'].includes(settings.autonomy)) return null;

 const todayKey = getLocalDateKey(now);
 const key = `proactive.briefing.${window.id}.${todayKey}`;
 const prefs = await getPreferenceMap(userId);
 if (prefs[key] === 'sent') return null;

 const text = await buildIntervalBriefing(userId, window, nativeContext, now);
 if (!text) return null;
 const briefing = await createBriefing(userId, {
 kind: `${window.id}_briefing`,
 title: window.label,
 body: text,
 source: 'schedule',
 metadata: { window: window.id, date: todayKey }
 });
 await setPreferenceValue(userId, key, 'sent');
 return { type: `${window.id}_briefing`, text: briefing.body };
}

```

```

async function maybeCreateHealthAlert(userId, nativeContext, now = new Date()) {
 const health = parseJsonObject(nativeContext?.health);
 const settings = parseJsonObject(nativeContext?.settings);
 if (!settings.healthAlerts) return null;
 const latest = Number(health.latestHeartRate);
 const resting = Number(health.restingHeartRate);
 const lowValue = [latest, resting].filter(Number.isFinite).find(value => value > 0 & & value < 45);
 if (!lowValue) return null;

 const todayKey = getLocalDateKey(now);
 const key = `proactive.health.low_hr.${todayKey}`;
 const prefs = await getPreferenceMap(userId);
 if (prefs[key] === 'sent') return null;

 const body = `Your heart rate data looks unusually low at ${Math.round(lowValue)} bpm. If that does not feel normal for you, check in with how
 const briefing = await createBriefing(userId, {
 kind: 'health_alert',
 title: 'Health check',

```

```

 body,
 source: 'healthkit',
 metadata: { heartRate: lowValue }
 });
 await setPreferenceValue(userId, key, 'sent');
 return { type: 'health_alert', text: briefing.body };
}

async function maybeCreateHomeFoodReminder(userId, nativeContext, now = new Date()) {
 const settings = parseJsonObject(nativeContext?.settings);
 if (!settings.locationReminders) return null;
 if (!['Active', 'Bold', 'High'].includes(settings.autonomy)) return null;
 const hour = getLocalHour(now);
 if (hour < 17 || hour > 21) return null;

 const location = parseJsonObject(nativeContext?.location);
 const home = parseJsonObject(settings.homeLocation);
 const lat = Number(location.latitude);
 const lng = Number(location.longitude);
 const homeLat = Number(home.latitude);
 const homeLng = Number(home.longitude);
 if (![lat, lng, homeLat, homeLng].every(Number.isFinite)) return null;

 const metres = haversineMetres(lat, lng, homeLat, homeLng);
 if (metres > 600) return null;

 const todayKey = getLocalDateKey(now);
 const key = `proactive.food.near_home.${todayKey}`;
 const prefs = await getPreferenceMap(userId);
 if (prefs[key] === 'sent') return null;

 const body = "You're close to home. If you haven't eaten yet, this is a good moment to sort food before you fully land.";
 const briefing = await createBriefing(userId, {
 kind: 'location_food_reminder',
 title: 'Food reminder',
 body,

 source: 'location',
 metadata: { distanceMetres: Math.round(metres) }
 });
 await setPreferenceValue(userId, key, 'sent');
 return { type: 'location_food_reminder', text: briefing.body };
}

function haversineMetres(lat1, lon1, lat2, lon2) {
 const radius = 6371000;
 const toRad = degrees => degrees * Math.PI / 180;
 const dLat = toRad(lat2 - lat1);
 const dLon = toRad(lon2 - lon1);
 const a = Math.sin(dLat / 2) ** 2 +
 Math.cos(toRad(lat1)) * Math.cos(toRad(lat2)) * Math.sin(dLon / 2) ** 2;
 return 2 * radius * Math.atan2(Math.sqrt(a), Math.sqrt(1 - a));
}

async function maybeCreateFailedActionFollowUp(userId, now = new Date()) {
 const prefs = await getPreferenceMap(userId);
 const { data: failedAction } = await supabase
 .from('action_log')
 .select('id, action, error, created_at')
 .eq('user_id', userId)
 .eq('status', 'failed')
 .order('created_at', { ascending: false })
 .limit(1)
 .maybeSingle();

 if (!failedAction?.id) return null;
 if (prefs[PROACTIVE_FAILURE_PREF] === failedAction.id) return null;

 const failedAt = new Date(failedAction.created_at).getTime();
 if (Number.isNaN(failedAt) || (now.getTime() - failedAt) > 90 * 60 * 1000) {
 return null;
 }

 const actionType = failedAction.action?.type || failedAction.action?.action?.type || failedAction.action?.action || failedAction.action?.type;
 if (!['find_place', 'get_directions', 'plan_trip', 'play_music', 'music_control', 'add_to_music_playlist'].includes(actionType)) {
 return null;
 }
 const actionLabel = humanizeActionType(actionType);
 const detail = String(failedAction.error || '').trim();
 if (!/(not_connected|reconnect|permission|authorized|authenticate|expired|revoked)/i.test(detail)) {
 return null;
 }
 const cleanDetail = detail
 .replace(/^\.\unknown$/i, '')
 .replace(/^\.\Maps error:/i, '')
 .replace(/^\.\Google error:/i, '')
 .replace(/\\s+/g, ' ');

```

```

 .trim();
const followUpText = cleanDetail
 ? `${actionLabel} needs attention: ${cleanDetail.slice(0, 100)}.`
 : `${actionLabel} needs attention before I can finish it.`;

await createBriefing(userId, {
 kind: 'failed_action_followup',
 title: 'Action follow-up',
 body: followUpText,
 source: 'action_log',
 metadata: { actionLogId: failedAction.id, actionType }
});
await setPreferenceValue(userId, PROACTIVE_FAILURE_PREF, failedAction.id);
return { type: 'failed_action_followup', text: followUpText };
}

function emptyProactiveSummary() {
 return {
 usersScanned: 0,
 briefings: 0,
 failureFollowUps: 0,
 healthAlerts: 0,
 locationReminders: 0,
 failures: 0
 };
}

async function runProactiveForUser(userId, logger = console, now = new Date()) {
 const summary = emptyProactiveSummary();
 summary.usersScanned = 1;
 try {
 const nativeContext = await getLatestNativeContext(userId);
 const [briefing, followUp, healthAlert, foodReminder] = await Promise.all([
 maybeCreateIntervalBriefing(userId, now),
 maybeCreateFailedActionFollowUp(userId, now),
 nativeContext ? maybeCreateHealthAlert(userId, nativeContext, now) : Promise.resolve(null),
 nativeContext ? maybeCreateHomeFoodReminder(userId, nativeContext, now) : Promise.resolve(null)
]);
 if (briefing) summary.briefings += 1;
 if (followUp) summary.failureFollowUps += 1;
 if (healthAlert) summary.healthAlerts += 1;
 if (foodReminder) summary.locationReminders += 1;
 const created = [briefing?.type, followUp?.type, healthAlert?.type, foodReminder?.type].filter(Boolean);
 if (created.length) logger.log(`[proactive] queued for ${userId}: ${created.join(', ')}`);
 } catch (sweepError) {
 summary.failures += 1;
 logger.error(`[proactive] failed for ${userId}:`, sweepError.message);
 }
 return summary;
}

function mergeProactiveSummary(target, source) {
 for (const key of Object.keys(emptyProactiveSummary())) {
 target[key] = (target[key] || 0) + (source[key] || 0);
 }
}

async function runProactiveSweep(logger = console) {
 const startedAt = Date.now();
 const summary = emptyProactiveSummary();

 const { data: users, error } = await supabase
 .from('users')
 .select('user_id');
 if (error) throw error;

 for (const user of users || []) {
 const userSummary = await runProactiveForUser(user.user_id, logger);
 mergeProactiveSummary(summary, userSummary);
 }

 summary.durationMs = Date.now() - startedAt;
 logger.log(`[proactive] sweep complete: ${JSON.stringify(summary)}`);
 return summary;
}

function canUseDirectActionSummary(actionResults) {
 return actionResults.length > 0 && actionResults.every(entry =>
 DIRECT_SUMMARY_ACTIONS.has(entry.action) && entry.result?.success && entry.result?.text
);
}

function summarizeActionResults(actionResults) {
 return actionResults
 .map(entry => entry.result?.text?.trim())
 .filter(Boolean)
 .join('\n\n');
}

```

```

}

function normalizeActionResultsForClient(actionResults) {
 const seen = new Set();
 const out = [];
 for (const entry of actionResults || []) {
 const error = entry?.result?.error || '';
 const text = entry?.result?.text || '';
 const key = `${entry?.action || ''}:${entry?.result?.success === false ? error : text}`;
 if (seen.has(key)) continue;
 seen.add(key);
 out.push(entry);
 }
 return out;
}

function userFacingActionFailure(entry) {
 const action = entry?.action || '';
 const rawError = String(entry?.result?.error || '').trim();
 if (action === 'book_uber' || action === 'find_place') {
 if (/Google Places is not ready|Places API|Google Places is not configured/i.test(rawError)) {
 return 'Nearby place ranking needs Google Places enabled on the server. Uber itself does not need an API key.';
 }
 if (/need your current location|enable location/i.test(rawError)) {
 return 'I need your current location to find that nearby place. Enable location and try again.';
 }
 if (/couldn't find a nearby|No place results found/i.test(rawError)) {
 return 'I could not find that nearby place from your current location. Try a different place name or enable location.';
 }
 if (/Geocoding error|No results found/i.test(rawError)) {
 return 'I could not find that destination. Try a different place name.';
 }
 }
 return rawError || 'That action failed.';
}

function toSingleSentence(text) {
 const cleaned = stripActionMarkupForDisplay(String(text || ''))
 .replace(/\\s+/g, ' ')
 .trim();
 if (!cleaned) return '';
 const match = cleaned.match(/[^\?]+\.[\?]+(?:["']\w+["']\?)+?/);
 const sentence = (match ? match[0] : cleaned).trim();
 return /[.!?]$/.test(sentence) ? sentence : `${sentence}.`;
}

function summarizeCompletedActionsConcise(actionResults) {
 const successful = actionResults.filter(entry => entry?.result?.success !== false);
 if (!successful.length) return '';
 const dataOnly = successful.every(entry => DATA_ACTIONS.has(entry.action));
 if (dataOnly) {
 return successful.every(entry => DIRECT_SUMMARY_ACTIONS.has(entry.action))
 ? summarizeActionResults(successful)
 : '';
 }

 const resultText = summarizeActionResults(successful);
 if (resultText) return resultText;

 const normalized = successful
 .map(entry => toSingleSentence(entry.result?.text || humanizeActionType(entry.action)))
 .filter(Boolean);
 if (!normalized.length) return '';
 if (normalized.length === 1) return normalized[0];
 return `${normalized
 .map(text => text.replace(/[.!?]+$/g, ''))
 .join('; ')}.`;
}

function summarizeFinishedActionsForUser(actionResults) {
 const normalizedResults = normalizeActionResultsForClient(actionResults);
 if (!normalizedResults.length) return '';
 const failures = normalizedResults.filter(entry => entry?.result?.success === false);
 if (failures.length) {
 return failures
 .map(entry => {
 const error = userFacingActionFailure(entry);
 return `${humanizeActionType(entry.action)} failed: ${error}`;
 })
 .join('\n');
 }
 const pending = normalizedResults.filter(entry => entry?.result?.pending);
 if (pending.length) {
 return pending
 .map(entry => entry.result?.text || `${reviewTitleForAction({ type: entry.action })}. Say "confirm" to continue or "cancel" to stop.`)
 .join('\n');
 }
}

```

```

 return summarizeCompletedActionsConcise(normalizedResults);
 }

function buildStructuredDataSummary(entry) {
 const result = entry?.result || {};
 if (entry?.action === 'get_emails' || entry?.action === 'search_emails') {
 if (!Array.isArray(result.emails) || result.emails.length === 0) return result.text || 'No emails found.';
 const emails = result.emails.map((email, index) => {
 ` ${index + 1}. From: ${email.from || 'Unknown sender'} | Subject: ${email.subject || '(No subject)'} ${email.date ? ` | Date: ${email.date}` : ''}`
));
 return `Email results:\n${emails.join('\n')}`;
 }
 if (entry?.action === 'get_calendar_events') {
 if (!Array.isArray(result.events) || result.events.length === 0) return result.text || 'No upcoming events found.';
 const events = result.events.map((event, index) => {
 ` ${index + 1}. ${event.title || 'Untitled'} ${event.start ? ` | Starts: ${event.start}` : ''} ${event.end ? ` | Ends: ${event.end}` : ''}`
));
 return `Upcoming events:\n${events.join('\n')}`;
 }
 if (entry?.action === 'get_telegram_contacts') {
 if (!Array.isArray(result.contacts) || result.contacts.length === 0) return result.text || 'No contacts found.';
 const contacts = result.contacts.map((contact, index) => ` ${index + 1}. ${contact.name || contact.username || 'Unnamed contact'}`);
 return `Telegram contacts:\n${contacts.join('\n')}`;
 }
 return result.text || '';
}

function getStructuredDataResults(actionResults) {
 return actionResults
 .filter(entry => DATA_ACTIONS.has(entry.action) && entry.result?.success)
 .map(entry => ({ action: entry.action, text: buildStructuredDataSummary(entry) }))
 .filter(entry => entry.text);
}

// Fire-and-forget post-response tasks (memory + style preferences)
function postResponseTasks(userId, message) {
 if (shouldSaveMemory(message)) {
 extractMemoryFact(userId, message).then(fact => {
 if (fact) saveMemory(userId, fact, 'fact');
 }).catch(() => {});
 }
 const styleCues = [
 { pattern: /too long|tl;dr|too short|not enough|be brief|be concise|more detail|explain more|less detail|shut up|stop rambling/i, key: 'response_preference' },
 { pattern: /be direct|be blunt|be nice|be polite|don't be rude|don't be sarcastic|more casual|more formal/i, key: 'tone_preference' },
 { pattern: /use bullet|use numbers|no bullet|no numbers|bullet points|step by step/i, key: 'format_preference' },
];
 for (const cue of styleCues) {
 if (cue.pattern.test(message)) {
 savePreference(userId, cue.key, `User said "${message}" - adapt accordingly`);
 }
 }
}

async function respondWithResult({ res, streaming, wantsTTS, settings, trace, userId, message, spoken, actionResults = [] }) {
 saveMessage(userId, 'assistant', { text: spoken, actions: actionResults }, trace)
 .catch(err => trace.log('supabase.conversations.insert_assistant.short_async_fail', err.message));
 postResponseTasks(userId, message);

 if (streaming) {
 res.setHeader('Content-Type', 'text/event-stream');
 res.setHeader('Cache-Control', 'no-cache');
 res.setHeader('Connection', 'keep-alive');
 res.flushHeaders();
 const sse = obj => res.write(`data: ${JSON.stringify(obj)}\n\n`);
 if (actionResults.length) sse({ type: 'actions', results: actionResults });
 sse({ type: 'replace', text: spoken });
 if (wantsTTS) {
 try {
 const audio = await trace.run('gemini.tts.generateSpeech.short_response', () => generateSpeech(buildVoiceExcerpt(spoken), settings.voice));
 if (audio) sse({ type: 'audio', data: audio, format: 'wav', mimeType: 'audio/wav', seq: 0, chunk: 0 });
 } catch (ttsErr) {
 console.error(['tts error'], ttsErr.message);
 sse({ type: 'tts-error', error: ttsErr.message });
 }
 }
 sse({ type: 'done' });
 res.end();
 return;
 }

 const result = { text: spoken, actions: actionResults };
 if (wantsTTS) {
 try {
 const audio = await trace.run('gemini.tts.generateSpeech.short_response_nonstream', () => generateSpeech(buildVoiceExcerpt(spoken), settings.voice));
 if (audio) {
 result.audio = audio;
 }
 }
 }
}

```

```

 result.audioFormat = 'wav';
 result.audioMimeType = 'audio/wav';
 }
} catch (ttsErr) {
 console.error('[tts error]', ttsErr.message);
 result.ttsError = ttsErr.message;
}
}
res.json(result);
}

app.post('/chat', chatRateLimiter, async (req, res) => {
 const streaming = req.query.stream === 'true';

 try {
 const { message, userId, settings = {}, location = null, nativeHints = null, chatStartedAt = null } = req.body;
 if (!requireMatchingUser(req, res, userId)) return;
 const wantsTTS = req.query.tts === 'true';

 if (!message?.trim()) {
 return res.status(400).json({ error: 'message is required.' });
 }

 const trace = createRequestTrace(`chat:${userId}:${Date.now()}`);
 trace.log(`request.start stream=${streaming} tts=${wantsTTS} msg=${JSON.stringify((message || '').slice(0, 80))}`);

 // Let the model start as soon as context is ready instead of waiting on the DB write.
 saveMessage(userId, 'user', message, trace).catch(err => trace.log('supabase.conversations.insert_user.async_fail', err.message));

 const pendingAction = await getPendingAction(userId);
 if (pendingAction && isPendingCancelMessage(message)) {
 await clearPendingAction(userId);
 await respondWithResult({
 res,
 streaming,
 wantsTTS,
 settings,
 trace,
 userId,
 message,
 spoken: 'Cancelled.'
 });
 return;
 }

 if (pendingAction && isPendingConfirmMessage(message)) {
 trace.log(`pending_action.confirm ${pendingAction.action.type}`);
 let actionResults = await executeActions(userId, [pendingAction.action], {
 userMessage: pendingAction.userMessage || message,
 location,

 nativeHints: pendingAction.nativeHints || nativeHints,
 bypassReview: true,
 trace
 }, trace);
 actionResults = normalizeActionResultsForClient(actionResults);
 await clearPendingAction(userId);
 const spoken = summarizeFinishedActionsForUser(actionResults) ||
 actionResults.map(a => a.result?.text || a.result?.error).filter(Boolean).join(' ') ||
 'Done.';
 await respondWithResult({
 res,
 streaming,
 wantsTTS,
 settings,
 trace,
 userId,
 message,
 spoken,
 actionResults
 });
 return;
 }

 const deterministicQuickReply = getDeterministicQuickReply(message);
 if (deterministicQuickReply) {
 saveMessage(userId, 'assistant', { text: deterministicQuickReply, actions: [] }, trace)
 .catch(err => trace.log('supabase.conversations.insert_assistant.quick_async_fail', err.message));
 postResponseTasks(userId, message);

 if (streaming) {
 res.setHeader('Content-Type', 'text/event-stream');
 res.setHeader('Cache-Control', 'no-cache');
 res.setHeader('Connection', 'keep-alive');
 res.flushHeaders();
 const sse = obj => res.write(`data: ${JSON.stringify(obj)}\n\n`);
 sse({ type: 'text', chunk: deterministicQuickReply });
 }
 }
 }
});

```

```

 if (wantsTTS) {
 try {
 const audio = await trace.run('gemini.tts.generateSpeech.quick', () => generateSpeech(buildVoiceExcerpt(deterministicQuickReply),
 if (audio) sse({ type: 'audio', data: audio, format: 'wav', mimeType: 'audio/wav', seq: 0, chunk: 0 });
 } catch (ttsErr) {
 console.error('[tts error]', ttsErr.message);
 sse({ type: 'tts-error', error: ttsErr.message });
 }
 }
 sse({ type: 'done' });
 res.end();
 return;
 }

 const result = { text: deterministicQuickReply, actions: [] };
 if (wantsTTS) {
 try {
 const audio = await trace.run('gemini.tts.generateSpeech.quick_nonstream', () => generateSpeech(buildVoiceExcerpt(deterministicQuickReply),
 if (audio) {
 result.audio = audio;
 result.audioMimeType = 'audio/wav';
 }
 } catch (ttsErr) {
 console.error('[tts error]', ttsErr.message);
 result.ttsError = ttsErr.message;
 }
 }
 return res.json(result);
}

const contextualTurn = await inferContextualDeterministicTurn(userId, message, settings, trace, {
 since: chatStartedAt
});
if (contextualTurn?.spokenOnly) {
 trace.log(`context_router.match ${contextualTurn.reason}`);
 await respondWithResult({
 res,
 streaming,
 wantsTTS,
 settings,
 trace,
 userId,
 message,
 spoken: contextualTurn.spoken,
 actionResults: []
 });
 return;
}

const deterministicAction = contextualTurn || inferDeterministicAction(message, { settings });
if (deterministicAction) {
 trace.log(`intent_router.match ${deterministicAction.reason}`);

 if (streaming) {
 res.setHeader('Content-Type', 'text/event-stream');
 res.setHeader('Cache-Control', 'no-cache');
 res.setHeader('Connection', 'keep-alive');
 res.flushHeaders();
 const sse = obj => res.write(`data: ${JSON.stringify(obj)}\n\n`);
 const sendStatus = (status, label, extra = {}) => sse({ type: 'status', status, label, ...extra });
 sendStatus('action_start', getActionStatusLabel(deterministicAction.actions[0].type, 'start'), { action: deterministicAction.actions[0].type });
 let actionResults = await executeActions(userId, deterministicAction.actions, { userMessage: message, location, nativeHints, trace }, {
 onActionComplete: (action, result) => sendStatus('action_complete', getActionStatusLabel(action.type, actionCompletionPhase(result)), {
 action: action.type,
 success: result?.success !== false
 })
 });
 actionResults = normalizeActionResultsForClient(actionResults);
 const spoken = summarizeFinishedActionsForUser(actionResults) || deterministicAction.spoken;
 sse({ type: 'actions', results: actionResults });
 sse({ type: 'replace', text: spoken });
 if (wantsTTS) {
 try {
 const audio = await trace.run('gemini.tts.generateSpeech.intent_router', () => generateSpeech(buildVoiceExcerpt(spoken), settings));
 if (audio) sse({ type: 'audio', data: audio, format: 'wav', mimeType: 'audio/wav', seq: 0, chunk: 0 });
 } catch (ttsErr) {
 console.error('[tts error]', ttsErr.message);
 sse({ type: 'tts-error', error: ttsErr.message });
 }
 }
 sse({ type: 'done' });
 res.end();

 saveMessage(userId, 'assistant', { text: spoken, actions: actionResults }, trace)

 .catch(err => trace.log('supabase.conversations.insert_assistant.intent_async_fail', err.message));
 postResponseTasks(userId, message);
 }
}

```



[illegible]

```

 heldDisplayText = heldDisplayText.slice(-8);
 if (visible) {
 hasStreamedText = true;
 sse({ type: 'text', chunk: visible });
 }
 }
};
const flushSafeDisplayText = () => {
 if (actionMarkupStarted || !heldDisplayText) return;
 hasStreamedText = true;
 sse({ type: 'text', chunk: heldDisplayText });
 heldDisplayText = '';
};
const ttsStreamer = wantsTTS ? createSentenceTtsStreamer({
 voiceName: settings.voice,
 sse,
 trace,
 onSpeakingStart: () => sendStatus('speaking_start', 'Speaking')
}) : null;
for await (const chunk of stream) {
 const text = chunk.text || '';
 if (text) {
 if (firstChunk) { trace.log('gemini.first_token'); firstChunk = false; }
 fullText += text;
 emitSafeDisplayText(text);
 // Kick off TTS for complete sentences as they arrive, not after full generation
 if (ttsStreamer && !actionMarkupStarted) ttsStreamer.ingest(fullText);
 }
}
flushSafeDisplayText();
trace.log('gemini.initial_complete');

if (!fullText.trim()) {
 fullText = await recoverEmptyModelResponse({ model: chatModel, initialRequest, message, trace });
 if (fullText) trace.log('gemini.empty_recovery_success');
}

let { spoken, actions } = parseActions(fullText);
spoken = stripActionMarkupForDisplay(spoken).trim();
if (!spoken && !actions.length) {
 const recovered = await recoverEmptyModelResponse({ model: chatModel, initialRequest, message, trace });
 if (recovered) {
 fullText = recovered;
 ({ spoken, actions } = parseActions(fullText));
 spoken = stripActionMarkupForDisplay(spoken).trim();
 trace.log('gemini.blank_spoken_recovery_success');
 }
}

// Execute actions in parallel
let actionResults = [];
if (actions.length > 0) {
 actionResults = await executeActions(userId, actions, { userMessage: message, location, nativeHints, trace }, trace, {
 onActionStart: action => sendStatus('action_start', getActionStatusLabel(action.type, 'start'), { action: action.type }),
 onActionComplete: (action, result) => sendStatus('action_complete', getActionStatusLabel(action.type, actionCompletionPhase(result)), { action: action.type, success: result?.success !== false })
 });
};
actionResults = normalizeActionResultsForClient(actionResults);
sse({ type: 'actions', results: actionResults });
trace.log('actions.complete');
}

// For data-fetching actions, stream a follow-up summary
const dataResults = getStructuredDataResults(actionResults);
if (canUseDirectActionSummary(actionResults)) {
 spoken = summarizeActionResults(actionResults);
 sse({ type: 'replace', text: spoken });
 if (ttsStreamer) ttsStreamer.ingest(spoken);
} else if (dataResults.length > 0) {
 sse({ type: 'replace', text: '' });
 const context = dataResults.map(a => a.text).join('\n\n');
 const followUpRequest = buildModernGenerateRequest({
 dynamicSystemPrompt,
 useSearch,
 cachedContentName,
 baseHistory,
 userContent: { role: 'user', parts: [{ text: message }] }
 });
 followUpRequest.contents.push(
 { role: 'model', parts: [{ text: spoken || '...' }] },
 { role: 'user', parts: [{ text: `Here are the results:\n\n${context}\n\nSpeak these back naturally and conversationally. Be concise.` }]
);
 const followUp = await trace.run('gemini.generateContentStream.followup', () => modernGenAI.models.generateContentStream({
 model: chatModel,
 contents: followUpRequest.contents,

```

[illegible]

```

let actionResults = [];
if (actions.length > 0) {
 actionResults = await executeActions(userId, actions, { userMessage: message, location, nativeHints, trace }, trace);
 actionResults = normalizeActionResultsForClient(actionResults);
}

// For data-fetching actions, re-prompt Gemini with results
const dataResults = getStructuredDataResults(actionResults);
if (canUseDirectActionSummary(actionResults)) {
 spoken = summarizeActionResults(actionResults);
} else if (dataResults.length > 0) {
 const context = dataResults.map(a => a.text).join('\n\n');
 const followUpRequest = buildModernGenerateRequest({
 dynamicSystemPrompt,
 useSearch,
 cachedContentName,
 baseHistory,
 userContent: { role: 'user', parts: [{ text: message }] }
 });
 followUpRequest.contents.push(
 { role: 'model', parts: [{ text: spoken || '...' }] },
 { role: 'user', parts: [{ text: `Here are the results:\n\n${context}\n\nSpeak these back naturally and conversationally. Be concise. Only` }
);
 const followUp = await trace.run('gemini.generateContent.followup_nonstream', () => modernGenAI.models.generateContent({
 model: chatModel,
 contents: followUpRequest.contents,
 config: followUpRequest.config
 }));
 spoken = parseActions(followUp.text || '').spoken || context;
}
const actionConfirmation = summarizeFinishedActionsForUser(actionResults);
if (actionConfirmation) spoken = actionConfirmation;

if (!spoken) {
 spoken = actionResults.length
 ? (actionResults.map(a => a.result?.text || a.result?.error).filter(Boolean).join(' ') || 'I could not complete that action.')
 : "I couldn't get a clean answer for that. Ask me again and I'll re-check it.";
}

// Don't block on saving assistant message
saveMessage(userId, 'assistant', { text: spoken, actions: actionResults }, trace)
 .catch(err => trace.log('supabase.conversations.insert_assistant.async_fail', err.message));

const result = { text: spoken, actions: actionResults };

if (wantsTTS) {
 try {
 const audio = await trace.run('gemini.tts.generateSpeech.nonstream', () => generateSpeech(buildVoiceExcerpt(spoken), settings.voice));
 if (audio) {
 console.log(`[audio][backend:chat-json] returning tts audio bytes=${Buffer.from(audio, 'base64').length} mime=audio/wav`);
 result.audio = audio;
 result.audioFormat = 'wav';
 result.audioMimeType = 'audio/wav';
 }
 } catch (ttsErr) {
 console.error('[tts error]', ttsErr.message);
 result.ttsError = ttsErr.message;
 }
}

trace.log('request.total');
res.json(result);
postResponseTasks(userId, message);

} catch (err) {
 console.log(`[trace:chat:unscoped] FAIL outer ${err.message}`);
 console.error('/chat error:', err.message);
 res.status(500).json({ error: err.message, text: `Error: ${err.message}` });
}
});

app.get('/preferences/:userId', async (req, res) => {
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 try {
 const { data, error } = await supabase
 .from('preferences')
 .select('*')
 .eq('user_id', req.params.userId)
 .order('updated_at', { ascending: false });
 if (error || !data) return res.json({ preferences: [] });
 res.json({ preferences: data });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

app.delete('/preferences/:userId', async (req, res) => {

```

Page 85 · Confidential

```

 if (!userId) {
 return res.status(400).send('Invalid OAuth state');
 }
 if (!code || typeof code !== 'string') {
 return res.status(400).send('Missing OAuth code');
 }

 try {
 const redirectUri = `${req.headers['x-forwarded-proto'] || req.protocol}://${req.get('host')}/auth/google/callback`;
 const resp = await axios.post('https://oauth2.googleapis.com/token', {
 code,
 client_id: process.env.GMAIL_CLIENT_ID,
 client_secret: process.env.GMAIL_CLIENT_SECRET,
 redirect_uri: redirectUri,
 grant_type: 'authorization_code'
 });

 const tokens = {
 access_token: resp.data.access_token,
 refresh_token: resp.data.refresh_token,
 expires_at: Date.now() + resp.data.expires_in * 1000,
 client_id: process.env.GMAIL_CLIENT_ID,
 client_secret: process.env.GMAIL_CLIENT_SECRET
 };

 const { error: upsertError } = await supabase.from('connectors').upsert(
 { user_id: userId, connector_id: 'google', enabled: true, tokens, updated_at: new Date().toISOString() },
 { onConflict: 'user_id,connector_id' }
);
 if (upsertError) throw upsertError;

 const appOrigin = process.env.APP_URL || `${req.headers['x-forwarded-proto'] || req.protocol}://${req.get('host')}`;
 res.send(`
 <html><body style="font-family:sans-serif;text-align:center;padding:40px;background:#0d0d0d;color:#fff">
 <p style="font-size:18px">✓ Google connected</p>
 <p style="color:#888;font-size:13px">You can close this window</p>
 <script>window.opener?.postMessage('google_auth_success',${JSON.stringify(appOrigin)});setTimeout(=>window.close(),1500);</script>
 </body></html>
 `);
 } catch (err) {
 console.error('/auth/google/callback error:', err.response?.data || err.message);
 const appOrigin = process.env.APP_URL || `${req.headers['x-forwarded-proto'] || req.protocol}://${req.get('host')}`;
 const errMsg = escapeHtml(err.response?.data?.error_description || err.message);
 res.send(`
 <html><body style="font-family:sans-serif;text-align:center;padding:40px;background:#0d0d0d;color:#fff">
 <p style="font-size:18px">X Connection failed</p>
 <p style="color:#888;font-size:13px">${errMsg}</p>
 <script>window.opener?.postMessage('google_auth_error',${JSON.stringify(appOrigin)});setTimeout(=>window.close(),3000);</script>
 </body></html>
 `);
 }
 });
});

app.get('/debug/:userId', async (req, res) => {
 const debugToken = req.headers['x-debug-token'];
 if (!process.env.DEBUG_SECRET) return res.status(404).json({ error: 'Not found' });
 if (debugToken !== process.env.DEBUG_SECRET) return res.status(403).json({ error: 'Forbidden' });
 if (!requireMatchingUser(req, res, req.params.userId)) return;
 const userId = req.params.userId;
 try {
 const enabledConnectors = await getEnabledConnectors(userId);
 const { data: connRow } = await supabase
 .from('connectors').select('connector_id, enabled, tokens').eq('user_id', userId);

 const [emailTest, calendarTest] = await Promise.all([
 dispatch(userId, 'get_emails', { max_results: 1 }).catch(e => ({ error: e.message })),
 dispatch(userId, 'get_calendar_events', { max_results: 1 }).catch(e => ({ error: e.message }))
]);

 res.json({
 userId,
 enabledConnectors,
 connectorRows: (connRow || []).map(row => ({
 connector_id: row.connector_id,
 enabled: row.enabled,
 hasTokens: !!row.tokens
 })),
 googleEmailTest: emailTest,
 googleCalendarTest: calendarTest,
 envHasGmailRefreshToken: !!process.env.GMAIL_REFRESH_TOKEN,
 envHasGeminKey: !!process.env.GEMINI_API_KEY
 });
 } catch (err) {
 res.status(500).json({ error: err.message });
 }
});

```

```

app.get('/health', async (_req, res) => {
 const missingEnv = getMissingRuntimeEnv();
 let dbStatus = 'ok';
 let dbLatencyMs = 0;
 try {
 const dbStart = Date.now();
 await supabase.from('users').select('id').limit(1);
 dbLatencyMs = Date.now() - dbStart;
 } catch (e) {
 dbStatus = 'error';
 }
 const mem = process.memoryUsage();
 const versionInfo = getRuntimeVersion();
 res.json({
 status: (missingEnv.length || dbStatus !== 'ok') ? 'degraded' : 'ok',
 db: { status: dbStatus, latencyMs: dbLatencyMs },
 memory: { heapUsedMb: Math.round(mem.heapUsed / 1024 / 1024), heapTotalMb: Math.round(mem.heapTotal / 1024 / 1024) },
 uptime: Math.round(process.uptime()),
 missingEnv,
 ...versionInfo
 });
});

app.get('/version', (_req, res) => {
 res.json(getRuntimeVersion());
});

app.get('/changelog', (req, res) => {
 res.json([
 { version: '2.0.0', date: '2026-05-28', changes: ['Multi-surface assistant: PWA + native iOS app', 'Gemini Live real-time voice', 'Proactive', 'Maps connector with Google Places', 'Trainline train search', 'Uber/UberEats/Deliveroo de', 'Per-user authentication', 'Session tokens', 'Connector health diagnostics'] },
 { version: '1.5.0', date: '2026-04-01', changes: ['Maps connector with Google Places', 'Trainline train search', 'Uber/UberEats/Deliveroo de', 'Per-user authentication', 'Session tokens', 'Connector health diagnostics'] },
 { version: '1.4.0', date: '2026-03-01', changes: ['Per-user authentication', 'Session tokens', 'Connector health diagnostics'] },
 { version: '1.3.0', date: '2026-02-01', changes: ['Telegram User API connector', 'Google Calendar integration', 'Action contracts and risk l', 'Gmail connector with OAuth', 'Context brain for conversation follow-ups', 'Prompt and con', 'Gmail connector with OAuth', 'Context brain for conversation follow-ups', 'Prompt and con'] },
 { version: '1.2.0', date: '2026-01-01', changes: ['Gmail connector with OAuth', 'Context brain for conversation follow-ups', 'Prompt and con', 'Gmail connector with OAuth', 'Context brain for conversation follow-ups', 'Prompt and con'] }
]);
});

function legalPage(title, body) {
 return `<!doctype html>
<html lang="en">
<head>
 <meta charset="utf-8">
 <meta name="viewport" content="width=device-width, initial-scale=1">
 <title>${escapeHtml(title)} · Oxy</title>
 <style>
 body{margin:0;background:#0b0b0c;color:#f4f0ec;font:16px/1.55 -apple-system,BlinkMacSystemFont,"Segoe UI",sans-serif}
 main{max-width:760px;margin:0 auto;padding:48px 20px 72px}
 h1{font-size:34px;line-height:1.1;margin:0 0 10px}
 h2{font-size:20px;margin:30px 0 10px}
 p,li{color:#cfc8c1}
 a{color:#e97961}
 .meta{color:#8f8781;font-size:14px;margin-bottom:30px}
 </style>
</head>
<body><main>${body}</main></body>
</html>`;
}

app.get('/privacy', (_req, res) => {
 res.setHeader('Content-Type', 'text/html; charset=utf-8');
 res.send(legalPage('Privacy Policy', `
 <h1>Privacy Policy</h1>
 <p class="meta">Last updated ${escapeHtml(getLocalDateKey())}</p>
 <h2>Data Controller</h2>
 <p>Oxy is operated by Chizi Gamonye-Wuchi. Contact: support@oxy.app
 <h2>What We Collect</h2>

 Chat messages and conversation history
 Voice audio (transcribed and discarded after processing)
 Location data (when location permission is granted)
 Contacts (when contacts permission is granted)
 Health data (when HealthKit permission is granted)
 Calendar and reminder data (when calendar permission is granted)
 Email content (when Gmail connector is connected)
 OAuth tokens for connected services
 Memories you ask Oxy to keep, plus stable facts inferred from conversations

 <h2>How We Use It</h2>

 Providing the AI assistant service and completing requested actions
 Improving the service through aggregated usage analytics

 <h2>Lawful Basis</h2>
 <p>Contract performance for account and assistant features. Legitimate interests for service improvement.</p>
 <h2>Third-Party Processors</h2>
 `);
});

```

Page 88 · Confidential



```

const cutoff = new Date(Date.now() - 180 * 24 * 60 * 60 * 1000).toISOString();
const { data: oldConvs, error } = await supabase.from('conversations').select('id, user_id, created_at').lt('created_at', cutoff);
if (error) return res.status(500).json({ error: error.message });

let deleted = 0;
const byUser = {};
for (const c of (oldConvs || [])) {
 if (!byUser[c.user_id]) byUser[c.user_id] = [];
 byUser[c.user_id].push(c.id);
}

for (const [userId, ids] of Object.entries(byUser)) {
 const { count } = await supabase.from('conversations').select('id', { count: 'exact', head: true }).eq('user_id', userId);
 if (count > 500) {
 const toDelete = ids.slice(0, Math.min(ids.length, count - 500));
 if (toDelete.length > 0) {
 const { error: delErr } = await supabase.from('conversations').delete().in('id', toDelete);
 if (!delErr) deleted += toDelete.length;
 }
 }
}

res.json({ deleted, message: `Cleaned up ${deleted} old conversations` });
});

app.get('/install-shortcut', (_req, res) => {
 const fs = require('fs');
 const path = require('path');
 const filePath = path.join(__dirname, '..', 'Oxy.shortcut');
 if (fs.existsSync(filePath)) {
 res.setHeader('Content-Type', 'application/octet-stream');
 res.setHeader('Content-Disposition', 'attachment; filename="Oxy.shortcut"');
 res.sendFile(filePath);
 } else {
 res.status(404).json({ error: 'Shortcut file not found' });
 }
});

app.get('/', (_req, res) => {
 res.setHeader('Content-Type', 'text/html; charset=utf-8');
 res.sendFile(path.resolve(__dirname, '..', 'index.html'));
});

// Sentry error handler must be last
if (process.env.SENTRY_DSN) {
 try {
 const Sentry = require('@sentry/node');
 app.use(Sentry.expressErrorHandler());
 } catch (e) {}
}

module.exports = app;
module.exports.runProactiveSweep = runProactiveSweep;

```

## live.js (Gemini Live WebSocket) — live.js

```

const { WebSocketServer } = require('ws');
const { GoogleGenAI, Modality } = require('@google/genai');
const { dispatch } = require('./connectors');
const { createSupabaseServiceClient } = require('./runtime');
const { verifySignedPayload } = require('./auth');

const supabase = createSupabaseServiceClient();

const LIVE_MODEL = process.env.OXY_LIVE_MODEL
 || (String(process.env.GOOGLE_GENAI_USE_ENTERPRISE || '').toLowerCase() === 'true'
 ? 'gemini-live-2.5-flash-preview-native-audio'
 : 'gemini-live-2.5-flash-preview');

const LIVE_VOICE_SET = new Set([
 'Zephyr', 'Puck', 'Charon', 'Kore', 'Fenrir', 'Leda', 'Orus', 'Aoede',
 'Callirrhoe', 'Autonoe', 'Enceladus', 'Iapetus', 'Umbriel', 'Algieba',
 'Despina', 'Erinome', 'Algenib', 'Rasalgethi', 'Laomedea', 'Achernar',
 'Alnilam', 'Schedar', 'Gacrux', 'Pulcherrima', 'Achird', 'Zubenelgenubi',
 'Vindemiatrix', 'Sadachbia', 'Sadaltager', 'Sulafat'
]);

const LIVE_FUNCTION_DECLARATIONS = [
 {
 name: 'create_reminder',
 description: 'Create a reminder for the user.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 title: { type: 'string' },
 due_date: { type: 'string' }
 },
 },
 },

```

```

 required: ['title'],
 additionalProperties: false
 }
},
{
 name: 'send_email',
 description: 'Send an email on the user's behalf.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 to: { type: 'string' },
 subject: { type: 'string' },
 body: { type: 'string' },
 thread_id: { type: 'string' },
 in_reply_to: { type: 'string' },
 references: { type: 'string' }
 },
 required: ['to', 'subject', 'body'],
 additionalProperties: false
 }
},
{
 name: 'get_emails',
 description: 'Get recent emails for the user.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 max_results: { type: 'number' },
 label: { type: 'string' },
 labels: {
 type: 'array',
 items: { type: 'string' }
 }
 },
 additionalProperties: false
 }
},
{
 name: 'search_emails',
 description: 'Search the user's email inbox.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 query: { type: 'string' },
 max_results: { type: 'number' }
 },
 required: ['query'],
 additionalProperties: false
 }
},
{
 name: 'create_calendar_event',
 description: 'Create a calendar event.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 title: { type: 'string' },
 start_date: { type: 'string' },
 end_date: { type: 'string' }
 },
 required: ['title', 'start_date'],
 additionalProperties: false
 }
},
{
 name: 'get_calendar_events',
 description: 'Get upcoming calendar events.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 max_results: { type: 'number' }
 },
 additionalProperties: false
 }
},
{
 name: 'book_uber',
 description: 'Open an Uber ride flow to a destination.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 destination: { type: 'string' }
 },
 required: ['destination'],
 additionalProperties: false
 }
},

```

```

{
 name: 'send_telegram',
 description: 'Send a Telegram message.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 contact: { type: 'string' },
 message: { type: 'string' }
 },
 required: ['contact', 'message'],
 additionalProperties: false
 }
},
{
 name: 'get_telegram_contacts',
 description: 'List Telegram contacts.',
 parametersJsonSchema: {
 type: 'object',
 properties: {},
 additionalProperties: false
 }
},
{
 name: 'search_trains',
 description: 'Search live trains between two stations.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 origin: { type: 'string' },
 destination: { type: 'string' }
 },
 required: ['origin', 'destination'],
 additionalProperties: false
 }
},
{
 name: 'order_uber_eats',
 description: 'Open an Uber Eats handoff for a dish, cuisine, or restaurant.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 query: { type: 'string' },
 restaurant: { type: 'string' },
 item: { type: 'string' }
 },
 required: ['query'],
 additionalProperties: false
 }
},
{
 name: 'order_deliveroo',
 description: 'Open a Deliveroo handoff for a dish, cuisine, or restaurant.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 query: { type: 'string' },
 restaurant: { type: 'string' },
 item: { type: 'string' }
 },
 required: ['query'],
 additionalProperties: false
 }
},
{
 name: 'search_netflix_title',
 description: 'Search for a Netflix title.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 title: { type: 'string' }
 },
 required: ['title'],
 additionalProperties: false
 }
},
{
 name: 'add_to_netflix_list',
 description: 'Open the flow to add a Netflix title to the user's list.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 title: { type: 'string' }
 },
 required: ['title'],
 additionalProperties: false
 }
}

```

```

 },
 {
 name: 'forget_memory',
 description: 'Delete something from Oxy memory.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 scope: { type: 'string' },
 query: { type: 'string' }
 },
 },
 additionalProperties: false
 }
],
 {
 name: 'generate_visual',
 description: 'Generate a contextual visual for the conversation.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 brief: { type: 'string' },
 style: { type: 'string' },
 usage: { type: 'string' }
 },
 required: ['brief'],
 additionalProperties: false
 }
 },
 {
 name: 'create_diagram',
 description: 'Create a diagram to explain a topic.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 topic: { type: 'string' },
 goal: { type: 'string' }
 },
 required: ['topic'],
 additionalProperties: false
 }
 },
 {
 name: 'create_presentation',
 description: 'Create a short presentation outline with visuals.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 topic: { type: 'string' },
 audience: { type: 'string' },
 objective: { type: 'string' },
 slide_count: { type: 'number' }
 },
 required: ['topic'],
 additionalProperties: false
 }
 },
 {
 name: 'play_music',
 description: 'Play a song, artist, album, or playlist on the user\'s phone.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 query: { type: 'string', description: 'What to play – song title, artist, genre, or playlist name' }
 },
 required: ['query'],
 additionalProperties: false
 }
 },
 {
 name: 'make_call',
 description: 'Make a phone call to a contact or number.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 contact: { type: 'string', description: 'Contact name or phone number to call' }
 },
 required: ['contact'],
 additionalProperties: false
 }
 }
];

function createLiveClient() {
 const useEnterprise = String(process.env.GOOGLE_GENAI_USE_ENTERPRISE || '').toLowerCase() === 'true'
 || (!!process.env.GOOGLE_CLOUD_PROJECT && !!process.env.GOOGLE_CLOUD_LOCATION);

 if (useEnterprise) {

```

```

 return new GoogleGenAI({
 enterprise: true,
 project: process.env.GOOGLE_CLOUD_PROJECT,
 location: process.env.GOOGLE_CLOUD_LOCATION || 'us-central1',
 apiVersion: process.env.GOOGLE_GENAI_API_VERSION || 'v1beta'
 });
 }

 return new GoogleGenAI({
 apiKey: process.env.GEMINI_API_KEY || process.env.GOOGLE_API_KEY,
 apiVersion: process.env.GOOGLE_GENAI_API_VERSION || 'v1beta'
 });
}

function createSocketError(socket, error) {
 if (socket.readyState !== 1) return;
 socket.send(JSON.stringify({ type: 'live-error', error: error?.message || String(error || 'Unknown realtime error') }));
}

function sendSocketEvent(socket, payload) {
 if (socket.readyState !== 1) return;
 socket.send(JSON.stringify(payload));
}

function getInputMimeType(mimeType = '') {
 const normalized = String(mimeType || '').toLowerCase();
 const rateMatch = normalized.match(/(?:rate|sample_rate)=[0-9]+/);
 const sampleRate = rateMatch ? Number(rateMatch[1]) : 16000;
 if (normalized.includes('audio/l16')) return `audio/l16;rate=${sampleRate}`;
 if (normalized.includes('audio/pcm')) return `audio/l16;rate=${sampleRate}`;
 if (normalized.startsWith('audio/')) return normalized;
 return `audio/l16;rate=${sampleRate}`;
}

function normalizeVoiceName(voiceName) {
 return LIVE_VOICE_SET.has(voiceName) ? voiceName : 'Schedar';
}

function mergeTranscript(previous, next) {
 const prev = String(previous || '').trim();
 const incoming = String(next || '').trim();
 if (!incoming) return prev;
 if (!prev) return incoming;
 if (incoming === prev) return prev;
 if (incoming.startsWith(prev)) return incoming;
 if (prev.startsWith(incoming)) return prev;

 const maxOverlap = Math.min(prev.length, incoming.length);
 for (let overlap = maxOverlap; overlap > 0; overlap -= 1) {
 if (prev.slice(-overlap) === incoming.slice(0, overlap)) {
 return `${prev}${incoming.slice(overlap)}`;
 }
 }
 return `${prev} ${incoming}`.replace(/\s+/g, ' ').trim();
}

function extractTextFromConversationContent(content) {
 if (content == null) return '';
 if (typeof content === 'string') return content;
 if (typeof content !== 'object') return String(content);
 return typeof content.text === 'string' ? content.text : '';
}

function serializeConversationContent(payload) {
 if (payload == null) return '';
 if (typeof payload === 'string') return payload;
 if (typeof payload !== 'object') return String(payload);

 const next = {};
 if (typeof payload.text === 'string') next.text = payload.text;

 if (typeof payload.image === 'string') next.image = payload.image;
 if (Array.isArray(payload.actions) && payload.actions.length) next.actions = payload.actions;
 if (typeof payload.audio === 'string') next.audio = payload.audio;
 if (typeof payload.kind === 'string') next.kind = payload.kind;

 return Object.keys(next).length === 1 && typeof next.text === 'string'
 ? next.text
 : JSON.stringify(next);
}

async function saveMessage(userId, role, content) {
 await supabase.from('conversations').insert({
 user_id: userId,
 role,
 content: serializeConversationContent(content),
 created_at: new Date().toISOString()
 });
}

```

```

 });
 }

 async function logAction(userId, action, input, result) {
 await supabase.from('action_log').insert({
 user_id: userId,
 action,
 input,
 result,
 status: result?.success === false ? 'failed' : 'executed',
 error: result?.success === false ? result.error || result.text || 'Action failed' : null,
 created_at: new Date().toISOString()
 });
 }

 async function getUserMemory(userId) {
 try {
 const { data } = await supabase
 .from('memories')
 .select('content, source, created_at')
 .eq('user_id', userId)
 .order('created_at', { ascending: false })
 .limit(6);

 if (!Array.isArray(data) || data.length === 0) return '';
 return data
 .map(row => {
 const prefix = row.source === 'manual_profile' ? 'Profile' : 'Memory';
 return `${prefix}: ${row.content}`;
 })
 .join('\n');
 } catch {
 return '';
 }
 }

 async function getRecentConversationText(userId) {
 try {
 const { data } = await supabase
 .from('conversations')
 .select('role, content, created_at')
 .eq('user_id', userId)
 .neq('role', 'system')
 .order('created_at', { ascending: false })
 .limit(8);

 if (!Array.isArray(data) || !data.length) return '';
 return data
 .reverse()
 .map(row => `${row.role === 'user' ? 'User' : 'Oxy':} ${extractTextFromConversationContent(row.content)}`)
 .filter(Boolean)
 .join('\n');
 } catch {
 return '';
 }
 }

 function buildLiveSystemInstruction(memory, recentConversation) {
 return [
 'You are Oxy, a proactive personal assistant speaking with the user in real time.',
 'Be concise, natural, and useful. Keep spoken replies fairly short unless the user asks for depth.',
 'The user leads the conversation. Follow what they just said instead of surfacing unrelated stored memory.',
 'Treat memory as background context for understanding. Only mention it when it is directly relevant to the user\'s current question, request, or action.',
 'If the user just greets you or makes a simple check-in, respond naturally to that message and do not bring up unrelated personal context.',
 'If the user asks you to do something and a tool is available, call the tool instead of merely describing what you would do.',
 'Google Search grounding is always available in this session. For current events, recent news, companies, public figures, schedules, prices, etc.',
 'If the user asks you to forget, delete, wipe, or remove something from memory, use forget_memory.',
 'Do not repeat successful past actions unless the user explicitly asks you to repeat them.',
 'If an action fails and the user asks to retry, retry only the failed action unless they ask otherwise.',
 'If information is uncertain, say that plainly rather than guessing.',
 'Do not repeat context you already stated earlier in the current conversation unless the user asks for it again.',
 memory ? `Known user context:\n${memory}` : '',
 recentConversation ? `Recent conversation context:\n${recentConversation}` : ''
].filter(Boolean).join('\n\n');
 }

 function summarizeToolResult(result) {
 if (!result) return 'No result returned.';
 if (result.success === false) return result.error || result.text || 'Action failed.';
 if (typeof result.text === 'string' && result.text.trim()) return result.text.trim();
 if (result.artifact?.summary) return result.artifact.summary;
 return 'Action completed successfully.';
 }

 async function forgetMemory(userId, args = {}) {
 const scope = String(args.scope || '').toLowerCase();
 const query = String(args.query || '').trim();
 }

```

```

 if (scope === 'all') {
 await supabase.from('memories').delete().eq('user_id', userId);
 return { success: true, text: 'I cleared what I had in memory.' };
 }

 if (scope === 'recent') {
 const { data } = await supabase
 .from('memories')
 .select('id')
 .eq('user_id', userId)
 .order('created_at', { ascending: false })
 .limit(1);
 if (!data?.length) return { success: true, text: 'There was nothing stored to forget.' };
 await supabase.from('memories').delete().eq('id', data[0].id);
 return { success: true, text: 'I forgot the most recent memory.' };
 }

 if (query) {
 const escaped = query.replace(/[\\%_]/g, match => `\\${match}`);
 const { data } = await supabase
 .from('memories')
 .select('id')
 .eq('user_id', userId)
 .ilike('content', `%${escaped}%`);
 if (!data?.length) return { success: true, text: `I couldn't find anything stored about "${query}"` };
 await supabase.from('memories').delete().in('id', data.map(row => row.id));
 return { success: true, text: `I removed what I had stored about "${query}"` };
 }

 return { success: false, error: 'forget_memory needs scope "recent" or "all", or a query.' };
 }

 async function executeFunctionCalls(userId, functionCalls = []) {
 const functionResponses = [];
 const actionResults = [];

 for (const call of functionCalls) {
 const name = call.name;
 const input = (call.arguments && typeof call.arguments === 'object') ? call.arguments : {};
 let result;

 try {
 if (name === 'forget_memory') {
 result = await forgetMemory(userId, input);
 } else if (name === 'play_music') {
 // Native iOS action – signal the client to handle it
 const query = String(input.query || '').trim();
 result = { success: true, text: `Playing ${query} || 'music' on your phone.`, nativeSignal: 'play_music', query };
 } else if (name === 'make_call') {
 // Native iOS action – signal the client to handle it
 const contact = String(input.contact || '').trim();
 result = { success: true, text: `Calling ${contact}.`, nativeSignal: 'make_call', contact };
 } else {
 result = await dispatch(userId, name, input);
 }
 } catch (error) {
 result = { success: false, error: error?.message || `Failed to execute ${name}` };
 }

 try {
 await logAction(userId, name, input, result);
 } catch {}

 actionResults.push({ action: name, result });
 functionResponses.push({
 id: call.id,
 name,
 response: result?.success === false
 ? { error: result.error || result.text || 'Action failed.' }
 : { output: summarizeToolResult(result) }
 });
 }

 return { functionResponses, actionResults };
 }

 async function createLiveSession(userId, voiceName, socket, state) {
 const ai = createLiveClient();
 const [memory, recentConversation] = await Promise.all([
 getUserMemory(userId),
 getRecentConversationText(userId)
]);

 return ai.live.connect({
 model: LIVE_MODEL,
 config: {
 responseModalities: [Modality.AUDIO],
 }
 });
 }

```

```

speechConfig: {
 languageCode: 'en-GB',
 voiceConfig: {
 prebuiltVoiceConfig: {
 voiceName: normalizeVoiceName(voiceName)
 }
 }
},
inputAudioTranscription: {},
outputAudioTranscription: {},
systemInstruction: buildLiveSystemInstruction(memory, recentConversation),
tools: [
 { googleSearch: {} },
 { functionDeclarations: LIVE_FUNCTION_DECLARATIONS }
]
},
callbacks: {
 onopen: () => {
 sendSocketEvent(socket, { type: 'live-open' });
 },
 onmessage: async message => {
 try {
 if (message.serverContent?.inputTranscription?.text) {
 state.userTranscript = mergeTranscript(
 state.userTranscript,
 message.serverContent.inputTranscription.text
);
 sendSocketEvent(socket, {
 type: 'user-transcript',
 text: state.userTranscript,
 final: !!message.serverContent.inputTranscription.finished
 });
 }

 if (message.serverContent?.outputTranscription?.text) {
 state.assistantTranscript = mergeTranscript(
 state.assistantTranscript,
 message.serverContent.outputTranscription.text
);
 sendSocketEvent(socket, {
 type: 'assistant-transcript',
 text: state.assistantTranscript,
 final: !!message.serverContent.outputTranscription.finished
 });
 }

 const parts = message.serverContent?.modelTurn?.parts || [];
 for (const part of parts) {
 if (typeof part.text === 'string' && part.text.trim()) {
 state.assistantTranscript = mergeTranscript(state.assistantTranscript, part.text);
 sendSocketEvent(socket, {
 type: 'assistant-transcript',
 text: state.assistantTranscript,
 final: false
 });
 }
 if (part.inlineData?.data) {
 sendSocketEvent(socket, {
 type: 'assistant-audio',
 data: part.inlineData.data,
 mimeType: part.inlineData.mimeType || 'audio/pcm;rate=24000'
 });
 }
 }

 if (message.toolCall?.functionCalls?.length) {
 const { functionResponses, actionResults } = await executeFunctionCalls(userId, message.toolCall.functionCalls);
 if (actionResults.length) {
 state.actionResults.push(...actionResults);
 sendSocketEvent(socket, { type: 'assistant-actions', results: actionResults });
 }
 if (state.session) {
 try {
 await state.session.sendToolResponse({ functionResponses });
 } catch (error) {
 console.warn('[live] sendToolResponse failed:', error?.message || error);
 }
 }
 }

 if (message.serverContent?.interrupted) {
 sendSocketEvent(socket, { type: 'live-interrupted' });
 }

 if (message.serverContent?.turnComplete) {
 const userText = state.userTranscript.trim();

```



```

 const assistantText = state.assistantTranscript.trim();
 const actionResults = state.actionResults.slice();
 if (userText) {
 saveMessage(userId, 'user', userText).catch(() => {});
 }
 if (assistantText || actionResults.length) {
 saveMessage(userId, 'assistant', { text: assistantText, actions: actionResults }).catch(() => {});
 }
 sendSocketEvent(socket, { type: 'live-turn-complete' });
 state.userTranscript = '';
 state.assistantTranscript = '';
 state.actionResults = [];
 }
} catch (error) {
 createSocketError(socket, error);
}
},
onerror: error => {
 createSocketError(socket, error?.error || error);
},
onclose: () => {
 sendSocketEvent(socket, { type: 'live-closed' });
}
});
}

function writeUnauthorized(socket) {
 socket.write('HTTP/1.1 401 Unauthorized\r\nConnection: close\r\n\r\n');
 socket.destroy();
}

async function flushPendingAudio(state) {
 while (state.pendingEvents.length > 0 && state.session) {
 const event = state.pendingEvents.shift();
 if (event.type === 'audio-chunk') {
 const bytes = Buffer.from(event.data || '', 'base64');
 state.session.sendRealtimeInput({
 audio: new Blob([bytes], { type: getInputMimeType(event.mimeType) })
 });
 continue;
 }
 if (event.type === 'audio-end') {
 state.session.sendRealtimeInput({ audioStreamEnd: true });
 }
 }
}

function attachRealtimeVoiceServer(server) {
 const wss = new WebSocketServer({ noServer: true });

 server.on('upgrade', (req, socket, head) => {
 const url = new URL(req.url, `http://${req.headers.host}`);
 if (url.pathname !== '/realtime-voice') return;
 wss.handleUpgrade(req, socket, head, ws => {
 wss.emit('connection', ws, req);
 });
 });

 wss.on('connection', (socket) => {
 const state = {
 userId: null,
 authenticated: false,
 session: null,
 sessionPromise: null,
 userTranscript: '',
 assistantTranscript: '',
 actionResults: [],
 pendingEvents: []
 };
 const authTimeout = setTimeout(() => {
 if (!state.authenticated) {
 createSocketError(socket, 'Unauthorized');
 socket.close();
 }
 }, 5000);

 socket.on('message', async raw => {
 try {
 if (Buffer.byteLength(raw) > 512 * 1024) {
 createSocketError(socket, 'Realtime message too large.');
```

```

 if (event.type === 'auth') {
 const payload = verifySignedPayload(event.token || '');
 if (!payload || payload.type !== 'session' || !payload.userId) {
 createSocketError(socket, 'Unauthorized');
 socket.close();
 return;
 }
 state.userId = payload.userId;
 state.authenticated = true;
 clearTimeout(authTimeout);
 sendSocketEvent(socket, { type: 'live-authenticated' });
 return;
 }

 if (!state.authenticated) {
 createSocketError(socket, 'Unauthorized');
 socket.close();
 return;
 }

 if (event.type === 'live-start') {
 if (!state.session && !state.sessionPromise) {
 state.sessionPromise = createLiveSession(state.userId, event.voice, socket, state)
 .then(async session => {
 state.session = session;
 state.sessionPromise = null;
 await flushPendingAudio(state);
 return session;
 })
 .catch(error => {
 state.sessionPromise = null;
 createSocketError(socket, error);
 throw error;
 });
 }
 return;
 }

 if (!state.session) {
 if (state.sessionPromise && (event.type === 'audio-chunk' || event.type === 'audio-end')) {
 state.pendingEvents.push(event);
 return;
 }
 createSocketError(socket, 'Live session has not been started yet.');
```

```

 return;
 }

 if (event.type === 'audio-chunk') {
 const bytes = Buffer.from(event.data || '', 'base64');
 state.session.sendRealtimeInput({
 audio: new Blob([bytes], { type: getInputMimeType(event.mimeType) })
 });
 return;
 }

 if (event.type === 'audio-end') {
 state.session.sendRealtimeInput({ audioStreamEnd: true });
 return;
 }

 if (event.type === 'live-stop') {
 state.session.close();
 state.session = null;
 }
 } catch (error) {
 createSocketError(socket, error);
 }
});

socket.on('close', () => {
 clearTimeout(authTimeout);
 try {
 state.session?.close();
 } catch {}
 state.session = null;
 state.sessionPromise = null;
 state.pendingEvents = [];
});

return wss;
}

module.exports = {
 attachRealtimeVoiceServer
};

```

**companion-live.js — companion-live.js**

```

const { performance } = require('perf_hooks');
const { WebSocketServer } = require('ws');
const { GoogleGenAI, Modality } = require('@google/genai');
const { dispatch } = require('./connectors');
const { createSupabaseServiceClient } = require('./runtime');
const { verifySignedPayload } = require('./auth');

const supabase = createSupabaseServiceClient();

const COMPANION_LIVE_PATH = '/companion-live';
const COMPANION_LIVE_MODEL = process.env.OXY_COMPANION_LIVE_MODEL
 || process.env.OXY_LIVE_MODEL
 || (String(process.env.GOOGLE_GENAI_USE_ENTERPRISE || '').toLowerCase() === 'true'
 ? 'gemini-live-2.5-flash-preview-native-audio'
 : 'gemini-live-2.5-flash-preview');

const LIVE_VOICE_SET = new Set([
 'Zephyr', 'Puck', 'Charon', 'Kore', 'Fenrir', 'Leda', 'Orus', 'Aoede',
 'Callirrhoe', 'Autonoe', 'Enceladus', 'Iapetus', 'Umbriel', 'Algieba',
 'Despina', 'Erinome', 'Algenib', 'Rasalgethi', 'Laomedea', 'Achernar',
 'Alnilam', 'Schedar', 'Gacrux', 'Pulcherrima', 'Achird', 'Zubenelgenubi',
 'Vindemiatrix', 'Sadachbia', 'Sadaltager', 'Sulafat'
]);

const LIVE_FUNCTION_DECLARATIONS = [
 {
 name: 'create_reminder',
 description: 'Create a reminder for the user.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 title: { type: 'string' },
 due_date: { type: 'string' }
 },
 required: ['title'],
 additionalProperties: false
 }
 },
 {
 name: 'send_email',
 description: 'Send an email on the user\'s behalf.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 to: { type: 'string' },
 subject: { type: 'string' },
 body: { type: 'string' },
 thread_id: { type: 'string' },
 in_reply_to: { type: 'string' },
 references: { type: 'string' }
 },
 required: ['to', 'subject', 'body'],
 additionalProperties: false
 }
 },
 {
 name: 'get_emails',
 description: 'Get recent emails for the user.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 max_results: { type: 'number' },
 label: { type: 'string' },
 labels: {
 type: 'array',
 items: { type: 'string' }
 }
 },
 additionalProperties: false
 }
 },
 {
 name: 'search_emails',
 description: 'Search the user\'s email inbox.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 query: { type: 'string' },
 max_results: { type: 'number' }
 },
 required: ['query'],
 additionalProperties: false
 }
 }
];

```

```

 name: 'create_calendar_event',
 description: 'Create a calendar event.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 title: { type: 'string' },
 start_date: { type: 'string' },
 end_date: { type: 'string' }
 },
 required: ['title', 'start_date'],
 additionalProperties: false
 }
 },
 {
 name: 'get_calendar_events',
 description: 'Get upcoming calendar events.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 max_results: { type: 'number' }
 },
 additionalProperties: false
 }
 },
 {
 name: 'book_uber',
 description: 'Open an Uber ride flow to a destination.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 destination: { type: 'string' }
 },
 required: ['destination'],
 additionalProperties: false
 }
 },
 {
 name: 'send_telegram',
 description: 'Send a Telegram message.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 contact: { type: 'string' },
 message: { type: 'string' }
 },
 required: ['contact', 'message'],
 additionalProperties: false
 }
 },
 {
 name: 'get_telegram_contacts',
 description: 'List Telegram contacts.',
 parametersJsonSchema: {
 type: 'object',
 properties: {},
 additionalProperties: false
 }
 },
 {
 name: 'search_trains',
 description: 'Search live trains between two stations.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 origin: { type: 'string' },
 destination: { type: 'string' }
 },
 required: ['origin', 'destination'],
 additionalProperties: false
 }
 },
 {
 name: 'order_uber_eats',
 description: 'Open an Uber Eats handoff for a dish, cuisine, or restaurant.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 query: { type: 'string' },
 restaurant: { type: 'string' },
 item: { type: 'string' }
 },
 required: ['query'],
 additionalProperties: false
 }
 },
 {
 {

```

```

 name: 'order_deliveroo',
 description: 'Open a Deliveroo handoff for a dish, cuisine, or restaurant.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 query: { type: 'string' },
 restaurant: { type: 'string' },
 item: { type: 'string' }
 },
 required: ['query'],
 additionalProperties: false
 }
 },
 {
 name: 'search_netflix_title',
 description: 'Search for a Netflix title.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 title: { type: 'string' }
 },
 required: ['title'],
 additionalProperties: false
 }
 },
 {
 name: 'add_to_netflix_list',
 description: 'Open the flow to add a Netflix title to the user\'s list.',
 parametersJsonSchema: {
 type: 'object',
 properties: {
 title: { type: 'string' }
 },
 required: ['title'],
 additionalProperties: false
 }
 }
];

function createLiveClient() {
 const useEnterprise = String(process.env.GOOGLE_GENAI_USE_ENTERPRISE || '').toLowerCase() === 'true'
 || (!!process.env.GOOGLE_CLOUD_PROJECT && !!process.env.GOOGLE_CLOUD_LOCATION);

 if (useEnterprise) {
 return new GoogleGenAI({
 enterprise: true,
 project: process.env.GOOGLE_CLOUD_PROJECT,
 location: process.env.GOOGLE_CLOUD_LOCATION || 'us-central1',
 apiVersion: process.env.GOOGLE_GENAI_API_VERSION || 'v1beta'
 });
 }

 return new GoogleGenAI({
 apiKey: process.env.GEMINI_API_KEY || process.env.GOOGLE_API_KEY,
 apiVersion: process.env.GOOGLE_GENAI_API_VERSION || 'v1beta'
 });
}

function createPrototypeTrace(label, socket) {
 const startedAt = performance.now();
 const prefix = `[trace:${label}]`;
 const sentMilestones = new Set();

 return {
 elapsed() {
 return Math.round(performance.now() - startedAt);
 },
 log(step, extra = '') {
 const suffix = extra ? ` ${extra}` : '';
 console.log(`${prefix} +${this.elapsed()}ms ${step}${suffix}`);
 },
 telemetry(stage, extra = {}) {
 if (socket.readyState !== 1) return;
 socket.send(JSON.stringify({
 type: 'telemetry',
 stage,
 elapsedMs: this.elapsed(),
 ...extra
 }));
 },
 milestone(stage, extra = {}) {
 if (sentMilestones.has(stage)) return;
 sentMilestones.add(stage);
 this.telemetry(stage, extra);
 }
 };
};

```

```

}

function sendSocketEvent(socket, payload) {
 if (socket.readyState !== 1) return;
 socket.send(JSON.stringify(payload));
}

function sendStatus(socket, status, label, extra = {}) {
 sendSocketEvent(socket, { type: 'status', status, label, ...extra });
}

function createSocketError(socket, error) {
 sendSocketEvent(socket, {
 type: 'error',
 error: error?.message || String(error || 'Unknown companion live error')
 });
}

function writeUnauthorized(socket) {
 socket.write('HTTP/1.1 401 Unauthorized\r\nConnection: close\r\n\r\n');
 socket.destroy();
}

function getInputMimeType(mimeType = '') {
 const normalized = String(mimeType || '').toLowerCase();
 const rateMatch = normalized.match(/(?:rate|sample_rate)=[0-9]+/);
 const sampleRate = rateMatch ? Number(rateMatch[1]) : 16000;
 if (normalized.includes('audio/l16')) return `audio/l16;rate=${sampleRate}`;
 if (normalized.includes('audio/pcm')) return `audio/l16;rate=${sampleRate}`;
 if (normalized.startsWith('audio/')) return normalized;
 return `audio/l16;rate=${sampleRate}`;
}

function normalizeVoiceName(voiceName) {
 return LIVE_VOICE_SET.has(voiceName) ? voiceName : 'Aoede';
}

function mergeTranscript(previous, next) {
 const prev = String(previous || '').trim();
 const incoming = String(next || '').trim();
 if (!incoming) return prev;
 if (!prev) return incoming;
 if (incoming === prev) return prev;
 if (incoming.startsWith(prev)) return incoming;
 if (prev.startsWith(incoming)) return prev;

 const maxOverlap = Math.min(prev.length, incoming.length);
 for (let overlap = maxOverlap; overlap > 0; overlap -= 1) {
 if (prev.slice(-overlap) === incoming.slice(0, overlap)) {
 return `${prev}${incoming.slice(overlap)}`;
 }
 }
 return `${prev} ${incoming}`.replace(/\s+/g, ' ').trim();
}

function extractTextFromConversationContent(content) {
 if (content == null) return '';
 if (typeof content === 'string') return content;
 if (typeof content !== 'object') return String(content);
 return typeof content.text === 'string' ? content.text : '';
}

function serializeConversationContent(payload) {
 if (payload == null) return '';
 if (typeof payload === 'string') return payload;
 if (typeof payload !== 'object') return String(payload);

 const next = {};
 if (typeof payload.text === 'string') next.text = payload.text;
 if (Array.isArray(payload.actions) && payload.actions.length) next.actions = payload.actions;
 return Object.keys(next).length === 1 && typeof next.text === 'string'
 ? next.text
 : JSON.stringify(next);
}

async function saveMessage(userId, role, content) {
 await supabase.from('conversations').insert({
 user_id: userId,
 role,
 content: serializeConversationContent(content),
 created_at: new Date().toISOString()
 });
}

async function logAction(userId, action, input, result) {
 await supabase.from('action_log').insert({
 user_id: userId,

```

```

 action,
 input,
 result,
 status: result?.success === false ? 'failed' : 'executed',
 error: result?.success === false ? result.error || result.text || 'Action failed' : null,
 created_at: new Date().toISOString()
 });
}

async function getUserContextSnapshot(userId) {
 const [memoriesRes, conversationRes, preferenceRes, connectorRes, actionRes] = await Promise.all([
 supabase
 .from('memories')
 .select('content, source, created_at')
 .eq('user_id', userId)
 .order('created_at', { ascending: false })

 .limit(6),
 supabase
 .from('conversations')
 .select('role, content, created_at')
 .eq('user_id', userId)
 .neq('role', 'system')
 .order('created_at', { ascending: false })
 .limit(8),
 supabase
 .from('preferences')
 .select('key, value, updated_at')
 .eq('user_id', userId)
 .order('updated_at', { ascending: false })
 .limit(8),
 supabase
 .from('connectors')
 .select('connector_id, enabled')
 .eq('user_id', userId)
 .eq('enabled', true)
 .order('connector_id', { ascending: true }),
 supabase
 .from('action_log')
 .select('action, status, error, created_at')
 .eq('user_id', userId)
 .order('created_at', { ascending: false })
 .limit(5)
]);

 const memory = Array.isArray(memoriesRes.data) && memoriesRes.data.length
 ? memoriesRes.data
 .map(row => ` ${row.source} === 'manual_profile' ? 'Profile' : 'Memory': ${row.content}`)
 .join('\n')
 : '';

 const recentConversation = Array.isArray(conversationRes.data) && conversationRes.data.length
 ? conversationRes.data
 .reverse()
 .map(row => ` ${row.role} === 'user' ? 'User' : 'Oxy': ${extractTextFromConversationContent(row.content)}`)
 .filter(Boolean)
 .join('\n')
 : '';

 const preferences = Array.isArray(preferenceRes.data) && preferenceRes.data.length
 ? preferenceRes.data.map(row => ` ${row.key}: ${row.value}`).join('\n')
 : '';

 const enabledConnectors = Array.isArray(connectorRes.data) && connectorRes.data.length
 ? connectorRes.data.map(row => row.connector_id).join(', ')
 : '';

 const recentActions = Array.isArray(actionRes.data) && actionRes.data.length
 ? actionRes.data
 .map(row => ` ${row.action}: ${row.status}${row.error ? ` (${row.error})` : ''}`)
 .join('\n')
 : '';

 return {
 memory,
 recentConversation,
 preferences,
 enabledConnectors,
 recentActions
 };
}

function buildCompanionSystemInstruction(snapshot) {
 return [
 'You are Oxy speaking to the user through a companion app in a real-time Gemini Live session.',
 'The session is audio-first: listen, think, act, and respond naturally with short spoken answers.',
 'Gemini Live handles the transcription, reasoning, and speech. Do not narrate internal steps.',
];
}

```

```

 'The user leads the conversation. Follow what they just said instead of surfacing unrelated stored memory.',
 'Treat memory as background context for understanding. Only mention it when it is directly relevant to the user\'s current question, request',
 'If the user just greets you or makes a simple check-in, respond naturally to that message and do not bring up unrelated personal context.',
 'If the user asks you to perform an action and a function is available, call the function instead of only describing what you would do.',
 'Google Search grounding is always available in this session. For current events, recent news, companies, public figures, schedules, prices,',
 'After a successful action such as sending a message, booking something, or creating an event, stop after one clear confirmation sentence.',
 'Do not repeat context you already stated earlier in the same conversation unless the user asks for it again.',
 'Prefer concise, warm spoken replies. For simple conversational questions, keep the response to a maximum of two sentences.',
 snapshot.enabledConnectors ? `Enabled connectors: ${snapshot.enabledConnectors}` : '',
 snapshot.preferences ? `User preferences:\n${snapshot.preferences}` : '',
 snapshot.memory ? `Known user context:\n${snapshot.memory}` : '',
 snapshot.recentActions ? `Recent action outcomes:\n${snapshot.recentActions}` : '',
 snapshot.recentConversation ? `Recent conversation:\n${snapshot.recentConversation}` : ''
].filter(Boolean).join('\n\n');
}

function summarizeToolResult(result) {
 if (!result) return 'No result returned.';
 if (result.success === false) return result.error || result.text || 'Action failed.';
 if (typeof result.text === 'string' && result.text.trim()) return result.text.trim();
 if (result.artifact?.summary) return result.artifact.summary;
 return 'Action completed successfully.';
}

async function executeFunctionCalls(userId, functionCalls, socket, trace) {
 const functionResponses = [];
 const actionResults = [];

 for (const call of functionCalls) {
 const name = call.name;
 const input = (call.arguments && typeof call.arguments === 'object') ? call.arguments : {};
 let result;

 sendStatus(socket, 'action_start', humanizeActionLabel(name, 'start'), { action: name });
 trace.log(`tool.start ${name}`);
 trace.telemetry('tool_start', { action: name });

 try {
 const allowedNames = new Set(LIVE_FUNCTION_DECLARATIONS.map(f => f.name));
 result = allowedNames.has(name)
 ? await dispatch(userId, name, input)
 : { success: false, error: `Function "${name}" is not available in this session.` };
 } catch (error) {
 result = { success: false, error: error?.message || `Failed to execute ${name}.` };
 }

 try {
 await logAction(userId, name, input, result);
 } catch (error) {
 trace.log(`action_log.fail ${name}`, error?.message || error);

 console.warn(`[companion-live] action_log failed:`, error?.message || error);
 }

 sendStatus(socket, 'action_complete', humanizeActionLabel(name, 'complete'), {
 action: name,
 success: result?.success !== false
 });
 trace.log(`tool.complete ${name}`, result?.success === false ? 'failed' : 'ok');
 trace.telemetry('tool_complete', { action: name, success: result?.success !== false });

 actionResults.push({ action: name, result });
 functionResponses.push({
 id: call.id,
 name,
 response: result?.success === false
 ? { error: result.error || result.text || 'Action failed.' }
 : { output: summarizeToolResult(result) }
 });
 }

 return { functionResponses, actionResults };
}

function humanizeActionLabel(action, phase) {
 const map = {
 send_email: 'email',
 get_emails: 'inbox',
 search_emails: 'email search',
 create_calendar_event: 'calendar event',
 get_calendar_events: 'calendar',
 create_reminder: 'reminder',
 book_uber: 'Uber',
 send_telegram: 'Telegram message',
 get_telegram_contacts: 'Telegram contacts',
 search_trains: 'train search',
 order_uber_eats: 'Uber Eats',
 };

```



```

 order_deliveroo: 'Deliveroo',
 search_netflix_title: 'Netflix search',
 add_to_netflix_list: 'Netflix list'
 };
 const label = map[action] || action.replace(/_/g, ' ');
 return phase === 'start'
 ? `Working on ${label}`
 : `${label.charAt(0).toUpperCase()}${label.slice(1)} ready`;
}

async function createCompanionLiveSession(userId, voiceName, socket, state) {
 const ai = createLiveClient();
 const snapshot = await getUserContextSnapshot(userId);

 state.trace.log('context.loaded');
 state.trace.telemetry('context_loaded', {
 memoryChars: snapshot.memory.length,
 recentConversationChars: snapshot.recentConversation.length
 });

 return ai.live.connect({
 model: COMPANION_LIVE_MODEL,
 config: {
 responseModalities: [Modality.AUDIO],
 speechConfig: {
 languageCode: 'en-GB',
 voiceConfig: {
 prebuiltVoiceConfig: {
 voiceName: normalizeVoiceName(voiceName)
 }
 }
 },
 inputAudioTranscription: {},
 outputAudioTranscription: {},
 systemInstruction: buildCompanionSystemInstruction(snapshot),
 tools: [
 { googleSearch: {} },
 { functionDeclarations: LIVE_FUNCTION_DECLARATIONS }
]
 },
 callbacks: {
 onopen: () => {
 state.trace.log('gemini.live.open');
 state.trace.milestone('live_open');
 sendSocketEvent(socket, {
 type: 'session.ready',
 model: COMPANION_LIVE_MODEL,
 voice: normalizeVoiceName(voiceName)
 });
 sendStatus(socket, 'thinking_start', 'Listening!');
 },
 onmessage: async (message) => {
 try {
 if (message.serverContent?.inputTranscription?.text) {
 state.userTranscript = mergeTranscript(
 state.userTranscript,
 message.serverContent.inputTranscription.text
);
 state.trace.milestone('first_user_transcript');
 sendSocketEvent(socket, {
 type: 'transcript.user',
 text: state.userTranscript,
 final: !!message.serverContent.inputTranscription.finished
 });
 }

 if (message.serverContent?.outputTranscription?.text) {
 state.assistantTranscript = mergeTranscript(
 state.assistantTranscript,
 message.serverContent.outputTranscription.text
);
 state.trace.milestone('first_assistant_transcript');
 sendSocketEvent(socket, {
 type: 'transcript.assistant',
 text: state.assistantTranscript,
 final: !!message.serverContent.outputTranscription.finished
 });
 }

 const parts = message.serverContent?.modelTurn?.parts || [];
 for (const part of parts) {
 if (typeof part.text === 'string' && part.text.trim()) {
 state.assistantTranscript = mergeTranscript(state.assistantTranscript, part.text);
 state.trace.milestone('first_assistant_transcript');

 sendSocketEvent(socket, {
 type: 'transcript.assistant',

```

```

 text: state.assistantTranscript,
 final: false
 });
}
if (part.inlineData?.data) {
 if (!state.hasStartedSpeaking) {
 state.hasStartedSpeaking = true;
 sendStatus(socket, 'speaking_start', 'Speaking');
 state.trace.milestone('first_assistant_audio');
 }
 sendSocketEvent(socket, {
 type: 'audio',
 data: part.inlineData.data,
 mimeType: part.inlineData.mimeType || 'audio/pcm;rate=24000'
 });
}
}

if (message.toolCall?.functionCalls?.length) {
 const { functionResponses, actionResults } = await executeFunctionCalls(userId, message.toolCall.functionCalls, socket, state.trace);
 if (actionResults.length) {
 state.actionResults.push(...actionResults);
 sendSocketEvent(socket, { type: 'actions', results: actionResults });
 }
 if (state.session) {
 try {
 await state.session.sendToolResponse({ functionResponses });
 } catch (error) {
 state.trace.log('tool_response.fail', error?.message || error);
 console.warn('[companion-live] sendToolResponse failed:', error?.message || error);
 }
 } else {
 state.trace.log('tool_response.skip session_closed');
 }
}

if (message.serverContent?.interrupted) {
 sendSocketEvent(socket, { type: 'interrupted' });
}

if (message.serverContent?.turnComplete) {
 const userText = state.userTranscript.trim();
 const assistantText = state.assistantTranscript.trim();
 const actionResults = state.actionResults.slice();
 if (userText) saveMessage(userId, 'user', userText).catch(() => {});
 if (assistantText || actionResults.length) {
 saveMessage(userId, 'assistant', { text: assistantText, actions: actionResults }).catch(() => {});
 }
 state.trace.milestone('turn_complete');
 sendSocketEvent(socket, { type: 'turn.complete' });
 state.userTranscript = '';
 state.assistantTranscript = '';
 state.actionResults = [];
 state.hasStartedSpeaking = false;
}
} catch (error) {
 createSocketError(socket, error);
}
},
onerror: error => {
 state.trace.log('gemini.live.error', error?.error?.message || error?.message || String(error));
 createSocketError(socket, error?.error || error);
},
onclose: () => {
 state.trace.log('gemini.live.closed');
 sendSocketEvent(socket, { type: 'session.closed' });
}
}
});
}

async function flushPendingAudio(state) {
 while (state.pendingEvents.length > 0 && state.session) {
 const event = state.pendingEvents.shift();
 if (event.type === 'audio.append') {
 const bytes = Buffer.from(event.data || '', 'base64');
 state.session.sendRealtimeInput({
 audio: new Blob([bytes], { type: getInputMimeType(event.mimeType) })
 });
 continue;
 }
 if (event.type === 'audio.end') {
 state.session.sendRealtimeInput({ audioStreamEnd: true });
 }
 }
}
}

```

```

function attachCompanionLivePrototypeServer(server) {
 const wss = new WebSocketServer({ noServer: true });

 server.on('upgrade', (req, socket, head) => {
 const url = new URL(req.url, `http://${req.headers.host || 'localhost'}`);
 if (url.pathname !== COMPANION_LIVE_PATH) return;
 wss.handleUpgrade(req, socket, head, ws => {
 wss.emit('connection', ws, req);
 });
 });

 wss.on('connection', socket => {
 const state = {
 userId: null,
 authenticated: false,
 trace: createPrototypeTrace(`companion-live:${Date.now()}`, socket),
 session: null,
 sessionPromise: null,
 userTranscript: '',
 assistantTranscript: '',
 actionResults: [],
 pendingEvents: [],
 hasStartedSpeaking: false
 };

 state.trace.log('socket.connected');
 sendSocketEvent(socket, { type: 'session.connecting', path: COMPANION_LIVE_PATH });

 const authTimeout = setTimeout(() => {
 if (!state.authenticated) {
 createSocketError(socket, 'Unauthorized');

 socket.close();
 }
 }, 5000);

 socket.on('message', async raw => {
 try {
 if (Buffer.byteLength(raw) > 512 * 1024) {
 createSocketError(socket, 'Companion live message too large.');
```

```

 if (!state.session) {
 if (state.sessionPromise &&& (event.type === 'audio.append' || event.type === 'audio.end')) {
 if (state.pendingEvents.length >= 50) {
 createSocketError(socket, 'Companion live: too many pending audio events.');
```

socket.close();

return;

}

state.pendingEvents.push(event);

return;

}

createSocketError(socket, 'Companion live session has not been started yet.');

return;

}

if (event.type === 'audio.append') {

const bytes = Buffer.from(event.data || '', 'base64');

state.session.sendRealtimeInput({

  audio: new Blob([bytes], { type: getInputMimeType(event.mimeType) })

});

return;

}

if (event.type === 'audio.end') {

  state.trace.log('audio.end');

  state.trace.milestone('audio\_end');

  state.session.sendRealtimeInput({ audioStreamEnd: true });

  return;

}

if (event.type === 'session.stop') {

  if (state.session) {

    state.session.close();

    state.session = null;

  } else if (state.sessionPromise) {

    state.sessionPromise.then(session => { try { session?.close(); } catch {} }).catch(() => {});

    state.sessionPromise = null;

  }

}

} catch (error) {

  createSocketError(socket, error);

}

});

socket.on('close', () => {

  clearTimeout(authTimeout);

  try {

    state.session?.close();

  } catch {}

  state.trace.log('socket.closed');

  state.pendingEvents = [];

  state.session = null;

  state.sessionPromise = null;

});

});

return wss;

}

module.exports = {

  attachCompanionLivePrototypeServer

};

## connectors/index.js — connectors/index.js

```

const google = require('./google');
const uber = require('./uber');
const ubereats = require('./ubereats');
const deliveroo = require('./deliveroo');
const netflix = require('./netflix');
const telegram = require('./telegram');
const trainline = require('./trainline');
const maps = require('./maps');
```

// Registry: action name → connector module

// To add a new connector: create connectors/myservice.js and register its actions here

const registry = {};

for (const action of google.SUPPORTED\_ACTIONS) registry[action] = google;

for (const action of uber.SUPPORTED\_ACTIONS) registry[action] = uber;

for (const action of ubereats.SUPPORTED\_ACTIONS) registry[action] = ubereats;

for (const action of deliveroo.SUPPORTED\_ACTIONS) registry[action] = deliveroo;

for (const action of netflix.SUPPORTED\_ACTIONS) registry[action] = netflix;

for (const action of telegram.SUPPORTED\_ACTIONS) registry[action] = telegram;

for (const action of trainline.SUPPORTED\_ACTIONS) registry[action] = trainline;

for (const action of maps.SUPPORTED\_ACTIONS) registry[action] = maps;

// Set of connector IDs that have a live implementation

```
const IMPLEMENTED_CONNECTORS = new Set(['google', 'uber', 'ubereats', 'deliveroo', 'netflix', 'telegram', 'trainline', 'maps']);

async function dispatch(userId, action, params) {
 const connector = registry[action];
 if (connector) return connector.execute(userId, action, params);
 return { success: false, error: `No connector registered for action: ${action}` };
}

module.exports = { dispatch, registry, IMPLEMENTED_CONNECTORS };
```

## auth.js — auth.js

```
const crypto = require('crypto');

const SESSION_TTL_MS = 7 * 24 * 60 * 60 * 1000;

function getSessionSecret() {
 return process.env.OXY_SESSION_SECRET || process.env.SESSION_SECRET || process.env.OXY_API_KEY || '';
}

function encodeBase64Url(value) {
 return Buffer.from(value, 'utf8').toString('base64url');
}

function decodeBase64Url(value) {
 return Buffer.from(value, 'base64url').toString('utf8');
}

function signPayload(payload, expiresInMs = SESSION_TTL_MS) {
 const secret = getSessionSecret();
 if (!secret) throw new Error('Server auth is not configured.');
```

```
 const enrichedPayload = {
 ...payload,
 exp: Date.now() + expiresInMs
 };
 const encodedPayload = encodeBase64Url(JSON.stringify(enrichedPayload));
 const sig = crypto.createHmac('sha256', secret).update(encodedPayload).digest('base64url');
 return `${encodedPayload}.${sig}`;
}

function verifySignedPayload(token) {
 if (!token || typeof token !== 'string') return null;
 const parts = token.split('.');
 if (parts.length !== 2) return null;
 const [encodedPayload, sig] = parts;
 const secret = getSessionSecret();
 if (!secret) return null;

 const expectedSig = crypto.createHmac('sha256', secret).update(encodedPayload).digest('base64url');
 const sigBuf = Buffer.from(sig);
 const expectedBuf = Buffer.from(expectedSig);
 if (sigBuf.length !== expectedBuf.length || !crypto.timingSafeEqual(sigBuf, expectedBuf)) {
 return null;
 }

 try {
 const payload = JSON.parse(decodeBase64Url(encodedPayload));
 if (!payload.exp || Date.now() > payload.exp) return null;
 return payload;
 } catch {
 return null;
 }
}

function createSessionToken(userId, tokenVersion) {
 const payload = { type: 'session', userId };
 if (tokenVersion !== undefined && tokenVersion !== null) {
 payload.tokenVersion = tokenVersion;
 }
 return signPayload(payload);
}

function getProvidedSessionToken(req) {
 const authHeader = req.get('Authorization');
 if (authHeader && authHeader.startsWith('Bearer ')) {
 return authHeader.slice('Bearer '.length).trim();
 }

 const headerToken = req.get('X-Session-Token');
 if (headerToken) return headerToken;

 return '';
}

function requireSessionAuth(req, res, next) {
 if (!getSessionSecret()) {
```

```

 return res.status(500).json({ error: 'Server auth is not configured.' });
 }

 const payload = verifySignedPayload(getProvidedSessionToken(req));
 if (!payload || payload.type !== 'session' || !payload.userId) {
 return res.status(401).json({ error: 'Unauthorized' });
 }

 req.auth = { userId: payload.userId, exp: payload.exp, tokenVersion: payload.tokenVersion };
 next();
}

function getAuthenticatedUserId(req) {
 return req.auth?.userId || null;
}

function hashPassword(password, salt = crypto.randomBytes(16).toString('hex')) {
 const derived = crypto.scryptSync(password, salt, 64).toString('hex');
 return `${salt}:${derived}`;
}

function verifyPassword(password, storedHash) {
 if (!storedHash || typeof storedHash !== 'string' || !storedHash.includes(':')) return false;
 const [salt, expectedHash] = storedHash.split(':', 2);
 const actualHash = crypto.scryptSync(password, salt, 64).toString('hex');
 const expectedBuf = Buffer.from(expectedHash, 'hex');
 const actualBuf = Buffer.from(actualHash, 'hex');
 return expectedBuf.length === actualBuf.length && crypto.timingSafeEqual(expectedBuf, actualBuf);
}

module.exports = {
 createSessionToken,
 getAuthenticatedUserId,
 getProvidedSessionToken,
 getSessionSecret,
 hashPassword,
 requireSessionAuth,
 signPayload,
 verifyPassword,
 verifySignedPayload
};

```

## runtime.js — runtime.js

```

const { createClient } = require('@supabase/supabase-js');
const { GoogleGenerativeAI } = require('@google/generative-ai');

const REQUIRED_RUNTIME_ENV = [
 'SUPABASE_URL',
 'SUPABASE_KEY',
 'GEMINI_API_KEY',
 'OXY_SESSION_SECRET'
];

let hasLoggedMissingRuntimeEnv = false;

function getMissingRuntimeEnv() {
 return REQUIRED_RUNTIME_ENV.filter(name => !process.env[name]);
}

function logMissingRuntimeEnvOnce(context = 'runtime') {
 const missing = getMissingRuntimeEnv();
 if (!missing.length || hasLoggedMissingRuntimeEnv) return missing;
 hasLoggedMissingRuntimeEnv = true;
 console.error(`[boot] Missing required env vars for ${context}: ${missing.join(', ')}`);
 return missing;
}

function createSupabaseServiceClient() {
 const missing = !process.env.SUPABASE_URL || !process.env.SUPABASE_KEY;
 if (missing) {
 logMissingRuntimeEnvOnce('supabase');
 throw new Error('SUPABASE_URL and SUPABASE_KEY must be configured before startup.');
```

```
createSupabaseServiceClient,
getMissingRuntimeEnv,
logMissingRuntimeEnvOnce
};
```

## package.json — package.json

```
{
 "name": "oxy",
 "version": "1.0.0",
 "description": "Oxy — AI-powered personal assistant",
 "main": "server.js",
 "scripts": {
 "start": "node server.js",
 "dev": "nodemon server.js",
 "test": "node --test",
 "smoke": "node --test test/smoke/*.test.js",
 "brain:evals": "node --test test/smoke/brain-evals.test.js",
 "release:check": "node --check api/index.js &&& node --check api/services/context-brain.js &&& node --test test/smoke/*.test.js",
 "proactive:job": "node proactive-job.js",
 "benchmark:voice": "node scripts/benchmark-voice-pipelines.js"
 },
 "dependencies": {
 "@google/genai": "^2.2.0",
 "@google/generative-ai": "^0.12.0",
 "@sentry/node": "^8.55.2",
 "@supabase/supabase-js": "^2.49.0",
 "archiver": "^7.0.1",
 "axios": "^1.9.0",
 "cors": "^2.8.5",
 "dotenv": "^16.5.0",
 "express": "^5.1.0",
 "multer": "^1.4.5-lts.2",
 "telegram": "^2.26.22",
 "ws": "^8.20.1"
 },
 "devDependencies": {
 "nodemon": "^3.1.10"
 }
}
```

# iOS App — Entry & Models

## OxyApp.swift — OxyApp/OxyApp/OxyApp.swift

```
import SwiftUI

@main
struct OxyApp: App {
 @UIApplicationDelegateAdaptor(AppDelegate.self) private var appDelegate
 @State private var appState = AppState()
 @AppStorage("oxy_appTheme") private var appTheme = "dark"

 var body: some Scene {
 WindowGroup {
 RootView()
 .environment(appState)
 .preferredColorScheme(preferredColorScheme)
 .tint(Color.oxyStone)
 }
 }

 private var preferredColorScheme: ColorScheme? {
 switch appTheme {
 case "light":
 return .light
 case "system":
 return nil
 default:
 return .dark
 }
 }
}

struct RootView: View {
 @Environment(AppState.self) private var appState
 @AppStorage("oxy_hasCompletedOnboarding") private var hasCompletedOnboarding = false

 var body: some View {
 Group {
 if appState.isAuthenticated {
 MainTabView()
 } else if hasCompletedOnboarding {
 LoginView()
 } else {
 OnboardingView {
 hasCompletedOnboarding = true
 }
 }
 }
 .animation(.easeInOut(duration: 0.35), value: appState.isAuthenticated)
 .animation(.easeInOut(duration: 0.35), value: hasCompletedOnboarding)
 .task {
 appState.restoreSession()
 }
 }
}

private struct OnboardingView: View {
 let onFinish: () -> Void
 @State private var pageIndex = 0
 @State private var motionIsLive = false
 @State private var controlsVisible = false
 @Environment(\.accessibilityReduceMotion) private var reduceMotion

 private let pages = [
 OxyOnboardingPage(
 imageName: "oxy_onboarding_hero",
 eyebrow: "OXY",
 title: "A personal AI you can wear.",
 subtitle: "Voice, memory, and action, close enough to be useful.",
 primaryAction: "Start",
 motion: .hero,
 imageAnchor: .center,
 baseScale: 1.06,
 drift: CGSize(width: -10, height: -18),
 glowPosition: UnitPoint(x: 0.48, y: 0.62)
),
 OxyOnboardingPage(
 imageName: "oxy_onboarding_glow",
 eyebrow: "Always With You",
 title: "The brain stays near.",
 subtitle: "Ask for what you need. Oxy keeps the thread, handles the follow-up, and moves with your day.",
 primaryAction: "Next",
 motion: .glow,
 imageAnchor: .center,

```



```

 baseScale: 1.08,
 drift: CGSize(width: 12, height: -14),
 glowPosition: UnitPoint(x: 0.48, y: 0.55)
),
 OxyOnboardingPage(
 imageName: "oxy_onboarding_anatomy",
 eyebrow: "Context",
 title: "Built to understand the back and forth.",
 subtitle: "It remembers what you meant, not just the exact words you typed.",
 primaryAction: "Next",
 motion: .signal,
 imageAnchor: .trailing,
 baseScale: 1.03,
 drift: CGSize(width: -14, height: 10),
 glowPosition: UnitPoint(x: 0.63, y: 0.23)
),
 OxyOnboardingPage(
 imageName: "oxy_onboarding_daily",
 eyebrow: "Connected",
 title: "Music, places, rides, messages, health, plans.",
 subtitle: "You choose the access. Oxy earns the trust screen by screen.",
 primaryAction: "Next",
 motion: .connected,
 imageAnchor: .center,
 baseScale: 1.04,
 drift: CGSize(width: 8, height: -12),
 glowPosition: UnitPoint(x: 0.52, y: 0.60)
),
 OxyOnboardingPage(
 imageName: "oxy_onboarding_sun",
 eyebrow: "Ready",
 title: "Use the app now. Pair hardware later.",
 subtitle: "When the device is ready, Oxy will become the thing you talk to without reaching for your phone.",
 primaryAction: "Continue",
 motion: .sun,
 imageAnchor: .center,
 baseScale: 1.05,
 drift: CGSize(width: -8, height: -16),
 glowPosition: UnitPoint(x: 0.47, y: 0.42)
)
]

var body: some View {
 GeometryReader { geometry in
 ZStack {
 Color.black.ignoresSafeArea()

 TabView(selection: $pageIndex) {
 ForEach(pages.indices, id: \.self) { index in
 OnboardingPageView(
 page: pages[index],
 size: geometry.size,
 isSelected: pageIndex == index,
 motionIsLive: motionIsLive,
 reduceMotion: reduceMotion
)
 }
 .tag(index)
 }
 }
 .tabViewStyle(.page(indexDisplayMode: .never))
 .ignoresSafeArea()

 VStack {
 HStack {
 Text(pageIndex == 0 ? "Oxy" : pages[pageIndex].eyebrow)
 .font(.system(size: pageIndex == 0 ? 34 : 13, weight: pageIndex == 0 ? .medium : .semibold))
 .tracking(pageIndex == 0 ? 0 : 2.4)
 .foregroundColor(.white)

 Spacer()

 Button("Skip") {
 onFinish()
 }
 .font(.system(size: 15, weight: .semibold))
 .foregroundColor(.white.opacity(0.82))
 }
 .padding(.horizontal, 32)
 .padding(.top, 68)

 Spacer()

 VStack(spacing: 22) {
 PageDots(count: pages.count, currentIndex: pageIndex)

 VStack(spacing: 12) {

```

```

 Button {
 advance()
 } label: {
 Text(pages[pageIndex].primaryAction)
 .font(.system(size: 18, weight: .semibold))
 .foregroundColor(Color.black)
 .frame(maxWidth: .infinity)
 .frame(height: 62)
 .background(Color.white)
 .clipShape(Capsule())
 }
 .buttonStyle(.plain)

 Button {
 if pageIndex == pages.count - 1 {
 onFinish()
 } else {
 withAnimation(.easeInOut(duration: 0.35)) {
 pageIndex = pages.count - 1
 }
 }
 } label: {
 Text(pageIndex == pages.count - 1 ? "I do not have hardware yet" : "Explore")
 .font(.system(size: 16, weight: .semibold))
 .foregroundColor(.white)
 .frame(maxWidth: .infinity)
 .frame(height: 58)
 .overlay(
 Capsule()
 .stroke(.white.opacity(0.72), lineWidth: 1.2)
)
 }
 .buttonStyle(.plain)
 }
 .opacity(controlsVisible ? 1 : 0)
 .offset(y: controlsVisible ? 0 : 18)
 .animation(.spring(response: 0.4, dampingFraction: 0.8).delay(0.30), value: controlsVisible)
}
.padding(.horizontal, 26)
.padding(.bottom, 42)
}
.ignoresSafeArea(edges: .top)
}
}
.onAppear {
 motionIsLive = true
 controlsVisible = true
}
.onChange(of: pageIndex) { _, _ in
 controlsVisible = false
 DispatchQueue.main.asyncAfter(deadline: .now() + 0.08) {
 controlsVisible = true
 }
}
}
}

private func advance() {
 if pageIndex >= pages.count - 1 {
 onFinish()
 return
 }
 withAnimation(.easeInOut(duration: 0.35)) {
 pageIndex += 1
 }
}

}

private struct OnboardingPageView: View {
 let page: OxyOnboardingPage
 let size: CGSize
 let isSelected: Bool
 let motionIsLive: Bool
 let reduceMotion: Bool

 @State private var copyVisible = false

 var body: some View {
 TimelineView(.animation(minimumInterval: 1.0 / 30.0, paused: reduceMotion || !motionIsLive || !isSelected)) { context in
 let phase = CGFloat(context.date.timeIntervalSinceReferenceDate)
 let pulse = reduceMotion || !isSelected ? 0 : (sin(phase * page.motion.speed) + 1) / 2
 let drift = reduceMotion || !isSelected ? 0 : sin(phase * 0.34)

 ZStack(alignment: .bottomLeading) {
 Image(page.imageName)
 .resizable()
 .scaledToFill()
 .frame(width: size.width, height: size.height)
 }
 }
 }
}

```

```

 .scaleEffect(page.baseScale + pulse * page.motion.zoomRange, anchor: page.imageAnchor)
 .offset(
 x: page.drift.width * drift,
 y: page.drift.height * drift
)
 .frame(width: size.width, height: size.height)
 .clipped()
 .overlay(
 LinearGradient(
 stops: [
 .init(color: .black.opacity(0.18), location: 0.0),
 .init(color: .black.opacity(0.05), location: 0.36),
 .init(color: .black.opacity(0.68), location: 0.74),
 .init(color: .black.opacity(0.88), location: 1.0)
],
 startPoint: .top,
 endPoint: .bottom
)
)
 .overlay(
 LinearGradient(
 colors: [.black.opacity(0.34), .clear],
 startPoint: .leading,
 endPoint: .trailing
)
)
)

motionOverlay(pulse: pulse, phase: phase)

VStack(alignment: .leading, spacing: 16) {
 Text(page.eyebrow)
 .font(.system(size: 12, weight: .semibold))
 .tracking(2.2)
 .foregroundColor(.white.opacity(0.72))
 .opacity(isSelected ? 1 : 0.45)

 Text(page.title)
 .font(.system(size: 44, weight: .light))
 .lineSpacing(2)
 .foregroundColor(.white)
 .fixedSize(horizontal: false, vertical: true)
 .opacity(copyVisible && isSelected ? 1 : 0)
 .offset(y: copyVisible && isSelected ? 0 : 22)
 .animation(.spring(response: 0.4, dampingFraction: 0.8), value: copyVisible)

 Text(page.subtitle)
 .font(.system(size: 16, weight: .regular))
 .lineSpacing(4)
 .foregroundColor(.white.opacity(0.78))
 .fixedSize(horizontal: false, vertical: true)
 .opacity(copyVisible && isSelected ? 1 : 0)
 .offset(y: copyVisible && isSelected ? 0 : 18)
 .animation(.spring(response: 0.4, dampingFraction: 0.8).delay(0.15), value: copyVisible)
}
 .padding(.horizontal, 32)
 .padding(.bottom, 210)
 .offset(y: isSelected && !reduceMotion ? -pulse * 5 : 0)
 .opacity(isSelected ? 1 : 0.72)
 .animation(.easeOut(duration: 0.45), value: isSelected)
}
}
.onAppear { updateCopyVisibility() }
.onChange(of: isSelected) { _, _ in updateCopyVisibility() }
}

private func updateCopyVisibility() {
 guard !reduceMotion else {
 copyVisible = isSelected
 return
 }
 copyVisible = false
 guard isSelected else { return }
 DispatchQueue.main.asyncAfter(deadline: .now() + 0.06) {
 copyVisible = true
 }
}

@ViewBuilder
private func motionOverlay(pulse: CGFloat, phase: CGFloat) -> some View {
 if let glowPosition = page.glowPosition {
 Circle()
 .fill(
 RadialGradient(
 colors: [
 Color.cyan.opacity(0.36 + pulse * 0.18),
 Color.cyan.opacity(0.12 + pulse * 0.12),
 Color.clear
]
)
)
 }
}

```

```

],
 center: .center,
 startRadius: 0,
 endRadius: size.width * 0.34
)
)
.frame(width: size.width * 0.72, height: size.width * 0.72)
.blur(radius: 22 + pulse * 10)
.blendMode(.screen)
.position(x: size.width * glowPosition.x, y: size.height * glowPosition.y)
.allowsHitTesting(false)
}

switch page.motion {
case .hero:
 LightSweepView(phase: phase, active: isSelected && !reduceMotion)
 .frame(width: size.width, height: size.height)
 .allowsHitTesting(false)
case .signal:
 SignalTraceView(progress: traceProgress(phase))

 .stroke(
 LinearGradient(
 colors: [
 Color.white.opacity(0.22),
 Color.cyan.opacity(0.92),
 Color.white.opacity(0.16)
],
 startPoint: .top,
 endPoint: .bottom
),
 style: StrokeStyle(lineWidth: 5, lineCap: .round, lineJoin: .round)
)
 .shadow(color: Color.cyan.opacity(0.72), radius: 18)
 .frame(width: size.width, height: size.height)
 .allowsHitTesting(false)
case .connected:
 UtilityChipCloud(phase: phase, active: isSelected && !reduceMotion)
 .padding(.horizontal, 28)
 .padding(.bottom, 360)
 .frame(width: size.width, height: size.height, alignment: .bottomLeading)
 .allowsHitTesting(false)
case .glow, .sun:
 EmptyView()
}
}

private func traceProgress(_ phase: CGFloat) -> CGFloat {
 guard isSelected && !reduceMotion else { return 0.82 }
 let cycle = phase.truncatingRemainder(dividingBy: 3.8) / 3.8
 if cycle < 0.18 {
 return 0
 } else if cycle < 0.72 {
 return min(1, (cycle - 0.18) / 0.54)
 } else {
 return 1
 }
}

private struct PageDots: View {
 let count: Int
 let currentIndex: Int

 var body: some View {
 HStack(spacing: 8) {
 ForEach(0..<count, id: \.self) { index in
 Capsule()
 .fill(index == currentIndex ? Color.white : Color.white.opacity(0.32))
 .frame(width: index == currentIndex ? 28 : 8, height: 8)
 .animation(.easeInOut(duration: 0.22), value: currentIndex)
 }
 }
 .accessibilityHidden(true)
 }
}

private struct OxyOnboardingPage {
 let imageName: String
 let eyebrow: String
 let title: String
 let subtitle: String
 let primaryAction: String
 let motion: OxyOnboardingMotionStyle
 let imageAnchor: UnitPoint
 let baseScale: CGFloat
 let drift: CGSize

```

```

 let glowPosition: UnitPoint?
}

private enum OxyOnboardingMotionStyle {
 case hero
 case glow
 case signal
 case connected
 case sun

 var speed: CGFloat {
 switch self {
 case .hero:
 return 0.22
 case .glow:
 return 0.72
 case .signal:
 return 0.42
 case .connected:
 return 0.32
 case .sun:
 return 0.46
 }
 }
}

var zoomRange: CGFloat {
 switch self {
 case .glow:
 return 0.026
 case .signal:
 return 0.018
 default:
 return 0.022
 }
}

private struct SignalTraceView: Shape {
 let progress: CGFloat

 func path(in rect: CGRect) -> Path {
 var path = Path()
 path.move(to: CGPoint(x: rect.width * 0.58, y: rect.height * 0.18))
 path.addCurve(
 to: CGPoint(x: rect.width * 0.66, y: rect.height * 0.38),
 control1: CGPoint(x: rect.width * 0.70, y: rect.height * 0.20),
 control2: CGPoint(x: rect.width * 0.70, y: rect.height * 0.30)
)
 path.addCurve(
 to: CGPoint(x: rect.width * 0.58, y: rect.height * 0.62),
 control1: CGPoint(x: rect.width * 0.61, y: rect.height * 0.48),
 control2: CGPoint(x: rect.width * 0.56, y: rect.height * 0.54)
)
 path.addCurve(
 to: CGPoint(x: rect.width * 0.70, y: rect.height * 0.84),
 control1: CGPoint(x: rect.width * 0.61, y: rect.height * 0.71),
 control2: CGPoint(x: rect.width * 0.67, y: rect.height * 0.76)
)
 return path.trimmedPath(from: 0, to: progress)
 }
}

private struct LightSweepView: View {
 let phase: CGFloat
 let active: Bool

 var body: some View {
 let offset = active ? sin(phase * 0.42) : -1
 Rectangle()
 .fill(
 LinearGradient(
 colors: [
 Color.clear,
 Color.white.opacity(0.12),
 Color.cyan.opacity(0.18),
 Color.clear
],
 startPoint: .top,
 endPoint: .bottom
)
)
 .frame(width: 180)
 .rotationEffect(.degrees(-18))
 .offset(x: offset * 160, y: -30)
 .blur(radius: 22)
 .blendMode(.screen)
 }
}

```

```

 .opacity(active ? 1 : 0.35)
 }
}

private struct UtilityChipCloud: View {
 let phase: CGFloat
 let active: Bool

 private let chips = ["Music", "Places", "Rides", "Memory"]

 var body: some View {
 VStack(alignment: .leading, spacing: 10) {
 ForEach(chips.indices, id: \.self) { index in
 Text(chips[index])
 .font(.system(size: 13, weight: .semibold))
 .foregroundColor(.white.opacity(0.88))
 .padding(.horizontal, 14)
 .padding(.vertical, 9)
 .background(.white.opacity(0.12), in: Capsule())
 .overlay(
 Capsule()
 .stroke(.white.opacity(0.16), lineWidth: 1)
)
 .offset(x: CGFloat(index % 2) * 34)
 .opacity(chipOpacity(index: index))
 .scaleEffect(chipScale(index: index))
 }
 }
 }

 private func chipOpacity(index: Int) -> Double {
 guard active else { return 0.45 }
 let cycle = (phase + CGFloat(index) * 0.24).truncatingRemainder(dividingBy: 2.8) / 2.8
 return Double(0.5 + min(cycle * 2.2, 1) * 0.5)
 }

 private func chipScale(index: Int) -> CGFloat {
 guard active else { return 0.96 }
 let cycle = (phase + CGFloat(index) * 0.22).truncatingRemainder(dividingBy: 2.8) / 2.8
 return 0.96 + min(cycle * 2.4, 1) * 0.04
 }
}

```

## AppState.swift — OxyApp/OxyApp/Models/AppState.swift

```

import Foundation
import Observation

@Observable
final class AppState {
 var isAuthenticated = false
 var userId: String = ""
 var token: String = ""
 var isLoading = false
 var errorMessage: String?

 private let keychain = KeychainHelper.shared
 @ObservationIgnored private var sessionExpiredObserver: NSObjectProtocol?

 init() {
 sessionExpiredObserver = NotificationCenter.default.addObserver(
 forName: .oxySessionExpired,
 object: nil,
 queue: .main
) { [weak self] _ in
 self?.logout()
 self?.errorMessage = "Session expired. Please sign in again."
 }
 }

 deinit {
 if let sessionExpiredObserver {
 NotificationCenter.default.removeObserver(sessionExpiredObserver)
 }
 }

 func restoreSession() {
 guard let savedToken = keychain.read(key: "session_token"),
 let savedUserId = keychain.read(key: "user_id"),
 !savedToken.isEmpty else {
 isAuthenticated = false
 return
 }
 token = savedToken
 userId = savedUserId
 isAuthenticated = true
 Task { @MainActor in

```

```

 NativeIntegrationManager.shared.bootstrap(userId: savedUserId)
 }
}

func login(userId: String, token: String) {
 self.userId = userId
 self.token = token
 let savedToken = keychain.save(key: "session_token", value: token)
 let savedUserId = keychain.save(key: "user_id", value: userId)
 guard savedToken, savedUserId else {
 keychain.delete(key: "session_token")
 keychain.delete(key: "user_id")
 self.token = ""
 self.userId = ""
 isAuthenticated = false
 errorMessage = "Could not save your session securely. Please unlock your device and try again."
 return
 }
 isAuthenticated = true
 errorMessage = nil
 Task { @MainActor in
 NativeIntegrationManager.shared.bootstrap(userId: userId)
 }
}

func logout() {
 keychain.delete(key: "session_token")
 keychain.delete(key: "user_id")
 token = ""
 userId = ""
 isAuthenticated = false
}
}

```

## Message.swift — OxyApp/OxyApp/Models/Message.swift

```

import Foundation

struct Message: Identifiable, Equatable {
 let id: UUID
 let dbId: String?
 let role: Role
 var content: String
 let timestamp: Date
 var actions: [ActionResult]
 var isStreaming: Bool

 enum Role: String, Codable {
 case user
 case assistant
 }

 init(
 id: UUID = UUID(),
 dbId: String? = nil,
 role: Role,
 content: String,
 timestamp: Date = Date(),
 actions: [ActionResult] = [],
 isStreaming: Bool = false
) {
 self.id = id
 self.dbId = dbId
 self.role = role
 self.content = content
 self.timestamp = timestamp
 self.actions = actions
 self.isStreaming = isStreaming
 }

 static func == (lhs: Message, rhs: Message) -> Bool {
 lhs.id == rhs.id
 && lhs.content == rhs.content
 && lhs.isStreaming == rhs.isStreaming
 && lhs.actions == rhs.actions
 }
}

extension Date {
 static func oxyParse(_ value: String?) -> Date? {
 guard let value, !value.isEmpty else { return nil }
 let fractional = ISO8601DateFormatter()
 fractional.formatOptions = [.withInternetDateTime, .withFractionalSeconds]
 if let date = fractional.date(from: value) { return date }
 if let date = ISO8601DateFormatter().date(from: value) { return date }

 let normalized = value.replacingOccurrences(of: " ", with: "T")
 }
}

```

```

 if let date = fractional.date(from: Self.trimFractionalSeconds(normalized)) { return date }
 if let date = ISO8601DateFormatter().date(from: Self.trimFractionalSeconds(normalized)) { return date }

 let formatter = DateFormatter()
 formatter.locale = Locale(identifier: "en_US_POSIX")
 formatter.timeZone = TimeZone(secondsFromGMT: 0)
 for format in [
 "yyyy-MM-dd'T'HH:mm:ss.SSSXXXX",
 "yyyy-MM-dd'T'HH:mm:ssXXXX",
 "yyyy-MM-dd'T'HH:mm:ss.SSSZ",
 "yyyy-MM-dd'T'HH:mm:ssZ"
] {
 formatter.dateFormat = format
 if let date = formatter.date(from: Self.trimFractionalSeconds(normalized)) {
 return date
 }
 }
 return nil
 }

 var oxyISO8601String: String {
 let formatter = ISO8601DateFormatter()
 formatter.formatOptions = [.withInternetDateTime, .withFractionalSeconds]
 return formatter.string(from: self)
 }

 private static func trimFractionalSeconds(_ value: String) -> String {
 value.replacingOccurrences(
 of: #"\\.\\d{3}\\d+([Zz]|[-]\\d{2}:?\\d{2})"#,
 with: ".$1$2",
 options: .regularExpression
)
 }
}

struct ActionResult: Codable, Identifiable, Equatable {
 var id: String { action + (text ?? "") }
 let action: String
 let success: Bool
 let text: String?
 let error: String?
 let deepLink: String?
 let webLink: String?
 let cardText: String?
 let actionSummary: String?
 let risk: String?
 let confirmation: String?
 let pending: Bool
 let connectorId: String?
 let healthStatus: String?

 enum CodingKeys: String, CodingKey {
 case action, result, success, text, error, deepLink, webLink, cardText, actionSummary, risk, confirmation, pending, connectorId, healthStatus
 }

 init(
 action: String,
 success: Bool,
 text: String? = nil,
 error: String? = nil,
 deepLink: String? = nil,
 webLink: String? = nil,
 cardText: String? = nil,
 actionSummary: String? = nil,
 risk: String? = nil,
 confirmation: String? = nil,
 pending: Bool = false,
 connectorId: String? = nil,
 healthStatus: String? = nil
) {
 self.action = action
 self.success = success
 self.text = text
 self.error = error
 self.deepLink = deepLink
 self.webLink = webLink
 self.cardText = cardText
 self.actionSummary = actionSummary
 self.risk = risk
 self.confirmation = confirmation
 self.pending = pending
 self.connectorId = connectorId
 self.healthStatus = healthStatus
 }

 init(native result: NativeLocalActionResult) {

```



```

 self.init(
 action: result.action,
 success: result.success,
 text: result.text,
 error: result.error,
 deepLink: result.deepLink,
 webLink: result.deepLink,
 cardText: result.cardText,
 actionSummary: result.actionSummary,
 risk: result.risk,
 confirmation: result.confirmation
)
 }

 init(from decoder: Decoder) throws {
 let container = try decoder.container(keyedBy: CodingKeys.self)
 action = try container.decodeIfPresent(String.self, forKey: .action) ?? "unknown"

 if let result = try? container.nestedContainer(keyedBy: CodingKeys.self, forKey: .result) {
 success = try result.decodeIfPresent(Bool.self, forKey: .success) ?? false
 text = try result.decodeIfPresent(String.self, forKey: .text)
 error = try result.decodeIfPresent(String.self, forKey: .error)
 deepLink = try result.decodeIfPresent(String.self, forKey: .deepLink)
 webLink = try result.decodeIfPresent(String.self, forKey: .webLink)
 cardText = try result.decodeIfPresent(String.self, forKey: .cardText)
 actionSummary = try result.decodeIfPresent(String.self, forKey: .actionSummary)
 risk = try result.decodeIfPresent(String.self, forKey: .risk)
 confirmation = try result.decodeIfPresent(String.self, forKey: .confirmation)
 pending = try result.decodeIfPresent(Bool.self, forKey: .pending) ?? false
 connectorId = try result.decodeIfPresent(String.self, forKey: .connectorId)
 healthStatus = try result.decodeIfPresent(String.self, forKey: .healthStatus)
 } else {
 success = try container.decodeIfPresent(Bool.self, forKey: .success) ?? false
 text = try container.decodeIfPresent(String.self, forKey: .text)
 error = try container.decodeIfPresent(String.self, forKey: .error)
 deepLink = try container.decodeIfPresent(String.self, forKey: .deepLink)
 webLink = try container.decodeIfPresent(String.self, forKey: .webLink)
 cardText = try container.decodeIfPresent(String.self, forKey: .cardText)
 actionSummary = try container.decodeIfPresent(String.self, forKey: .actionSummary)
 risk = try container.decodeIfPresent(String.self, forKey: .risk)
 confirmation = try container.decodeIfPresent(String.self, forKey: .confirmation)
 pending = try container.decodeIfPresent(Bool.self, forKey: .pending) ?? false
 connectorId = try container.decodeIfPresent(String.self, forKey: .connectorId)
 healthStatus = try container.decodeIfPresent(String.self, forKey: .healthStatus)
 }
 }

 func encode(to encoder: Encoder) throws {
 var container = encoder.container(keyedBy: CodingKeys.self)
 try container.encode(action, forKey: .action)
 try container.encode(success, forKey: .success)
 try container.encodeIfPresent(text, forKey: .text)
 try container.encodeIfPresent(error, forKey: .error)
 try container.encodeIfPresent(deepLink, forKey: .deepLink)
 try container.encodeIfPresent(webLink, forKey: .webLink)
 try container.encodeIfPresent(cardText, forKey: .cardText)
 try container.encodeIfPresent(actionSummary, forKey: .actionSummary)
 try container.encodeIfPresent(risk, forKey: .risk)
 try container.encodeIfPresent(confirmation, forKey: .confirmation)
 try container.encode(pending, forKey: .pending)
 try container.encodeIfPresent(connectorId, forKey: .connectorId)
 try container.encodeIfPresent(healthStatus, forKey: .healthStatus)
 }
}

struct AuthResponse: Codable {
 let success: Bool
 let token: String?
 let userId: String?
 let error: String?
}

struct HistoryEntry: Codable, Identifiable {
 let id: String?
 let role: String
 let content: String
 let createdAt: String?
 let actions: [ActionResult]?

 enum CodingKeys: String, CodingKey {
 case id
 case role
 case content
 case actions
 case createdAt = "created_at"
 }
}

```

```
 var stableId: String {
 id ?? UUID().uuidString
 }
}

struct HistoryResponse: Codable {
 let history: [HistoryEntry]
}

struct Briefing: Codable, Identifiable, Equatable {
 let id: String
 let kind: String
 let title: String?
 let body: String
 let source: String?
 let read: Bool?

 let createdAt: String?

 enum CodingKeys: String, CodingKey {
 case id, kind, title, body, source, read
 case createdAt = "created_at"
 }

 var isUnread: Bool {
 read == false
 }
}

struct BriefingsResponse: Codable {
 let briefings: [Briefing]
}
```

# iOS App — Services

## APIClient.swift — OxyApp/OxyApp/Services/APIClient.swift

```
import Foundation

extension Notification.Name {
 static let oxySessionExpired = Notification.Name("oxy.sessionExpired")
 static let oxyJumpToChat = Notification.Name("oxy.jumpToChat")
}

final class APIClient: @unchecked Sendable {
 static let shared = APIClient()

 /// Base URL of the Oxy backend.
 /// Priority: UserDefaults override > OXY_BASE_URL build setting > hardcoded default.
 /// Users can set a custom URL via Settings → Diagnostics or the `oxy_custom_backend_url` UserDefaults key.
 var baseURL: String {
 if let custom = UserDefaults.standard.string(forKey: "oxy_custom_backend_url"), !custom.isEmpty {
 return custom.trimmingCharacters(in: CharacterSet(charactersIn: "/"))
 }
 if let envURL = Bundle.main.infoDictionary?["OXY_BASE_URL"] as? String, !envURL.isEmpty {
 return envURL.trimmingCharacters(in: CharacterSet(charactersIn: "/"))
 }
 #if DEBUG
 return "https://oxy-151340634966.europe-west2.run.app"
 #else
 return "https://oxy-151340634966.europe-west2.run.app"
 #endif
 }

 private init() {}

 // MARK: - Token

 private var token: String {
 KeychainHelper.shared.read(key: "session_token") ?? ""
 }

 // MARK: - Requests

 func request(
 path: String,
 method: String = "GET",
 body: [String: Any]? = nil,
 queryItems: [URLQueryItem]? = nil
) async throws -> Data {
 guard var components = URLComponents(string: "\(baseURL)\(path)") else {
 throw APIError.invalidURL
 }
 if let queryItems {
 components.queryItems = (components.queryItems ?? []) + queryItems
 }
 guard let url = components.url else {
 throw APIError.invalidURL
 }

 var req = URLRequest(url: url)
 req.httpMethod = method
 req.setValue("application/json", forHTTPHeaderField: "Content-Type")
 if !token.isEmpty {
 req.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
 }
 if let body {
 req.httpBody = try JSONSerialization.data(withJSONObject: body)
 }
 req.timeoutInterval = 60

 let (data, response) = try await URLSession.shared.data(for: req)
 guard let http = response as? HTTPURLResponse else {
 throw APIError.unknown
 }
 guard (200...299).contains(http.statusCode) else {
 let errorBody = try? JSONDecoder().decode(ErrorBody.self, from: data)
 let message = errorBody?.error ?? "Request failed"
 if http.statusCode == 401 {
 handleUnauthorized()
 throw APIError.server(401, message)
 }
 throw APIError.server(http.statusCode, message)
 }
 return data
 }

 func exportUserData(userId: String) async throws -> Data {
```

```

 try await request(path: "/user/\(userId)/export")
 }

func deleteAccount(userId: String) async throws {
 _ = try await request(path: "/user/\(userId)", method: "DELETE")
}

func multipartRequest(
 path: String,
 method: String = "POST",
 fields: [String: String],
 fileField: String,
 fileName: String,
 mimeType: String,
 fileData: Data,
 queryItems: [URLQueryItem]? = nil
) async throws -> Data {
 guard var components = URLComponents(string: "\(baseUrl)\(path)") else {
 throw APIError.invalidURL
 }
 if let queryItems {
 components.queryItems = (components.queryItems ?? []) + queryItems
 }
 guard let url = components.url else {
 throw APIError.invalidURL
 }

 let boundary = "Boundary-\(UUID().uuidString)"
 var body = Data()
 for (key, value) in fields {
 body.append("--\(\boundary)\r\n")
 body.append("Content-Disposition: form-data; name=\\"\(key)\"\\r\n\r\n")
 body.append("\(\value)\r\n")
 }
 body.append("--\(\boundary)\r\n")
 body.append("Content-Disposition: form-data; name=\\"\(fileField)\"\\r\n\r\n")
 body.append("Content-Type: \(\mimeType)\r\n\r\n")
 body.append(fileData)
 body.append("\r\n--\(\boundary)--\r\n")

 var req = URLRequest(url: url)
 req.httpMethod = method
 req.setValue("multipart/form-data; boundary=\(\boundary)", forHTTPHeaderField: "Content-Type")
 if !token.isEmpty {
 req.setValue("Bearer \(\token)", forHTTPHeaderField: "Authorization")
 }
 req.httpBody = body
 req.timeoutInterval = 120

 let (data, response) = try await URLSession.shared.data(for: req)
 guard let http = response as? HTTPURLResponse else {
 throw APIError.unknown
 }
 guard (200...299).contains(http.statusCode) else {
 let errorBody = try? JSONDecoder().decode(ErrorBody.self, from: data)
 let message = errorBody?.error ?? "Upload failed"
 if http.statusCode == 401 {
 handleUnauthorized()
 throw APIError.server(401, message)
 }
 throw APIError.server(http.statusCode, message)
 }
 return data
}

/// Opens an SSE stream and yields parsed events via an AsyncStream.
func sseStream(
 path: String,
 method: String = "POST",
 body: [String: Any]? = nil,
 queryItems: [URLQueryItem]? = nil
) -> AsyncStream<SSEEvent> {
 let baseUrl = self.baseUrl
 let token = self.token
 let bodyDataResult: Result<Data?, Error> = Result {
 try body.map { try JSONSerialization.data(withJSONObject: $0) }
 }
 return AsyncStream<SSEEvent> { continuation in
 Task {
 do {
 guard var components = URLComponents(string: "\(baseUrl)\(path)") else {
 continuation.yield(.error("Invalid URL"))
 continuation.finish()
 return
 }
 if let queryItems {
 components.queryItems = (components.queryItems ?? []) + queryItems
 }
 }
 }
 }
}

```

```

 }
 guard let url = components.url else {
 continuation.yield(.error("Invalid URL"))
 continuation.finish()
 return
 }

 var req = URLRequest(url: url)
 req.httpMethod = method
 req.setValue("application/json", forHTTPHeaderField: "Content-Type")
 if !token.isEmpty {
 req.setValue("Bearer \(token)", forHTTPHeaderField: "Authorization")
 }
 switch bodyDataResult {
 case .success(let bodyData):
 req.httpBody = bodyData
 case .failure(let error):
 continuation.yield(.error(error.localizedDescription))
 continuation.finish()
 return
 }
 req.timeoutInterval = 120

 let (bytes, response) = try await URLSession.shared.bytes(for: req)
 guard let http = response as? HTTPURLResponse else {
 continuation.yield(.error("Server error"))
 continuation.finish()
 return
 }
 guard (200...299).contains(http.statusCode) else {
 if http.statusCode == 401 {
 self.handleUnauthorized()
 continuation.yield(.error("Session expired. Please sign in again."))
 } else {
 continuation.yield(.error("Server error"))
 }
 continuation.finish()
 return
 }
}

var buffer = ""
for try await line in bytes.lines {
 if line.hasPrefix("data: ") {
 let jsonStr = String(line.dropFirst(6))
 buffer = ""
 if let data = jsonStr.data(using: .utf8) {
 let event = SSEEvent.parse(data)
 continuation.yield(event)
 if case .done = event { break }
 }
 } else if line.isEmpty {
 buffer = ""
 } else {
 buffer += line
 }
}
continuation.finish()
} catch {
 continuation.yield(.error(error.localizedDescription))
 continuation.finish()
}
}
}

private func handleUnauthorized() {
 KeychainHelper.shared.delete(key: "session_token")
 KeychainHelper.shared.delete(key: "user_id")
 NotificationCenter.default.post(name: .oxySessionExpired, object: nil)
}

private extension Data {
 mutating func append(_ string: String) {
 if let data = string.data(using: .utf8) {
 append(data)
 }
 }
}

// MARK: - SSE Event Types

enum SSEEvent {
 case text(String)
 case replace(String)
 case actions([ActionResult])
 case status(String, String)

```

```

case audio(String, String)
case transcription(String)
case transcriptionError(String)
case ttsError(String)
case done
case error(String)

static func parse(_ data: Data) -> SSEEvent {
 guard let json = try? JSONSerialization.jsonObject(with: data) as? [String: Any],
 let type = json["type"] as? String else {
 return .error("Invalid event data")
 }
 switch type {
 case "text":
 return .text(json["chunk"] as? String ?? "")
 case "replace":
 return .replace(json["text"] as? String ?? "")
 case "actions":
 if let resultsData = try? JSONSerialization.data(withJSONObject: json["results"] ?? []),
 let results = try? JSONDecoder().decode([ActionResult].self, from: resultsData) {
 return .actions(results)
 }
 return .actions([])
 case "status":
 return .status(
 json["status"] as? String ?? "",
 json["label"] as? String ?? ""
)
 case "audio":
 return .audio(
 json["data"] as? String ?? "",
 json["mimeType"] as? String ?? "audio/wav"
)
 case "transcription":
 return .transcription(json["text"] as? String ?? "")
 case "transcription-error":
 return .transcriptionError(json["error"] as? String ?? "Transcription failed")
 case "tts-error":
 return .ttsError(json["error"] as? String ?? "TTS failed")
 case "done":
 return .done
 case "error":
 return .error(json["error"] as? String ?? "Unknown error")
 default:
 return .error("Unknown event type: \(type)")
 }
}
}

// MARK: - Error Types

enum APIError: LocalizedError {
 case invalidURL
 case server(Int, String)
 case unknown

 var errorDescription: String? {
 switch self {
 case .invalidURL:
 return "Invalid URL"
 case .server(_, let message):
 return message
 case .unknown:
 return "An unknown error occurred"
 }
 }
}

var isUnauthorized: Bool {
 if case .server(401, _) = self { return true }
 return false
}
}

private struct ErrorBody: Codable {
 let error: String
}
}

```

## AuthService.swift — OxyApp/OxyApp/Services/AuthService.swift

```

import Foundation

struct AuthService {
 private let api = APIClient.shared

 func register(userId: String, password: String) async throws -> AuthResponse {
 let data = try await api.request(
 path: "/auth/register",

```

```

 method: "POST",
 body: ["userId": userId, "password": password]
)
 return try JSONDecoder().decode(AuthResponse.self, from: data)
}

func login(userId: String, password: String) async throws -> AuthResponse {
 let data = try await api.request(
 path: "/auth/login",
 method: "POST",
 body: ["userId": userId, "password": password]
)
 return try JSONDecoder().decode(AuthResponse.self, from: data)
}
}

```

## ChatService.swift — OxyApp/OxyApp/Services/ChatService.swift

```
import Foundation
```

```

struct ChatService {
 private let api = APIClient.shared

 func backendVersion() async throws -> BackendVersion {
 let data = try await api.request(path: "/version")
 return try JSONDecoder().decode(BackendVersion.self, from: data)
 }

 func sendMessage(
 userId: String,
 message: String,
 chatStartedAt: String? = nil,
 settings: OxySettings? = nil,
 location: [String: Double]? = nil,
 nativeHints: [String: Any]? = nil,
 tts: Bool = false
) -> AsyncStream<SSEEvent> {
 var body: [String: Any] = [
 "userId": userId,
 "message": message
]

 if let chatStartedAt {
 body["chatStartedAt"] = chatStartedAt
 }

 if let settings {
 body["settings"] = [
 "name": settings.name,
 "voice": settings.voice,
 "voiceOn": settings.voiceOn,
 "voiceEngine": settings.voiceEngine,
 "autonomy": settings.autonomy,
 "preferredMapsApp": settings.preferredMapsApp,
 "preferredTransportMode": settings.preferredTransportMode,
 "reviewBeforeOpeningApps": settings.reviewBeforeOpeningApps,
 "confirmSensitiveAppOpens": settings.confirmSensitiveAppOpens
]
 }

 if let location {
 body["location"] = location
 }

 if let nativeHints, !nativeHints.isEmpty {
 body["nativeHints"] = nativeHints
 }

 var queryItems = [URLQueryItem(name: "stream", value: "true")]
 if tts || (settings?.voiceOn == true) {
 queryItems.append(URLQueryItem(name: "tts", value: "true"))
 }

 return api.sseStream(
 path: "/chat",
 method: "POST",
 body: body,
 queryItems: queryItems
)
 }

 func loadHistory(userId: String, limit: Int = 50, since: String? = nil) async throws -> [HistoryEntry] {
 var queryItems = [URLQueryItem(name: "limit", value: String(limit))]
 if let since {
 queryItems.append(URLQueryItem(name: "since", value: since))
 }
 let data = try await api.request(

```

```

 path: "/history/\(userId)",
 queryItems: queryItems
)
 return try JSONDecoder().decode(HistoryResponse.self, from: data).history
}

func loadHistoryAround(userId: String, createdAt: String? = nil, messageId: String? = nil) async throws -> [HistoryEntry] {
 var queryItems = [
 URLQueryItem(name: "before", value: "60"),
 URLQueryItem(name: "after", value: "60")
]
 if let createdAt {
 queryItems.append(URLQueryItem(name: "createdAt", value: createdAt))
 }
 if let messageId {
 queryItems.append(URLQueryItem(name: "messageId", value: messageId))
 }
 let data = try await api.request(
 path: "/history/\(userId)/around",
 queryItems: queryItems
)
 return try JSONDecoder().decode(HistoryResponse.self, from: data).history
}

func logNativeLocalAction(userId: String, message: String, result: ActionResult, chatStartedAt: String? = nil) async {
 var body: [String: Any] = [
 "userId": userId,
 "message": message,
 "result": result.dictionary
]
 if let chatStartedAt {
 body["chatStartedAt"] = chatStartedAt
 }
 _ = try? await api.request(
 path: "/native/local-action",
 method: "POST",
 body: body
)
}

func sendImageMessage(
 userId: String,
 message: String,
 imageData: Data,
 fileName: String,
 mimeType: String,
 chatStartedAt: String? = nil,
 settings: OxySettings? = nil
) async throws -> ImageChatResponse {
 var fields: [String: String] = [
 "userId": userId,
 "message": message
]
 if let chatStartedAt {
 fields["chatStartedAt"] = chatStartedAt
 }
 if let settings {
 let payload: [String: Any] = [
 "name": settings.name,
 "voice": settings.voice,
 "voiceOn": settings.voiceOn,
 "voiceEngine": settings.voiceEngine,
 "autonomy": settings.autonomy,
 "preferredMapsApp": settings.preferredMapsApp,
 "preferredTransportMode": settings.preferredTransportMode,
 "reviewBeforeOpeningApps": settings.reviewBeforeOpeningApps,
 "confirmSensitiveAppOpens": settings.confirmSensitiveAppOpens
]
 if let data = try? JSONSerialization.data(withJSONObject: payload),
 let json = String(data: data, encoding: .utf8) {
 fields["settings"] = json
 }
 }

 var queryItems: [URLQueryItem] = []
 if settings?.voiceOn == true {
 queryItems.append(URLQueryItem(name: "tts", value: "true"))
 }

 let data = try await api.multipartRequest(
 path: "/chat-with-image",
 fields: fields,
 fileField: "image",
 fileName: fileName,
 mimeType: mimeType,
 fileData: imageData,

```



```

 queryItems: queryItems
)
 return try JSONDecoder().decode(ImageChatResponse.self, from: data)
}

func loadBriefings(userId: String, limit: Int = 30) async throws -> [Briefing] {
 let data = try await api.request(
 path: "/briefings/\(userId)",
 queryItems: [URLQueryItem(name: "limit", value: String(limit))]
)
 return try JSONDecoder().decode(BriefingsResponse.self, from: data).briefings
}

func markBriefingRead(userId: String, briefingId: String) async {
 _ = try? await api.request(
 path: "/briefings/\(briefingId)/read",
 method: "POST",
 body: ["userId": userId]
)
}

func runProactiveCheck(userId: String) async throws {
 _ = try await api.request(
 path: "/proactive/\(userId)/run",
 method: "POST",
 body: ["userId": userId]
)
}
}

struct BackendVersion: Decodable {
 let app: String?
 let packageVersion: String?
 let gitCommit: String?
 let gitBranch: String?
 let cloudRunRevision: String?
 let deployId: String?
 let buildTime: String?
 let environment: String?
}

struct ImageChatResponse: Codable {
 let text: String
 let actions: [ActionResult]?
 let audio: String?
 let audioMimeType: String?
 let ttsError: String?
}

private extension Encodable {
 var dictionary: [String: Any] {
 guard let data = try? JSONEncoder().encode(self),
 let object = try? JSONSerialization.jsonObject(with: data) as? [String: Any] else {
 return [:]
 }
 return object
 }
}

```

### **VoiceInputManager.swift — OxyApp/OxyApp/Services/VoiceInputManager.swift**

```

import AVFoundation
import Speech
import Observation

@Observable
@MainActor
final class VoiceInputManager {
 var isRecording = false
 var isPreparing = false
 var transcript = ""
 var errorMessage: String?

 private var audioEngine: AVAudioEngine?
 private var recognitionTask: SFSpeechRecognitionTask?
 private var recognitionRequest: SFSpeechAudioBufferRecognitionRequest?
 private var startTask: Task<Void, Never>?

 var speechAuthStatus: SFSpeechRecognizerAuthorizationStatus {
 SFSpeechRecognizer.authorizationStatus()
 }

 var micAuthStatus: AVAudioSession.RecordPermission {
 AVAudioSession.sharedInstance().recordPermission
 }

 var isAuthorized: Bool {

```

```

 speechAuthStatus == .authorized && micAuthStatus == .granted
 }

 func requestPermissions() {
 SFSpeechRecognizer.requestAuthorization { _ in }
 AVAudioSession.sharedInstance().requestRecordPermission { _ in }
 }

 func startRecording() {
 guard !isRecording, !isPreparing else { return }

 startTask?.cancel()
 startTask = Task { [weak self] in
 await self?.beginRecording()
 }
 }

 private func beginRecording() async {
 isPreparing = true
 errorMessage = nil
 transcript = ""

 guard await ensurePermissions() else {
 isPreparing = false
 errorMessage = "Enable microphone and speech access to use voice."
 return
 }

 let recognizer = SFSpeechRecognizer(locale: Locale.current)
 guard let recognizer, recognizer.isAvailable else {
 isPreparing = false
 errorMessage = "Speech recognition not available"
 return
 }

 let request = SFSpeechAudioBufferRecognitionRequest()
 request.shouldReportPartialResults = true

 let engine = AVAudioEngine()
 let inputNode = engine.inputNode
 let recordingFormat = inputNode.outputFormat(forBus: 0)

 inputNode.installTap(onBus: 0, bufferSize: 1024, format: recordingFormat) { buffer, _ in
 request.append(buffer)
 }

 do {
 let session = AVAudioSession.sharedInstance()
 try session.setCategory(.record, mode: .measurement, options: .duckOthers)
 try session.setActive(true, options: .notifyOthersOnDeactivation)

 engine.prepare()
 try engine.start()
 } catch {
 inputNode.removeTap(onBus: 0)
 isPreparing = false
 errorMessage = "Audio setup failed: \(error.localizedDescription)"
 return
 }

 recognitionTask = recognizer.recognitionTask(with: request) { [weak self] result, error in
 Task { @MainActor in
 guard let self else { return }
 if let result {
 self.transcript = result.bestTranscription.formattedString
 }
 if error != nil || (result?.isFinal == true) {
 self.stopRecording()
 }
 }
 }
 }

 audioEngine = engine
 recognitionRequest = request
 isPreparing = false
 isRecording = true
}

func stopRecording() {
 startTask?.cancel()
 startTask = nil
 audioEngine?.stop()
 audioEngine?.inputNode.removeTap(onBus: 0)
 recognitionRequest?.endAudio()
 recognitionTask?.cancel()

 audioEngine = nil

```

```

 recognitionRequest = nil
 recognitionTask = nil
 isPreparing = false
 isRecording = false

 try? AVAudioSession.sharedInstance().setActive(false, options: .notifyOthersOnDeactivation)
 }

 private func ensurePermissions() async -> Bool {
 let speech = await requestSpeechAuthorizationIfNeeded()
 let mic = await requestMicrophoneAuthorizationIfNeeded()
 return speech == .authorized && mic == .granted
 }

 private func requestSpeechAuthorizationIfNeeded() async -> SFSpeechRecognizerAuthorizationStatus {
 let current = SFSpeechRecognizer.authorizationStatus()
 guard current == .notDetermined else { return current }
 return await withCheckedContinuation { continuation in
 SFSpeechRecognizer.requestAuthorization { status in
 continuation.resume(returning: status)
 }
 }
 }

 private func requestMicrophoneAuthorizationIfNeeded() async -> AVAudioSession.RecordPermission {
 let session = AVAudioSession.sharedInstance()
 let current = session.recordPermission
 guard current == .undetermined else { return current }
 return await withCheckedContinuation { continuation in
 session.requestRecordPermission { granted in
 continuation.resume(returning: granted ? .granted : .denied)
 }
 }
 }
}

```

### KeychainHelper.swift — OxyApp/OxyApp/Services/KeychainHelper.swift

```

import Foundation
import Security

final class KeychainHelper: @unchecked Sendable {
 static let shared = KeychainHelper()
 private let service = "ai.oxy.app"

 private init() {}

 @discardableResult
 func save(key: String, value: String) -> Bool {
 guard let data = value.data(using: .utf8) else { return false }
 delete(key: key)
 let query: [String: Any] = [
 kSecClass as String: kSecClassGenericPassword,
 kSecAttrService as String: service,
 kSecAttrAccount as String: key,
 kSecValueData as String: data,
 kSecAttrAccessible as String: kSecAttrAccessibleAfterFirstUnlock
]
 let status = SecItemAdd(query as CFDictionary, nil)
 return status == errSecSuccess
 }

 func read(key: String) -> String? {
 let query: [String: Any] = [
 kSecClass as String: kSecClassGenericPassword,
 kSecAttrService as String: service,
 kSecAttrAccount as String: key,
 kSecReturnData as String: true,
 kSecMatchLimit as String: kSecMatchLimitOne
]
 var result: AnyObject?
 let status = SecItemCopyMatching(query as CFDictionary, &result)
 guard status == errSecSuccess, let data = result as? Data else { return nil }
 return String(data: data, encoding: .utf8)
 }

 func delete(key: String) {
 let query: [String: Any] = [
 kSecClass as String: kSecClassGenericPassword,
 kSecAttrService as String: service,
 kSecAttrAccount as String: key
]
 SecItemDelete(query as CFDictionary)
 }
}

```

### LocationManager.swift — OxyApp/OxyApp/Services/LocationManager.swift

```

import CoreLocation
import Observation

@Observable
@MainActor
final class LocationManager: NSObject, @preconcurrency CLLocationManagerDelegate {
 static let shared = LocationManager()

 var lastLocation: CLLocation?
 var authorizationStatus: CLAuthorizationStatus = .notDetermined
 var isAuthorized: Bool {
 authorizationStatus == .authorizedWhenInUse || authorizationStatus == .authorizedAlways
 }

 private let manager = CLLocationManager()
 private var pendingLocationContinuations: [CheckedContinuation<String: Double>?, Never<String: Double>?] = []

 override init() {
 super.init()
 manager.delegate = self
 manager.desiredAccuracy = kCLLocationAccuracyNearestTenMeters
 manager.distanceFilter = 25
 authorizationStatus = manager.authorizationStatus
 }

 func requestPermission() {
 manager.requestWhenInUseAuthorization()
 }

 func requestAlwaysPermission() {
 if authorizationStatus == .authorizedWhenInUse {
 manager.requestAlwaysAuthorization()
 } else {
 manager.requestWhenInUseAuthorization()
 }
 }

 func requestLocation() {
 if isAuthorized {
 manager.requestLocation()
 } else {
 requestPermission()
 }
 }

 func currentLocationForLocalRequest(timeoutNanoseconds: UInt64 = 900_000_000) async -> [String: Double]? {
 if let lastLocation, abs(lastLocation.timestamp.timeIntervalSinceNow) < 180 {
 return locationDict
 }
 if !isAuthorized {
 requestPermission()
 return nil
 }

 manager.requestLocation()
 return await withCheckedContinuation { continuation in
 pendingLocationContinuations.append(continuation)
 Task { [weak self] in
 try? await Task.sleep(nanoseconds: timeoutNanoseconds)
 self?.resolvePendingLocationContinuations(with: self?.locationDict)
 }
 }
 }

 var locationDict: [String: Double]? {
 guard let loc = lastLocation else { return nil }
 return [
 "latitude": loc.coordinate.latitude,
 "longitude": loc.coordinate.longitude
]
 }

 // MARK: - CLLocationManagerDelegate

 func locationManager(_ manager: CLLocationManager, didUpdateLocations locations: [CLLocation]) {
 lastLocation = locations.last
 resolvePendingLocationContinuations(with: locationDict)
 }

 func locationManager(_ manager: CLLocationManager, didFailWithError error: Error) {
 resolvePendingLocationContinuations(with: nil)
 }

 func locationManagerDidChangeAuthorization(_ manager: CLLocationManager) {
 authorizationStatus = manager.authorizationStatus
 if isAuthorized {
 manager.requestLocation()
 }
 }

```

```

 } else {
 resolvePendingLocationContinuations(with: nil)
 }
 }

private func resolvePendingLocationContinuations(with location: [String: Double]?) {
 guard !pendingLocationContinuations.isEmpty else { return }
 let continuations = pendingLocationContinuations
 pendingLocationContinuations.removeAll()
 continuations.forEach { $0.resume(returning: location) }
}
}

```

### AppDelegate.swift — OxyApp/OxyApp/Services/AppDelegate.swift

```

import UIKit

final class AppDelegate: NSObject, UIApplicationDelegate {
 func application(
 _ application: UIApplication,
 didRegisterForRemoteNotificationsWithDeviceToken deviceToken: Data
) {
 guard let userId = KeychainHelper.shared.read(key: "user_id"), !userId.isEmpty else { return }
 Task { @MainActor in
 await NativeIntegrationManager.shared.registerPushToken(deviceToken, userId: userId)
 }
 }
}

```

### NativeIntegrationManager.swift — OxyApp/OxyApp/Services/NativeIntegrationManager.swift

```

import Contacts
import CoreBluetooth
import CoreLocation
import EventKit
import Foundation
import HealthKit
import MapKit
import MediaPlayer
import MessageUI
import AVFoundation
import MusicKit
import Observation
import UIKit
import UserNotifications

struct NativeContactHint: Codable, Equatable {
 let displayName: String
 let phone: String?
 let email: String?
}

struct NativePlaceResult: Equatable {
 let name: String
 let address: String
 let distanceMeters: CLLocationDistance?
 let mapURL: URL
}

struct NativeLocalActionResult: Equatable, Sendable {
 let action: String
 let text: String
 let cardText: String
 let actionSummary: String
 let deepLink: String?
 let success: Bool
 let error: String?
 let risk: String?
 let confirmation: String?

 init(
 action: String,
 text: String,
 cardText: String,
 actionSummary: String,
 deepLink: String?,
 success: Bool = true,
 error: String? = nil,
 risk: String? = nil,
 confirmation: String? = nil
) {
 self.action = action
 self.text = text
 self.cardText = cardText
 self.actionSummary = actionSummary
 self.deepLink = deepLink
 self.success = success
 }
}

```

```

 self.error = error
 self.risk = risk
 self.confirmation = confirmation
 }
}

struct NativeHealthSnapshot: Codable {
 var latestHeartRate: Double?
 var restingHeartRate: Double?
 var stepCountToday: Double?
 var sleepMinutesLastNight: Double?
 var recentWorkouts: [NativeWorkoutSummary]?
}

struct NativeWorkoutSummary: Codable, Equatable {
 let activity: String
 let durationMinutes: Double
 let energyKilocalories: Double?
 let distanceMeters: Double?
 let endedAt: Date
}

private struct ITunesSongResult: Decodable {
 let resultCount: Int
 let results: [ITunesSong]
}

private struct ITunesSong: Decodable {
 let trackId: Int?
 let trackName: String?
 let artistName: String?
 let trackViewUrl: String?
}

private struct ITunesAlbumResult: Decodable {
 let resultCount: Int
 let results: [ITunesAlbum]
}

private struct ITunesAlbum: Decodable {
 let collectionId: Int?
 let collectionName: String?
 let artistName: String?
 let collectionViewUrl: String?
}

private struct ITunesAppResult: Decodable {
 let resultCount: Int
 let results: [ITunesApp]
}

private struct ITunesApp: Decodable {
 let trackId: Int?
 let trackName: String?
 let sellerName: String?
 let trackViewUrl: String?
}

private struct NativeAppShortcut {
 let displayName: String
 let aliases: [String]
 let universallinks: [String]
 let schemes: [String]
 let fallbackURL: String?

 let sensitive: Bool

 init(
 displayName: String,
 aliases: [String],
 universallinks: [String] = [],
 schemes: [String] = [],
 fallbackURL: String? = nil,
 sensitive: Bool = false
) {
 self.displayName = displayName
 self.aliases = aliases
 self.universallinks = universallinks
 self.schemes = schemes
 self.fallbackURL = fallbackURL
 self.sensitive = sensitive
 }
}

struct NativeCapabilities: Codable {
 var notifications: Bool
 var healthKit: Bool
}

```

```

 var musicKit: Bool
 var contacts: Bool
 var reminders: Bool
 var locationAlways: Bool
}

@MainActor
final class NativeIntegrationManager: NSObject {
 static let shared = NativeIntegrationManager()

 private let healthStore = HKHealthStore()
 private let contactStore = CNContactStore()
 private let eventStore = EKEEventStore()
 private let detector = try? NSDataDetector(types: NSTextCheckingResult.CheckingType.date.rawValue)
 private var lastMusicQuery: String?
 private var lastMusicArtist: String?
 private var lastMusicError: String?
 private var musicHistory: [String] = []
 private var lastReminderIdentifier: String?
 private var lastReminderTitle: String?
 private var lastReminderDueDate: Date?
 let pendant = PendantBLEManager()

 private override init() {
 super.init()
 }

 func bootstrap(userId: String) {
 guard !userId.isEmpty else { return }
 Task {
 await requestNotificationPermission(userId: userId)
 await syncNativeContext(userId: userId)
 }
 }

 func requestNotificationPermission(userId: String) async {
 do {
 let granted = try await UNUserNotificationCenter.current().requestAuthorization(options: [.alert, .sound, .badge])
 if granted {
 UIApplication.shared.registerForRemoteNotifications()
 }
 } catch {}
 }

 func registerPushToken(_ deviceToken: Data, userId: String) async {
 let token = deviceToken.map { String(format: "%02x", $0) }.joined()
 UserDefaults.standard.set(token, forKey: "oxy_push_token")
 do {
 _ = try await APIClient.shared.request(
 path: "/devices/register",
 method: "POST",
 body: [
 "userId": userId,
 "platform": "ios",
 "pushToken": token,
 "timezone": TimeZone.current.identifier
]
)
 } catch {}
 }

 func requestNativePermissions(userId: String) async {
 LocationManager.shared.requestAlwaysPermission()
 await requestHealthPermission()
 await requestContactsPermission()
 await requestReminderPermission()
 await requestCalendarPermission()
 await syncNativeContext(userId: userId)
 }

 func syncNativeContext(userId: String) async {
 guard !userId.isEmpty else { return }
 let settings = loadSettings()
 let health = await healthSnapshot()
 let capabilities = await nativeCapabilities()
 var body: [String: Any] = [
 "userId": userId,
 "health": health.dictionary,
 "capabilities": capabilities.dictionary,
 "settings": settings.nativeDictionary
]
 if let location = LocationManager.shared.locationDict {
 body["location"] = location
 }
 do {
 _ = try await APIClient.shared.request(path: "/native/context", method: "POST", body: body)
 } catch {}
 }
}

```

```

}

func markCurrentLocationAsHome(userId: String) async {
 LocationManager.shared.requestLocation()
 guard let location = LocationManager.shared.locationDict else { return }
 var settings = loadSettings()
 settings.homeLatitude = location["latitude"]
 settings.homeLongitude = location["longitude"]
 saveSettings(settings)
 await syncNativeContext(userId: userId)
}

func openMessage(recipient: String, body: String) {
 let encodedRecipient = recipient.addingPercentEncoding(withAllowedCharacters: .urlPathAllowed) ?? recipient
 let encodedBody = body.addingPercentEncoding(withAllowedCharacters: .urlQueryAllowed) ?? body
 if let url = URL(string: "sms:\(encodedRecipient)&body=\(encodedBody)") {
 UIApplication.shared.open(url)
 }
}

func openFaceTime(recipient: String) {
 let encoded = recipient.addingPercentEncoding(withAllowedCharacters: .urlPathAllowed) ?? recipient
 if let url = URL(string: "facetime://\(encoded)") {
 UIApplication.shared.open(url)
 }
}

func localContextHints(for message: String) async -> [String: Any] {
 var hints: [String: Any] = [:]
 if isContactResolutionRequest(message) {
 let contacts = await contactHints(in: contactQuery(from: message))
 if !contacts.isEmpty {
 hints["contacts"] = contacts.map(\.dictionary)
 }
 }
 return hints
}

func resetConversationContext() {
 lastMusicQuery = nil
 lastMusicArtist = nil
 lastMusicError = nil
 musicHistory.removeAll()
 lastReminderIdentifier = nil
 lastReminderTitle = nil
 lastReminderDueDate = nil
}

func executeLocalRequest(_ message: String) async -> NativeLocalActionResult? {
 let normalized = message.trimmingCharacters(in: .whitespacesAndNewlines)
 guard !normalized.isEmpty else { return nil }
 let lower = normalized.lowercased()
 if let result = await nativeDiagnostics(for: normalized) {
 return result
 }
 if isPersonalHealthDataRequest(normalized),
 let result = await answerNativeHealthRequest(normalized) {
 return result
 }
 if let result = await updateLastNativeReminder(from: normalized) {
 return result
 }
 if shouldLetBrainHandleFirst(normalized) {
 return nil
 }
 if let result = await openNativeApp(from: normalized) {
 return result
 }
 if lower.contains("uber") || lower.contains("taxi") || lower.contains("ride") {
 return nil
 }

 if let result = await prepareNativeMessage(from: normalized) {
 return result
 }

 if let result = await answerNativeHealthRequest(normalized) {
 return result
 }

 if let result = await createNativeReminder(from: normalized) {
 return result
 }

 if let result = await createNativeCalendarEvent(from: normalized) {
 return result
 }
}

```



```

 if !requiresOnlineMusicResolution(normalized),
 let result = await handleNativeMusicRequest(normalized) {
 return result
 }

 if shouldUseNativePlaceSearch(normalized), let result = await findNativePlace(for: normalized) {
 return result
 }

 return nil
 }

private func shouldLetBrainHandleFirst(_ message: String) -> Bool {
 let lower = message.lowercased()
 let trimmed = lower.trimmingCharacters(in: .whitespacesAndNewlines)
 if requiresOnlineMusicResolution(message) {
 return true
 }
 if isAmbiguousContextReference(trimmed) {
 return true
 }
 if lower.contains("last song i asked")
 || lower.contains("last track i asked")
 || lower.contains("song i asked you")
 || lower.contains("track i asked you")
 || lower.contains("thing i asked you to play") {
 return true
 }
 if trimmed == "look it up"
 || trimmed == "search it"
 || trimmed == "google it"
 || trimmed == "check online" {
 return true
 }
 if lower.contains("bruh")
 || lower.contains("huh")
 || lower.contains("what do you mean")
 || lower.contains("that is wrong")
 || lower.contains("that's wrong")
 || lower.contains("not what i")
 || lower.contains("i mean")
 || lower.contains("try again") {
 return true
 }
 if isPersonalMemoryStatement(trimmed) {
 return true
 }
 if lower.contains("remind me")
 || lower.contains("create a reminder")
 || lower.contains("set a reminder")
 || lower.contains("text ")
 || lower.contains("message ")
 || lower.contains("call ")
 || lower.contains("facetime")
 || lower.contains("add to calendar")
 || lower.contains("schedule ")
 || lower.contains("calendar event") {
 return false
 }
 if lower.contains("latest")
 || lower.contains("current")
 || lower.contains("right now")
 || lower.contains("today")
 || lower.contains("news")
 || lower.contains("price")
 || lower.contains("popular")
 || lower.contains("best")
 || lower.contains("top")
 || lower.contains("chart")
 || lower.contains("ranking")
 || lower.contains("weather")
 || lower.contains("forecast") {
 return true
 }
 if trimmed.range(of: #"^(who|what|when|where|why|how|is|are|do|does|did|can|could|should|would|will)\b"#, options: .regularExpression) != nil {
 return true
 }
 return false
}

private func isPersonalMemoryStatement(_ trimmed: String) -> Bool {
 if trimmed.hasPrefix("remember my ")
 || trimmed.hasPrefix("remember that my ")
 || trimmed.hasPrefix("save my ")
 || trimmed.contains("my usual ")
 || trimmed.contains("my preferred ")

```

```

 || trimmed.contains("my default ") {
 return true
 }
 if trimmed.range(of: #"^(my|our)\s+[^\?!\]]{2,80}\s+(is|are)\s+[^\?!\]]{2,120}$"#, options: .regularExpression) != nil {
 return true
 }
 return false
}

private func isPersonalHealthDataRequest(_ message: String) -> Bool {
 let lower = message.lowercased()
 let hasHealthTerm = lower.contains("health")
 || lower.contains("steps")
 || lower.contains("step count")
 || lower.contains("heart rate")
 || lower.contains("bpm")
 || lower.contains("resting heart")
 || lower.contains("sleep")
 || lower.contains("slept")
 || lower.contains("workout")
 || lower.contains("workouts")
 || lower.contains("exercise")
 guard hasHealthTerm else { return false }

 if lower.contains("my ")
 || lower.contains("i ")
 || lower.contains("i've")
 || lower.contains("ive ")
 || lower.contains("me ")
 || lower.contains("today")
 || lower.contains("yesterday")
 || lower.contains("last night")
 || lower.contains("this week")
 || lower.contains("latest")
 || lower.contains("current")
 || lower.contains("check health")
 || lower.contains("health snapshot")
 || lower.contains("health data") {
 return true
 }

 return false
}

private func isAmbiguousContextReference(_ trimmed: String) -> Bool {
 if [
 "it", "that", "this", "there", "same", "again", "do it", "do that",
 "book it", "book that", "open it", "open that", "send it",
 "play it", "play that", "play this", "play that one", "play this one",
 "that one", "this one", "the other one", "last one"
].contains(trimmed) {
 return true
 }
 if trimmed.contains("what about")
 || trimmed.contains("no i mean")
 || trimmed.contains("nah i mean")
 || trimmed.contains("actually i mean")
 || trimmed.contains("is that right")
 || trimmed.contains("are you sure")
 || trimmed.contains("send it to")
 || trimmed.contains("open the nearest one")
 || trimmed.contains("book that") {
 return true
 }
 return false
}

func playResolvedMusicQuery(_ query: String) async -> NativeLocalActionResult? {
 let cleaned = query.trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
 guard !cleaned.isEmpty else { return nil }
 return await playNativeMusicQuery(cleaned)
}

private func nativeDiagnostics(for message: String) async -> NativeLocalActionResult? {
 let lower = message.lowercased()
 guard lower.contains("diagnose oxy")
 || lower.contains("native diagnostics")

 || lower.contains("oxy diagnostics")
 || lower.contains("why is oxy not working") else { return nil }

 let contactsStatus = CNContactStore.authorizationStatus(for: .contacts)
 let musicStatus = MusicAuthorization.currentStatus
 let canSendText = MFMComposeViewController.canSendText()
 var lines = [
 "Contacts: \(authorizationLabel(contactsStatus))",
 "Can send iMessage/SMS: \(canSendText ? "yes" : "no")",
]
}

```

```

 "MusicKit: \(musicAuthorizationLabel(musicStatus))",
 "Apple Music app installed: \(UIApplication.shared.canOpenURL(URL(string: "music://")!) ? "yes" : "no")"
]
 if musicStatus == .authorized {
 do {
 let subscription = try await MusicSubscription.current
 lines.append("Can play Apple Music catalog: \(subscription.canPlayCatalogContent ? "yes" : "no")")
 lines.append("Cloud library enabled: \(subscription.hasCloudLibraryEnabled ? "yes" : "no")")
 } catch {
 lines.append("Music subscription check failed: \(String(describing: error))")
 }
 }
 if let lastMusicError, !lastMusicError.isEmpty {
 lines.append("Last music error: \(lastMusicError)")
 }
 return NativeLocalActionResult(
 action: "native_diagnostics",
 text: lines.joined(separator: "\n"),
 cardText: lines.joined(separator: "\n"),
 actionSummary: "Native diagnostics",
 deepLink: nil
)
}

private func authorizationLabel(_ status: CNAuthorizationStatus) -> String {
 switch status {
 case .authorized: return "authorized"
 case .denied: return "denied"
 case .restricted: return "restricted"
 case .notDetermined: return "not requested"
 case .limited: return "limited"
 @unknown default: return "unknown"
 }
}

private func musicAuthorizationLabel(_ status: MusicAuthorization.Status) -> String {
 switch status {
 case .authorized: return "authorized"
 case .denied: return "denied"
 case .restricted: return "restricted"
 case .notDetermined: return "not requested"
 @unknown default: return "unknown"
 }
}

private func prepareNativeMessage(from message: String) async -> NativeLocalActionResult? {
 guard let parsed = parseMessageRequest(message) else { return nil }
 let contacts = await contactHints(in: parsed.contact)
 let matched = contacts.first
 let recipient = matched?.phone ?? matched?.email ?? normalizedMessageAddress(parsed.contact)
 guard let recipient, !recipient.isEmpty else {
 return NativeLocalActionResult(
 action: "send_message",
 text: "I need a phone number for \(parsed.contact). Turn on Contacts access for Oxy or include the number.",
 cardText: "No phone number found for \(parsed.contact)",
 actionSummary: "Message needs contact",
 deepLink: nil,
 success: false,
 error: "No message address found."
)
 }

 let label = matched?.displayName ?? parsed.contact
 let encodedRecipient = recipient.addingPercentEncoding(withAllowedCharacters: .urlPathAllowed) ?? recipient
 let encodedBody = parsed.body.addingPercentEncoding(withAllowedCharacters: .urlQueryAllowed) ?? parsed.body
 return NativeLocalActionResult(
 action: "send_message",
 text: "Message ready for \(label). Review and tap Send.",
 cardText: "To \(label) · \(parsed.body)",
 actionSummary: "Message ready",
 deepLink: "sms:\(encodedRecipient)?&body=\(encodedBody)"
)
}

private func parseMessageRequest(_ message: String) -> (contact: String, body: String)? {
 let patterns = [
 #"^(?i)^(?:please\s+)?(?:send\s+)?(?:a\s+)?message\s+to\s+(.+) \s+(?:saying|that says|and say|say) \s+(.+) $"#,
 #"^(?i)^(?:please\s+)?(?:text|message) \s+(.+) \s+(?:saying|that says|and say|say) \s+(.+) $"#,
 #"^(?i)^(?:please\s+)?send\s+(.+) \s+(?:a\s+)?message\s+(?:saying|that says|and say|say) \s+(.+) $"#
]
 for pattern in patterns {
 guard let regex = try? NSRegularExpression(pattern: pattern) else { continue }
 let range = NSRange(message.startIndex...message.endIndex, in: message)
 guard let match = regex.firstMatch(in: message, range: range), match.numberOfRanges >= 3,
 let contactRange = Range(match.range(at: 1), in: message),
 let bodyRange = Range(match.range(at: 2), in: message) else { continue }
 let contact = String(message[contactRange]).trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
 }
}

```

```

 let body = String(message[bodyRange]).trimmingCharacters(in: .whitespacesAndNewlines)
 if !contact.isEmpty, !body.isEmpty {
 return (contact, body)
 }
 }
 return nil
}

private func normalizedMessageAddress(_ text: String) -> String? {
 let trimmed = text.trimmingCharacters(in: .whitespacesAndNewlines)
 if trimmed.range(of: #"^\\+?[0-9][0-9\\s().-]{5,}$"#, options: .regularExpression) != nil {
 return trimmed
 }
 if trimmed.range(of: #"^[^\\s]+@[^\\s]+\\.^[^\\s]+$"#, options: .regularExpression) != nil {
 return trimmed
 }
 return nil
}

private func openNativeApp(from message: String) async -> NativeLocalActionResult? {
 guard let target = requestedAppName(from: message),
 !isContextualOpenTarget(target) else { return nil }

 if let app = nativeAppShortcuts().first(where: { shortcut in
 shortcut.aliases.contains { alias in
 target == alias || target == "\\(alias) app"
 }
 }) {
 let link = bestEffortAppOpenLink(for: target)
 guard let link else {
 return NativeLocalActionResult(
 action: "open_app",
 text: "I couldn't open \\(app.displayName) from here.",
 cardText: app.displayName,
 actionSummary: "App unavailable",
 deepLink: nil,
 success: false,
 error: "\\(app.displayName) URL scheme is unavailable."
)
 }

 return NativeLocalActionResult(
 action: "open_app",
 text: app.sensitive ? "Open \\(app.displayName)? Say yes to continue." : "Opening \\(app.displayName).",
 cardText: app.displayName,
 actionSummary: app.sensitive ? "Confirm app open" : "App opening",
 deepLink: link,
 risk: app.sensitive ? "sensitive_app" : nil,
 confirmation: app.sensitive ? "open_app_confirmation" : nil
)
 }

 let displayName = titleCasedAppName(target)
 // iOS has no public "open any installed app by name" API, so unknown apps use a best-effort launch handoff.
 return NativeLocalActionResult(
 action: "open_app",
 text: "Opening the best app match for \\(displayName).",
 cardText: displayName,
 actionSummary: "App opening",
 deepLink: bestEffortAppOpenLink(for: target)
)
}

private func requestedAppName(from message: String) -> String? {
 let lower = message.lowercased().trimmingCharacters(in: .whitespacesAndNewlines)
 guard lower.range(of: #"^(open|launch|start)\\s+"#, options: .regularExpression) != nil else { return nil }
 let target = lower
 .replacingOccurrences(of: #"^(open|launch|start)\\s+(the\\s+)?"#, with: "", options: .regularExpression)
 .replacingOccurrences(of: #"\\s+for\\s+me$"#, with: "", options: .regularExpression)
 .replacingOccurrences(of: #"\\s+please$"#, with: "", options: .regularExpression)
 .trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
 guard !target.isEmpty, target.count <= 40 else { return nil }
 return target
}

private func isContextualOpenTarget(_ target: String) -> Bool {
 let trimmed = target.trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
 if [
 "it", "that", "this", "there", "same", "again", "that one", "this one",
 "the other one", "last one", "nearest one", "nearest", "the nearest one",
 "the nearest", "nearby one", "the nearby one"
].contains(trimmed) {
 return true
 }
 return trimmed.contains("what about")
 || trimmed.contains("no i mean")
}

```

```

 || trimmed.contains("actually")
 }

private func bestEffortAppOpenLink(for target: String) -> String? {
 var components = URLComponents()
 components.scheme = "oxy-open-app"
 components.host = "open"
 components.queryItems = [URLQueryItem(name: "name", value: target)]
 return components.url?.absoluteString
}

func openBestEffortApp(from url: URL) async {
 guard url.scheme == "oxy-open-app" else {
 _ = await openURLIfAccepted(url)
 return
 }
 let components = URLComponents(url: url, resolvingAgainstBaseURL: false)
 guard let target = components?.queryItems?.first(where: { $0.name == "name" })?.value,
 !target.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty else { return }
 await openBestEffortApp(named: target)
}

private func openBestEffortApp(named rawTarget: String) async {
 let target = rawTarget.trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
 guard !target.isEmpty else { return }

 if let known = nativeAppShortcuts().first(where: { shortcut in
 shortcut.aliases.contains { alias in
 target == alias || target == "\(alias) app"
 }
 }) {
 for link in known.universallinks {
 if let url = URL(string: link), await openUniversalLinkIfAssociated(url) { return }
 }
 for scheme in known.schemes {
 if let url = URL(string: scheme), await openURLIfAccepted(url) { return }
 }
 if let fallback = known.fallbackURL, let url = URL(string: fallback) {
 _ = await openURLIfAccepted(url)
 }
 return
 }

 for scheme in generatedAppSchemeCandidates(for: target) {
 if let url = URL(string: scheme), await openURLIfAccepted(url) { return }
 }

 if let app = try? await searchITunesApp(target),
 let appURL = app.storeURL(for: app) ?? URL(string: app.trackViewUrl ?? "") {
 _ = await openURLIfAccepted(appURL)
 return
 }

 if let searchURL = appStoreSearchURL(for: target) {
 _ = await openURLIfAccepted(searchURL)
 }
}

private func openUniversalLinkIfAssociated(_ url: URL) async -> Bool {
 await withCheckedContinuation { continuation in
 UIApplication.shared.open(
 url,
 options: [.universalLinksOnly: true]
) { accepted in
 continuation.resume(returning: accepted)
 }
 }
}

private func openURLIfAccepted(_ url: URL) async -> Bool {
 await withCheckedContinuation { continuation in
 UIApplication.shared.open(url, options: [:]) { accepted in
 continuation.resume(returning: accepted)
 }
 }
}

private func generatedAppSchemeCandidates(for target: String) -> [String] {
 let words = target
 .folding(options: [.caseInsensitive, .diacriticInsensitive], locale: nil)
 .lowercased()
 .components(separatedBy: CharacterSet.alphanumerics.inverted)
 .filter { !$0.isEmpty && $0 != "app" }
 guard !words.isEmpty else { return [] }
 let compact = words.joined()
 let dashed = words.joined(separator: "-")
 let underscored = words.joined(separator: "_")

```

```

let candidates = [
 "\\(compact)://",
 "\\(dashed)://",
 "\\(underscored)://",
 "\\(compact)app://",
 "\\(compact)-app://"
]

var seen = Set<String>()
return candidates.filter { seen.insert($0).inserted }
}

private fun titleCasedAppName(_ target: String) -> String {
 target
 .split(separator: " ")
 .map { word in
 let lower = word.lowercased()
 return lower.prefix(1).uppercased() + String(lower.dropFirst())
 }
 .joined(separator: " ")
}

private fun nativeAppShortcuts() -> [NativeAppShortcut] {
 [
 NativeAppShortcut(displayName: "TikTok", aliases: ["tiktok", "tik tok"], universalLinks: ["https://www.tiktok.com/"], schemes: ["tiktok"]),
 NativeAppShortcut(displayName: "Instagram", aliases: ["instagram", "insta"], universalLinks: ["https://www.instagram.com/"], schemes: ["instagram"]),
 NativeAppShortcut(displayName: "YouTube", aliases: ["youtube", "you tube"], universalLinks: ["https://www.youtube.com/"], schemes: ["youtube"]),
 NativeAppShortcut(displayName: "WhatsApp", aliases: ["whatsapp", "whats app"], universalLinks: ["https://wa.me/"], schemes: ["whatsapp"]),
 NativeAppShortcut(displayName: "Chrome", aliases: ["chrome", "google chrome"], schemes: ["googlechrome://", "googlechromes://"], fallbackURL: "https://www.google.com/chrome"),
 NativeAppShortcut(displayName: "Spotify", aliases: ["spotify"], universalLinks: ["https://open.spotify.com/"], schemes: ["spotify://"]),
 NativeAppShortcut(displayName: "Apple Music", aliases: ["apple music", "music"], universalLinks: ["https://music.apple.com/"], schemes: ["apple music"]),
 NativeAppShortcut(displayName: "Uber", aliases: ["uber"], schemes: ["uber://"], fallbackURL: "https://m.uber.com/ul/"),
 NativeAppShortcut(displayName: "Netflix", aliases: ["netflix"], universalLinks: ["https://www.netflix.com/"], schemes: ["netflix://"]),
 NativeAppShortcut(displayName: "Telegram", aliases: ["telegram"], universalLinks: ["https://t.me/"], schemes: ["tg://"], fallbackURL: "https://www.telegram.com"),
 NativeAppShortcut(displayName: "Deliveroo", aliases: ["deliveroo"], universalLinks: ["https://deliveroo.co.uk/"], schemes: ["deliveroo"]),
 NativeAppShortcut(displayName: "Trainline", aliases: ["trainline", "the trainline"], universalLinks: ["https://www.thetrainline.com/"], schemes: ["trainline"]),
 NativeAppShortcut(displayName: "Monzo", aliases: ["monzo"], universalLinks: ["https://monzo.com/"], schemes: ["monzo://"], fallbackURL: "https://www.monzo.com"),
 NativeAppShortcut(displayName: "Revolut", aliases: ["revolut"], universalLinks: ["https://www.revolut.com/"], schemes: ["revolut://"]),
 NativeAppShortcut(displayName: "Starling", aliases: ["starling", "starling bank"], universalLinks: ["https://www.starlingbank.com/"], schemes: ["starling"]),
 NativeAppShortcut(displayName: "Maps", aliases: ["maps", "apple maps"], universalLinks: ["https://maps.apple.com/"], schemes: ["maps"])
]
}

private fun handleNativeMusicRequest(_ message: String) async -> NativeLocalActionResult? {
 let lower = message.lowercased()
 guard lower.contains("music") else { return nil }

 || lower.contains("song")
 || lower.contains("playlist")
 || lower.contains("album")
 || lower == "pause"
 || lower.hasPrefix("pause ")
 || lower == "resume"
 || lower.hasPrefix("resume ")
 || lower == "resume it"
 || lower == "unpause"
 || lower.hasPrefix("unpause ")
 || lower == "next"
 || lower == "previous"
 || lower == "back"
 || lower.contains("pause playback")
 || lower.contains("resume playback")
 || lower.contains("skip")
 || lower.hasPrefix("play ")
 || lower.hasPrefix("listen to ")
 || isMusicAddRequest(lower) else { return nil }

 if let result = await handleMusicTransportCommand(lower) {
 return result
 }

 if lower.contains("playlist") || isMusicAddRequest(lower) {
 return await addNativeMusicItem(from: message)
 }
 return await playNativeMusic(from: message)
}

private fun isMusicAddRequest(_ lower: String) -> Bool {
 guard lower.hasPrefix("add ") else { return false }
 return lower.contains("playlist")
 || lower.contains("library")
 || lower.contains("music")
 || lower.contains("song")
 || lower.contains("track")
 || lower.contains("album")
 || lower.contains("apple music")
}

private fun requiresOnlineMusicResolution(_ message: String) -> Bool {

```

```

 let lower = message.lowercased()
 guard lower.contains("play") || lower.contains("song") || lower.contains("music") || lower.contains("track") else {
 return false
 }

 return lower.contains("billboard")
 || lower.contains("hot 100")
 || lower.contains("chart")
 || lower.contains("number one")
 || lower.contains("no. 1")
 || lower.contains("top song")
 || lower.contains("top track")
 || lower.contains("most popular")
 || lower.contains("most streamed")
 || lower.contains("viral")
 || lower.contains("right now")
 || lower.contains("currently")
 || lower.contains("today")
 || lower.contains("latest")
 || lower.contains("trending")
}

private func requestMusicPermission() async -> Bool {
 switch MusicAuthorization.currentStatus {
 case .authorized:
 return true
 case .notDetermined:
 return await MusicAuthorization.request() == .authorized
 default:
 return false
 }
}

private func handleMusicTransportCommand(_ lower: String) async -> NativeLocalActionResult? {
 let systemPlayer = MPMusicPlayerController.systemMusicPlayer
 let appPlayer = MPMusicPlayerController.applicationMusicPlayer

 if lower == "previous" || lower == "back" || lower.contains("previous song") || lower.contains("last track") {
 try? await ApplicationMusicPlayer.shared.skipToPreviousEntry()
 try? await SystemMusicPlayer.shared.skipToPreviousEntry()
 systemPlayer.skipToPreviousItem()
 appPlayer.skipToPreviousItem()
 return NativeLocalActionResult(
 action: "music_control",
 text: "Going back.",
 cardText: "Previous track",
 actionSummary: "Previous track",
 deepLink: nil
)
 }

 guard !lower.hasPrefix("play ") && !lower.hasPrefix("listen to ") else { return nil }

 if lower == "pause" || lower.hasPrefix("pause ") || lower.contains("pause music") || lower.contains("pause playback") {
 ApplicationMusicPlayer.shared.pause()
 SystemMusicPlayer.shared.pause()
 systemPlayer.pause()
 appPlayer.pause()
 return NativeLocalActionResult(
 action: "music_control",
 text: "Paused.",
 cardText: "Playback paused",
 actionSummary: "Music paused",
 deepLink: nil
)
 }

 if lower == "resume"
 || lower == "resume it"
 || lower == "play"
 || lower == "unpause"
 || lower.hasPrefix("resume ")
 || lower.hasPrefix("unpause ")
 || lower.contains("resume music")
 || lower.contains("resume playback")
 || lower.contains("unpause music")
 || lower.contains("unpause playback") {
 try? await ApplicationMusicPlayer.shared.play()
 try? await SystemMusicPlayer.shared.play()
 systemPlayer.play()
 appPlayer.play()
 return NativeLocalActionResult(
 action: "music_control",
 text: "Resumed.",
 cardText: "Playback resumed",
 actionSummary: "Music resumed",
 deepLink: nil
)
 }
}

```

```

)
}

if lower == "next"
 || lower == "skip"
 || lower == "skip it"
 || lower == "skip track"
 || lower == "skip the track"
 || lower == "skip song"
 || lower == "skip the song"
 || lower.contains("next song")
 || lower.contains("next track")
 || lower.contains("skip this")
 || lower.contains("skip song")
 || lower.contains("skip track") {
 try? await ApplicationMusicPlayer.shared.skipToNextEntry()
 try? await SystemMusicPlayer.shared.skipToNextEntry()
 systemPlayer.skipToNextItem()
 appPlayer.skipToNextItem()
 return NativeLocalActionResult(
 action: "music_control",
 text: "Skipped.",
 cardText: "Next track",
 actionSummary: "Music skipped",
 deepLink: nil
)
}

return nil
}

private func playNativeMusic(from message: String) async -> NativeLocalActionResult? {
 if isAlbumPlaybackRequest(message) {
 let query = resolvedAlbumQuery(from: message)
 guard !query.isEmpty else { return nil }
 return await playNativeAlbumQuery(query)
 }
 let query = resolvedMusicQuery(from: message)

 guard !query.isEmpty else { return nil }
 return await playNativeMusicQuery(query)
}

private func playNativeAlbumQuery(_ query: String) async -> NativeLocalActionResult? {
 guard await requestMusicPermission() else {
 return NativeLocalActionResult(
 action: "play_music",
 text: "Turn on Apple Music access and I can play that album natively.",
 cardText: "Enable Apple Music access",
 actionSummary: "Music needs access",
 deepLink: nil,
 success: false,
 error: "Apple Music permission is not authorized."
)
 }
}

var failures: [String] = []
for candidate in albumSearchCandidates(for: query) {
 do {
 if let iTunesAlbum = try await withMusicTimeout(seconds: 4, operation: { try await self.searchITunesAlbum(candidate, originalQuery: query) }) {
 let collectionId = iTunesAlbum.collectionId {
 try await playWithStoreID(String(collectionId))
 }
 let title = iTunesAlbum.collectionName ?? query
 let artist = iTunesAlbum.artistName ?? "Apple Music"
 recordPlayedMusic("\(title) \(artist)", artist: artist)
 lastMusicError = nil
 return NativeLocalActionResult(
 action: "play_music",
 text: "Playing \(title) by \(artist).",
 cardText: "\(title) · \(artist)",
 actionSummary: "Album playing",
 deepLink: iTunesAlbum.collectionViewUrl
)
 }
 } catch {
 failures.append("iTunes album \(candidate): \(String(describing: error))")
 }
}

do {
 if let album = try await withMusicTimeout(seconds: 4, operation: { try await self.searchCatalogAlbum(candidate, originalQuery: query) }) {
 try await playWithStoreID(album.id.rawValue)
 recordPlayedMusic("\(album.title) \(album.artistName)", artist: album.artistName)
 lastMusicError = nil
 return NativeLocalActionResult(
 action: "play_music",
 text: "Playing \(album.title) by \(album.artistName).",
 cardText: "\(album.title) · \(album.artistName)",

```



```

 actionSummary: "Album playing",
 deepLink: album.url?.absoluteString
)
}
} catch {
 failures.append("MusicKit album \(candidate): \(String(describing: error))")
}
}

do {
 try await playLocalLibraryAlbum(matching: query)
 recordPlayedMusic(query)
 lastMusicError = nil
 return NativeLocalActionResult(
 action: "play_music",
 text: "Playing \(query).",
 cardText: query,
 actionSummary: "Album playing",
 deepLink: nil
)
} catch {
 failures.append("Local album: \(String(describing: error))")
}

let detail = failures.joined(separator: " | ")
lastMusicError = detail
return NativeLocalActionResult(
 action: "play_music",
 text: "I couldn't find that album clearly enough to play it.",
 cardText: query,
 actionSummary: "Album not found",
 deepLink: nil,
 success: false,
 error: detail.isEmpty ? "No album result for \(query)." : detail
)
}

private func playNativeMusicQuery(_ query: String) async -> NativeLocalActionResult? {
 guard await requestMusicPermission() else {
 return NativeLocalActionResult(
 action: "play_music",
 text: "Turn on Apple Music access and I can play that natively.",
 cardText: "Enable Apple Music access",
 actionSummary: "Music needs access",
 deepLink: nil,
 success: false,
 error: "Apple Music permission is not authorized."
)
 }
}

var failures: [String] = []

do {
 if let iTunesSong = try await withMusicTimeout(seconds: 4, operation: { try await self.searchITunesSong(query) }) {
 guard let trackId = iTunesSong.trackId else {
 throw NSError(domain: "OxyMusic", code: 5, userInfo: [NSLocalizedStringKey: "Missing iTunes track ID"])
 }
 try await playWithStoreID(String(trackId))
 let title = iTunesSong.trackName ?? query
 let artist = iTunesSong.artistName ?? "Apple Music"
 recordPlayedMusic("\(title) \(artist)", artist: artist)
 lastMusicError = nil
 return NativeLocalActionResult(
 action: "play_music",
 text: "Playing \(title) by \(artist).",
 cardText: "\(title) · \(artist)",
 actionSummary: "Music playing",
 deepLink: iTunesSong.trackViewUrl
)
 }
} catch {
 failures.append("iTunes store queue: \(String(describing: error))")
}

do {
 if let song = try await withMusicTimeout(seconds: 4, operation: { try await self.searchCatalogSong(query) }) {
 try await playResolvedSong(song, query: query)
 recordPlayedMusic("\(song.title) \(song.artistName)", artist: song.artistName)
 lastMusicError = nil
 return NativeLocalActionResult(
 action: "play_music",
 text: "Playing \(song.title) by \(song.artistName).",
 cardText: "\(song.title) · \(song.artistName)",
 actionSummary: "Music playing",
 deepLink: song.url?.absoluteString
)
 }
}

```

```

 }
} catch {
 failures.append("MusicKit catalog: \(String(describing: error))")
}

do {
 try await playLocalLibraryItem(matching: query)
 recordPlayedMusic(query)
 lastMusicError = nil
 return NativeLocalActionResult(
 action: "play_music",
 text: "Playing \(query).",
 cardText: query,
 actionSummary: "Music playing",
 deepLink: nil
)
} catch {
 failures.append("Local library: \(String(describing: error))")
}

let detail = failures.joined(separator: " | ")
lastMusicError = detail
return NativeLocalActionResult(
 action: "play_music",
 text: "I found the request, but iOS did not start native playback.",
 cardText: query,
 actionSummary: "Music failed",
 deepLink: nil,
 success: false,
 error: detail.isEmpty ? "Native playback failed for \(query)." : detail
)
}

private func playResolvedSong(_ song: Song, query: String) async throws {
 var failures: [String] = []
 do {
 try await playWithMediaPlayer(song)
 return
 } catch {
 failures.append("Store queue: \(String(describing: error))")
 }

 do {
 try await playLocalLibraryItem(matching: "\(song.title) \(song.artistName)")
 return
 } catch {
 failures.append("Local library: \(String(describing: error))")
 }

 do {
 try await playLocalLibraryItem(matching: query)
 return
 } catch {
 failures.append("Local query: \(String(describing: error))")
 }

 throw NSError(
 domain: "OxyMusic",
 code: 2,
 userInfo: [NSLocalizedStringKey: failures.joined(separator: " | ")]
)
}

private func playWithMediaPlayer(_ song: Song) async throws {
 let storeID = song.id.rawValue
 guard !storeID.isEmpty else {
 throw NSError(domain: "OxyMusic", code: 1, userInfo: [NSLocalizedStringKey: "Missing Apple Music store ID"])
 }
 try await playWithStoreID(storeID)
}

private func playWithStoreID(_ storeID: String) async throws {
 guard !storeID.isEmpty, storeID != "0" else {
 throw NSError(domain: "OxyMusic", code: 6, userInfo: [NSLocalizedStringKey: "Missing playable store ID"])
 }
 var failures: [String] = []
 for attempt in [
 ("system music player", MPMusicPlayerController.systemMusicPlayer),
 ("app music player", MPMusicPlayerController.applicationMusicPlayer)
] {
 do {
 try await playStoreID(storeID, player: attempt.1, label: attempt.0)
 return
 } catch {
 failures.append("\(attempt.0): \(String(describing: error))")
 }
 }
}

```

```

 throw NSError(
 domain: "OxyMusic",
 code: 13,
 userInfo: [NSLocalizedStringKey: failures.joined(separator: " | ")]
)
 }

private func playStoreID(_ storeID: String, player: MPMusicPlayerController, label: String) async throws {
 try prepareAudioSessionForMusic()
 let descriptor = MPMusicPlayerStoreQueueDescriptor(storeIDs: [storeID])
 player.setQueue(with: descriptor)
 try await withMusicTimeout(seconds: 5) {
 try await player.prepareToPlay()
 }
 player.play()
 try await withMusicTimeout(seconds: 4) {
 try await self.waitForMediaPlayerToStart(player, label: label)
 }
}

private func waitForMediaPlayerToStart(_ player: MPMusicPlayerController, label: String) async throws {
 for _ in 0..

```

```

 }
}

private func playLocalLibraryItem(matching query: String) async throws {
 let normalizedQuery = normalizeMusicText(query)
 guard !normalizedQuery.isEmpty else {
 throw NSError(domain: "OxyMusic", code: 3, userInfo: [NSLocalizedStringKey: "Empty music query"])
 }
 let mediaQuery = MPMediaQuery.songs()
 let item = mediaQuery.items?.first { item in
 let title = normalizeMusicText(item.title ?? "")
 let artist = normalizeMusicText(item.artist ?? "")
 let combined = normalizeMusicText("\(item.title ?? "") \(item.artist ?? "")")
 return combined.contains(normalizedQuery)
 || normalizedQuery.contains(combined)
 || title.contains(normalizedQuery)
 || normalizedQuery.contains(title)
 || (!artist.isEmpty && normalizedQuery.contains(artist) && !title.isEmpty && normalizedQuery.contains(title))
 }
 guard let item else {
 throw NSError(domain: "OxyMusic", code: 4, userInfo: [NSLocalizedStringKey: "No matching local library item"])
 }
 try prepareAudioSessionForMusic()
 let player = MPMusicPlayerController.applicationMusicPlayer
 let collection = MPMediaItemCollection(items: [item])
 player.setQueue(with: collection)
 player.play()
 try await withMusicTimeout(seconds: 4) {
 try await self.waitForMediaPlayerToStart(player, label: "local library")
 }
}

private func playLocalLibraryAlbum(matching query: String) async throws {
 let normalizedQuery = normalizeMusicText(query)
 guard !normalizedQuery.isEmpty else {
 throw NSError(domain: "OxyMusic", code: 14, userInfo: [NSLocalizedStringKey: "Empty album query"])
 }
 let albumQuery = MPMediaQuery.albums()
 let collection = albumQuery.collections?.first { collection in
 guard let item = collection.representativeItem else { return false }
 let albumTitle = normalizeMusicText(item.albumTitle ?? "")
 let artist = normalizeMusicText(item.albumArtist ?? item.artist ?? "")
 let combined = normalizeMusicText("\(item.albumTitle ?? "") \(item.albumArtist ?? item.artist ?? "")")
 return combined.contains(normalizedQuery)
 || normalizedQuery.contains(combined)
 || albumTitle.contains(normalizedQuery)
 || normalizedQuery.contains(albumTitle)
 || compactMusicText(combined).contains(compactMusicText(normalizedQuery))
 || compactMusicText(albumTitle).contains(compactMusicText(normalizedQuery))
 || (!artist.isEmpty && normalizedQuery.contains(artist) && !albumTitle.isEmpty && normalizedQuery.contains(albumTitle))
 }
 guard let collection else {
 throw NSError(domain: "OxyMusic", code: 15, userInfo: [NSLocalizedStringKey: "No matching local library album"])
 }
 try prepareAudioSessionForMusic()
 let player = MPMusicPlayerController.applicationMusicPlayer
 player.setQueue(with: collection)
 player.play()
 try await withMusicTimeout(seconds: 4) {
 try await self.waitForMediaPlayerToStart(player, label: "local album")
 }
}

private func prepareAudioSessionForMusic() throws {
 let session = AVAudioSession.sharedInstance()
 try session.setCategory(.playback, mode: .default)
 try session.setActive(true)
}

private func searchITunesSong(_ query: String) async throws -> ITunesSong? {
 var components = URLComponents(string: "https://itunes.apple.com/search")
 components?.queryItems = [
 URLQueryItem(name: "term", value: query),
 URLQueryItem(name: "media", value: "music"),
 URLQueryItem(name: "entity", value: "song"),
 URLQueryItem(name: "limit", value: "1"),
 URLQueryItem(name: "country", value: Locale.current.region?.identifier ?? "GB")
]
 guard let url = components?.url else {
 throw NSError(domain: "OxyMusic", code: 7, userInfo: [NSLocalizedStringKey: "Invalid iTunes search URL"])
 }
 let (data, response) = try await withMusicTimeout(seconds: 4) {
 try await URLSession.shared.data(from: url)
 }
 if let http = response as? HTTPURLResponse, !(200..<300).contains(http.statusCode) {
 throw NSError(domain: "OxyMusic", code: http.statusCode, userInfo: [NSLocalizedStringKey: "iTunes search failed with HTTP \(http.statusCode)"])
 }
}

```

```

 }
 let decoded = try JSONDecoder().decode(ITunesSongResult.self, from: data)
 guard decoded.resultCount > 0 else { return nil }
 return decoded.results.first { $0.trackId != nil }
}

private func searchITunesAlbum(_ query: String, originalQuery: String) async throws -> ITunesAlbum? {
 var components = URLComponents(string: "https://itunes.apple.com/search")
 components?.queryItems = [
 URLQueryItem(name: "term", value: query),
 URLQueryItem(name: "media", value: "music"),
 URLQueryItem(name: "entity", value: "album"),
 URLQueryItem(name: "limit", value: "10"),
 URLQueryItem(name: "country", value: Locale.current.region?.identifier ?? "GB")
]
 guard let url = components?.url else {
 throw NSError(domain: "OxyMusic", code: 16, userInfo: [NSLocalizedStringKey: "Invalid iTunes album search URL"])
 }
 let (data, response) = try await withMusicTimeout(seconds: 4) {
 try await URLSession.shared.data(from: url)
 }
 if let http = response as? HTTPURLResponse, !(200..<300).contains(http.statusCode) {
 throw NSError(domain: "OxyMusic", code: http.statusCode, userInfo: [NSLocalizedStringKey: "iTunes album search failed with HTTP"])
 }
 let decoded = try JSONDecoder().decode(ITunesAlbumResult.self, from: data)
 guard decoded.resultCount > 0 else { return nil }
 return decoded.results.first {
 $0.collectionId != nil && iTunesAlbumMatches($0, originalQuery: originalQuery, candidate: query)
 }
}

private func searchITunesApp(_ query: String) async throws -> ITunesApp? {
 var components = URLComponents(string: "https://itunes.apple.com/search")
 components?.queryItems = [
 URLQueryItem(name: "term", value: query),
 URLQueryItem(name: "media", value: "software"),
 URLQueryItem(name: "entity", value: "software"),
 URLQueryItem(name: "limit", value: "5"),
 URLQueryItem(name: "country", value: Locale.current.region?.identifier ?? "GB")
]
 guard let url = components?.url else {
 throw NSError(domain: "OxyApps", code: 1, userInfo: [NSLocalizedStringKey: "Invalid app search URL"])
 }
 let (data, response) = try await URLSession.shared.data(from: url)
 if let http = response as? HTTPURLResponse, !(200..<300).contains(http.statusCode) {
 throw NSError(domain: "OxyApps", code: http.statusCode, userInfo: [NSLocalizedStringKey: "App search failed with HTTP \ \(http.st
)
 let decoded = try JSONDecoder().decode(ITunesAppResult.self, from: data)
 guard decoded.resultCount > 0 else { return nil }
 let normalizedQuery = normalizeAppSearchText(query)
 return decoded.results.first { app in
 guard app.trackId != nil else { return false }
 let name = normalizeAppSearchText(app.trackName ?? "")
 return name == normalizedQuery
 || name.contains(normalizedQuery)
 || normalizedQuery.contains(name)
 } ?? decoded.results.first { $0.trackId != nil }
}

private func appStoreURL(for app: ITunesApp) -> URL? {
 guard let trackId = app.trackId else { return URL(string: app.trackViewUrl ?? "") }
 return URL(string: "itms-apps://itunes.apple.com/app/id\ \(trackId)")
}

private func appStoreSearchURL(for query: String) -> URL? {
 var components = URLComponents(string: "https://apps.apple.com/search")
 components?.queryItems = [
 URLQueryItem(name: "term", value: query)
]
 return components?.url
}

private func normalizeAppSearchText(_ text: String) -> String {
 text
 .folding(options: [.caseInsensitive, .diacriticInsensitive], locale: nil)
 .lowercased()
 .replacingOccurrences(of: #"[^a-z0-9]+"#, with: " ", options: .regularExpression)
 .trimmingCharacters(in: .whitespacesAndNewlines)
}

private func musicPlaybackErrorMessage(_ error: Error) -> String {
 let details = String(describing: error)
 if details.localizedCaseInsensitiveContains("No matching local library item") {
 return "I found the song, but iOS would not start Apple Music playback. Add it to your library or try another song."
 }
 if details.localizedCaseInsensitiveContains("playback state") {

```

```

 return "I found the song, but iOS did not actually start native playback."
 }
 return "I found the song, but iOS would not start native playback."
}

private func musicAddErrorMessage(_ error: Error) -> String {
 let details = String(describing: error)
 if details.localizedCaseInsensitiveContains("Playlist not found") {
 return "I couldn't find that playlist in your Apple Music library."
 }
 if details.localizedCaseInsensitiveContains("developerTokenRequestFailed") {
 return "Apple Music would not allow playlist editing through MusicKit yet, but playback still works."
 }
 return "Apple Music couldn't add that yet."
}

private func addNativeMusicItem(from message: String) async -> NativeLocalActionResult? {
 guard await requestMusicPermission() else {
 return NativeLocalActionResult(
 action: "add_to_music_playlist",
 text: "Turn on Apple Music access and I can add music natively.",
 cardText: "Enable Apple Music access",
 actionSummary: "Music needs access",
 deepLink: "music://",
 success: false,
 error: "Apple Music permission is not authorized."
)
 }

 let parsedRequest = parseMusicAddRequest(message)
 let parsed = (
 query: resolvedMusicReference(parsedRequest.query),
 playlistName: parsedRequest.playlistName
)
 guard !parsed.query.isEmpty else { return nil }

 do {
 if let song = try? await searchCatalogSong(parsed.query) {
 if let playlistName = parsed.playlistName, !playlistName.isEmpty {
 guard let playlist = try await findLibraryPlaylist(named: playlistName) else {
 return NativeLocalActionResult(
 action: "add_to_music_playlist",
 text: "I found \(song.title), but couldn't find a playlist called \(playlistName).",
 cardText: "\(song.title) · Missing playlist: \(playlistName)",
 actionSummary: "Playlist not found",
 deepLink: "music://",
 success: false,
 error: "Playlist not found: \(playlistName)."
)
 }
 try await MusicLibrary.shared.add(song, to: playlist)
 return NativeLocalActionResult(
 action: "add_to_music_playlist",
 text: "Added \(song.title) by \(song.artistName) to \(playlist.name).",
 cardText: "\(song.title) · \(playlist.name)",
 actionSummary: "Added to playlist",
 deepLink: playlist.url?.absoluteString ?? song.url?.absoluteString ?? "music://"
)
 }

 try await MusicLibrary.shared.add(song)
 return NativeLocalActionResult(
 action: "add_to_music_playlist",
 text: "Added \(song.title) by \(song.artistName) to your Apple Music library.",
 cardText: "\(song.title) · Library",
 actionSummary: "Added to library",
 deepLink: song.url?.absoluteString ?? "music://"
)
 }

 guard let iTunesSong = try await searchITunesSong(parsed.query),
 let trackId = iTunesSong.trackId else {
 return NativeLocalActionResult(
 action: "add_to_music_playlist",
 text: "I couldn't find \(parsed.query) in Apple Music.",
 cardText: parsed.query,
 actionSummary: "Music not found",
 deepLink: "music://",
 success: false,
 error: "No Apple Music or iTunes result for \(parsed.query)."
)
 }

 let title = iTunesSong.trackName ?? parsed.query
 let artist = iTunesSong.artistName ?? "Apple Music"
 let storeID = String(trackId)
 if let playlistName = parsed.playlistName, !playlistName.isEmpty {

```

```

 let addedPlaylistName = try await addStoreID(storeID, toPlaylistNamed: playlistName)
 return NativeLocalActionResult(
 action: "add_to_music_playlist",
 text: "Added \((title) by \((artist) to \((addedPlaylistName).",
 cardText: "\((title) · \((addedPlaylistName)",
 actionSummary: "Added to playlist",
 deepLink: iTunesSong.trackViewUrl ?? "music://"
)
 }

 try await addStoreIDToLibrary(storeID)
 return NativeLocalActionResult(
 action: "add_to_music_playlist",
 text: "Added \((title) by \((artist) to your Apple Music library.",
 cardText: "\((title) · Library",
 actionSummary: "Added to library",
 deepLink: iTunesSong.trackViewUrl ?? "music://"
)
} catch {
 return NativeLocalActionResult(
 action: "add_to_music_playlist",
 text: musicAddErrorMessage(error),
 cardText: parsed.query,
 actionSummary: "Music add failed",
 deepLink: nil,
 success: false,
 error: String(describing: error)
)
}
}

private func searchCatalogSong(_ query: String) async throws -> Song? {
 var request = MusicCatalogSearchRequest(term: query, types: [Song.self])

 request.limit = 1
 let response = try await request.response()
 return response.songs.first
}

private func searchCatalogAlbum(_ query: String, originalQuery: String) async throws -> Album? {
 var request = MusicCatalogSearchRequest(term: query, types: [Album.self])
 request.limit = 5
 let response = try await request.response()
 return response.albums.first { catalogAlbumMatches($0, originalQuery: originalQuery, candidate: query) }
}

private func withMusicTimeout<T>(seconds: Double, operation: @escaping () async throws -> T) async throws -> T {
 try await withThrowingTaskGroup(of: T.self) { group in
 group.addTask {
 try await operation()
 }
 group.addTask {
 try await Task.sleep(for: .seconds(seconds))
 throw NSError(
 domain: "OxyMusic",
 code: 408,
 userInfo: [NSLocalizedStringKey: "Apple Music timed out."]
)
 }
 guard let value = try await group.next() else {
 throw NSError(
 domain: "OxyMusic",
 code: 408,
 userInfo: [NSLocalizedStringKey: "Apple Music timed out."]
)
 }
 group.cancelAll()
 return value
 }
}

private func findLibraryPlaylist(named name: String) async throws -> Playlist? {
 var request = MusicLibraryRequest<Playlist>()
 request.limit = 100
 let response = try await request.response()
 let normalized = normalizeMusicText(name)
 return response.items.first { playlist in
 let playlistName = normalizeMusicText(playlist.name)
 return playlistName == normalized || playlistName.contains(normalized) || normalized.contains(playlistName)
 }
}

private func cleanMusicQuery(_ message: String) -> String {
 var text = message
 let patterns = [
 #"(?i)^play\s+"#,
 #"(?i)^listen to\s+"#,
]

```

```

 #"(?i)\bon apple music\b"#,
 #"(?i)\bin apple music\b"#,
 #"(?i)\bthe song\b"#,
 #"(?i)\bsong\b"#,
 #"(?i)\bmusic\b"#
]
 for pattern in patterns {
 text = text.replacingOccurrences(of: pattern, with: " ", options: .regularExpression)
 }
 return text.replacingOccurrences(of: "\\s+", with: " ", options: .regularExpression)
 .trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
}

private func cleanAlbumQuery(_ message: String) -> String {
 cleanMusicQuery(message)
 .replacingOccurrences(of: #"(?i)\bthe album\b"#, with: " ", options: .regularExpression)
 .replacingOccurrences(of: #"(?i)\balbum\b"#, with: " ", options: .regularExpression)
 .replacingOccurrences(of: #"(?i)\brecord\b"#, with: " ", options: .regularExpression)
 .replacingOccurrences(of: "\\s+", with: " ", options: .regularExpression)
 .trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
}

private func resolvedMusicQuery(from message: String) -> String {
 let query = cleanMusicQuery(message)
 return resolvedMusicReference(query)
}

private func resolvedAlbumQuery(from message: String) -> String {
 let query = cleanAlbumQuery(message)
 return resolvedMusicReference(query)
}

private func resolvedMusicReference(_ query: String) -> String {
 let normalized = normalizeMusicText(query)
 if isPreviousMusicReference(normalized), let previous = previousMusicQuery {
 return previous
 }
 if isLastMusicReference(normalized), let lastMusicQuery, !lastMusicQuery.isEmpty {
 return lastMusicQuery
 }
 if ["it", "that", "this", "that one", "this one"].contains(normalized),
 let lastMusicQuery,
 !lastMusicQuery.isEmpty {
 return lastMusicQuery
 }
 return query
}

private func isAlbumPlaybackRequest(_ message: String) -> Bool {
 let lower = message.lowercased()
 guard lower.hasPrefix("play ") || lower.hasPrefix("listen to ") else { return false }
 return lower.contains(" album") || lower.contains("album ") || lower.contains(" record") || lower.contains("record ")
}

private func albumSearchCandidates(for query: String) -> [String] {
 var candidates: [String] = []
 let normalized = normalizeMusicText(query)
 if let lastMusicArtist,
 !lastMusicArtist.isEmpty,
 !normalized.contains(normalizeMusicText(lastMusicArtist)),
 query.range(of: #"\\b(by|from)\\b"#, options: .regularExpression) == nil {
 candidates.append("\(query) \(lastMusicArtist)")
 }
 candidates.append(query)
 var seen = Set<String>()
 return candidates.filter { candidate in
 let key = normalizeMusicText(candidate)

 guard !key.isEmpty, !seen.contains(key) else { return false }
 seen.insert(key)
 return true
 }
}

private var previousMusicQuery: String? {
 guard musicHistory.count >= 2 else { return nil }
 return musicHistory[musicHistory.count - 2]
}

private func recordPlayedMusic(_ query: String, artist: String? = nil) {
 let trimmed = query.trimmingCharacters(in: .whitespacesAndNewlines)
 guard !trimmed.isEmpty else { return }
 lastMusicQuery = trimmed
 if let artist = artist?.trimmingCharacters(in: .whitespacesAndNewlines), !artist.isEmpty {
 lastMusicArtist = artist
 }
 if musicHistory.last != trimmed {

```



```

 musicHistory.append(trimmed)
 }
 if musicHistory.count > 12 {
 musicHistory.removeFirst(musicHistory.count - 12)
 }
}

private func isLastMusicReference(_ normalized: String) -> Bool {
 ["last", "last song", "last again", "last song again", "the last", "the last again", "the last song", "the last song again", "same", "sa
}

private func isPreviousMusicReference(_ normalized: String) -> Bool {
 ["previous", "previous song", "previous again", "previous song again", "the previous", "the previous song", "the previous song again"].c
}

private func parseMusicAddRequest(_ message: String) -> (query: String, playlistName: String?) {
 var text = message
 .replacingOccurrences(of: #"(?i)^add\s+"#, with: "", options: .regularExpression)
 .replacingOccurrences(of: #"(?i)\bthe song\b"#, with: " ", options: .regularExpression)
 .replacingOccurrences(of: #"(?i)\bon apple music\b"#, with: " ", options: .regularExpression)
 .replacingOccurrences(of: #"(?i)\bin apple music\b"#, with: " ", options: .regularExpression)

 if let range = text.range(of: #"(?i)\s+to\s+(my\s+)?playlist\s+"#, options: .regularExpression) {
 let query = String(text[..

```

```

guard isPersonalHealthDataRequest(message) else { return nil }

guard HKHealthStore.isHealthDataAvailable() else {
 return NativeLocalActionResult(
 action: "check_health",
 text: "Health data is not available on this device.",
 cardText: "HealthKit unavailable",
 actionSummary: "Health unavailable",
 deeplink: nil
)
}

await requestHealthPermission()
let snapshot = await healthSnapshot()

if lower.contains("steps") || lower.contains("step count") {
 let range = healthDateRange(from: lower)
 guard let steps = await stepCount(start: range.start, end: range.end) else {
 return noHealthDataResult("steps \(range.label)")
 }
 let count = Int(steps.rounded())
 return NativeLocalActionResult(
 action: "check_health",
 text: "Apple Health shows \(count.formatted()) steps \(range.label).",
 cardText: "\(count.formatted()) steps \(range.label)",
 actionSummary: "Steps checked",
 deeplink: "x-apple-health://"
)
}

if lower.contains("sleep") || lower.contains("slept") {
 guard let minutes = snapshot.sleepMinutesLastNight else {
 return noHealthDataResult("sleep last night")
 }
 let formatted = formatDurationMinutes(minutes)
 return NativeLocalActionResult(
 action: "check_health",
 text: "Apple Health shows \(formatted) of sleep for your latest overnight sleep window.",
 cardText: "\(formatted) sleep",
 actionSummary: "Sleep checked",
 deeplink: "x-apple-health://"
)
}

if lower.contains("heart rate") || lower.contains("bpm") || lower.contains("resting heart") {
 let latest = snapshot.latestHeartRate.map { "\(Int($0.rounded())) bpm" }
 let resting = snapshot.restingHeartRate.map { "resting \(Int($0.rounded())) bpm" }
 let parts = [latest, resting].compactMap { $0 }
 guard !parts.isEmpty else {
 return noHealthDataResult("heart rate")
 }
 return NativeLocalActionResult(
 action: "check_health",
 text: "Your latest Health heart data is \(parts.joined(separator: ", ")).",
 cardText: parts.joined(separator: " · "),
 actionSummary: "Heart checked",
 deeplink: "x-apple-health://"
)
}

if lower.contains("workout") || lower.contains("workouts") || lower.contains("exercise") {
 let workouts = snapshot.recentWorkouts ?? []
 guard !workouts.isEmpty else {
 return noHealthDataResult("recent workouts")
 }
 let first = workouts[0]
 let text = "Your latest workout was \(first.activity.lowercased()) for \(formatDurationMinutes(first.durationMinutes))."
 let detail = workouts.prefix(3).map(formatWorkoutSummary).joined(separator: "\n")
 return NativeLocalActionResult(
 action: "check_health",
 text: text,
 cardText: detail,
 actionSummary: "Workouts checked",
 deeplink: "x-apple-health://"
)
}

let parts = [
 snapshot.stepCountToday.map { "\(Int($0.rounded()).formatted()) steps today" },
 snapshot.sleepMinutesLastNight.map { "\(formatDurationMinutes($0)) sleep" },
 snapshot.latestHeartRate.map { "\(Int($0.rounded())) bpm latest heart rate" },
 snapshot.recentWorkouts?.first.map { "latest workout: \($0.activity.lowercased(), \(formatDurationMinutes($0.durationMinutes))" }
].compactMap { $0 }
guard !parts.isEmpty else {
 return noHealthDataResult("Health")
}
return NativeLocalActionResult(

```

```

 action: "check_health",
 text: "Here is your latest Health snapshot: \(parts.joined(separator: "; ").",
 cardText: parts.joined(separator: "\n"),
 actionSummary: "Health checked",
 deepLink: "x-apple-health://"
)
}

private func noHealthDataResult(_ label: String) -> NativeLocalActionResult {
 NativeLocalActionResult(
 action: "check_health",
 text: "I could not read \(label) from Health yet. Open Health permissions for Oxy, then try again.",
 cardText: "Enable Health access for Oxy",
 actionSummary: "Health needs access",
 deepLink: "x-apple-health://"
)
}

private func requestHealthPermission() async {
 guard HKHealthStore.isHealthDataAvailable() else { return }
 let identifiers: [HKQuantityTypeIdentifier] = [.heartRate, .restingHeartRate, .stepCount]
 let quantityTypes = identifiers.compactMap { HKObjectType.quantityType(forIdentifier: $0) }
 let sleepType = HKObjectType.categoryType(forIdentifier: .sleepAnalysis)
 let workoutType = HKObjectType.workoutType()
 var readTypes = Set<HKObjectType>(quantityTypes)
 if let sleepType { readTypes.insert(sleepType) }
 readTypes.insert(workoutType)
 do {
 try await healthStore.requestAuthorization(toShare: [], read: readTypes)
 } catch {}
}

private func requestContactsPermission() async {
 _ = try? await contactStore.requestAccess(for: .contacts)
}

private func requestReminderPermission() async {
 if #available(iOS 17.0, *) {
 _ = try? await eventStore.requestFullAccessToReminders()
 } else {
 _ = try? await eventStore.requestAccess(to: .reminder)
 }
}

private func requestCalendarPermission() async {
 if #available(iOS 17.0, *) {
 _ = try? await eventStore.requestFullAccessToEvents()
 } else {
 _ = try? await eventStore.requestAccess(to: .event)
 }
}

private func nativeCapabilities() async -> NativeCapabilities {
 let notificationSettings = await UNUserNotificationCenter.current().notificationSettings()
 let contactStatus = CNContactStore.authorizationStatus(for: .contacts)
 let reminderStatus = EKEEventStore.authorizationStatus(for: .reminder)
 let locationStatus = LocationManager.shared.authorizationStatus
 let remindersAuthorized: Bool
 if #available(iOS 17.0, *) {
 remindersAuthorized = reminderStatus == .fullAccess || reminderStatus == .writeOnly
 } else {
 remindersAuthorized = reminderStatus == .authorized
 }
 return NativeCapabilities(
 notifications: notificationSettings.authorizationStatus == .authorized || notificationSettings.authorizationStatus == .provisional,
 healthKit: HKHealthStore.isHealthDataAvailable(),
 musicKit: MusicAuthorization.currentStatus == .authorized,
 contacts: contactStatus == .authorized,
 reminders: remindersAuthorized,
 locationAlways: locationStatus == .authorizedAlways
)
}

private func healthSnapshot() async -> NativeHealthSnapshot {
 guard HKHealthStore.isHealthDataAvailable() else { return NativeHealthSnapshot() }
 return NativeHealthSnapshot(
 latestHeartRate: await latestQuantity(.heartRate, unit: HKUnit.count().unitDivided(by: .minute())),
 restingHeartRate: await latestQuantity(.restingHeartRate, unit: HKUnit.count().unitDivided(by: .minute())),
 stepCountToday: await sumQuantityToday(.stepCount, unit: .count()),
 sleepMinutesLastNight: await sleepMinutesLastNight(),
 recentWorkouts: await recentWorkouts(limit: 3)
)
}

private func latestQuantity(_ identifier: HKQuantityTypeIdentifier, unit: HKUnit) async -> Double? {
 guard let type = HKObjectType.quantityType(forIdentifier: identifier) else { return nil }

```

```

 return await withCheckedContinuation { continuation in
 let sort = NSSortDescriptor(key: HKSampleSortIdentifierEndDate, ascending: false)
 let query = HKSampleQuery(sampleType: type, predicate: nil, limit: 1, sortDescriptors: [sort]) { _, samples, _ in
 let value = (samples?.first as? HKQuantitySample)?.quantity.doubleValue(for: unit)
 continuation.resume(returning: value)
 }
 healthStore.execute(query)
 }
 }

private func sumQuantityToday(_ identifier: HKQuantityTypeIdentifier, unit: HKUnit) async -> Double? {
 if identifier == .stepCount {
 return await stepCount(start: Calendar.current.startOfDay(for: Date()), end: Date())
 }
 return await sumQuantity(identifier, unit: unit, start: Calendar.current.startOfDay(for: Date()), end: Date())
}

private func sumQuantity(_ identifier: HKQuantityTypeIdentifier, unit: HKUnit, start: Date, end: Date) async -> Double? {
 guard let type = HKObjectType.quantityType(forIdentifier: identifier) else { return nil }
 let predicate = HKQuery.predicateForSamples(withStart: start, end: end)
 return await withCheckedContinuation { continuation in
 let query = HKStatisticsQuery(quantityType: type, quantitySamplePredicate: predicate, options: .cumulativeSum) { _, stats, _ in
 continuation.resume(returning: stats?.sumQuantity()?.doubleValue(for: unit))
 }
 healthStore.execute(query)
 }
}

private func stepCount(start: Date, end: Date) async -> Double? {
 guard let type = HKObjectType.quantityType(forIdentifier: .stepCount), end > start else { return nil }
 let calendar = Calendar.current
 let anchorDate = calendar.startOfDay(for: start)
 var interval = DateComponents()
 interval.day = 1

 return await withCheckedContinuation { continuation in
 let query = HKStatisticsCollectionQuery(
 quantityType: type,
 quantitySamplePredicate: nil,
 options: .cumulativeSum,
 anchorDate: anchorDate,
 intervalComponents: interval
)
 query.initialResultsHandler = { _, collection, _ in
 guard let collection else {
 continuation.resume(returning: nil)
 return
 }
 var total = 0.0
 collection.enumerateStatistics(from: start, to: end) { statistics, _ in
 total += statistics.sumQuantity()?.doubleValue(for: .count()) ?? 0
 }
 continuation.resume(returning: total)
 }
 healthStore.execute(query)
 }
}

private func healthDateRange(from lower: String) -> (start: Date, end: Date, label: String) {
 let calendar = Calendar.current
 let now = Date()
 let todayStart = calendar.startOfDay(for: now)

 if lower.contains("yesterday") {
 let start = calendar.date(byAdding: .day, value: -1, to: todayStart) ?? todayStart
 return (start, todayStart, "yesterday")
 }

 if lower.contains("this morning") {
 let noon = calendar.date(byAdding: .hour, value: 12, to: todayStart) ?? now
 return (todayStart, min(noon, now), "this morning")
 }

 return (todayStart, now, "today")
}

private func sleepMinutesLastNight() async -> Double? {
 guard let type = HKObjectType.categoryType(forIdentifier: .sleepAnalysis) else { return nil }
 let calendar = Calendar.current
 let now = Date()
 let todayStart = calendar.startOfDay(for: now)

 let broadStart = calendar.date(byAdding: .hour, value: -54, to: todayStart) ?? calendar.date(byAdding: .day, value: -2, to: now) ?? now
 let predicate = HKQuery.predicateForSamples(withStart: broadStart, end: now, options: .strictEndDate)
 return await withCheckedContinuation { continuation in
 let query = HKSampleQuery(sampleType: type, predicate: predicate, limit: HKObjectQueryNoLimit, sortDescriptors: nil) { _, samples, _ in
 let sleepSamples = (samples as? [HKCategorySample] ?? [])
 }
 }
}

```

```

 .filter { Self.isAsleepSample($0) }

 let candidateMinutes = [0, -1, -2].compactMap { dayOffset -> Double? in
 guard let dayStart = calendar.date(byAdding: .day, value: dayOffset, to: todayStart),
 let windowStart = calendar.date(byAdding: .hour, value: -6, to: dayStart),
 let plannedWindowEnd = calendar.date(byAdding: .hour, value: 18, to: dayStart) else {
 return nil
 }
 let windowEnd = min(plannedWindowEnd, now)
 let intervals = sleepSamples.compactMap { sample -> (Date, Date)? in
 let start = max(sample.startDate, windowStart)
 let end = min(sample.endDate, windowEnd)
 guard end > start else { return nil }
 return (start, end)
 }
 return Self.mergedMinutes(intervals)
 }

 let latestMeaningful = candidateMinutes.first { $0 >= 90 }
 let fallback = candidateMinutes.max()
 let minutes = latestMeaningful ?? fallback ?? 0
 continuation.resume(returning: minutes > 0 ? minutes : nil)
}
healthStore.execute(query)
}
}

private nonisolated static func isAsleepSample(_ sample: HKCategorySample) -> Bool {
 sample.value == HKCategoryValueSleepAnalysis.asleepCore.rawValue
 || sample.value == HKCategoryValueSleepAnalysis.asleepDeep.rawValue
 || sample.value == HKCategoryValueSleepAnalysis.asleepREM.rawValue
 || sample.value == HKCategoryValueSleepAnalysis.asleepUnspecified.rawValue
}

private nonisolated static func mergedMinutes(_ intervals: [(Date, Date)]) -> Double {
 let sorted = intervals.sorted { $0.0 < $1.0 }
 var merged: [(Date, Date)] = []
 for interval in sorted {
 guard let last = merged.last else {
 merged.append(interval)
 continue
 }
 if interval.0 <= last.1 {
 merged[merged.count - 1] = (last.0, max(last.1, interval.1))
 } else {
 merged.append(interval)
 }
 }
 return merged.reduce(0.0) { total, interval in
 total + interval.1.timeIntervalSince(interval.0) / 60.0
 }
}

private func recentWorkouts(limit: Int) async -> [NativeWorkoutSummary] {
 let type = HKObjectType.workoutType()
 let start = Calendar.current.date(byAdding: .day, value: -14, to: Date()) ?? Date()
 let predicate = HKQuery.predicateForSamples(withStart: start, end: Date())
 let sort = NSSortDescriptor(key: HKSampleSortIdentifierEndDate, ascending: false)
 return await withCheckedContinuation { continuation in
 let query = HKSampleQuery(sampleType: type, predicate: predicate, limit: limit, sortDescriptors: [sort]) { _, samples, _ in
 let workouts = (samples as? [HKWorkout]) ?? [].map { workout in
 NativeWorkoutSummary(
 activity: workout.workoutActivityType.displayName,
 durationMinutes: workout.duration / 60.0,
 energyKilocalories: workout.totalEnergyBurned?.doubleValue(for: .kilocalorie()),
 distanceMeters: workout.totalDistance?.doubleValue(for: .meter()),
 endedAt: workout.endDate
)
 }
 continuation.resume(returning: workouts)
 }
 healthStore.execute(query)
 }
}

private func contactHints(in message: String) async -> [NativeContactHint] {
 switch CNContactStore.authorizationStatus(for: .contacts) {
 case .authorized:
 break
 case .notDetermined:
 guard (try? await contactStore.requestAccess(for: .contacts)) == true else { return [] }
 default:
 return []
 }

 let query = message.trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
 guard !query.isEmpty else { return [] }

```

```

return await withTaskGroup(of: [NativeContactHint].self) { group in
 group.addTask {
 Self.lookupContacts(matching: query)
 }
 group.addTask {
 try? await Task.sleep(nanoseconds: 2_000_000_000)
 return []
 }
 let result = await group.next() ?? []
 group.cancelAll()
 return result
}
}

private nonisolated static func lookupContacts(matching query: String) -> [NativeContactHint] {
 let keys: [CNKeyDescriptor] = [
 CNContactGivenNameKey as CNKeyDescriptor,
 CNContactFamilyNameKey as CNKeyDescriptor,
 CNContactNicknameKey as CNKeyDescriptor,
 CNContactPhoneNumbersKey as CNKeyDescriptor,
 CNContactEmailAddressesKey as CNKeyDescriptor
]
 let lower = query.lowercased()
 let store = CNContactStore()

 func hint(from contact: CNContact) -> NativeContactHint {
 let names = [
 "\(contact.givenName) \(contact.familyName)".trimmingCharacters(in: .whitespaces),

 contact.givenName,
 contact.familyName,
 contact.nickname
].filter { !$0.isEmpty }
 return NativeContactHint(
 displayName: names.first ?? "Contact",
 phone: contact.phoneNumbers.first?.value.stringValue,
 email: contact.emailAddresses.first.map { String($0.value) }
)
 }

 func ranked(_ hints: [NativeContactHint]) -> [NativeContactHint] {
 hints.sorted { lhs, rhs in
 let lhsHasPhone = lhs.phone?.isEmpty == false
 let rhsHasPhone = rhs.phone?.isEmpty == false
 if lhsHasPhone != rhsHasPhone { return lhsHasPhone }
 let lhsExact = lhs.displayName.lowercased() == lower
 let rhsExact = rhs.displayName.lowercased() == lower
 if lhsExact != rhsExact { return lhsExact }
 return lhs.displayName.count < rhs.displayName.count
 }
 }

 do {
 let predicate = CNContact.predicateForContacts(matchingName: query)
 let contacts = try store.unifiedContacts(matching: predicate, keysToFetch: keys)
 let hints = ranked(contacts.map(hint))
 if !hints.isEmpty {
 return Array(hints.prefix(5))
 }
 } catch {}

 let request = CNContactFetchRequest(keysToFetch: keys)
 var matches: [NativeContactHint] = []
 do {
 try store.enumerateContacts(with: request) { contact, stop in
 let names = [
 contact.givenName,
 contact.familyName,
 "\(contact.givenName) \(contact.familyName)".trimmingCharacters(in: .whitespaces),
 contact.nickname
].filter { !$0.isEmpty }
 guard names.contains(where: { name in
 let normalized = name.lowercased()
 return normalized == lower || normalized.contains(lower) || lower.contains(normalized)
 }) else { return }
 matches.append(hint(from: contact))
 if matches.count >= 5 { stop.pointee = true }
 }
 } catch {
 return []
 }
 return ranked(matches)
}

private func isLocalPlaceRequest(_ text: String) -> Bool {
 let lower = text.lowercased()
 return lower.contains("nearest")
}

```

```

 || lower.contains("closest")
 || lower.contains("near me")
 || lower.contains("nearby")
 || lower.hasPrefix("where is")
 || lower.hasPrefix("where's")
 }

private func shouldUseNativePlaceSearch(_ text: String) -> Bool {
 let lower = text.lowercased()
 return lower.contains("open maps")
 || lower.contains("open in maps")
 || lower.contains("show me on maps")
 || lower.contains("apple maps")
}

private func isContactResolutionRequest(_ text: String) -> Bool {
 let lower = text.lowercased()
 return lower.contains("text ")
 || lower.contains("message ")
 || lower.contains("call ")
 || lower.contains("facetime")
 || lower.contains("email ")
 || lower.contains("send ")
}

private func contactQuery(from text: String) -> String {
 let trimmed = text.trimmingCharacters(in: .whitespacesAndNewlines)
 if let range = trimmed.range(of: #"(?i)\bto\s+([A-Za-z][A-Za-z .'-]{1,60})(?:[?!.]|$)"#, options: .regularExpression) {
 var contact = String(trimmed[range])
 contact = contact.replacingOccurrences(of: #"(?i)^\s*\s+"#, with: "", options: .regularExpression)
 return contact.trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
 }
 return trimmed
}

private func findNativePlace(for message: String) async -> NativeLocalActionResult? {
 guard let place = await searchPlace(for: message) else { return nil }
 let distance = place.distanceMeters.map(formatDistance) ?? ""
 let text = "I found \(place.name)\(place.address.isEmpty ? "" : ", \(place.address)")\((distance.isEmpty ? "." : ", \(distance) away.")"
 let detail = [place.address, distance.isEmpty ? nil : "\((distance) away)"].compactMap { $0 }.joined(separator: " · ")
 return NativeLocalActionResult(
 action: "find_place",
 text: text,
 cardText: detail.isEmpty ? "Open in Maps" : detail,
 actionSummary: "Place found",
 deepLink: place.mapURL.absoluteString
)
}

private func searchPlace(for message: String) async -> NativePlaceResult? {
 let query = cleanPlaceQuery(message)
 guard !query.isEmpty else { return nil }
 let request = MKLocalSearch.Request()
 request.naturalLanguageQuery = query
 let origin = LocationManager.shared.lastLocation
 if let origin {
 request.region = MKCoordinateRegion(center: origin.coordinate, latitudinalMeters: 25_000, longitudinalMeters: 25_000)
 }
 guard let response = try? await MKLocalSearch(request: request).start() else { return nil }
 let sortedItems = response.mapItems.sorted { lhs, rhs in
 guard let origin else { return false }
 let left = CLLocation(latitude: lhs.placemark.coordinate.latitude, longitude: lhs.placemark.coordinate.longitude).distance(from: origin)
 let right = CLLocation(latitude: rhs.placemark.coordinate.latitude, longitude: rhs.placemark.coordinate.longitude).distance(from: origin)
 return left < right
 }
 guard let item = sortedItems.first else { return nil }

 let placemark = item.placemark
 let address = [placemark.thoroughfare, placemark.locality, placemark.postalCode]
 .compactMap { $0 }
 .joined(separator: ", ")
 let distance = origin.map { CLLocation(latitude: placemark.coordinate.latitude, longitude: placemark.coordinate.longitude).distance(from: $0) }
 let label = (item.name ?? query).addingPercentEncoding(withAllowedCharacters: .urlQueryAllowed) ?? query
 let url = URL(string: "https://maps.apple.com/?ll=\(placemark.coordinate.latitude),\((placemark.coordinate.longitude)&q=\(label))")!
 return NativePlaceResult(
 name: item.name ?? query,
 address: address,
 distanceMeters: distance,
 mapURL: url
)
}

private func cleanPlaceQuery(_ message: String) -> String {
 var text = message.lowercased()
 let replacements = [
 "can you tell me where the",
]

```

```

 "where's the",
 "where is the",
 "nearest",
 "closest",
 "near me",
 "nearby",
 "directions to",
 "navigate to",
 "route to",
 "bus to",
 "buses to",
 "get me to",
 "i need to be at",
 "what bus can i take"
]
 for replacement in replacements {
 text = text.replacingOccurrences(of: replacement, with: " ")
 }
 text = text.replacingOccurrences(of: "\\bby\\b", with: " ", options: .regularExpression)
 text = text.replacingOccurrences(of: "\\bis\\b", with: " ", options: .regularExpression)
 text = text.replacingOccurrences(of: "\\b\\d{1,2}:\\d{2}\\b", with: " ", options: .regularExpression)
 text = text.replacingOccurrences(of: "\\s+", with: " ", options: .regularExpression)
 return text.trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
}

private func createNativeReminder(from message: String) async -> NativeLocalActionResult? {
 let lower = message.lowercased()
 guard lower.hasPrefix("remind me") || lower.hasPrefix("reminder") || lower.hasPrefix("create a reminder") else { return nil }
 await requestReminderPermission()
 guard remindersAuthorized else {
 return NativeLocalActionResult(action: "create_reminder", text: "Turn on Reminders access and I can create that natively.", cardText: "")
 }

 let parsedDate = detectedDate(in: message)
 let title = cleanReminderTitle(message, date: parsedDate)
 guard !title.isEmpty else { return nil }
 let reminder = EKReminder(eventStore: eventStore)
 reminder.title = title
 reminder.calendar = eventStore.defaultCalendarForNewReminders()
 if let parsedDate {
 reminder.dueDateComponents = Calendar.current.dateComponents([.year, .month, .day, .hour, .minute], from: parsedDate)
 let alarm = EKAlarm(absoluteDate: parsedDate)
 reminder.addAlarm(alarm)
 }
 do {
 try eventStore.save(reminder, commit: true)
 lastReminderIdentifier = reminder.calendarItemIdentifier
 lastReminderTitle = title
 lastReminderDueDate = parsedDate
 let timeText = parsedDate.map { DateFormatter.localizedString(from: $0, dateStyle: .none, timeStyle: .short) }
 return NativeLocalActionResult(
 action: "create_reminder",
 text: timeText.map { "Reminder set for \($0): \(title)." } ?? "Reminder set: \(title).",
 cardText: timeText.map { "\(title) · \($0)" } ?? title,
 actionSummary: "Reminder created",
 deepLink: "x-apple-reminderkit://"
)
 } catch {
 return nil
 }
}

private func updateLastNativeReminder(from message: String) async -> NativeLocalActionResult? {
 guard isReminderUpdateFollowUp(message),
 let identifier = lastReminderIdentifier,
 let reminder = eventStore.calendarItem(withIdentifier: identifier) as? EKReminder else { return nil }
 await requestReminderPermission()
 guard remindersAuthorized else {
 return NativeLocalActionResult(action: "create_reminder", text: "Turn on Reminders access and I can update that natively.", cardText: "")
 }
 guard let dueDate = reminderFollowUpDate(in: message) else { return nil }
 reminder.dueDateComponents = Calendar.current.dateComponents([.year, .month, .day, .hour, .minute], from: dueDate)
 reminder.alarms?.forEach { reminder.removeAlarm($0) }
 reminder.addAlarm(EKAlarm(absoluteDate: dueDate))

 do {
 try eventStore.save(reminder, commit: true)
 lastReminderIdentifier = reminder.calendarItemIdentifier
 lastReminderTitle = reminder.title
 lastReminderDueDate = dueDate
 let label = formatReminderFollowUpDate(dueDate)
 return NativeLocalActionResult(
 action: "create_reminder",
 text: "Reminder moved to \(label): \(reminder.title ?? lastReminderTitle ?? "reminder").",
 cardText: "\(reminder.title ?? lastReminderTitle ?? "Reminder") · \(label)",
 actionSummary: "Reminder updated",
 deepLink: "x-apple-reminderkit://"
)
 }
}

```



```

)
 } catch {
 return nil
 }
}

private func isReminderUpdateFollowUp(_ message: String) -> Bool {
 let lower = message.lowercased().trimmingCharacters(in: .whitespacesAndNewlines)
 guard lastReminderIdentifier != nil else { return false }

 guard lower.contains("tomorrow")
 || lower.contains("today")
 || lower.range(of: #"\\b(at|by)\\s+\\d{1,2}(?:\\d{2})?\\s*(am|pm)?\\b"#, options: .regularExpression) != nil
 || lower.range(of: #"\\b\\d{1,2}(?:\\d{2})?\\s*(am|pm)\\b"#, options: .regularExpression) != nil
 else { return false }
 return lower.contains("that")
 || lower.contains("it")
 || lower.contains("reminder")
 || lower.hasPrefix("actually")
 || lower.hasPrefix("make ")
 || lower.hasPrefix("change ")
 || lower.hasPrefix("move ")
}

private func reminderFollowUpDate(in message: String) -> Date? {
 let lower = message.lowercased()
 let calendar = Calendar.current
 let parsed = detectedDate(in: message)
 let hasExplicitTime = lower.range(of: #"\\b(at|by)\\s+\\d{1,2}(?:\\d{2})?\\s*(am|pm)?\\b"#, options: .regularExpression) != nil
 || lower.range(of: #"\\b\\d{1,2}(?:\\d{2})?\\s*(am|pm)\\b"#, options: .regularExpression) != nil

 if lower.contains("tomorrow") || lower.contains("today") {
 let dayOffset = lower.contains("tomorrow") ? 1 : 0
 guard let targetDay = calendar.date(byAdding: .day, value: dayOffset, to: calendar.startOfDay(for: Date())) else {
 return parsed
 }
 let timeSource = hasExplicitTime ? parsed : lastReminderDueDate
 let time = timeSource.map { calendar.dateComponents([.hour, .minute], from: $0) }
 var components = calendar.dateComponents([.year, .month, .day], from: targetDay)
 components.hour = time?.hour ?? 9
 components.minute = time?.minute ?? 0
 return calendar.date(from: components) ?? parsed
 }

 return parsed
}

private func formatReminderFollowUpDate(_ date: Date) -> String {
 let time = DateFormatter.localizedString(from: date, dateStyle: .none, timeStyle: .short)
 if Calendar.current.isDateInTomorrow(date) {
 return "tomorrow at \(time)"
 }
 if Calendar.current.isDateInToday(date) {
 return "today at \(time)"
 }
 return DateFormatter.localizedString(from: date, dateStyle: .medium, timeStyle: .short)
}

private func createNativeCalendarEvent(from message: String) async -> NativeLocalActionResult? {
 let lower = message.lowercased()
 let looksLikeDatedAdd = lower.hasPrefix("add ") && detectedDate(in: message) != nil && !isMusicAddRequest(lower)
 guard lower.contains("calendar") || lower.contains("add event") || lower.contains("schedule") || looksLikeDatedAdd else { return nil }
 await requestCalendarPermission()
 guard calendarAuthorized else {
 return NativeLocalActionResult(action: "create_calendar_event", text: "Turn on Calendar access and I can create that natively.", cardText: "Turn on Calendar access and I can create that natively.")
 }
 guard let start = detectedDate(in: message) else { return nil }
 let title = cleanCalendarTitle(message, date: start)
 guard !title.isEmpty else { return nil }
 let event = EKEvent(eventStore: eventStore)
 event.title = title
 event.startDate = start
 event.endDate = Calendar.current.date(byAdding: .hour, value: 1, to: start) ?? start.addingTimeInterval(3600)
 event.calendar = eventStore.defaultCalendarForNewEvents
 do {
 try eventStore.save(event, span: .thisEvent, commit: true)
 let timeText = DateFormatter.localizedString(from: start, dateStyle: .medium, timeStyle: .short)
 return NativeLocalActionResult(
 action: "create_calendar_event",
 text: "Added \(title) to Calendar for \(timeText).",
 cardText: "\(title) · \(timeText)",
 actionSummary: "Calendar updated",
 deepLink: "calshow://"
)
 }
} catch {
 return nil
}

```

```

}

private var remindersAuthorized: Bool {
 let status = EKEEventStore.authorizationStatus(for: .reminder)
 if #available(iOS 17.0, *) {
 return status == .fullAccess || status == .writeOnly
 }
}

private var calendarAuthorized: Bool {
 let status = EKEEventStore.authorizationStatus(for: .event)
 if #available(iOS 17.0, *) {
 return status == .fullAccess || status == .writeOnly
 }
 return status == .authorized
}

private func detectedDate(in message: String) -> Date? {
 let range = NSRange(message.startIndex..&message.endIndex, in: message)
 return detector?.firstMatch(in: message, options: [], range: range)?.date
}

private func cleanReminderTitle(_ message: String, date: Date?) -> String {
 var title = message
 .replacingOccurrences(of: #"(?i)^remind me to\s+"#, with: "", options: .regularExpression)
 .replacingOccurrences(of: #"(?i)^remind me\s+"#, with: "", options: .regularExpression)
 .replacingOccurrences(of: #"(?i)^create a reminder to\s+"#, with: "", options: .regularExpression)
 .replacingOccurrences(of: #"(?i)^reminder to\s+"#, with: "", options: .regularExpression)
 if let match = dateMatch(in: message) {
 title = title.replacingOccurrences(of: match, with: "")
 }
 title = title.replacingOccurrences(of: #"(?i)\s+\b(at|on|by|for)\s+"#, with: "", options: .regularExpression)
 return title.trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
}

private func cleanCalendarTitle(_ message: String, date: Date?) -> String {
 var title = message
 .replacingOccurrences(of: #"(?i)add\s+"#, with: "", options: .regularExpression)
 .replacingOccurrences(of: #"(?i)to (my)?calendar"#, with: "", options: .regularExpression)
 .replacingOccurrences(of: #"(?i)schedule\s+"#, with: "", options: .regularExpression)
 .replacingOccurrences(of: #"(?i)event\s+"#, with: "", options: .regularExpression)
 if let match = dateMatch(in: message) {
 title = title.replacingOccurrences(of: match, with: "")
 }
 return title.trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
}

private func dateMatch(in message: String) -> String? {
 let range = NSRange(message.startIndex..&message.endIndex, in: message)
 guard let match = detector?.firstMatch(in: message, options: [], range: range),
 let swiftRange = Range(match.range, in: message) else { return nil }
 return String(message[swiftRange])
}

private func formatDistance(_ meters: CLLocationDistance) -> String {
 if meters < 1000 {
 return "\(Int(meters.rounded())) m"
 }
 return String(format: "%.1f km", meters / 1000)
}

private func formatDurationMinutes(_ minutes: Double) -> String {
 let total = max(0, Int(minutes.rounded()))
 let hours = total / 60
 let remainder = total % 60
 if hours > 0 & remainder > 0 {
 return "\(hours)h \(remainder)m"
 }
 if hours > 0 {
 return "\(hours)h"
 }
 return "\(remainder)m"
}

private func formatWorkoutSummary(_ workout: NativeWorkoutSummary) -> String {
 let date = DateFormatter.localizedString(from: workout.endedAt, dateStyle: .short, timeStyle: .none)
 var parts = ["\(workout.activity)", formatDurationMinutes(workout.durationMinutes)]
 if let energy = workout.energyKilocalories {
 parts.append("\(Int(energy.rounded())) kcal")
 }
 if let distance = workout.distanceMeters, distance > 0 {
 parts.append(formatDistance(distance))
 }
 return "\(date): \(parts.joined(separator: " · "))"
}

```

```

private func loadSettings() -> OxySettings {
 if let data = UserDefaults.standard.data(forKey: "oxy_settings"),
 let saved = try? JSONDecoder().decode(OxySettings.self, from: data) {
 return saved
 }
 return OxySettings()
}

private func saveSettings(_ settings: OxySettings) {
 if let data = try? JSONEncoder().encode(settings) {
 UserDefaults.standard.set(data, forKey: "oxy_settings")
 }
}

private extension Encodable {
 var dictionary: [String: Any] {
 guard let data = try? JSONEncoder().encode(self),
 let object = try? JSONSerialization.jsonObject(with: data) as? [String: Any] else {
 return [:]
 }
 return object
 }
}

extension OxySettings {
 var nativeDictionary: [String: Any] {
 var dict = dictionary
 if let homeLatitude, let homeLongitude {
 dict["homeLocation"] = ["latitude": homeLatitude, "longitude": homeLongitude]
 }
 return dict
 }
}

private extension HKWorkoutActivityType {
 var displayName: String {
 switch self {
 case .running: return "Run"
 case .walking: return "Walk"
 case .cycling: return "Cycle"
 case .traditionalStrengthTraining: return "Strength training"
 case .functionalStrengthTraining: return "Functional strength"
 case .highIntensityIntervalTraining: return "HIIT"
 case .yoga: return "Yoga"
 case .swimming: return "Swim"
 case .dance: return "Dance"
 case .mindAndBody: return "Mind and body"
 case .coreTraining: return "Core training"
 case .pilates: return "Pilates"
 case .rowing: return "Rowing"
 case .elliptical: return "Elliptical"
 case .stairClimbing: return "Stair climbing"
 case .soccer: return "Football"
 case .basketball: return "Basketball"
 case .tennis: return "Tennis"
 case .other: return "Workout"
 default: return "Workout"
 }
 }
}

// MARK: - PendantBLEManager

enum PendantConnectionState: String {
 case disconnected
 case scanning
 case connecting
 case connected
 case error
}

@Observable
final class PendantBLEManager: NSObject {
 static let didReceiveData = Notification.Name("PendantBLEManagerDidReceiveData")
 static let didConnect = Notification.Name("PendantBLEManagerDidConnect")
 static let didDisconnect = Notification.Name("PendantBLEManagerDidDisconnect")

 static let namePrefix = "Oxy"
 private static let pairedPeripheralKey = "oxy_paired_pendant_uuid"
 private static let connectionTimeout: TimeInterval = 10

 private enum UART {
 static let service = CBUUID(string: "6E400001-B5A3-F393-E0A9-E50E24DCCA9E")
 static let rx = CBUUID(string: "6E400002-B5A3-F393-E0A9-E50E24DCCA9E") // central writes
 static let tx = CBUUID(string: "6E400003-B5A3-F393-E0A9-E50E24DCCA9E") // central subscribes
 }
}

```

```

private(set) var connectionState: PendantConnectionState = .disconnected
private(set) var peripheralName: String?
private(set) var lastError: String?

@ObservationIgnored private var central: CBCentralManager!
@ObservationIgnored private(set) var peripheral: CBPeripheral?
@ObservationIgnored private(set) var rxCharacteristic: CBCharacteristic?
@ObservationIgnored private(set) var txCharacteristic: CBCharacteristic?

@ObservationIgnored private var connectionTimer: Timer?
@ObservationIgnored private var retryCount = 0
private static let maxRetries = 3

var isConnected: Bool { connectionState == .connected }

@ObservationIgnored private var centralReady = false

override init() {
 super.init()
 central = CBCentralManager(delegate: self, queue: .main, options: [
 CBCentralManagerOptionShowPowerAlertKey: false
])
}

/// Send a short ASCII command to the pendant (e.g. "THINK", "DONE", "PING").
func sendCommand(_ cmd: String) {
 guard let p = peripheral, let rx = rxCharacteristic,
 p.state == .connected else { return }
 let data = Data(cmd.utf8)
 // Use withoutResponse for low-latency feedback; pendant doesn't need ACK
 p.writeValue(data, for: rx, type: .withoutResponse)
}

func startScan() {
 guard central.state == .poweredOn else { return }

 if let savedUUID = pairedPeripheralUUID {
 let known = central.retrievePeripherals(withIdentifiers: [savedUUID])
 if let cached = known.first, cached.state == .disconnected {
 print("[Pendant] Reconnecting to saved peripheral \(cached.name ?? savedUUID.uuidString)")
 connectionState = .connecting
 peripheral = cached
 cached.delegate = self
 startConnectionTimer()
 central.connect(cached, options: nil)
 return
 }
 }

 print("[Pendant] Scanning for Nordic UART Service \(UART.service)")
 connectionState = .scanning
 lastError = nil
 central.scanForPeripherals(
 withServices: [UART.service],
 options: [CBCentralManagerScanOptionAllowDuplicatesKey: false]
)
}

func stopScan() {
 central.stopScan()
 cancelConnectionTimer()
 if connectionState == .scanning {
 connectionState = .disconnected
 }
}

func unpair() {
 cancelConnectionTimer()
 if let p = peripheral, p.state == .connected || p.state == .connecting {
 central.cancelPeripheralConnection(p)
 }
 clearPairedPeripheralUUID()
 rxCharacteristic = nil
 txCharacteristic = nil
 peripheral = nil
 peripheralName = nil
 connectionState = .disconnected
 lastError = nil
 retryCount = 0
 print("[Pendant] Unpaired")
}

// MARK: - Peripheral identity persistence

private var pairedPeripheralUUID: UUID? {
 get {
 guard let str = UserDefaults.standard.string(forKey: Self.pairedPeripheralKey) else { return nil }

```

```

 return UUID(uuidString: str)
 }
 set {
 if let uuid = newValue {
 UserDefaults.standard.set(uuid.uuidString, forKey: Self.pairedPeripheralKey)
 } else {
 UserDefaults.standard.removeObject(forKey: Self.pairedPeripheralKey)
 }
 }
}

private func clearPairedPeripheralUUID() {
 UserDefaults.standard.removeObject(forKey: Self.pairedPeripheralKey)
}

// MARK: - Connection timeout

private func startConnectionTimer() {
 cancelConnectionTimer()
 connectionTimer = Timer.scheduledTimer(withTimeInterval: Self.connectionTimeout, repeats: false) { [weak self] _ in
 self?.handleConnectionTimeout()
 }
}

private func cancelConnectionTimer() {
 connectionTimer?.invalidate()
 connectionTimer = nil
}

private func handleConnectionTimeout() {
 guard connectionState == .connecting, let p = peripheral else { return }
 print("[Pendant] Connection timed out for \(p.name ?? p.identifier.uuidString)")
 central.cancelPeripheralConnection(p)
 rxCharacteristic = nil
 txCharacteristic = nil
 peripheral = nil

 retryCount += 1
 if retryCount < Self.maxRetries {
 print("[Pendant] Retrying scan (attempt \(retryCount + 1)/\(Self.maxRetries))")
 startScan()
 } else {
 connectionState = .error
 lastError = "Connection timed out after \(Self.maxRetries) attempts"
 retryCount = 0
 print("[Pendant] \(lastError!)")
 }
}
}

extension PendantBLEManager: CBCentralManagerDelegate {
 func centralManagerDidUpdateState(_ central: CBCentralManager) {
 print("[Pendant] CBCentralManager state: \(central.state.rawValue)")
 centralReady = central.state == .poweredOn
 if central.state == .poweredOn {
 if pairedPeripheralUUID != nil {
 startScan()
 }
 } else {
 connectionState = .disconnected
 }
 }
}

func centralManager(_ central: CBCentralManager,
 didDiscover peripheral: CBPeripheral,
 advertisementData: [String: Any],
 rssi RSSI: NSNumber) {
 let name = peripheral.name ?? advertisementData[CBAdvertisementDataLocalNameKey] as? String
 print("[Pendant] Discovered peripheral - name: \(name ?? "<nil>"), UUID: \(peripheral.identifier), RSSI: \(RSSI)")

 let nameMatches: Bool
 if pairedPeripheralUUID != nil {
 nameMatches = peripheral.identifier == pairedPeripheralUUID
 } else if let name {
 nameMatches = name.hasPrefix(Self.namePrefix)
 } else {
 nameMatches = false
 }

 guard nameMatches else {
 print("[Pendant] Ignoring peripheral - name does not start with \(Self.namePrefix)")
 return
 }

 central.stopScan()
 self.peripheral = peripheral
 self.peripheralName = name

```

```

 peripheral.delegate = self
 connectionState = .connecting
 retryCount = 0
 startConnectionTimer()
 print("[Pendant] Connecting to \(name ?? peripheral.identifier.uuidString)...")
 central.connect(peripheral, options: nil)
 }

 func centralManager(_ central: CBCentralManager, didConnect peripheral: CBPeripheral) {
 cancelConnectionTimer()
 print("[Pendant] Connected to \(peripheral.name ?? peripheral.identifier.uuidString)")
 pairedPeripheralUUID = peripheral.identifier
 peripheral.discoverServices([UART.service])
 }

 func centralManager(_ central: CBCentralManager,
 didDisconnectPeripheral peripheral: CBPeripheral,
 error: Error?) {
 cancelConnectionTimer()
 if let error {
 print("[Pendant] Disconnected from \(peripheral.name ?? peripheral.identifier.uuidString) with error: \(error.localizedDescription)")
 } else {
 print("[Pendant] Disconnected from \(peripheral.name ?? peripheral.identifier.uuidString)")
 }
 rxCharacteristic = nil
 txCharacteristic = nil
 self.peripheral = nil
 connectionState = .disconnected
 NotificationCenter.default.post(name: PendantBLEManager.didDisconnect, object: nil)

 if pairedPeripheralUUID != nil {
 print("[Pendant] Will attempt reconnection to paired pendant")
 DispatchQueue.main.asyncAfter(deadline: .now() + 1.0) { [weak self] in
 self?.startScan()
 }
 }
 }

 func centralManager(_ central: CBCentralManager,
 didFailToConnect peripheral: CBPeripheral,
 error: Error?) {
 cancelConnectionTimer()
 let desc = error?.localizedDescription ?? "unknown error"
 print("[Pendant] Failed to connect to \(peripheral.name ?? peripheral.identifier.uuidString): \(desc)")
 rxCharacteristic = nil
 txCharacteristic = nil
 self.peripheral = nil

 retryCount += 1
 if retryCount < Self.maxRetries {
 print("[Pendant] Retrying scan (attempt \(retryCount + 1)/\(Self.maxRetries))")
 startScan()
 } else {
 connectionState = .error
 lastError = "Failed to connect: \(desc)"
 retryCount = 0
 }
 }
}

extension PendantBLEManager: CBPeripheralDelegate {
 func peripheral(_ peripheral: CBPeripheral, didDiscoverServices error: Error?) {
 if let error {
 print("[Pendant] Service discovery error: \(error.localizedDescription)")
 connectionState = .error
 lastError = "Service discovery failed: \(error.localizedDescription)"
 return
 }
 guard let service = peripheral.services?.first(where: { $0.uuid == UART.service }) else {
 print("[Pendant] Nordic UART Service not found among discovered services: \(peripheral.services?.map(\.uuid) ?? [])")
 connectionState = .error
 lastError = "UART service not found on device"
 return
 }
 print("[Pendant] Discovered Nordic UART Service – discovering characteristics")
 peripheral.discoverCharacteristics([UART.rx, UART.tx], for: service)
 }

 func peripheral(_ peripheral: CBPeripheral,
 didDiscoverCharacteristicsFor service: CBService,
 error: Error?) {
 if let error {
 print("[Pendant] Characteristic discovery error: \(error.localizedDescription)")
 connectionState = .error
 lastError = "Characteristic discovery failed: \(error.localizedDescription)"
 return
 }
 }
}

```

```

 }
 for characteristic in service.characteristics ?? [] {
 switch characteristic.uuid {
 case UART.rx:
 print("[Pendant] Found RX characteristic (write) \(characteristic.uuid)")
 rxCharacteristic = characteristic
 case UART.tx:
 print("[Pendant] Found TX characteristic (notify) \(characteristic.uuid) – subscribing")
 txCharacteristic = characteristic
 peripheral.setNotifyValue(true, for: characteristic)
 default:
 break
 }
 }
}

func peripheral(_ peripheral: CBPeripheral,
 didUpdateValueFor characteristic: CBCharacteristic,
 error: Error?) {
 guard error == nil, characteristic.uuid == UART.tx,
 let data = characteristic.value else { return }

 NotificationCenter.default.post(name: PendantBLEManager.didReceiveData, object: data)
}

func peripheral(_ peripheral: CBPeripheral,
 didUpdateNotificationStateFor characteristic: CBCharacteristic,
 error: Error?) {
 if let error {
 print("[Pendant] Failed to subscribe to \(characteristic.uuid): \(error.localizedDescription)")
 connectionState = .error
 lastError = "Failed to subscribe to notifications"
 } else {
 print("[Pendant] Subscribed to TX notifications – ready")
 connectionState = .connected
 NotificationCenter.default.post(name: PendantBLEManager.didConnect, object: nil)
 }
}
}
}

```

### PendantLiveService.swift (NEW) — OxyApp/OxyApp/Services/PendantLiveService.swift

```

import AVFoundation
import Foundation
import Observation
import UIKit

extension Notification.Name {
 static let pendantNativeAction = Notification.Name("oxy.pendantNativeAction")
}

/// Streams raw BLE audio from the pendant directly to the Gemini Live
/// `~/realtime-voice` WebSocket for end-to-end STT + reasoning + TTS.
///
/// This replaces the SFSpeechRecognizer path entirely: audio never goes
/// through Apple's on-device recogniser. Gemini Live handles transcription,
/// conversation, tool execution, and TTS in a single low-latency pipeline.
@Observable
@MainActor
final class PendantLiveService {

 enum LiveState {
 case idle
 case connecting
 case listening // session active, audio streaming
 case processing // Gemini turn in progress
 }

 private(set) var liveState: LiveState = .idle
 private(set) var userTranscript: String?
 private(set) var assistantTranscript: String?
 private(set) var errorMessage: String?

 @ObservationIgnored private var socket: URLSessionWebSocketTask?
 @ObservationIgnored private var isSessionOpen = false
 @ObservationIgnored private var pendingChunks: [Data] = [] // buffered before session opens
 @ObservationIgnored private let audioPlayer = PendantAudioOutputPlayer()

 @ObservationIgnored private var bleDataObserver: NSObjectProtocol?
 @ObservationIgnored private var connectObserver: NSObjectProtocol?
 @ObservationIgnored private var disconnectObserver: NSObjectProtocol?

 init() {
 bleDataObserver = NotificationCenter.default.addObserver(
 forName: PendantBLEManager.didReceiveData,
 object: nil,
 queue: .main
)
 }
}

```

```

) { [weak self] n in
 guard let data = n.object as? Data else { return }
 Task { @MainActor in self?.forwardAudioChunk(data) }
 }

 connectObserver = NotificationCenter.default.addObserver(
 forName: PendantBLEManager.didConnect,
 object: nil,
 queue: .main
) { [weak self] _ in
 Task { @MainActor in self?.openSession() }
 }

 disconnectObserver = NotificationCenter.default.addObserver(
 forName: PendantBLEManager.didDisconnect,
 object: nil,
 queue: .main
) { [weak self] _ in
 Task { @MainActor in self?.closeSession() }
 }
}

deinit {
 [bleDataObserver, connectObserver, disconnectObserver]
 .compactMap { $0 }
 .forEach { NotificationCenter.default.removeObserver($0) }
}

// MARK: - Session lifecycle

private func openSession() {
 guard liveState == .idle else { return }
 liveState = .connecting

 let base = APIClient.shared.baseURL
 let wsBase = base
 .replacingOccurrences(of: "https://", with: "wss://")
 .replacingOccurrences(of: "http://", with: "ws://")

 guard let url = URL(string: "\(wsBase)/realtime-voice") else {
 errorMessage = "Invalid WebSocket URL"
 liveState = .idle
 return
 }

 do {
 let session = AVAudioSession.sharedInstance()
 try session.setCategory(.playback, mode: .default, options: [.defaultToSpeaker])
 try session.setActive(true, options: .notifyOthersOnDeactivation)
 } catch {
 print("[PendantLive] Audio session: \(error.localizedDescription)")
 }

 let task = URLSession.shared.webSocketTask(with: url)
 socket = task
 task.resume()

 // Authenticate immediately
 let token = KeychainHelper.shared.read(key: "session_token") ?? ""
 send(["type": "auth", "token": token])

 Task { await receiveLoop() }
 print("[PendantLive] WebSocket opened, authenticating...")
}

private func closeSession() {
 socket?.cancel(with: .normalClosure, reason: nil)
 socket = nil
 isSessionOpen = false
 pendingChunks = []
 liveState = .idle
 userTranscript = nil
 assistantTranscript = nil
 audioPlayer.stop()
 print("[PendantLive] Session closed")
}

// MARK: - Audio forwarding

private func forwardAudioChunk(_ data: Data) {
 let event: [String: Any] = [
 "type": "audio-chunk",
 "data": data.base64EncodedString(),
 "mimeType": "audio/l16;rate=16000"
]
}

```



```

 if isSessionOpen {
 send(event)
 } else if liveState == .connecting {
 // Buffer up to ~1 s of audio while session negotiates
 if pendingChunks.count < 80 { pendingChunks.append(data) }
 }
 }

private func flushPendingChunks() {
 for chunk in pendingChunks {
 send([
 "type": "audio-chunk",
 "data": chunk.base64EncodedString(),
 "mimeType": "audio/l16;rate=16000"
])
 }
 pendingChunks = []
}

// MARK: - WebSocket receive loop

private func receiveLoop() async {
 guard let task = socket else { return }
 while true {
 do {
 let msg = try await task.receive()
 let text: String
 switch msg {
 case .string(let s): text = s
 case .data(let d): text = String(data: d, encoding: .utf8) ?? ""
 @unknown default: continue
 }
 await MainActor.run { handleEvent(text) }
 } catch {
 await MainActor.run {
 if liveState != .idle {
 print("[PendantLive] Receive error: \(error.localizedDescription)")
 closeSession()
 }
 }
 break
 }
 }
}

// MARK: - Server event handling

private func handleEvent(_ raw: String) {
 guard let data = raw.data(using: .utf8),
 let json = try? JSONSerialization.jsonObject(with: data) as? [String: Any],
 let type = json["type"] as? String else { return }

 switch type {

 case "live-authenticated":
 // Start the Gemini Live session with the user's chosen voice
 let voice = UserDefaults.standard.string(forKey: "oxy_live_voice") ?? "Schedar"
 send(["type": "live-start", "voice": voice])

 case "live-open":
 isSessionOpen = true
 liveState = .listening
 flushPendingChunks()
 print("[PendantLive] Gemini Live session open")

 case "user-transcript":
 let text = json["text"] as? String ?? ""
 userTranscript = text
 if json["final"] as? Bool == true && !text.isEmpty {
 liveState = .processing
 NativeIntegrationManager.shared.pendant.sendCommand("THINK")
 print("[PendantLive] User said: \(text)")
 }

 case "assistant-transcript":
 assistantTranscript = json["text"] as? String

 case "assistant-audio":
 guard let b64 = json["data"] as? String,
 let audioData = Data(base64Encoded: b64) else { return }
 let mimeType = json["mimeType"] as? String ?? "audio/pcm;rate=24000"
 audioPlayer.enqueue(audioData, mimeType: mimeType)

 case "assistant-actions":
 if let results = json["results"] as? [[String: Any]] {
 handleActionResults(results)
 }
 }
}

```

```

 case "live-turn-complete":
 liveState = .listening
 userTranscript = nil
 assistantTranscript = nil
 NativeIntegrationManager.shared.pendant.sendCommand("DONE")
 print("[PendantLive] Turn complete")

 case "live-interrupted":
 audioPlayer.stop()

 case "live-error":
 let msg = json["error"] as? String ?? "Unknown error"
 errorMessage = msg
 print("[PendantLive] Error: \(msg)")
 // Brief pause then reconnect
 DispatchQueue.main.asyncAfter(deadline: .now() + 2) { [weak self] in
 self?.closeSession()
 self?.openSession()
 }

 case "live-closed":
 closeSession()

 default:
 break
 }
}

// MARK: - Native action dispatch

private func handleActionResults(_ results: [[String: Any]]) {
 for result in results {
 guard let signal = result["nativeSignal"] as? String else { continue }
 NotificationCenter.default.post(
 name: .pendantNativeAction,
 object: nil,
 userInfo: ["signal": signal, "data": result]
)
 }
}

// MARK: - WebSocket send

private func send(_ dict: [String: Any]) {
 guard let task = socket,
 let body = try? JSONSerialization.data(withJSONObject: dict),
 let text = String(data: body, encoding: .utf8) else { return }
 task.send(.string(text)) { error in
 if let error {
 print("[PendantLive] Send error: \(error.localizedDescription)")
 }
 }
}

// MARK: - PCM audio output player

/// Plays back raw 16-bit PCM audio chunks from Gemini Live (typically 24 kHz mono).
final class PendantAudioOutputPlayer {

 private var engine: AVAudioEngine?
 private var playerNode: AVAudioPlayerNode?
 private var pcmFormat: AVAudioFormat?
 private var configuredSampleRate: Double = 0

 func enqueue(_ data: Data, mimeType: String) {
 let sampleRate = parseSampleRate(mimeType)
 ensureEngine(sampleRate: sampleRate)

 guard let format = pcmFormat else { return }

 let sampleCount = data.count / MemoryLayout<Int16>.size
 guard sampleCount > 0,
 let buf = AVAudioPCMBuffer(pcmFormat: format, frameCapacity: UInt32(sampleCount)) else { return }
 buf.frameLength = UInt32(sampleCount)

 data.withUnsafeBytes { raw in
 guard let src = raw.baseAddress?.assumingMemoryBound(to: Int16.self),
 let dst = buf.floatChannelData?[0] else { return }
 for i in 0..

```

```

func stop() {
 playerNode?.stop()
 engine?.stop()
 engine = nil
 playerNode = nil
 pcmFormat = nil
 configuredSampleRate = 0
}

private func ensureEngine(sampleRate: Double) {
 if engine != nil && configuredSampleRate == sampleRate { return }
 stop()

 let eng = AVAudioEngine()
 let node = AVAudioPlayerNode()
 guard let fmt = AVAudioFormat(
 commonFormat: .pcmFormatFloat32,
 sampleRate: sampleRate,
 channels: 1,
 interleaved: false
) else { return }

 eng.attach(node)
 eng.connect(node, to: eng.mainMixerNode, format: fmt)

 do {
 try eng.start()
 node.play()
 engine = eng
 playerNode = node
 pcmFormat = fmt
 configuredSampleRate = sampleRate
 } catch {
 print("[PendantAudioPlayer] Engine start failed: \(error.localizedDescription)")
 }
}

private func parseSampleRate(_ mimeType: String) -> Double {
 if let match = mimeType.range(of: #"rate=(\d+)#", options: .regularExpression) {
 let s = String(mimeType[match]).replacingOccurrences(of: "rate=", with: "")
 return Double(s) ?? 24000
 }
 return 24000
}
}

```

## PendantAudioBridge.swift — OxyApp/OxyApp/Services/PendantAudioBridge.swift

```

import AVFoundation
import Foundation
import Observation
import Speech

/// Bridges continuous pendant BLE audio to the chat pipeline.
///
/// Audio flow: pendant PDM (16 kHz Int16) → BLE chunks → accumulated buffer →
/// Float32 conversion → SFSpeechRecognizer. Energy-based VAD signals end-of-
/// utterance so the recognizer commits a final result promptly rather than
/// waiting for its own timeout.
@Observable
@MainActor
final class PendantAudioBridge {

 enum BridgeState: String {
 case idle
 case listening
 case transcribing
 }

 private(set) var state: BridgeState = .idle
 private(set) var lastTranscript: String?
 private(set) var errorMessage: String?

 @ObservationIgnored private var observer: NSObjectProtocol?
 @ObservationIgnored private var connectionObserver: NSObjectProtocol?
 @ObservationIgnored private var connectObserver: NSObjectProtocol?
 @ObservationIgnored private let sampleRate: Double = 16000

 // Speech recognition
 @ObservationIgnored private var recognitionRequest: SFSpeechAudioBufferRecognitionRequest?
 @ObservationIgnored private var recognitionTask: SFSpeechRecognitionTask?
 @ObservationIgnored private var recognizer: SFSpeechRecognizer?
 @ObservationIgnored private var isSessionActive = false

 // BLE chunk accumulation – pendant sends 20-byte chunks; we batch to ~256 ms
 @ObservationIgnored private var pendingPCMDData = Data()
 @ObservationIgnored private let minSamplesPerBatch: Int = 4096 // ~256 ms @ 16 kHz

```

[illegible]

```

 resetVAD()
 pendingPCMDData = Data()
 startRecognitionSession()
 }

// MARK: - Audio handling

private func handleIncomingAudio(_ data: Data) {
 if !isSessionActive { startRecognitionSession() }
 pendingPCMDData.append(data)
 let sampleCount = pendingPCMDData.count / MemoryLayout<Int16>.size
 if sampleCount >= minSamplesPerBatch {
 flushAudioBuffer()
 }
}

private func flushAudioBuffer() {
 guard let request = recognitionRequest else { return }
 let data = pendingPCMDData
 pendingPCMDData = Data()

 let sampleCount = data.count / MemoryLayout<Int16>.size
 guard sampleCount > 0 else { return }

 guard let pcmBuf = AVAudioPCMBuffer(pcmFormat: floatFormat, frameCapacity: UInt32(sampleCount)) else { return }
 pcmBuf.frameLength = UInt32(sampleCount)

 var sumSquares: Float = 0
 data.withUnsafeBytes { raw in
 guard let src = raw.baseAddress?.assumingMemoryBound(to: Int16.self) else { return }
 guard let dst = pcmBuf.floatChannelData?[0] else { return }
 for i in 0..

```

```

// MARK: - Speech recognition

private func startRecognitionSession() {
 guard !isSessionActive else { return }
 guard let recognizer, recognizer.isAvailable else {
 errorMessage = "Speech recognition not available"
 return
 }

 guard SFSpeechRecognizer.authorizationStatus() == .authorized else {
 Task {
 let status = await requestSpeechAuthorization()
 if status == .authorized { startRecognitionSession() }
 else { errorMessage = "Speech recognition not authorized" }
 }
 return
 }

 recognitionTask?.cancel()
 recognitionRequest = nil
 pendingPCMDData = Data()
 resetVAD()

 let request = SFSpeechAudioBufferRecognitionRequest()
 request.shouldReportPartialResults = true
 // Let on-device recognition run when available for lower latency

 if #available(iOS 17, *) {
 request.requiresOnDeviceRecognition = false
 }
 recognitionRequest = request
 isSessionActive = true
 state = .listening
 errorMessage = nil

 print("[PendantBridge] Recognition session started")

 recognitionTask = recognizer.recognitionTask(with: request) { [weak self] result, error in
 Task { @MainActor in
 guard let self else { return }

 if let result {
 let transcript = result.bestTranscription.formattedString
 self.lastTranscript = transcript

 if result.isFinal {
 print("[PendantBridge] Final: \(transcript)")
 self.isSessionActive = false
 if !transcript.trimmingCharacters(in: .whitespaces).isEmpty {
 self.deliverTranscript(transcript)
 }
 // Start fresh session for next utterance
 self.startRecognitionSession()
 }
 }

 if let error {
 print("[PendantBridge] Recognition error: \(error.localizedDescription)")
 self.isSessionActive = false
 DispatchQueue.main.asyncAfter(deadline: .now() + self.errorRestartDelay) { [weak self] in
 guard let self, self.state != .idle else { return }
 self.startRecognitionSession()
 }
 }
 }
 }

 restartTimer?.invalidate()
 restartTimer = Timer.scheduledTimer(withTimeInterval: sessionTimeout, repeats: false) { [weak self] _ in
 Task { @MainActor in self?.restartSession() }
 }
}

private func restartSession() {
 print("[PendantBridge] Session timeout – restarting")
 flushAudioBuffer()
 recognitionRequest?.endAudio()
 recognitionTask?.cancel()
 recognitionTask = nil
 recognitionRequest = nil
 isSessionActive = false
 startRecognitionSession()
}

private func deliverTranscript(_ transcript: String) {
 print("[PendantBridge] Delivering: \(transcript)")
 lastTranscript = transcript
}

```

```

 state = .transcribing

 // Notify pendant that AI is now processing
 NativeIntegrationManager.shared.pendant.sendCommand("THINK")

 onTranscript?(transcript)

 DispatchQueue.main.asyncAfter(deadline: .now() + 0.5) { [weak self] in
 if self?.state == .transcribing { self?.state = .listening }
 }
 }

 private func stopRecognition() {
 print("[PendantBridge] Stopping recognition")
 restartTimer?.invalidate()
 restartTimer = nil
 isSessionActive = false
 flushAudioBuffer()
 recognitionRequest?.endAudio()
 recognitionTask?.cancel()
 recognitionTask = nil
 recognitionRequest = nil
 pendingPCMDData = Data()
 resetVAD()
 state = .idle
 }

 private func requestSpeechAuthorization() async -> SFSpeechRecognizerAuthorizationStatus {
 await withCheckedContinuation { continuation in
 SFSpeechRecognizer.requestAuthorization { continuation.resume(returning: $0) }
 }
 }
}

```

# iOS App — Views

## MainTabView.swift — OxyApp/OxyApp/Views/MainTabView.swift

```
import SwiftUI

struct MainTabView: View {
 @Environment(AppState.self) private var appState
 @AppStorage("oxy_accentColor") private var accentColor = "stone"
 @State private var selectedTab = Tab.chat

 enum Tab: String {
 case chat, proactive, history, connectors, settings
 }

 var body: some View {
 TabView(selection: $selectedTab) {
 ChatView()
 .tabItem {
 Image(systemName: "bubble.left.fill")
 Text("Chat")
 }
 .tag(Tab.chat)

 ProactiveView()
 .tabItem {
 Image(systemName: "sparkles")
 Text("Today")
 }
 .tag(Tab.proactive)

 HistoryView()
 .tabItem {
 Image(systemName: "clock.fill")
 Text("Chats")
 }
 .tag(Tab.history)

 ConnectorsView()
 .tabItem {
 Image(systemName: "link")
 Text("Connectors")
 }
 .tag(Tab.connectors)

 SettingsView()
 .tabItem {
 Image(systemName: "gearshape.fill")
 Text("Settings")
 }
 .tag(Tab.settings)
 }
 .tint(Color.oxyStone)
 .id(accentColor)
 .onReceive(NotificationCenter.default.publisher(for: .oxyJumpToChat)) { _ in
 withAnimation { selectedTab = .chat }
 }
 }
}

#Preview {
 MainTabView()
 .environment(AppState())
}
```

## ChatView.swift — OxyApp/OxyApp/Views/Chat/ChatView.swift

```
import MessageUI
import Network
import PhotosUI
import SwiftUI
import UIKit
import UniformTypeIdentifiers

struct ChatView: View {
 @Environment(AppState.self) private var appState
 @State private var viewModel = ChatViewModel()
 @State private var voiceInput = VoiceInputManager()
 @FocusState private var isInputFocused: Bool
 @State private var showSearch = false
 @State private var isIncognito = false
 @State private var pendingReviewAction: ActionResult?
 @State private var messageDraft: MessageDraft?
 @State private var messageComposerAlert: String?
 @State private var handledReviewActionIDs = Set<String>()
```



```

@State private var handledMessageComposeActionIDs = Set<String>()
@State private var showPhotoPicker = false
@State private var showFileImporter = false
@State private var showAttachMenu = false
@State private var selectedPhotoItem: PhotosPickerItem?
@State private var pendingImageData: Data?
@State private var pendingImageName: String?
@State private var pendingImageMimeType = "image/jpeg"
@State private var pendingIsImage = true
@State private var isOffline = false
@State private var pendantLive = PendantLiveService()
private let networkMonitor = NWPathMonitor()

var body: some View {
 NavigationStack {
 ZStack {
 Color.oxyBg.ignoresSafeArea()

 VStack(spacing: 0) {
 // Offline banner
 if isOffline {
 HStack(spacing: 8) {
 Image(systemName: "wifi.slash")
 .font(.system(size: 12, weight: .semibold))
 Text("No internet connection")
 .font(.system(size: 12, weight: .medium))
 }
 .foregroundColor(Color.white)
 .frame(maxWidth: .infinity)
 .padding(.vertical, 7)
 .background(Color(red: 0.85, green: 0.62, blue: 0.22))
 .transition(.opacity.combined(with: .move(edge: .top)))
 }

 // Incognito banner
 if isIncognito {
 HStack(spacing: 8) {
 Image(systemName: "eye.slash.fill")
 .font(.system(size: 12))
 Text("Vanish mode – messages won't be saved")
 .font(.system(size: 12, weight: .medium))
 }
 .foregroundColor(Color.oxyStone)
 .frame(maxWidth: .infinity)
 .padding(.vertical, 6)
 .background(Color.oxyStone.opacity(0.1))
 }

 if viewModel.isViewingHistorySnapshot {
 HStack(spacing: 10) {
 Image(systemName: "clock.arrow.circlepath")
 .font(.system(size: 12, weight: .semibold))
 VStack(alignment: .leading, spacing: 1) {
 Text("Viewing history")
 .font(.system(size: 12, weight: .semibold))
 if let label = viewModel.historySnapshotLabel {
 Text(label)
 .font(.system(size: 11))
 }
 }
 Spacer()
 Button("Current Chat") {
 Task {
 await viewModel.returnToCurrentChat(userId: appState.userId)
 }
 }
 .font(.system(size: 12, weight: .semibold))
 }
 .foregroundColor(Color.oxyStone)
 .padding(.horizontal, 14)
 .padding(.vertical, 8)
 .background(Color.oxyStone.opacity(0.1))
 .transition(.opacity.combined(with: .move(edge: .top)))
 }

 if let networkError = viewModel.networkError {
 ErrorBanner(
 message: networkError,
 onRetry: {
 viewModel.retryLastFailedMessage(userId: appState.userId)
 },
 onDismiss: {
 viewModel.networkError = nil
 }
)
 .padding(.top, 8)
 .transition(.opacity.combined(with: .move(edge: .top)))
 }
 }
 }
 }
}

```

```

 }

 // Messages
 ScrollViewReader { proxy in
 ScrollView {
 LazyVStack(spacing: 2) {
 if viewModel.messages.isEmpty && !viewModel.isSending {
 WelcomeCard(onQuickAction: { action in
 viewModel.inputText = action
 viewModel.sendMessage(userId: appState.userId)
 })
 .padding(.top, 40)
 .padding(.bottom, 20)
 }

 ForEach(viewModel.messages) { message in
 MessageBubble(
 message: message,
 showsTypingIndicator: viewModel.statusLabel == nil,
 onActionCommand: { command in
 viewModel.sendCommand(command, userId: appState.userId)
 },
 onOpenAction: { action in
 handleActionOpen(action)
 }
)
 .id(message.id)
 .transition(.opacity.combined(with: .move(edge: .bottom)))
 }

 if let status = viewModel.statusLabel {
 StatusIndicator(label: status)
 .id("status")
 .padding(.horizontal, 16)
 .padding(.top, 4)
 }
 }
 .padding(.vertical, 12)
 .animation(.spring(response: 0.4, dampingFraction: 0.8), value: viewModel.messages.count)
 }
 .scrollDismissesKeyboard(.interactively)
 .onChange(of: viewModel.messages.count) {
 guard viewModel.scrollTargetMessageID == nil else { return }
 withAnimation(.spring(response: 0.4, dampingFraction: 0.8)) {
 if let lastId = viewModel.messages.last?.id {
 proxy.scrollTo(lastId, anchor: .bottom)
 } else {
 proxy.scrollTo("status", anchor: .bottom)
 }
 }
 }
 .onChange(of: viewModel.messages.last?.content) {
 guard viewModel.scrollTargetMessageID == nil else { return }
 withAnimation(.spring(response: 0.4, dampingFraction: 0.8)) {
 if let lastId = viewModel.messages.last?.id {
 proxy.scrollTo(lastId, anchor: .bottom)
 } else {
 proxy.scrollTo("status", anchor: .bottom)
 }
 }
 }
 .onChange(of: viewModel.scrollTargetMessageID) { _, targetID in
 guard let targetID else { return }
 Task { @MainActor in
 try? await Task.sleep(for: .milliseconds(150))
 withAnimation(.easeInOut(duration: 0.25)) {
 proxy.scrollTo(targetID, anchor: .center)
 }
 try? await Task.sleep(for: .milliseconds(700))
 viewModel.scrollTargetMessageID = nil
 }
 }
 .onChange(of: viewModel.messages) {
 presentPendingReviewIfNeeded()
 presentMessageComposerIfNeeded()
 }
 }

 // Voice recording overlay
 if voiceInput.isRecording {
 VoiceRecordingBar(
 transcript: voiceInput.transcript,
 onStop: {
 voiceInput.stopRecording()
 if !voiceInput.transcript.isEmpty {
 viewModel.inputText = voiceInput.transcript
 viewModel.sendMessage(userId: appState.userId)
 }
 }
)
 }

```

```

 }
 },
 onCancel: {
 voiceInput.stopRecording()
 }
)
 }

 // Pendant listening indicator
 if pendantLive.liveState != .idle {
 PendantListeningBar(state: pendantLive.liveState, transcript: pendantLive.userTranscript)
 }

 // Input bar
 ChatInputBar(
 text: $viewModel.inputText,
 isSending: viewModel.isSending || isOffline,
 isRecording: voiceInput.isRecording,
 isPreparingVoice: voiceInput.isPreparing,
 attachmentLabel: pendingImageName,
 attachmentData: pendingImageData,
 attachmentIsImage: pendingIsImage,
 isFocused: $isInputFocused,
 onSend: {
 sendCurrentDraft()
 },
 onVoice: {
 if voiceInput.isRecording {
 voiceInput.stopRecording()
 if !voiceInput.transcript.isEmpty {
 viewModel.inputText = voiceInput.transcript
 viewModel.sendMessage(userId: appState.userId)
 }
 } else {
 voiceInput.startRecording()
 }
 },
 onAttach: {
 showAttachMenu = true
 },
 onRemoveAttachment: {
 pendingImageData = nil
 pendingImageName = nil
 pendingIsImage = true
 selectedPhotoItem = nil
 }
)
}

.navigationBarTitleDisplayMode(.inline)
.toolbar {
 ToolbarItem(placement: .topBarLeading) {
 Button(action: { showSearch = true }) {
 Image(systemName: "magnifyingglass")
 .font(.system(size: 17))
 .foregroundColor(Color.oxySub)
 }
 }

 ToolbarItem(placement: .principal) {
 HStack(spacing: 10) {
 ZStack {
 Circle()
 .fill(
 LinearGradient(
 colors: [Color.oxyStone.opacity(0.3), Color.oxyStone.opacity(0.15)],
 startPoint: .topLeading,
 endPoint: .bottomTrailing
)
)
 .frame(width: 34, height: 34)

 Image(systemName: "sparkles")
 .font(.system(size: 14, weight: .medium))
 .foregroundColor(Color.oxyStone)
 }

 VStack(alignment: .leading, spacing: 1) {
 Text("Oxy")
 .font(.system(size: 16, weight: .semibold))
 .foregroundColor(Color.oxyText)
 HStack(spacing: 4) {
 Circle()
 .fill(Color.oxyGreen)
 .frame(width: 6, height: 6)
 Text("Online")
 }
 }
 }
 }
}

```

```

 .font(.system(size: 11))
 .foregroundColor(Color.oxySub)
 }
 }
 }
}

ToolbarItem(placement: .topBarTrailing) {
 Menu {
 Button(action: {
 withAnimation(.easeInOut(duration: 0.2)) {
 viewModel.startNewChat(userId: appState.userId)
 }
 }) {
 Label("New Chat", systemImage: "square.and.pencil")
 }
 Divider()
 Button(action: {
 withAnimation(.easeInOut(duration: 0.2)) {
 isIncognito.toggle()
 }
 }) {
 Label(
 isIncognito ? "Turn Off Vanish Mode" : "Vanish Mode",
 systemImage: isIncognito ? "eye.fill" : "eye.slash.fill"
)
 }
 Button(action: {
 viewModel.requestLocationAccess()
 }) {
 Label("Share Location", systemImage: "location.fill")
 }
 Divider()
 Button(role: .destructive, action: { appState.logout() }) {
 Label("Sign Out", systemImage: "rectangle.portrait.and.arrow.right")
 }
 } label: {
 Image(systemName: "ellipsis.circle")
 .font(.system(size: 18))
 .foregroundColor(Color.oxySub)
 }
}

.toolbarBackground(Color.oxySurface1, for: .navigationBar)
.toolbarBackground(.visible, for: .navigationBar)
.sheet(isPresented: $showSearch) {
 ChatSearchView(userId: appState.userId) { result in
 guard let createdAt = result.createdAt else { return }
 showSearch = false
 Task {
 await viewModel.loadHistoryAround(userId: appState.userId, createdAt: createdAt, messageId: result.messageId)
 }
 }
}

.sheet(item: $pendingReviewAction) { action in
 ActionReviewSheet(
 action: action,
 onConfirm: {
 handledReviewActionIDs.insert(action.id)
 pendingReviewAction = nil
 viewModel.sendCommand("confirm", userId: appState.userId)
 },
 onCancel: {
 handledReviewActionIDs.insert(action.id)
 pendingReviewAction = nil
 viewModel.sendCommand("cancel", userId: appState.userId)
 }
)
 .presentationDetents([.height(340), .medium])
 .presentationDragIndicator(.visible)
}

.sheet(item: $messageDraft) { draft in
 MessageComposeSheet(draft: draft) { result in
 messageDraft = nil
 handleMessageComposeResult(result)
 }
}

.alert("Messages unavailable", isPresented: Binding(
 get: { messageComposerAlert != nil },
 set: { if !$0 { messageComposerAlert = nil } }
)) {
 Button("OK", role: .cancel) {}
}

message: {
 Text(messageComposerAlert ?? "")
}

.confirmationDialog("Add attachment", isPresented: $showAttachMenu, titleVisibility: .visible) {

```

```

 Button("Photo Library") { showPhotoPicker = true }
 Button("Files") { showFileImporter = true }
 Button("Cancel", role: .cancel) {}
 }
 .photosPicker(isPresented: $showPhotoPicker, selection: $selectedPhotoItem, matching: .images)
 .fileImporter(
 isPresented: $showFileImporter,
 allowedContentTypes: [.pdf, .plainText, .commaSeparatedText, .json, .image, .data],
 allowsMultipleSelection: false
) { result in
 guard let url = try? result.get().first else { return }
 let didAccess = url.startAccessingSecurityScopedResource()
 defer { if didAccess { url.stopAccessingSecurityScopedResource() } }
 guard let data = try? Data(contentsOf: url) else { return }
 pendingImageData = data
 pendingImageName = url.lastPathComponent
 pendingImageMimeType = mimeType(for: url)
 pendingIsImage = pendingImageMimeType.hasPrefix("image/")
 }
 .onChange(of: selectedPhotoItem) { _, item in
 guard let item else { return }
 Task {
 if let data = try? await item.loadTransferable(type: Data.self) {
 await MainActor.run {
 pendingImageData = data
 pendingImageName = "Photo"
 pendingImageMimeType = data.starts(with: [0x89, 0x50, 0x4E, 0x47]) ? "image/png" : "image/jpeg"
 pendingIsImage = true
 }
 }
 }
 }
 .onChange(of: pendingReviewAction) { oldValue, newValue in
 if let oldValue, newValue == nil {
 handledReviewActionIDs.insert(oldValue.id)
 }
 }
}
.task {
 await viewModel.prepareChat(userId: appState.userId)
}
.onAppear {
 viewModel.requestLocationAccess()
 networkMonitor.pathUpdateHandler = { path in
 DispatchQueue.main.async {
 withAnimation(.easeInOut(duration: 0.3)) {
 isOffline = path.status != .satisfied
 }
 }
 }
}
networkMonitor.start(queue: DispatchQueue(label: "oxy.networkMonitor"))

let vm = viewModel
let state = appState
// Native actions signalled by Gemini Live (play_music, make_call)
// are dispatched here so ChatViewModel can use its existing native handlers.
NotificationCenter.default.addObserver(
 forName: .pendantNativeAction,
 object: nil,
 queue: .main
) { notification in
 guard let signal = notification.userInfo?["signal"] as? String else { return }
 switch signal {
 case "play_music":
 let query = (notification.userInfo?["data"] as? [String: Any])?["query"] as? String ?? ""
 vm.executeSilently("play \(query)", userId: state.userId)
 case "make_call":
 let contact = (notification.userInfo?["data"] as? [String: Any])?["contact"] as? String ?? ""
 if let url = URL(string: "tel://\(contact.filter { $0.isNumber || $0 == "+" })") {
 UIApplication.shared.open(url)
 }
 default:
 break
 }
}
}
.onReceive(NotificationCenter.default.publisher(for: .oxyJumpToChat)) { notification in
 guard let lastAt = notification.userInfo?["lastAt"] as? String, !lastAt.isEmpty else { return }
 let messageId = notification.userInfo?["messageId"] as? String
 Task {
 await viewModel.loadHistoryAround(userId: appState.userId, createdAt: lastAt, messageId: messageId)
 }
}
}

private func sendCurrentDraft() {
 if let pendingImageData {

```

```

 let defaultName = pendingImageMimeType == "image/png" ? "photo.png" : "photo.jpg"
 viewModel.sendMessage(
 userId: appState.userId,
 imageData: pendingImageData,
 fileName: (pendingIsImage ? nil : pendingImageName) ?? defaultName,
 mimeType: pendingImageMimeType,
 isImage: pendingIsImage
)
 self.pendingImageData = nil
 pendingImageName = nil
 pendingIsImage = true
 selectedPhotoItem = nil
 } else {
 viewModel.sendMessage(userId: appState.userId)
 }
}

private func mimeType(for url: URL) -> String {
 switch url.pathExtension.lowercased() {
 case "pdf": return "application/pdf"
 case "txt", "text": return "text/plain"
 case "md", "markdown": return "text/markdown"
 case "csv": return "text/csv"
 case "json": return "application/json"
 case "png": return "image/png"
 case "jpg", "jpeg": return "image/jpeg"
 case "heic": return "image/heic"
 case "webp": return "image/webp"
 case "doc": return "application/msword"

 case "docx": return "application/vnd.openxmlformats-officedocument.wordprocessingml.document"
 case "xls": return "application/vnd.ms-excel"
 case "xlsx": return "application/vnd.openxmlformats-officedocument.spreadsheetml.sheet"
 default: return "application/octet-stream"
 }
}

private func presentPendingReviewIfNeeded() {
 guard pendingReviewAction == nil else { return }
 guard let action = viewModel.messages
 .suffix(3)
 .flatMap(\.actions)
 .last(where: { $0.pending && !handledReviewActionIDs.contains($0.id) }) else {
 return
 }
 handledReviewActionIDs.insert(action.id)
 pendingReviewAction = action
}

private func presentMessageComposerIfNeeded() {
 guard pendingReviewAction == nil, messageDraft == nil else { return }
 let cutoff = Date().addingTimeInterval(-90)
 guard let action = viewModel.messages
 .suffix(3)
 .filter({ $0.timestamp >= cutoff })
 .flatMap(\.actions)
 .last(where: {
 $0.action == "send_message"
 && $0.success
 && !$0.pending
 && !handledMessageComposeActionIDs.contains($0.id)
 }) else {
 return
 }
 handledMessageComposeActionIDs.insert(action.id)
 guard let draft = MessageDraft(action: action) else { return }
 Task { @MainActor in
 try? await Task.sleep(nanoseconds: 250_000_000)
 guard pendingReviewAction == nil, messageDraft == nil else { return }
 presentMessageDraft(draft)
 }
}

private func handleActionOpen(_ action: ActionResult) {
 if action.action == "send_message", let draft = MessageDraft(action: action) {
 presentMessageDraft(draft)
 return
 }
 viewModel.openActionLink(action)
}

private func presentMessageDraft(_ draft: MessageDraft) {
 guard MFMessageComposeViewController.canSendText() else {
 messageComposerAlert = "This device cannot send text messages from an in-app composer."
 return
 }
 messageDraft = draft
}

```

```

 }

 private func handleMessageComposeResult(_ result: MessageComposeResult) {
 switch result {
 case .sent:
 viewModel.statusLabel = "Message sent"
 case .cancelled:
 viewModel.statusLabel = nil
 case .failed:
 viewModel.statusLabel = "Message failed"
 @unknown default:
 viewModel.statusLabel = nil
 }
 }
}

struct MessageDraft: Identifiable, Equatable {
 let id = UUID()
 let recipients: [String]
 let body: String

 init(recipients: [String], body: String) {
 self.recipients = recipients
 self.body = body
 }

 init?(action: ActionResult) {
 guard let link = action.deepLink ?? action.webLink,
 link.lowercased().hasPrefix("sms:") else { return nil }
 let rawPayload = String(link.dropFirst(4))
 let parts = rawPayload.split(separator: "?", maxSplits: 1, omittingEmptySubsequences: false)
 let rawRecipientAndMaybeBody = String(parts.first ?? "")
 let rawQuery = parts.count > 1 ? String(parts[1]) : ""
 let legacyPieces = rawRecipientAndMaybeBody.split(separator: "&", maxSplits: 1, omittingEmptySubsequences: false)
 let rawRecipient = String(legacyPieces.first ?? "")
 let query = rawQuery.isEmpty && legacyPieces.count > 1 ? String(legacyPieces[1]) : rawQuery
 let recipient = rawRecipient.removingPercentEncoding ?? rawRecipient
 let body = MessageDraft.body(from: query)
 guard !recipient.isEmpty || !body.isEmpty else { return nil }
 self.recipients = recipient.isEmpty ? [] : [recipient]
 self.body = body
 }

 private static func body(from query: String) -> String {
 guard !query.isEmpty else { return "" }
 var components = URLComponents()
 components.percentEncodedQuery = query.trimmingCharacters(in: CharacterSet(charactersIn: "&?"))
 return components.queryItems?.first(where: { $0.name == "body" })?.value ?? ""
 }
}

private struct MessageComposeSheet: UIViewControllerRepresentable {
 let draft: MessageDraft
 let onFinish: @MainActor @Sendable (MessageComposeResult) -> Void

 func makeUIViewController(context: Context) -> MFMessageComposeViewController {
 let controller = MFMessageComposeViewController()
 controller.messageComposeDelegate = context.coordinator
 controller.recipients = draft.recipients
 controller.body = draft.body
 return controller
 }

 func updateUIViewController(_ viewController: MFMessageComposeViewController, context: Context) {}

 func makeCoordinator() -> Coordinator {
 Coordinator(onFinish: onFinish)
 }

 final class Coordinator: NSObject, MFMessageComposeViewControllerDelegate {
 let onFinish: @MainActor @Sendable (MessageComposeResult) -> Void

 init(onFinish: @escaping @MainActor @Sendable (MessageComposeResult) -> Void) {
 self.onFinish = onFinish
 }

 func messageComposeViewController(
 _ controller: MFMessageComposeViewController,
 didFinishWith result: MessageComposeResult
) {
 let finish = onFinish
 Task { @MainActor in
 controller.dismiss(animated: true)
 finish(result)
 }
 }
 }
}

```

```

}

private struct ActionReviewSheet: View {
 let action: ActionResult
 let onConfirm: () -> Void
 let onCancel: () -> Void

 private var title: String {
 action.actionSummary ?? {
 switch action.action {
 case "send_email": return "Review email"
 case "send_message": return "Review message"
 case "send_telegram": return "Review Telegram"
 case "make_call": return "Review call"
 default: return "Review action"
 }
 }()
 }

 private var detail: String {
 action.cardText ?? action.text ?? "Ready for review."
 }

 var body: some View {
 VStack(alignment: .leading, spacing: 16) {
 Capsule()
 .fill(Color.oxyLine2)
 .frame(width: 36, height: 4)
 .frame(maxWidth: .infinity)

 HStack(spacing: 12) {
 Image(systemName: iconName)
 .font(.system(size: 15, weight: .bold))
 .foregroundColor(Color.oxyStone)
 .frame(width: 34, height: 34)
 .background(Color.oxyStone.opacity(0.12))
 .clipShape(Circle())

 VStack(alignment: .leading, spacing: 2) {
 Text(title)
 .font(.system(size: 19, weight: .semibold))
 .foregroundColor(Color.oxyText)
 Text("One tap when it looks right.")
 .font(.system(size: 12))
 .foregroundColor(Color.oxySub)
 }
 Spacer()
 }

 Text(detail)
 .font(.system(size: 15))
 .foregroundColor(Color.oxyText)
 .lineSpacing(5)
 .frame(maxWidth: .infinity, alignment: .leading)
 .padding(14)
 .background(Color.oxySurface2.opacity(0.72))
 .clipShape(RoundedRectangle(cornerRadius: 16))
 .overlay(
 RoundedRectangle(cornerRadius: 16)
 .stroke(Color.oxyLine, lineWidth: 1)
)

 HStack(spacing: 10) {
 Button(action: onCancel) {
 Label("Cancel", systemImage: "xmark")
 .font(.system(size: 15, weight: .semibold))
 .frame(maxWidth: .infinity)
 .padding(.vertical, 13)
 }
 .buttonStyle(.plain)
 .foregroundColor(Color.oxySub)
 .background(Color.oxySurface3)
 .clipShape(RoundedRectangle(cornerRadius: 12))

 Button(action: onConfirm) {
 Label("Send", systemImage: "arrow.up")
 .font(.system(size: 15, weight: .semibold))
 .frame(maxWidth: .infinity)
 .padding(.vertical, 13)
 }
 .buttonStyle(.plain)
 .foregroundColor(Color.oxyOnAccent)
 .background(Color.oxyStone)
 .clipShape(RoundedRectangle(cornerRadius: 12))
 }
 }
 .padding(18)
 }
}

```



```

 .frame(maxWidth: .infinity, maxHeight: .infinity, alignment: .topLeading)
 .background(Color.oxySurface1)
 }

 private var iconName: String {
 switch action.action {
 case "send_email": return "envelope.fill"
 case "send_message": return "message.fill"
 case "send_telegram": return "paperplane.fill"

 case "make_call": return "phone.fill"
 default: return "checkmark.seal.fill"
 }
 }
}

// MARK: - Voice Recording Bar

private struct VoiceRecordingBar: View {
 let transcript: String
 let onStop: () -> Void
 let onCancel: () -> Void

 @State private var pulse = false

 var body: some View {
 VStack(spacing: 8) {
 Divider().overlay(Color.oxyLine2)

 HStack(spacing: 12) {
 Button(action: onCancel) {
 Image(systemName: "xmark.circle.fill")
 .font(.system(size: 24))
 .foregroundColor(Color.oxyDim)
 }

 VStack(alignment: .leading, spacing: 2) {
 HStack(spacing: 6) {
 Circle()
 .fill(Color.oxyRed)
 .frame(width: 8, height: 8)
 .scaleEffect(pulse ? 1.2 : 0.8)
 .animation(.easeInOut(duration: 0.6).repeatForever(autoreverses: true), value: pulse)
 Text("Listening...")
 .font(.system(size: 13, weight: .medium))
 .foregroundColor(Color.oxyText)
 }

 if !transcript.isEmpty {
 Text(transcript)
 .font(.system(size: 13))
 .foregroundColor(Color.oxySub)
 .lineLimit(2)
 }
 }
 }

 Spacer()

 Button(action: onStop) {
 Image(systemName: "arrow.up")
 .font(.system(size: 15, weight: .semibold))
 .foregroundColor(Color.oxyOnAccent)
 .frame(width: 36, height: 36)
 .background(Color.oxyStone)
 .clipShape(Circle())
 }
 }
 .padding(.horizontal, 12)
 .padding(.vertical, 8)
 }
 .background(Color.oxySurface1)
 .onAppear { pulse = true }
}

// MARK: - Chat Search View

struct ChatSearchView: View {
 let userId: String
 let onSelect: (SearchResult) -> Void
 @Environment(\.dismiss) private var dismiss
 @State private var query = ""
 @State private var results: [SearchResult] = []
 @State private var sessions: [ChatSessionSummary] = []
 @State private var isSearching = false
 @State private var isLoadingSessions = true
 @State private var searchTask: Task<Void, Never>?

```

```

var body: some View {
 NavigationStack {
 ZStack {
 Color.oxyBg.ignoresSafeArea()

 if query.isEmpty {
 if isLoadingSessions {
 ProgressView()
 .tint(Color.oxyStone)
 } else if sessions.isEmpty {
 VStack(spacing: 12) {
 Image(systemName: "magnifyingglass")
 .font(.system(size: 36))
 .foregroundColor(Color.oxyDim)
 Text("Search conversations")
 .font(.system(size: 15))
 .foregroundColor(Color.oxySub)
 Text("Find messages from your chat history")
 .font(.system(size: 13))
 .foregroundColor(Color.oxyDim)
 }
 } else {
 ScrollView {
 LazyVStack(alignment: .leading, spacing: 0) {
 Text("Recent chats")
 .font(.system(size: 12, weight: .semibold))
 .foregroundColor(Color.oxyDim)
 .textCase(.uppercase)
 .tracking(0.4)
 .padding(.horizontal, 16)
 .padding(.vertical, 10)

 ForEach(sessions) { session in
 Button {
 onSelect(SearchResult(
 role: "user",
 content: session.title,
 createdAt: session.lastAt
))
 dismiss()
 } label: {
 ChatSessionRow(session: session)
 }

 .buttonStyle(.plain)
 .padding(.horizontal, 16)
 .padding(.vertical, 12)

 if session.id != sessions.last?.id {
 Divider()
 .overlay(Color.oxyLine)
 .padding(.horizontal, 16)
 }
 }
 }
 .padding(.vertical, 8)
 }
 }
 } else if results.isEmpty && !isSearching && !query.isEmpty {
 VStack(spacing: 12) {
 Image(systemName: "doc.text.magnifyingglass")
 .font(.system(size: 36))
 .foregroundColor(Color.oxyDim)
 Text("No results found")
 .font(.system(size: 15))
 .foregroundColor(Color.oxySub)
 }
 } else {
 ScrollView {
 LazyVStack(spacing: 0) {
 ForEach(results) { result in
 Button {
 onSelect(result)
 dismiss()
 } label: {
 SearchResultRow(result: result)
 }
 .buttonStyle(.plain)
 .padding(.horizontal, 16)
 .padding(.vertical, 12)

 if result.id != results.last?.id {
 Divider()
 .overlay(Color.oxyLine)
 .padding(.horizontal, 16)
 }
 }
 }
 }
 }
 }
 }
}

```

```

 }
 .padding(.vertical, 8)
 }
}

if isSearching {
 ProgressView()
 .tint(Color.oxyStone)
}

}
.navigationTitle("Search")
.navigationBarTitleDisplayMode(.inline)
.toolbarBackground(Color.oxySurface1, for: .navigationBar)
.toolbarBackground(.visible, for: .navigationBar)
.toolbar {
 ToolbarItem(placement: .cancellationAction) {
 Button("Done") { dismiss() }
 .foregroundColor(Color.oxyStone)
 }
}
.searchable(text: $query, placement: .navigationBarDrawer(displayMode: .always), prompt: "Search messages...")
.task {
 await loadSessions()
}
.onChange(of: query) { _, newValue in
 let trimmed = newValue.trimmingCharacters(in: .whitespacesAndNewlines)
 searchTask?.cancel()
 guard !trimmed.isEmpty else {
 results = []
 isSearching = false
 return
 }
 searchTask = Task {
 try? await Task.sleep(for: .milliseconds(300))
 if Task.isCancelled { return }
 await search(trimmed)
 }
}
.onDisappear { searchTask?.cancel() }
}
}

private func search(_ q: String) async {
 isSearching = true
 defer { if !Task.isCancelled { isSearching = false } }
 do {
 let data = try await APIClient.shared.request(
 path: "/history/\(userId)/search",
 queryItems: [URLQueryItem(name: "q", value: q)]
)
 // Drop stale results: a newer keystroke cancelled this task while in flight.
 if Task.isCancelled { return }
 let response = try JSONDecoder().decode(SearchResponse.self, from: data)
 results = response.results
 } catch {
 if !Task.isCancelled { results = [] }
 }
}

private func loadSessions() async {
 isLoadingSessions = true
 do {
 let data = try await APIClient.shared.request(path: "/history/\(userId)/sessions")
 let response = try JSONDecoder().decode(ChatSessionsResponse.self, from: data)
 sessions = response.sessions
 } catch {
 sessions = []
 }
 isLoadingSessions = false
}

private struct SearchResultRow: View {
 let result: SearchResult

 var body: some View {
 VStack(alignment: .leading, spacing: 5) {
 HStack {
 Text(result.role == "user" ? "YOU" : "OXY")
 .font(.system(size: 10, weight: .semibold))
 .foregroundColor(Color.oxySub)
 .tracking(0.5)

 Spacer()

 if let date = result.formattedDate {

```

```

 Text(date)
 .font(.system(size: 11))
 .foregroundColor(Color.oxyDim)
 }
 }

 Text(result.content)
 .font(.system(size: 14))
 .foregroundColor(Color.oxyText)
 .lineLimit(3)
 }
}

struct SearchResult: Codable, Identifiable {
 let messageId: String?
 let role: String
 let content: String
 let createdAt: String?

 var id: String { messageId ?? ((createdAt ?? "") + role + String(content.prefix(20))) }

 init(messageId: String? = nil, role: String, content: String, createdAt: String?) {
 self.messageId = messageId
 self.role = role
 self.content = content
 self.createdAt = createdAt
 }

 enum CodingKeys: String, CodingKey {
 case messageId = "id"
 case role, content
 case createdAt = "created_at"
 }

 var formattedDate: String? {
 let date = Date.oxyParse(createdAt)
 guard let date else { return nil }
 let fmt = DateFormatter()
 fmt.dateFormat = "d MMM · HH:mm"
 return fmt.string(from: date)
 }
}

struct SearchResponse: Codable {
 let results: [SearchResult]
}

private struct ChatSessionRow: View {
 let session: ChatSessionSummary

 var body: some View {
 HStack(spacing: 0) {
 Text(session.title)
 .font(.system(size: 15, weight: .medium))
 .foregroundColor(Color.oxyText)
 .lineLimit(1)

 Spacer(minLength: 12)

 if let date = session.formattedDate {
 Text(date)
 .font(.system(size: 13))
 .foregroundColor(Color.oxyDim)
 }
 }
 }
}

struct ChatSessionSummary: Codable, Identifiable {
 let id: String
 let title: String
 let preview: String
 let startedAt: String?
 let lastAt: String?
 let messageCount: Int

 enum CodingKeys: String, CodingKey {
 case id, title, preview
 case startedAt = "started_at"
 case lastAt = "last_at"
 case messageCount = "message_count"
 }

 var formattedDate: String? {
 let date = Date.oxyParse(lastAt ?? startedAt)
 guard let date else { return nil }
 }
}

```

```

 let fmt = DateFormatter()
 fmt.dateFormat = "d MMM · HH:mm"
 return fmt.string(from: date)
 }
}

struct ChatSessionsResponse: Codable {
 let sessions: [ChatSessionSummary]
}

// MARK: - Welcome Card

private struct WelcomeCard: View {
 let onQuickAction: (String) -> Void

 var body: some View {
 VStack(spacing: 16) {
 ZStack {
 Circle()
 .fill(
 LinearGradient(
 colors: [Color.oxyStone.opacity(0.2), Color.oxyStone.opacity(0.05)],
 startPoint: .topLeading,
 endPoint: .bottomTrailing
)
)
 .frame(width: 72, height: 72)

 Image(systemName: "sparkles")
 .font(.system(size: 36))
 .foregroundColor(Color.oxyStone)
 }

 VStack(spacing: 6) {
 Text("Hey, I'm Oxy")
 .font(.system(size: 18, weight: .semibold))
 .foregroundColor(Color.oxyText)

 Text("What can I help with?")
 .font(.system(size: 13))
 .foregroundColor(Color.oxySub)
 .multilineTextAlignment(.center)
 }

 HStack(spacing: 8) {
 QuickChip(icon: "envelope.fill", label: "Check emails") {
 onQuickAction("Check my emails")
 }
 QuickChip(icon: "calendar", label: "My schedule") {
 onQuickAction("What's my schedule today?")
 }
 QuickChip(icon: "car.fill", label: "Book a ride") {
 onQuickAction("Book me a ride")
 }
 }
 }
 .padding(.horizontal, 32)
 }
}

private struct QuickChip: View {
 let icon: String
 let label: String
 let onTap: () -> Void

 var body: some View {
 Button(action: onTap) {
 VStack(spacing: 6) {
 Image(systemName: icon)
 .font(.system(size: 16))
 .foregroundColor(Color.oxyStone)
 Text(label)
 .font(.system(size: 10, weight: .medium))
 .foregroundColor(Color.oxySub)
 }
 .frame(maxWidth: .infinity)
 .padding(.vertical, 14)
 .background(Color.oxySurface2)
 .clipShape(RoundedRectangle(cornerRadius: 12))
 .overlay(
 RoundedRectangle(cornerRadius: 12)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
 }
 .buttonStyle(.plain)
 }
}

```

```

}

// MARK: - Chat Input Bar

private struct ChatInputBar: View {
 @Binding var text: String
 let isSending: Bool
 let isRecording: Bool
 let isPreparingVoice: Bool
 let attachmentLabel: String?
 let attachmentData: Data?
 let attachmentIsImage: Bool
 var isFocused: FocusState<Bool> & Binding
 let onSend: () -> Void
 let onVoice: () -> Void
 let onAttach: () -> Void
 let onRemoveAttachment: () -> Void
 @State private var voicePulse = false

 var body: some View {
 VStack(spacing: 0) {
 Divider()
 .overlay(Color.oxyLine2)

 if let attachmentLabel {
 HStack(spacing: 10) {
 if attachmentIsImage, let attachmentData, let uiImage = UIImage(data: attachmentData) {
 Image(uiImage: uiImage)
 .resizable()
 .scaledToFill()
 .frame(width: 38, height: 38)
 .clipShape(RoundedRectangle(cornerRadius: 8))
 } else {
 Image(systemName: attachmentIsImage ? "photo.fill" : "doc.fill")
 .font(.system(size: 16, weight: .semibold))
 .foregroundColor(Color.oxyStone)
 .frame(width: 38, height: 38)
 .background(Color.oxySurface3)
 .clipShape(RoundedRectangle(cornerRadius: 8))
 }
 VStack(alignment: .leading, spacing: 1) {
 Text(attachmentLabel)
 .font(.system(size: 13, weight: .medium))
 .foregroundColor(Color.oxyText)
 .lineLimit(1)
 Text(attachmentIsImage ? "Ready for analysis" : "Ready to read")
 .font(.system(size: 11))
 .foregroundColor(Color.oxyDim)
 }
 Spacer()
 Button(action: onRemoveAttachment) {
 Image(systemName: "xmark.circle.fill")
 .font(.system(size: 16))
 .foregroundColor(Color.oxyDim)
 }
 .buttonStyle(.plain)
 }
 .padding(.horizontal, 12)

 .padding(.vertical, 8)
 .background(Color.oxySurface2)
 .clipShape(RoundedRectangle(cornerRadius: 12))
 .padding(.horizontal, 12)
 .padding(.top, 8)
 }

 HStack(alignment: .bottom, spacing: 10) {
 Button(action: onAttach) {
 Image(systemName: "plus")
 .font(.system(size: 20, weight: .medium))
 .foregroundColor(Color.oxySub)
 .frame(width: 36, height: 36)
 }
 .disabled(isSending)

 HStack(spacing: 8) {
 TextField("Message Oxy...", text: $text, axis: .vertical)
 .font(.system(size: 15))
 .foregroundColor(Color.oxyText)
 .lineLimit(1..5)
 .focused(isFocused)
 .onSubmit {
 if canSend { onSend() }
 }
 }
 .padding(.horizontal, 16)
 .padding(.vertical, 10)
 }
 }
 }
}

```

```

 .background(Color.oxySurface2)
 .clipShape(RoundedRectangle(cornerRadius: 22))
 .overlay(
 RoundedRectangle(cornerRadius: 22)
 .stroke(Color.oxyLine2, lineWidth: 1)
)

 Button(action: canSend ? onSend : onVoice) {
 ZStack {
 if isRecording &&& !canSend {
 Circle()
 .stroke(Color.oxyGreen.opacity(0.30), lineWidth: 2)
 .frame(width: 50, height: 50)
 .scaleEffect(voicePulse ? 1.32 : 0.78)
 .opacity(voicePulse ? 0 : 0.85)
 Circle()
 .stroke(Color.oxyGreen.opacity(0.20), lineWidth: 1.5)
 .frame(width: 62, height: 62)
 .scaleEffect(voicePulse ? 1.18 : 0.72)
 .opacity(voicePulse ? 0 : 0.65)
 }
 if isPreparingVoice &&& !canSend {
 ProgressView()
 .controlSize(.small)
 .tint(Color.oxyOnAccent)
 } else {
 Image(systemName: canSend ? "arrow.up" : (isRecording ? "stop.fill" : "mic.fill"))
 .font(.system(size: 15, weight: .semibold))
 }
 }
 .foregroundColor(canAct ? Color.oxyOnAccent : Color.oxyDim)
 .frame(width: 36, height: 36)
 .background(canAct ? (isRecording &&& !canSend ? Color.oxyRed : Color.oxyStone) : Color.oxySurface3)
 .clipShape(Circle())
 .scaleEffect(isRecording &&& !canSend &&& voicePulse ? 1.05 : 1.0)
 }
 .disabled(!canAct)
 .animation(.spring(response: 0.4, dampingFraction: 0.8), value: canAct)
 .animation(.easeInOut(duration: 1.05).repeatForever(autoreverses: false), value: voicePulse)
 }
 .padding(.horizontal, 12)
 .padding(.vertical, 8)
 .background(Color.oxySurface1)
}

}
.onAppear { voicePulse = true }
.onChange(of: isRecording) { _, recording in
 if recording {
 voicePulse = false
 DispatchQueue.main.async { voicePulse = true }
 }
}
}

private var canSend: Bool {
 !isSending &&& (!text.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty || attachmentLabel != nil)
}

private var canAct: Bool {
 !isSending &&& !isPreparingVoice
}
}

// MARK: - Status Indicator

private struct StatusIndicator: View {
 let label: String

 var body: some View {
 HStack {
 OxyThinkingIndicator(label: label, compact: true)
 .padding(.horizontal, 11)
 .padding(.vertical, 7)
 .background(Color.oxyStone.opacity(0.08))
 .clipShape(Capsule())
 Spacer(minLength: 60)
 }
 .frame(maxWidth: .infinity, alignment: .leading)
 .padding(.leading, 2)
 }
}

// MARK: - Pendant Listening Bar

private struct PendantListeningBar: View {
 let state: PendantLiveService.LiveState
 let transcript: String?
 @State private var pulse = false

```

```

private var dotColor: Color {
 switch state {
 case .idle: Color.oxyStone
 case .connecting: Color.oxyStone

 case .listening: Color.oxyGreen
 case .processing: Color.oxyStone
 }
}

private var label: String {
 switch state {
 case .idle: ""
 case .connecting: "Pendant connecting..."
 case .listening: "Pendant listening..."
 case .processing: "Processing..."
 }
}

var body: some View {
 HStack(spacing: 10) {
 Circle()
 .fill(dotColor)
 .frame(width: 10, height: 10)
 .scaleEffect(pulse && state == .listening ? 1.3 : 1.0)
 .animation(.easeInOut(duration: 0.6).repeatForever(autoreverses: true), value: pulse)

 VStack(alignment: .leading, spacing: 2) {
 Text(label)
 .font(.system(size: 13, weight: .semibold))
 .foregroundColor(Color.oxyText)
 if let transcript, !transcript.isEmpty {
 Text(transcript)
 .font(.system(size: 12))
 .foregroundColor(Color.oxySub)
 .lineLimit(1)
 }
 }
 Spacer()
 if state == .connecting || state == .processing {
 ProgressView()
 .controlSize(.small)
 .tint(Color.oxyStone)
 }
 }
 .padding(.horizontal, 16)
 .padding(.vertical, 10)
 .background(Color.oxySurface2)
 .onAppear { pulse = true }
}

#Preview {
 ChatView()
 .environment(AppState())
}

```

### ChatViewModel.swift — OxyApp/OxyApp/Views/Chat/ChatViewModel.swift

```

import Foundation
import Observation
import AVFoundation
import UIKit

private final class NativeActionTimeoutBox: @unchecked Sendable {
 private let lock = NSLock()
 private var didResume = false

 func resume(_ continuation: CheckedContinuation<NativeLocalActionResult?, Never>;, with result: NativeLocalActionResult?) {
 lock.lock()
 defer { lock.unlock() }
 guard !didResume else { return }
 didResume = true
 continuation.resume(returning: result)
 }
}

@Observable
@MainActor
final class ChatViewModel {
 var messages: [Message] = []
 var inputText = ""
 var isSending = false
 var statusLabel: String?
 var scrollTargetMessageID: UUID?
 var isViewingHistorySnapshot = false
 var historySnapshotLabel: String?

```



```

var networkError: String?

/// Called on the main actor when a silent pendant execution finishes.
var onSilentExecComplete: (() -> Void)?

@ObservationIgnored private let audioPlayback = AudioPlaybackManager()
@ObservationIgnored private var currentSendTask: Task<Void, Never>?
@ObservationIgnored private var sendWatchdogTask: Task<Void, Never>?
@ObservationIgnored private var activeChatStartedAt: String?
@ObservationIgnored private var pendingLocalAction: ActionResult?
@ObservationIgnored private var lastFailedText: String?

private let chatService = ChatService()
private let locationManager = LocationManager.shared
private let nativeManager = NativeIntegrationManager.shared

private var currentSettings: OxySettings {
 if let data = UserDefaults.standard.data(forKey: "oxy_settings"),
 let saved = try? JSONDecoder().decode(OxySettings.self, from: data) {
 return saved
 }
 return OxySettings()
}

private static let autoOpenActions: Set<String> = [
 "book_uber", "order_deliveroo", "order_uber_eats",
 "search_netflix_title", "add_to_netflix_list",
 "make_call", "add_to_music_playlist", "open_app"
]

private static let localRequestTerms = [
 "uber", "ride", "taxi", "nearest", "closest", "near me", "nearby",
 "place", "maps", "mcdonald", "john lewis", "gym", "restaurant", "cafe", "coffee",
 "shop", "supermarket", "store", "pharmacy", "station", "cinema",
 "bank", "atm", "directions", "navigate", "route", "bus", "buses", "transit",
 "public transport", "walk", "walking", "drive", "driving"
]

func prepareChat(userId: String) async {
 activeChatStartedAt = chatStartedAt(for: userId)
 await loadHistory(userId: userId)
}

func loadHistory(userId: String) async {
 do {
 let entries = try await chatService.loadHistory(userId: userId, since: activeChatStartedAt)
 let loaded = messages(from: entries)
 await MainActor.run {
 messages = loaded
 scrollTargetMessageID = nil
 isViewingHistorySnapshot = false
 historySnapshotLabel = nil
 }
 } catch {}
}

func loadHistoryAround(userId: String, createdAt: String, messageId: String? = nil) async {
 do {
 let entries = try await chatService.loadHistoryAround(userId: userId, createdAt: createdAt, messageId: messageId)
 let loaded = messages(from: entries)
 let targetID = closestMessageID(in: loaded, to: createdAt, messageId: messageId)
 let label = historyLabel(for: createdAt)
 await MainActor.run {
 messages = loaded
 statusLabel = nil
 isViewingHistorySnapshot = true
 historySnapshotLabel = label
 scrollTargetMessageID = targetID
 }
 } catch {}
}

func returnToCurrentChat(userId: String) async {
 activeChatStartedAt = chatStartedAt(for: userId)
 await loadHistory(userId: userId)
}

func startNewChat(userId: String) {
 clearChat()
 let startedAt = Date().oxyISO8601String
 activeChatStartedAt = startedAt
 UserDefaults.standard.set(startedAt, forKey: chatStartedAtKey(userId))
 isViewingHistorySnapshot = false
 historySnapshotLabel = nil
 nativeManager.resetConversationContext()
}

```

```

func sendMessage(userId: String) {
 let text = inputText.trimmingCharacters(in: .whitespacesAndNewlines)
 guard !text.isEmpty, !isSending else { return }
 if isViewingHistorySnapshot {
 startNewChat(userId: userId)
 }
 if activeChatStartedAt == nil {
 activeChatStartedAt = chatStartedAt(for: userId)
 }
 let pendingDecision = localActionDecision(for: text)

 inputText = ""
 isSending = true
 statusLabel = nil
 networkError = nil

 let userMessage = Message(role: .user, content: text)
 messages.append(userMessage)

 let assistantMessage = Message(role: .assistant, content: "", isStreaming: true)
 messages.append(assistantMessage)
 let assistantID = assistantMessage.id

 let settings = currentSettings
 let needsFreshLocation = shouldFetchLocation(for: text)
 startSendWatchdog(assistantID: assistantID)

 currentSendTask = Task {
 defer {
 Task { @MainActor in
 sendWatchdogTask?.cancel()
 sendWatchdogTask = nil
 }
 }
 }

 if let pendingAction = pendingLocalAction, let pendingDecision {
 await MainActor.run {
 pendingLocalAction = nil
 _ = updateAssistantMessage(id: assistantID) {
 if pendingDecision {
 $0.content = "Opening \(pendingAction.cardText ?? "that app")."
 $0.actions = [pendingAction]
 openActionLink(pendingAction)
 } else {
 $0.content = "Cancelled."
 $0.actions = []
 }
 }
 $0.isStreaming = false
 }
 statusLabel = nil
 isSending = false
 currentSendTask = nil
 }
 if pendingDecision {
 await chatService.logNativeLocalAction(
 userId: userId,
 message: text,
 result: pendingAction,
 chatStartedAt: activeChatStartedAt
)
 }
 return
} else if pendingDecision == nil {
 await MainActor.run {
 pendingLocalAction = nil
 }
}

var location = locationManager.locationDict
if needsFreshLocation {
 await MainActor.run {
 statusLabel = "Checking location"
 }
 location = await locationManager.currentLocationForLocalRequest() ?? location
}

if let localResult = await executeLocalRequestWithTimeout(text) {
 let actionResult = ActionResult(native: localResult)
 let holdForConfirmation = shouldHoldForLocalConfirmation(actionResult, settings: settings)
 await MainActor.run {
 _ = updateAssistantMessage(id: assistantID) {
 $0.content = holdForConfirmation ? confirmationPrompt(for: actionResult) : localResult.text
 $0.actions = [actionResult]
 $0.isStreaming = false
 }
 statusLabel = nil
 }
}

```

```

 isSending = false
 currentSendTask = nil
 if holdForConfirmation {
 pendingLocalAction = actionResult
 } else if shouldAutoOpen(actionResult, settings: settings) {
 openActionLink(actionResult)
 }
 }
}
await chatService.logNativeLocalAction(
 userId: userId,
 message: text,
 result: actionResult,
 chatStartedAt: activeChatStartedAt
)
return
}

let nativeHints = await nativeManager.localContextHints(for: text)

let stream = chatService.sendMessage(
 userId: userId,
 message: text,
 chatStartedAt: activeChatStartedAt,
 settings: settings,
 location: location,
 nativeHints: nativeHints
)
var fullText = ""

for await event in stream {
 if Task.isCancelled { break }
 await MainActor.run {
 switch event {
 case .text(let chunk):
 fullText += chunk
 guard updateAssistantMessage(id: assistantID, { $0.content = fullText }) else { return }
 statusLabel = nil

 case .replace(let replacement):

 fullText = replacement
 guard updateAssistantMessage(id: assistantID, { $0.content = fullText }) else { return }

 case .actions(let results):
 guard updateAssistantMessage(id: assistantID, { $0.actions = results }) else { return }
 openDeepLinks(results)
 Task {
 await playBackendMusicActions(results, assistantID: assistantID, userId: userId, originalMessage: text)
 }

 case .status(let status, let label):
 setStatus(status, label)

 case .transcription:
 break

 case .transcriptionError(let error):
 guard updateAssistantMessage(id: assistantID, { $0.content = error }) else { return }
 statusLabel = nil

 case .audio(let base64Audio, _):
 playAudio(base64Audio)

 case .ttsError(let error):
 statusLabel = "Voice unavailable: \(error)"

 case .done:
 _ = updateAssistantMessage(id: assistantID, { $0.isStreaming = false })
 statusLabel = nil
 networkError = nil
 lastFailedText = nil
 isSending = false
 currentSendTask = nil

 case .error(let error):
 lastFailedText = text
 networkError = friendlyNetworkError(error)
 if fullText.isEmpty {
 _ = updateAssistantMessage(id: assistantID, { $0.content = "Something went wrong: \(error)" })
 }
 _ = updateAssistantMessage(id: assistantID, { $0.isStreaming = false })
 statusLabel = nil
 isSending = false
 currentSendTask = nil
 }
 }
}
}

```

```

 await MainActor.run {
 _ = updateAssistantMessage(id: assistantID, { $0.isStreaming = false })
 isSending = false
 statusLabel = nil
 currentSendTask = nil
 }
 }
}

func sendCommand(_ command: String, userId: String) {
 guard !isSending else { return }
 inputText = command
 sendMessage(userId: userId)
}

/// Execute a voice command silently – runs through local actions + API
/// but does NOT add user/assistant message bubbles to the chat.
func executeSilently(_ command: String, userId: String) {
 let text = command.trimmingCharacters(in: .whitespacesAndNewlines)
 guard !text.isEmpty, !isSending else { return }
 print("[ChatVM] Silent exec: \(text)")

 if activeChatStartedAt == nil {
 activeChatStartedAt = chatStartedAt(for: userId)
 }

 isSending = true
 let settings = currentSettings

 Task {
 defer {
 Task { @MainActor in
 self.isSending = false
 self.onSilentExecComplete?()
 }
 }

 // Try local actions first (music, reminders, etc.)
 if let localResult = await executeLocalRequestWithTimeout(text) {
 let actionResult = ActionResult(native: localResult)
 print("[ChatVM] Silent local result: \(localResult.text)")
 await MainActor.run {
 if shouldAutoOpen(actionResult, settings: settings) {
 openActionLink(actionResult)
 }
 }
 await chatService.logNativeLocalAction(
 userId: userId,
 message: text,
 result: actionResult,
 chatStartedAt: activeChatStartedAt
)
 return
 }

 // Fall back to API
 let nativeHints = await nativeManager.localContextHints(for: text)
 let location = locationManager.locationDict

 let stream = chatService.sendMessage(
 userId: userId,
 message: text,
 chatStartedAt: activeChatStartedAt,
 settings: settings,
 location: location,
 nativeHints: nativeHints
)

 for await event in stream {
 if Task.isCancelled { break }
 await MainActor.run {
 switch event {

 case .actions(let results):
 self.openDeepLinks(results)
 // For silent exec, resolve music actions directly
 for result in results where result.action == "play_music" && result.success {
 let query = (result.cardText ?? result.text ?? "")
 .replacingOccurrences(of: #"(?i)^playing\s+"#, with: "", options: .regularExpression)
 .trimmingCharacters(in: .whitespacesAndNewLines.union(.punctuationCharacters))
 if !query.isEmpty {
 Task {
 _ = await self.executeNativeMusicWithTimeout(query: query)
 }
 }
 }

 case .audio(let base64Audio, _):

```

```

 self.playAudio(base64Audio)
 case .done, .error:
 break
 default:
 break
 }
 }
}

func retryLastFailedMessage(userId: String) {
 guard let lastFailedText, !isSending else { return }
 networkError = nil
 inputText = lastFailedText
 sendMessage(userId: userId)
}

func sendImageMessage(userId: String, imageData: Data, fileName: String, mimeType: String, isImage: Bool = true) {
 let typed = inputText.trimmingCharacters(in: .whitespacesAndNewlines)
 let text = typed.isEmpty
 ? (isImage ? "Look at this image and tell me what you see." : "Read this file and tell me what's in it.")
 : typed
 guard !isSending else { return }
 if isViewingHistorySnapshot {
 startNewChat(userId: userId)
 }
 if activeChatStartedAt == nil {
 activeChatStartedAt = chatStartedAt(for: userId)
 }

 inputText = ""
 isSending = true
 statusLabel = isImage ? "Looking at image" : "Reading file"
 networkError = nil

 let attachmentTag = isImage ? "[Image attached]" : "[File attached: \(fileName)]"
 let userMessage = Message(role: .user, content: "\(text)\n\(attachmentTag)")
 messages.append(userMessage)

 let assistantMessage = Message(role: .assistant, content: "", isStreaming: true)
 messages.append(assistantMessage)
 let assistantID = assistantMessage.id
 let settings = currentSettings
 startSendWatchdog(assistantID: assistantID)

 currentSendTask = Task {
 defer {
 Task { @MainActor in
 sendWatchdogTask?.cancel()
 sendWatchdogTask = nil
 }
 }
 do {
 let response = try await chatService.sendImageMessage(
 userId: userId,
 message: text,
 imageData: imageData,
 fileName: fileName,
 mimeType: mimeType,
 chatStartedAt: activeChatStartedAt,
 settings: settings
)
 let actions = response.actions ?? []
 await MainActor.run {
 _ = updateAssistantMessage(id: assistantID) {
 $0.content = response.text
 $0.actions = actions
 $0.isStreaming = false
 }
 openDeepLinks(actions)
 if let audio = response.audio {
 playAudio(audio)
 } else if let ttsError = response.ttsError {
 statusLabel = "Voice unavailable: \(ttsError)"
 } else {
 statusLabel = nil
 }
 networkError = nil
 lastFailedText = nil
 isSending = false
 currentSendTask = nil
 }
 } catch {
 await MainActor.run {
 lastFailedText = text
 networkError = friendlyNetworkError(error.localizedDescription)
 }
 }
 }
}

```

```

 _ = updateAssistantMessage(id: assistantID) {
 $0.content = "Something went wrong with that \$(isImage ? "image" : "file"): \$(error.localizedDescription)"
 $0.isStreaming = false
 }
 statusLabel = nil
 isSending = false
 currentSendTask = nil
 }
}

}

}

func clearChat() {
 currentSendTask?.cancel()
 currentSendTask = nil
 sendWatchdogTask?.cancel()
 sendWatchdogTask = nil
 pendingLocalAction = nil
 messages.removeAll()

 inputText = ""
 isSending = false
 statusLabel = nil
 networkError = nil
 scrollTargetMessageID = nil
 isViewingHistorySnapshot = false
 historySnapshotLabel = nil
}

func requestLocationAccess() {
 locationManager.requestPermission()
}

private func messages(from entries: [HistoryEntry]) -> [Message] {
 entries.compactMap { entry -> Message? in
 guard let role = Message.Role(rawValue: entry.role) else { return nil }
 return Message(
 dbId: entry.id,
 role: role,
 content: entry.content,
 timestamp: Date().oxyParse(entry.createdAt) ?? Date(),
 actions: entry.actions ?? []
)
 }
}

private func chatStartedAt(for userId: String) -> String {
 let key = chatStartedAtKey(userId)
 if let saved = UserDefaults.standard.string(forKey: key),
 let savedDate = Date().oxyParse(saved),
 Date().timeIntervalSince(savedDate) < TimeInterval(12 * 60 * 60) {
 return saved
 }
 let startedAt = Date().oxyISO8601String
 UserDefaults.standard.set(startedAt, forKey: key)
 return startedAt
}

private func chatStartedAtKey(_ userId: String) -> String {
 "oxy_current_chat_started_at_\$(userId)"
}

private func closestMessageID(in messages: [Message], to createdAt: String, messageId: String? = nil) -> UUID? {
 if let messageId, let exact = messages.first(where: { $0.dbId == messageId }) {
 return exact.id
 }
 guard let target = Date().oxyParse(createdAt) else { return nil }
 return messages
 .min(by: { abs($0.timestamp.timeIntervalSince(target)) < abs($1.timestamp.timeIntervalSince(target)) })?
 .id
}

private func historyLabel(for createdAt: String) -> String? {
 guard let date = Date().oxyParse(createdAt) else { return nil }
 let formatter = DateFormatter()
 formatter.dateFormat = "d MMM yyyy · HH:mm"
 return formatter.string(from: date)
}

@discardableResult
private func updateAssistantMessage(id: UUID, _ update: (inout Message) -> Void) -> Bool {
 guard let index = messages.firstIndex(where: { $0.id == id }) else { return false }
 update(&messages[index])
 return true
}

private func shouldFetchLocation(for text: String) -> Bool {

```

```

 let lower = text.lowercased()
 if lower.hasPrefix("remember ") || lower.hasPrefix("save ") || lower.hasPrefix("note down ") {
 return false
 }
 if lower.contains("play ") || lower.contains("pause") || lower.contains("playlist") || lower.contains("song") {
 return false
 }
 return Self.localRequestTerms.contains { lower.contains($0) }
 }

 private func setStatus(_ status: String, _ label: String) {
 let hiddenStatuses = ["thinking_start", "action_complete", "speaking_start"]
 guard !hiddenStatuses.contains(status), !label.isEmpty else {
 if status == "action_complete" || status == "speaking_start" {
 statusLabel = nil
 }
 return
 }
 statusLabel = label
 clearStatusSoon()
 }

 private func clearStatusSoon() {
 let current = statusLabel
 Task { @MainActor in
 try? await Task.sleep(for: .seconds(8))
 if statusLabel == current {
 statusLabel = nil
 }
 }
 }

 private func startSendWatchdog(assistantID: UUID) {
 sendWatchdogTask?.cancel()
 sendWatchdogTask = Task { @MainActor in
 try? await Task.sleep(for: .seconds(45))
 guard !Task.isCancelled, isSending else { return }
 currentSendTask?.cancel()
 currentSendTask = nil
 _ = updateAssistantMessage(id: assistantID) { message in
 if message.content.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty {
 message.content = "I got stuck there. Try that again and I'll keep it tighter."
 }
 message.isStreaming = false
 }
 statusLabel = nil
 networkError = "Oxy got stuck waiting for the network. Try again."
 lastFailedText = messages.reversed().first(where: { $0.role == .user })?.content
 isSending = false
 }
 }

 private func friendlyNetworkError(_ error: String) -> String {

 let lower = error.lowercased()
 if lower.contains("network") || lower.contains("internet") || lower.contains("lost") || lower.contains("offline") {
 return "Network connection was lost. Try again."
 }
 if lower.contains("timed out") || lower.contains("timeout") {
 return "That took too long. Try again."
 }
 if lower.contains("session expired") {
 return "Session expired. Please sign in again."
 }
 return "Something went wrong. Try again."
 }

 nonisolated private static func nativeMusicTimeoutResult() -> NativeLocalActionResult {
 NativeLocalActionResult(
 action: "play_music",
 text: "Native music took too long to respond, so I stopped waiting.",
 cardText: "Native music timed out",
 actionSummary: "Music timed out",
 deepLink: nil,
 success: false,
 error: "Native music request timed out."
)
 }

 nonisolated private static func nativeHealthTimeoutResult() -> NativeLocalActionResult {
 NativeLocalActionResult(
 action: "check_health",
 text: "Apple Health took too long to respond, so I stopped waiting.",
 cardText: "Health timed out",
 actionSummary: "Health timed out",
 deepLink: "x-apple-health://",
 success: false,

```

```

 error: "Native health request timed out."
)
}

nonisolated private static func isLikelyNativeHealthRequest(_ text: String) -> Bool {
 let lower = text.lowercased()
 let hasHealthTerm = lower.contains("health")
 || lower.contains("steps")
 || lower.contains("step count")
 || lower.contains("heart rate")
 || lower.contains("bpm")
 || lower.contains("resting heart")
 || lower.contains("sleep")
 || lower.contains("slept")
 || lower.contains("workout")
 || lower.contains("workouts")
 || lower.contains("exercise")
 guard hasHealthTerm else { return false }

 return lower.contains("my ")
 || lower.contains("i ")
 || lower.contains("i've")
 || lower.contains("ive ")
 || lower.contains("me ")
 || lower.contains("today")
 || lower.contains("yesterday")
 || lower.contains("last night")
 || lower.contains("this week")
 || lower.contains("latest")
 || lower.contains("current")
 || lower.contains("check health")
 || lower.contains("health snapshot")
 || lower.contains("health data")
}

private func runNativeActionWithHardTimeout(
 seconds: Double,
 timeoutResult: NativeLocalActionResult?,
 operation: @escaping () async -> NativeLocalActionResult?
) async -> NativeLocalActionResult? {
 let operationTask = Task {
 await operation()
 }

 return await withTaskCancellationHandler {
 await withCheckedContinuation { continuation in
 let box = NativeActionTimeoutBox()

 Task.detached {
 let result = await operationTask.value
 box.resume(continuation, with: result)
 }

 DispatchQueue.global(qos: .userInitiated).asyncAfter(deadline: .now() + seconds) {
 operationTask.cancel()
 box.resume(continuation, with: timeoutResult)
 }
 }
 } onCancel: {
 operationTask.cancel()
 }
}

private func executeNativeMusicWithTimeout(query: String) async -> NativeLocalActionResult? {
 await runNativeActionWithHardTimeout(seconds: 7, timeoutResult: Self.nativeMusicTimeoutResult()) {
 await NativeIntegrationManager.shared.playResolvedMusicQuery(query)
 }
}

private func executeLocalRequestWithTimeout(_ text: String) async -> NativeLocalActionResult? {
 let lower = text.lowercased()
 let isMusicRequest = lower.contains("play")
 || lower.contains("music")
 || lower.contains("song")
 || lower.contains("playlist")

 if isMusicRequest {
 return await runNativeActionWithHardTimeout(seconds: 7, timeoutResult: Self.nativeMusicTimeoutResult()) {
 await NativeIntegrationManager.shared.executeLocalRequest(text)
 }
 }

 let timeoutResult = Self.isLikelyNativeHealthRequest(text) ? Self.nativeHealthTimeoutResult() : nil
 return await runNativeActionWithHardTimeout(seconds: 7, timeoutResult: timeoutResult) {
 await NativeIntegrationManager.shared.executeLocalRequest(text)
 }
}

```



```

private func playBackendMusicActions(
 _ results: [ActionResult],
 assistantID: UUID,
 userID: String,
 originalMessage: String
) async {
 guard let action = results.first(where: { $0.action == "play_music" && $0.success }) else { return }
 let query = (action.cardText ?? action.text ?? "")
 .replacingOccurrences(of: #"(?i)^playing\s+"#, with: "", options: .regularExpression)
 .trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
 guard !query.isEmpty, !musicQueryStillNeedsResolution(query) else { return }
 guard let nativeResult = await executeNativeMusicWithTimeout(query: query) else { return }
 let nativeAction = ActionResult(native: nativeResult)
 _ = updateAssistantMessage(id: assistantID) {
 $0.content = nativeResult.text
 $0.actions = [nativeAction]
 $0.isStreaming = false
 }
 await chatService.logNativeLocalAction(
 userID: userID,
 message: originalMessage,
 result: nativeAction,
 chatStartedAt: activeChatStartedAt
)
}

private func musicQueryStillNeedsResolution(_ query: String) -> Bool {
 let lower = query.lowercased()
 return lower.contains("billboard")
 || lower.contains("hot 100")
 || lower.contains("chart")
 || lower.contains("most popular")
 || lower.contains("top song")
 || lower.contains("top track")
 || lower.contains("number one")
 || lower.contains("right now")
 || lower.contains("currently")
 || lower.contains("today")
 || lower.contains("latest")
 || lower.contains("trending")
}

private func localActionDecision(for text: String) -> Bool? {
 let lower = text
 .lowercased()
 .trimmingCharacters(in: .whitespacesAndNewlines.union(.punctuationCharacters))
 let confirmCommands = [
 "yes", "yep", "yeah", "yea", "sure", "confirm", "continue",
 "go ahead", "do it", "open it", "open", "ok", "okay"
]
 if confirmCommands.contains(lower) { return true }
 if lower.range(of: #"^(yes|yeah|yep|sure|confirm|continue|go ahead|open it|do it)\b"#, options: .regularExpression) != nil {
 return true
 }
 let cancelCommands = ["no", "nope", "cancel", "stop", "never mind", "nevermind", "don't", "dont"]
 if cancelCommands.contains(lower) { return false }
 if lower.range(of: #"^(no|nope|cancel|stop|don't)\b"#, options: .regularExpression) != nil {
 return false
 }
 return nil
}

private func shouldHoldForLocalConfirmation(_ action: ActionResult, settings: OxySettings) -> Bool {
 guard action.action == "open_app" else { return false }
 if settings.reviewBeforeOpeningApps { return true }
 return action.risk == "sensitive_app" && settings.confirmSensitiveAppOpens
}

private func confirmationPrompt(for action: ActionResult) -> String {
 let target = action.cardText ?? "that app"
 if action.risk == "sensitive_app" {
 return "Open \(target)? Say yes to continue."
 }
 return "Open \(target)?"
}

private func shouldAutoOpen(_ action: ActionResult, settings: OxySettings) -> Bool {
 guard Self.autoOpenActions.contains(action.action) else { return false }
 return !shouldHoldForLocalConfirmation(action, settings: settings)
}

// MARK: - Deep Links

private func openDeepLinks(_ results: [ActionResult]) {
 let settings = currentSettings
 if settings.reviewBeforeOpeningApps { return }
 for result in results {

```

```

 guard shouldAutoOpen(result, settings: settings) else { continue }
 openActionLink(result)
 }
}

func openActionLink(_ result: ActionResult) {
 if let link = result.deepLink, let url = URL(string: link) {
 if url.scheme == "oxy-open-app" {
 Task { @MainActor in
 await NativeIntegrationManager.shared.openBestEffortApp(from: url)
 }
 } else {
 UIApplication.shared.open(url)
 }
 } else if let link = result.webLink, let url = URL(string: link) {
 UIApplication.shared.open(url)
 }
}

// MARK: - Audio Playback

private func playAudio(_ base64Audio: String) {
 audioPlayback.play(base64Audio) { [weak self] message in
 Task { @MainActor in
 self?.statusLabel = message
 }
 }
}

private final class AudioPlaybackManager: NSObject, AVAudioPlayerDelegate {
 private var audioPlayer: AVAudioPlayer?

 private var pendingAudio: [Data] = []
 private var onError: ((String) -> Void)?
 private var sessionConfigured = false

 func play(_ base64Audio: String, onError: @escaping (String) -> Void) {
 guard let data = Data(base64Encoded: base64Audio) else { return }
 self.onError = onError
 pendingAudio.append(data)
 playNextAudioIfNeeded()
 }

 // Configure the audio session once. Reconfiguring on every chunk added latency
 // and audible gaps between streamed sentence clips.
 private func ensureSessionActive() {
 guard !sessionConfigured else { return }
 do {
 let session = AVAudioSession.sharedInstance()
 try session.setCategory(.playback, mode: .spokenAudio, options: [.duckOthers])
 try session.setActive(true)
 sessionConfigured = true
 } catch {
 // Non-fatal: playback may still work with the default session.
 }
 }

 private func playNextAudioIfNeeded() {
 guard audioPlayer?.isPlaying != true, !pendingAudio.isEmpty else { return }
 let data = pendingAudio.removeFirst()

 ensureSessionActive()
 do {
 let player = try AVAudioPlayer(data: data)
 player.delegate = self
 player.prepareToPlay()
 audioPlayer = player
 player.play()
 } catch {
 onError?("Voice playback failed")
 playNextAudioIfNeeded()
 }
 }

 func audioPlayerDidFinishPlaying(_ player: AVAudioPlayer, successfully flag: Bool) {
 if audioPlayer == player {
 audioPlayer = nil
 }
 playNextAudioIfNeeded()
 }
}

```

## MessageBubble.swift — OxyApp/OxyApp/Views/Chat/MessageBubble.swift

```
import SwiftUI
```

```

struct MessageBubble: View {
 let message: Message
 var showsTypingIndicator: Bool = true
 var onActionCommand: ((String) -> Void)? = nil
 var onOpenAction: ((ActionResult) -> Void)? = nil

 private var isUser: Bool { message.role == .user }
 private var bubbleStyle: String {
 guard let data = UserDefaults.standard.data(forKey: "oxy_settings"),
 let settings = try? JSONDecoder().decode(OxySettings.self, from: data) else {
 return "comfort"
 }
 return settings.bubbleStyle
 }
 private var isCompact: Bool { bubbleStyle == "compact" }
 private var visibleActions: [ActionResult] {
 message.actions.filter { action in
 !action.pending &&&
 !(action.action == "send_message" &&& action.success)
 }
 }

 var body: some View {
 VStack(alignment: isUser ? .trailing : .leading, spacing: isCompact ? 2 : 4) {
 // Message content
 if !message.content.isEmpty {
 HStack {
 if isUser { Spacer(minLength: 60) }

 VStack(alignment: .leading, spacing: 0) {
 if message.isStreaming &&& !isUser {
 StreamingWordText(
 text: message.content,
 fontSize: isCompact ? 14 : 15,
 lineSpacing: isCompact ? 2 : 4
)
 } else {
 Text(message.content)
 .font(.system(size: isCompact ? 14 : 15))
 .foregroundColor(isUser ? .white : Color.oxyText)
 .lineSpacing(isCompact ? 2 : 4)
 }
 }
 .padding(.horizontal, isCompact ? 13 : 16)
 .padding(.vertical, isCompact ? 8 : 11)
 .background(
 isUser
 ? AnyShapeStyle(
 LinearGradient(
 colors: [Color.oxyStone, Color.oxyStone.opacity(0.85)],
 startPoint: .topLeading,
 endPoint: .bottomTrailing
)
)
 : AnyShapeStyle(Color.oxySurface2)
)
 .clipShape(
 UnevenRoundedRectangle(
 topLeadingRadius: isUser ? 20 : 6,
 bottomLeadingRadius: isCompact ? 16 : 20,
 bottomTrailingRadius: isUser ? 6 : (isCompact ? 16 : 20),
 topTrailingRadius: isCompact ? 16 : 20
)
)

 if !isUser { Spacer(minLength: 60) }
 }
 }

 // Streaming indicator
 if message.isStreaming &&& message.content.isEmpty &&& showsTypingIndicator {
 HStack {
 OxyThinkingIndicator()
 .padding(.horizontal, 13)
 .padding(.vertical, 9)
 .background(Color.oxySurface2.opacity(0.72))
 .clipShape(
 UnevenRoundedRectangle(
 topLeadingRadius: 6,
 bottomLeadingRadius: 18,
 bottomTrailingRadius: 18,
 topTrailingRadius: 18
)
)
 Spacer(minLength: 60)
 }
 }
 }
 }
}

```

```

 // Action cards
 if !visibleActions.isEmpty {
 VStack(spacing: 6) {
 ForEach(visibleActions) { action in
 ActionCard(action: action, onCommand: onActionCommand, onOpenAction: onOpenAction)
 }
 }
 }

 // Timestamp
 HStack(spacing: 4) {
 if isUser {
 Spacer()
 if !message.isStreaming {
 Image(systemName: "checkmark")
 .font(.system(size: 9, weight: .bold))
 .foregroundColor(Color.oxyDim)
 }
 }
 Text(message.timestamp, style: .time)
 .font(.system(size: 10))
 .foregroundColor(Color.oxyDim)
 if !isUser { Spacer() }
 }
 .padding(.horizontal, 4)
 }
 .padding(.horizontal, 16)
 .padding(.vertical, isCompact ? 2 : 4)
 .transition(.opacity.combined(with: .move(edge: .bottom)))
 .animation(.spring(response: 0.4, dampingFraction: 0.8), value: message.content)
}

private struct StreamingWordText: View {
 let text: String
 let fontSize: CGFloat
 let lineSpacing: CGFloat

 private var words: [String] {
 text.split(separator: " ", omittingEmptySubsequences: false).map(String.init)
 }

 var body: some View {
 Text(attributedString)
 .font(.system(size: fontSize))
 .foregroundColor(Color.oxyText)
 .lineSpacing(lineSpacing)
 .animation(.easeOut(duration: 0.28), value: words.count)
 }

 private var attributedText: AttributedString {
 var output = AttributedString()
 for (index, word) in words.enumerated() {
 var part = AttributedString(index == words.count - 1 ? word : "\\(word) ")
 let distanceFromEnd = words.count - index
 part.foregroundColor = Color.oxyText.opacity(distanceFromEnd <= 4 ? 1 : 0.78)
 output += part
 }
 return output
 }
}

// MARK: - Action Card

struct ActionCard: View {
 let action: ActionResult
 var onCommand: ((String) -> Void)? = nil
 var onOpenAction: ((ActionResult) -> Void)? = nil

 private var hasLink: Bool {
 action.deepLink != nil || action.webLink != nil
 }

 private var detailText: String? {
 guard let raw = action.cardText ?? action.text ?? action.error else { return nil }
 let compact = raw
 .replacingOccurrences(of: "\n", with: " ")
 .replacingOccurrences(of: " ", with: " ")
 if !action.success && action.action == "book_uber" {
 if compact.localizedCaseInsensitiveContains("need your current location") ||
 compact.localizedCaseInsensitiveContains("enable location") {
 return "Enable location and try again."
 }
 }
 if compact.localizedCaseInsensitiveContains("places api") ||
 compact.localizedCaseInsensitiveContains("google places is not ready") {
 return "Nearby place ranking needs Google Places."
 }
 }
}

```

```

 }
 if compact.localizedCaseInsensitiveContains("nearby match") ||
 compact.localizedCaseInsensitiveContains("couldn't find a nearby") ||
 compact.localizedCaseInsensitiveContains("geocoding error") ||
 compact.localizedCaseInsensitiveContains("no results found") {
 return "Try a different place name or enable location."
 }
}
return compact
}

var body: some View {
 Group {
 if action.pending {
 cardContent
 } else {
 Button(action: openLink) {
 cardContent
 }
 .buttonStyle(.plain)
 .disabled(!hasLink)
 }
 }
}

private var cardContent: some View {
 VStack(alignment: .leading, spacing: action.pending ? 10 : 0) {
 HStack(spacing: 9) {
 Image(systemName: action.success ? "checkmark" : "exclamationmark")
 .font(.system(size: 10, weight: .bold))
 .foregroundColor(action.success ? Color.oxyGreen : Color.oxyRed)
 .frame(width: 22, height: 22)
 .background((action.success ? Color.oxyGreen : Color.oxyRed).opacity(0.12))
 .clipShape(Circle())

 VStack(alignment: .leading, spacing: 3) {
 Text(actionSummary)
 .font(.system(size: 12, weight: .semibold))
 .foregroundColor(Color.oxyText)

 if let text = detailText {
 Text(text)
 .font(.system(size: 11))
 .foregroundColor(Color.oxySub)
 .lineLimit(action.action == "plan_trip" ? 2 : 1)
 .multilineTextAlignment(.leading)
 }
 }

 Spacer()

 if hasLink {
 Image(systemName: "arrow.up.right")
 .font(.system(size: 12, weight: .bold))
 .foregroundColor(Color.oxyStone)
 } else {
 Image(systemName: iconForAction(action.action))
 .font(.system(size: 12, weight: .semibold))
 .foregroundColor(Color.oxyDim)
 }
 }
 }

 if action.pending, let onCommand {
 HStack(spacing: 8) {
 Button {
 onCommand("confirm")
 } label: {
 Label("Confirm", systemImage: "checkmark")
 .font(.system(size: 12, weight: .semibold))
 .frame(maxWidth: .infinity)
 }
 .buttonStyle(.plain)
 .foregroundColor(Color.oxyOnAccent)
 .padding(.vertical, 8)
 .background(Color.oxyStone)
 .clipShape(RoundedRectangle(cornerRadius: 9))

 Button {
 onCommand("cancel")
 } label: {
 Label("Cancel", systemImage: "xmark")
 .font(.system(size: 12, weight: .semibold))
 .frame(maxWidth: .infinity)
 }
 .buttonStyle(.plain)
 .foregroundColor(Color.oxySub)
 }
 }
}

```

```

 .padding(.vertical, 8)
 .background(Color.oxySurface3)
 .clipShape(RoundedRectangle(cornerRadius: 9))
 }
}
}
.padding(.horizontal, 11)
.padding(.vertical, 8)
.background(Color.oxySurface2.opacity(0.58))
.clipShape(RoundedRectangle(cornerRadius: 14))
.overlay(
 RoundedRectangle(cornerRadius: 14)
 .stroke(action.success ? Color.oxyLine.opacity(0.8) : Color.oxyRed.opacity(0.2), lineWidth: 1)
)
.shadow(color: Color.black.opacity(0.04), radius: 8, x: 0, y: 3)
}

private func openLink() {
 guard !action.pending else { return }
 if let onOpenAction {
 onOpenAction(action)
 return
 }
 if let link = action.deepLink, let url = URL(string: link) {
 UIApplication.shared.open(url)
 } else if let link = action.webLink, let url = URL(string: link) {
 UIApplication.shared.open(url)
 }
}

private func humanize(_ type: String) -> String {
 type.replacingOccurrences(of: "_", with: " ").capitalized
}

private var actionSummary: String {
 if let summary = action.actionSummary, !summary.isEmpty { return summary }
 switch action.action {
 case "send_email": return action.success ? "Email sent" : "Email failed"
 case "send_message": return action.success ? "Message ready" : "Message failed"
 case "send_telegram": return action.success ? "Telegram sent" : "Telegram failed"
 case "book_uber": return action.success ? "Uber opened" : "Uber needs attention"
 case "find_place": return action.success ? "Place found" : "Place search failed"
 case "get_directions": return action.success ? "Directions ready" : "Directions failed"
 case "plan_trip": return action.success ? "Trip planned" : "Trip failed"
 case "search_trains": return action.success ? "Trainline ready" : "Train search failed"
 case "station_board": return action.success ? "Station board ready" : "Station board failed"
 case "order_uber_eats": return action.success ? "Uber Eats opened" : "Uber Eats failed"
 case "order_deliveroo": return action.success ? "Deliveroo opened" : "Deliveroo failed"
 case "open_app": return action.success ? "App opened" : "App unavailable"
 case "play_music": return action.success ? "Music opened" : "Music failed"
 case "add_to_music_playlist": return action.success ? "Music added" : "Music add failed"
 case "create_reminder": return action.success ? "Reminder created" : "Reminder failed"
 case "create_calendar_event": return action.success ? "Calendar updated" : "Calendar failed"
 case "check_health": return action.success ? "Health checked" : "Health unavailable"
 default: return action.success ? "\(humanize(action.action)) done" : "\(humanize(action.action)) failed"
 }
}

private func iconForAction(_ type: String) -> String {
 switch type {
 case "send_email", "get_emails", "search_emails": return "envelope.fill"
 case "create_calendar_event", "get_calendar_events": return "calendar"
 case "book_uber": return "car.fill"
 case "find_place", "get_directions": return "map.fill"
 case "plan_trip": return "tram.fill"
 case "send_telegram", "get_telegram_contacts": return "paperplane.fill"
 case "search_trains", "station_board": return "tram.fill"
 case "order_uber_eats", "order_deliveroo": return "fork.knife"
 case "search_netflix_title", "add_to_netflix_list": return "play.tv.fill"
 case "create_reminder": return "bell.fill"
 case "send_message": return "message.fill"
 case "open_app": return "app.fill"
 case "play_music", "add_to_music_playlist": return "music.note"
 case "check_health": return "heart.fill"
 default: return "bolt.fill"
 }
}

#Preview {
 ScrollView {
 VStack(spacing: 4) {
 MessageBubble(message: Message(role: .user, content: "Book me an Uber to Kings Cross"))
 MessageBubble(message: Message(
 role: .assistant,
 content: "On it! Opening Uber for you.",
 actions: [ActionResult(

```

```

 action: "book_uber",
 success: true,
 text: "Opening Uber to Kings Cross",
 deepLink: "uber://?action=setPickup",
 webLink: "https://m.uber.com/ul/"
 })
})
MessageBubble(message: Message(role: .assistant, content: "", isStreaming: true))
}
}
.background(Color.oxyBg)
}

```

## SettingsView.swift — OxyApp/OxyApp/Views/Settings/SettingsView.swift

```

import AVFoundation
import SwiftUI
import UIKit

struct SettingsView: View {
 @Environment(AppState.self) private var appState
 @AppStorage("oxy_appTheme") private var appTheme = "dark"
 @State private var settings = OxySettings()
 @State private var showSignOutConfirm = false
 @State private var showSignOutAllConfirm = false
 @State private var showDeleteAccountConfirm = false
 @State private var showAccentPicker = false
 @State private var showBackendURLEditor = false
 @State private var versionTapCount = 0
 @State private var voicePreview = VoicePreviewPlayer()
 @State private var backendVersionText = "Checking backend..."
 @State private var accountStatusText: String?
 @State private var isExportingData = false
 @State private var isDeletingAccount = false
 @State private var isSigningOutAll = false
 @State private var sharePayload: SharePayload?
 @AppStorage("oxy_custom_backend_url") private var customBackendURL = ""

 var body: some View {
 NavigationStack {
 ZStack {
 Color.oxyBg.ignoresSafeArea()

 ScrollView {
 VStack(spacing: 18) {
 // Identity
 settingsSection(title: "Profile") {
 settingRow(label: "Assistant Name", description: nil) {
 TextField("Oxy", text: $settings.name)
 .font(.system(size: 15))
 .foregroundColor(Color.oxyText)
 .multilineTextAlignment(.trailing)
 .frame(width: 120)
 .onChange(of: settings.name) { _, _ in saveSettings() }
 }
 }

 settingsSection(title: "Personalisation") {
 NavigationLink {
 MemoryView(embedded: true)
 } label: {
 settingsNavigationRow(
 label: "Memory",
 description: "Saved facts and preferences",
 icon: "brain.head.profile"
)
 }
 .buttonStyle(.plain)
 }

 settingsSection(title: "Appearance") {
 settingRow(label: "Accent", description: selectedAccentLabel) {
 Button {
 showAccentPicker = true
 } label: {
 HStack(spacing: 8) {
 Circle()
 .fill(selectedAccentColor)
 .frame(width: 14, height: 14)
 Text(selectedAccentLabel)
 .font(.system(size: 13, weight: .semibold))
 }
 Image(systemName: "chevron.down")
 .font(.system(size: 10, weight: .bold))
 }
 .foregroundColor(Color.oxyText)
 .padding(.horizontal, 11)
 }
 }
 }
 }
 }
 }
 }
}

```

```

 .padding(.vertical, 7)
 .background(Color.oxySurface1)
 .clipShape(Capsule())
 }
 .buttonStyle(.plain)
}

Divider().overlay(Color.oxyLine)

settingRow(label: "Theme", description: nil) {
 Picker("Theme", selection: $settings.appTheme) {
 Text("Light").tag("light")
 Text("Dark").tag("dark")
 Text("System").tag("system")
 }
 .labelsHidden()
 .pickerStyle(.menu)
 .tint(Color.oxyStone)
 .onChange(of: settings.appTheme) { _, _ in saveSettings() }
}

Divider().overlay(Color.oxyLine)

settingRow(label: "Bubbles", description: nil) {
 Picker("Bubbles", selection: $settings.bubbleStyle) {
 Text("Comfort").tag("comfort")
 Text("Compact").tag("compact")
 }
 .labelsHidden()
 .pickerStyle(.menu)
 .tint(Color.oxyStone)
 .onChange(of: settings.bubbleStyle) { _, _ in saveSettings() }
}

// Voice
settingsSection(title: "Voice") {
 settingRow(label: "Voice Replies", description: nil) {
 Toggle("", isOn: $settings.voiceOn)
 .labelsHidden()
 .tint(Color.oxyGreen)
 .onChange(of: settings.voiceOn) { _, _ in saveSettings() }
 }

 Divider().overlay(Color.oxyLine)

 voicePickerRow
}

// Autonomy
settingsSection(title: "Assistant") {
 InitiativeScroller(selection: $settings.autonomy, onChange: saveSettings)

 Divider().overlay(Color.oxyLine)

 settingRow(label: "Briefings", description: nil) {
 Toggle("", isOn: $settings.proactiveBriefings)
 .labelsHidden()
 .tint(Color.oxyGreen)
 .onChange(of: settings.proactiveBriefings) { _, _ in saveSettings() }
 }
}

settingsSection(title: "Action Defaults") {
 settingRow(label: "Preferred Maps", description: nil) {
 Picker("Preferred Maps", selection: $settings.preferredMapsApp) {
 Text("Apple Maps").tag("apple")
 Text("Google Maps").tag("google")
 }
 .labelsHidden()
 .pickerStyle(.menu)
 .tint(Color.oxyStone)
 .onChange(of: settings.preferredMapsApp) { _, _ in saveSettings() }
 }

 Divider().overlay(Color.oxyLine)

 settingRow(label: "Transport", description: nil) {
 Picker("Transport", selection: $settings.preferredTransportMode) {
 Text("Driving").tag("driving")
 Text("Transit").tag("transit")
 Text("Walking").tag("walking")
 }
 .labelsHidden()
 .pickerStyle(.menu)
 .tint(Color.oxyStone)
 .onChange(of: settings.preferredTransportMode) { _, _ in saveSettings() }
 }
}

```



```

 }

 Divider().overlay(Color.oxyLine)

 settingRow(label: "Confirm Sensitive Apps", description: nil) {
 Toggle("", isOn: $settings.confirmSensitiveAppOpens)
 .labelsHidden()
 .tint(Color.oxyGreen)
 .onChange(of: settings.confirmSensitiveAppOpens) { _, _ in saveSettings() }
 }
}

// Pendant / Hardware
PendantSettingsSection(pendant: NativeIntegrationManager.shared.pendant)

settingsSection(title: "Account") {
 Button(action: exportMyData) {
 HStack {
 Image(systemName: isExportingData ? "hourglass" : "square.and.arrow.up")
 .font(.system(size: 14))
 Text(isExportingData ? "Preparing Export" : "Export My Data")
 .font(.system(size: 15, weight: .medium))
 Spacer()
 Image(systemName: "chevron.right")
 .font(.system(size: 12, weight: .semibold))
 .foregroundColor(Color.oxyDim)
 }
 .foregroundColor(Color.oxyText)
 .padding(.vertical, 4)
 }
 .buttonStyle(.plain)
 .disabled(isExportingData || isDeletingAccount)
 .accessibilityLabel("Export my data")

 Divider().overlay(Color.oxyLine)

 Button(action: { showDeleteAccountConfirm = true }) {
 HStack {
 Image(systemName: isDeletingAccount ? "hourglass" : "trash.fill")
 .font(.system(size: 14))
 Text(isDeletingAccount ? "Deleting Account" : "Delete Account")
 .font(.system(size: 15, weight: .medium))
 Spacer()
 Image(systemName: "chevron.right")
 .font(.system(size: 12, weight: .semibold))
 .foregroundColor(Color.oxyDim)
 }
 .foregroundColor(Color.oxyRed)
 .padding(.vertical, 4)
 }
 .buttonStyle(.plain)
 .disabled(isExportingData || isDeletingAccount)
 .accessibilityLabel("Delete account")

 Divider().overlay(Color.oxyLine)

 Button(action: { showSignOutConfirm = true }) {
 HStack {
 Image(systemName: "rectangle.portrait.and.arrow.right")
 .font(.system(size: 14))
 Text("Sign Out")
 .font(.system(size: 15, weight: .medium))
 Spacer()
 Image(systemName: "chevron.right")
 .font(.system(size: 12, weight: .semibold))
 .foregroundColor(Color.oxyDim)
 }
 .foregroundColor(Color.oxyRed)
 .padding(.vertical, 4)
 }
 .buttonStyle(.plain)
 .accessibilityLabel("Sign out")

 Divider().overlay(Color.oxyLine)

 Button(action: { showSignOutAllConfirm = true }) {
 HStack {
 Image(systemName: isSigningOutAll ? "hourglass" : "rectangle.portrait.and.arrow.right.fill")
 .font(.system(size: 14))
 Text(isSigningOutAll ? "Signing Out..." : "Sign Out All Devices")
 .font(.system(size: 15, weight: .medium))
 Spacer()
 Image(systemName: "chevron.right")
 .font(.system(size: 12, weight: .semibold))
 .foregroundColor(Color.oxyDim)
 }
 }
}

```

```

 .foregroundColor(Color.oxyRed)
 .padding(.vertical, 4)
 }
 .buttonStyle(.plain)
 .disabled(isSigningOutAll)
 .accessibilityLabel("Sign out all devices")

 if let accountStatusText {
 Divider().overlay(Color.oxyLine)
 Text(accountStatusText)
 .font(.system(size: 12))
 .foregroundColor(Color.oxySub)
 .lineSpacing(3)
 .frame(maxWidth: .infinity, alignment: .leading)
 .padding(.top, 4)
 }
}

settingsSection(title: "Support & Legal") {
 legallink(label: "Support", path: "/support", icon: "questionmark.circle.fill")
 Divider().overlay(Color.oxyLine)
 legallink(label: "Privacy Policy", path: "/privacy", icon: "hand.raised.fill")
 Divider().overlay(Color.oxyLine)
 legallink(label: "Terms of Use", path: "/terms", icon: "doc.text.fill")
 Divider().overlay(Color.oxyLine)
 Button(action: handleVersionTap) {
 HStack {
 Text("Version")
 .font(.system(size: 14, weight: .medium))
 .foregroundColor(Color.oxyText)
 Spacer()
 Text(backendVersionText)
 .font(.system(size: 12))
 .foregroundColor(Color.oxyDim)
 .lineLimit(1)
 }
 .padding(.vertical, 4)
 }
 .buttonStyle(.plain)
}

Spacer().frame(height: 32)
}
.padding(16)
}

.navigationTitle("Settings")
.navigationBarTitleDisplayMode(.large)
.toolbarBackground(Color.oxySurface1, for: .navigationBar)
.toolbarBackground(.visible, for: .navigationBar)
.alert("Sign Out", isPresented: $showSignOutConfirm) {
 Button("Sign Out", role: .destructive) { appState.logout() }
 Button("Cancel", role: .cancel) {}
} message: {
 Text("Are you sure you want to sign out?")
}
.alert("Sign Out All Devices", isPresented: $showSignOutAllConfirm) {
 Button("Sign Out All", role: .destructive) { signOutAllDevices() }
 Button("Cancel", role: .cancel) {}
} message: {
 Text("This will invalidate all active sessions on every device. You will be signed out here too.")
}
.alert("Delete Account", isPresented: $showDeleteAccountConfirm) {
 Button("Delete", role: .destructive) { deleteAccount() }
 Button("Cancel", role: .cancel) {}
} message: {
 Text("This permanently deletes your Oxy account data, including conversations, memories, connectors, preferences, and action his
}
.sheet(isPresented: $showAccentPicker) {
 AccentPickerSheet(selection: $settings.accentColor) {
 saveSettings()
 showAccentPicker = false
 }
 .presentationDetents([.medium])
 .presentationDragIndicator(.visible)
}
.sheet(isPresented: $showBackendURLEditor) {
 BackendURLEditorSheet(currentURL: $customBackendURL) {
 showBackendURLEditor = false
 loadBackendVersion()
 }
 .presentationDetents([.medium])
 .presentationDragIndicator(.visible)
}
.sheet(item: $sharePayload) { payload in
 ShareSheet(activityItems: [payload.url])
}

```

```

 .onAppear {
 loadSettings()
 loadBackendVersion()
 }
 }
}

// MARK: - Helpers

private var selectedVoiceLabel: String {
 OxySettings.voiceOptions.first(where: { $0.value == settings.voice })?.label ?? settings.voice
}

private var selectedAccentLabel: String {
 OxySettings.accentOptions.first(where: { $0.value == settings.accentColor })?.label ?? "Stone"
}

private var selectedAccentColor: Color {
 OxySettings.accentOptions.first(where: { $0.value == settings.accentColor })?.color ?? Color.oxyDefaultStone
}

private func previewVoice(_ voice: String) {
 Task {
 await voicePreview.preview(voice: voice)
 }
}

private func loadBackendVersion() {
 backendVersionText = "Checking backend..."

 Task {
 do {
 let version = try await ChatService().backendVersion()
 let commit = version.gitCommit?.isEmpty == false && version.gitCommit != "unknown"
 ? version.gitCommit!
 : (version.deployId?.isEmpty == false ? version.deployId! : "unknown")
 let branch = version.gitBranch?.isEmpty == false ? version.gitBranch! : "unknown"
 let environment = version.environment?.isEmpty == false ? version.environment! : "env unknown"
 await MainActor.run {
 backendVersionText = "\(commit) · \(branch) · \(environment)"
 }
 } catch {
 await MainActor.run {
 backendVersionText = "Backend version unavailable"
 }
 }
 }
}

private func exportMyData() {
 guard !isExportingData else { return }
 accountStatusText = nil
 isExportingData = true
 Task {
 do {
 let data = try await APIClient.shared.exportUserData(userId: appState.userId)
 let url = FileManager.default.temporaryDirectory
 .appendingPathComponent("oxy-data-export-\(Int(Date().timeIntervalSince1970)).json")
 try data.write(to: url, options: .atomic)
 await MainActor.run {
 isExportingData = false
 sharePayload = SharePayload(url: url)
 accountStatusText = "Export ready."
 }
 } catch {
 await MainActor.run {
 isExportingData = false
 accountStatusText = "Could not export your data: \(\error.localizedDescription)"
 }
 }
 }
}

private func signOutAllDevices() {
 guard !isSigningOutAll else { return }
 accountStatusText = nil
 isSigningOutAll = true
 Task {
 do {
 _ = try await APIClient.shared.request(path: "/auth/logout-all", method: "POST")
 await MainActor.run {
 isSigningOutAll = false
 appState.logout()
 }
 } catch {
 await MainActor.run {
 isSigningOutAll = false
 }
 }
 }
}

```

```

 accountStatusText = "Could not sign out all devices: \${error.localizedDescription}"
 }
}
}

private func deleteAccount() {
 guard !isDeletingAccount else { return }
 accountStatusText = nil
 isDeletingAccount = true
 Task {
 do {
 try await APIClient.shared.deleteAccount(userId: appState.userId)
 await MainActor.run {
 isDeletingAccount = false
 appState.logout()
 }
 } catch {
 await MainActor.run {
 isDeletingAccount = false
 accountStatusText = "Could not delete your account: \${error.localizedDescription}"
 }
 }
 }
}

private func settingsSection<Content: View>(title: String, @ViewBuilder content: () -> Content) -> some View {
 VStack(alignment: .leading, spacing: 12) {
 Text(title)
 .font(.system(size: 13, weight: .semibold))
 .foregroundColor(Color.oxySub)
 .textCase(.uppercase)
 .tracking(0.5)
 .scrollTransition(axis: .vertical) { content, phase in
 content
 .opacity(phase.isIdentity ? 1 : 0)
 .offset(y: phase.isIdentity ? 0 : 12)
 }

 VStack(spacing: 0) {
 content()
 }
 .padding(16)
 .background(Color.oxySurface2)
 .clipShape(RoundedRectangle(cornerRadius: 16))
 .overlay(
 RoundedRectangle(cornerRadius: 16)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
 .transition(.opacity.combined(with: .move(edge: .bottom)))
 }
 .animation(.spring(response: 0.4, dampingFraction: 0.8), value: title)
}

private func settingRow<Accessory: View>(
 label: String,
 description: String?,
 @ViewBuilder accessory: () -> Accessory
) -> some View {
 HStack {
 VStack(alignment: .leading, spacing: 2) {
 Text(label)
 .font(.system(size: 14, weight: .medium))

 .foregroundColor(Color.oxyText)
 if let description {
 Text(description)
 .font(.system(size: 12))
 .foregroundColor(Color.oxySub)
 }
 }
 Spacer()
 accessory()
 }
 .padding(.vertical, 4)
}

private func settingsNavigationRow(label: String, description: String?, icon: String) -> some View {
 HStack(spacing: 12) {
 Image(systemName: icon)
 .font(.system(size: 15, weight: .semibold))
 .foregroundColor(Color.oxyStone)
 .frame(width: 24)

 VStack(alignment: .leading, spacing: 2) {
 Text(label)
 .font(.system(size: 15, weight: .medium))

```

```

 .foregroundStyle(Color.oxyText)
 if let description {
 Text(description)
 .font(.system(size: 12))
 .foregroundStyle(Color.oxySub)
 }
}

Spacer()

Image(systemName: "chevron.right")
 .font(.system(size: 12, weight: .semibold))
 .foregroundStyle(Color.oxyDim)
}
.padding(.vertical, 4)
}

private var voicePickerRow: some View {
 VStack(alignment: .leading, spacing: 10) {
 HStack {
 VStack(alignment: .leading, spacing: 2) {
 Text("Voice")
 .font(.system(size: 14, weight: .medium))
 .foregroundStyle(Color.oxyText)
 Text(selectedVoiceLabel)
 .font(.system(size: 12))
 .foregroundStyle(Color.oxySub)
 }
 Spacer()
 Button {
 previewVoice(settings.voice)
 } label: {
 Image(systemName: voicePreview.isLoading ? "hourglass" : (voicePreview.isPlaying ? "speaker.wave.2.fill" : "play.fill"))
 .font(.system(size: 12, weight: .bold))
 .foregroundStyle(Color.oxyOnAccent)
 .frame(width: 32, height: 32)
 .background(Color.oxyStone)
 .clipShape(Circle())
 }
 .buttonStyle(.plain)
 .disabled(voicePreview.isLoading)
 }
 }

 Menu {
 ForEach(OxySettings.voiceOptions, id: \.value) { voice in
 Button {
 settings.voice = voice.value
 saveSettings()
 } label: {
 if settings.voice == voice.value {
 Label(voice.label, systemImage: "checkmark")
 } else {
 Text(voice.label)
 }
 }
 }
 } label: {
 HStack {
 Text(selectedVoiceLabel)
 .font(.system(size: 15, weight: .semibold))
 .foregroundStyle(Color.oxyText)
 Spacer()
 Image(systemName: "chevron.down")
 .font(.system(size: 11, weight: .bold))
 .foregroundStyle(Color.oxySub)
 }
 .frame(maxWidth: .infinity)
 .padding(.horizontal, 13)
 .padding(.vertical, 11)
 .background(Color.oxySurface1)
 .clipShape(RoundedRectangle(cornerRadius: 12))
 .overlay(
 RoundedRectangle(cornerRadius: 12)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
 }
 .buttonStyle(.plain)
}
.padding(.vertical, 4)
}

private func legalLink(label: String, path: String, icon: String) -> some View {
 Button {
 guard let url = URL(string: "\(APIClient.shared.baseURL)\(path)") else { return }
 UIApplication.shared.open(url)
 } label: {
 HStack {

```

```

 Image(systemName: icon)
 .font(.system(size: 14, weight: .semibold))
 Text(label)
 .font(.system(size: 15, weight: .medium))
 Spacer()
 Image(systemName: "arrow.up.right")
 .font(.system(size: 12, weight: .semibold))
 .foregroundColor(Color.oxyDim)
 }
 .foregroundColor(Color.oxyText)

 .padding(.vertical, 4)
}
.buttonStyle(.plain)
}

private func loadSettings() {
 if let data = UserDefaults.standard.data(forKey: "oxy_settings"),
 let saved = try? JSONDecoder().decode(OxySettings.self, from: data) {
 settings = saved
 }
 normalizeSettings()
 appTheme = settings.appTheme
 UserDefaults.standard.set(settings.accentColor, forKey: "oxy_accentColor")
}

private func saveSettings() {
 normalizeSettings()
 appTheme = settings.appTheme
 UserDefaults.standard.set(settings.accentColor, forKey: "oxy_accentColor")
 if let data = try? JSONEncoder().encode(settings) {
 UserDefaults.standard.set(data, forKey: "oxy_settings")
 }
 Task {
 await NativeIntegrationManager.shared.syncNativeContext(userId: appState.userId)
 }
}

private func handleVersionTap() {
 versionTapCount += 1
 if versionTapCount >= 5 {
 versionTapCount = 0
 showBackendURLEditor = true
 }
}

private func normalizeSettings() {
 settings.appTheme = OxySettings.normalizedTheme(settings.appTheme)
 settings.autonomy = OxySettings.normalizedAutonomy(settings.autonomy)
 if !OxySettings.voiceOptions.contains(where: { $0.value == settings.voice }) {
 settings.voice = "Aoede"
 }
 if !OxySettings.accentOptions.contains(where: { $0.value == settings.accentColor }) {
 settings.accentColor = "stone"
 }
 settings.designPalette = settings.accentColor
}

private struct SharePayload: Identifiable {
 let id = UUID()
 let url: URL
}

private struct BackendURLEditorSheet: View {
 @Binding var currentURL: String
 let onDone: () -> Void
 @State private var draft = ""

 var body: some View {
 NavigationStack {
 ZStack {
 Color.oxyBg.ignoresSafeArea()
 VStack(alignment: .leading, spacing: 16) {
 Text("Custom backend URL")
 .font(.system(size: 13, weight: .semibold))
 .foregroundColor(Color.oxySub)
 .textCase(.uppercase)
 .tracking(0.4)

 TextField("https://your-backend.run.app", text: $draft)
 .font(.system(size: 15))
 .foregroundColor(Color.oxyText)
 .autocorrectionDisabled()
 .textInputAutocapitalization(.never)
 .keyboardType(.URL)
 .padding(14)
 }
 }
 }
 }
}

```

```

 .background(Color.oxySurface2)
 .clipShape(RoundedRectangle(cornerRadius: 12))
 .overlay(
 RoundedRectangle(cornerRadius: 12)
 .stroke(Color.oxyLine2, lineWidth: 1)
)

 Text("Leave blank to use the default Cloud Run backend.")
 .font(.system(size: 12))
 .foregroundStyle(Color.oxySub)

 if !currentURL.isEmpty {
 Button(role: .destructive) {
 draft = ""
 currentURL = ""
 onDone()
 } label: {
 Text("Reset to default")
 .font(.system(size: 14, weight: .medium))
 }
 }

 Spacer()
 }
 .padding(20)
}

.navigationBarTitle("Backend URL")
.navigationBarTitleDisplayMode(.inline)
.toolbar {
 ToolbarItem(placement: .cancellationAction) {
 Button("Cancel") { onDone() }
 .foregroundStyle(Color.oxyStone)
 }
 ToolbarItem(placement: .confirmationAction) {
 Button("Save") {
 currentURL = draft.trimmingCharacters(in: .whitespacesAndNewlines)
 onDone()
 }
 .font(.system(size: 15, weight: .semibold))
 .foregroundStyle(Color.oxyStone)
 }
}
}
.onAppear { draft = currentURL }
}

}

private struct ShareSheet: UIViewControllerRepresentable {
 let activityItems: [Any]

 func makeUIViewController(context: Context) -> UIActivityViewController {
 UIActivityViewController(activityItems: activityItems, applicationActivities: nil)
 }

 func updateUIViewController(_ uiViewController: UIActivityViewController, context: Context) {}
}

private struct AccentPickerSheet: View {
 @Environment(\.dismiss) private var dismiss
 @Binding var selection: String
 let onSelect: () -> Void

 var body: some View {
 NavigationStack {
 ZStack {
 Color.oxyBg.ignoresSafeArea()

 ScrollView {
 VStack(spacing: 8) {
 ForEach(OxySettings.accentOptions) { option in
 Button {
 selection = option.value
 onSelect()
 } label: {
 HStack(spacing: 14) {
 Circle()
 .fill(option.color)
 .frame(width: 22, height: 22)
 .overlay(
 Circle()
 .stroke(Color.oxyLine2, lineWidth: 1)
)
 Text(option.label)
 .font(.system(size: 17, weight: .semibold))
 .foregroundStyle(Color.oxyText)
 }
 }
 }
 }
 }
 }
 }
 }
}

```

Page 216 · Confidential



```

 nonisolated func audioPlayerDidFinishPlaying(_ player: AVAudioPlayer, successfully flag: Bool) {
 Task { @MainActor in
 self.isPlaying = false
 }
 }
 }

private struct TTSPreviewResponse: Codable {
 let audio: String
}

// MARK: - Initiative

private struct InitiativeScroller: View {
 @Binding var selection: String
 let onChange: () -> Void

 private var value: Binding<Double> {
 Binding(
 get: {
 Double(OxySettings.autonomyLevels.firstIndex(of: OxySettings.normalizedAutonomy(selection)) ?? 2)
 },
 set: { newValue in
 let rounded = min(max(Int(newValue.rounded()), 0), OxySettings.autonomyLevels.count - 1)
 let next = OxySettings.autonomyLevels[rounded]
 if next != selection {
 selection = next
 onChange()
 }
 }
)
 }

 var body: some View {
 VStack(alignment: .leading, spacing: 12) {
 HStack {
 VStack(alignment: .leading, spacing: 2) {
 Text("Initiative")
 .font(.system(size: 14, weight: .medium))
 .foregroundColor(Color.oxyText)
 Text(description)
 .font(.system(size: 12))
 .foregroundColor(Color.oxySub)
 }
 Spacer()
 Text(selection)
 .font(.system(size: 13, weight: .semibold))
 .foregroundColor(Color.oxyStone)
 }

 Slider(value: value, in: 0...Double(OxySettings.autonomyLevels.count - 1), step: 1)
 .tint(Color.oxyStone)

 HStack {
 Text("Quiet")
 Spacer()
 Text("Steady")
 Spacer()
 Text("Active")
 }
 .font(.system(size: 11, weight: .medium))
 .foregroundColor(Color.oxySub)
 }
 }

 private var description: String {
 switch selection {
 case "Quiet": return "Only speaks when asked."
 case "Low": return "Light nudges, mostly reactive."
 case "Active": return "Looks for useful openings."
 case "Bold": return "More opinionated and proactive."
 default: return "Helpful without being noisy."
 }
 }
}

// MARK: - Settings Model

struct OxySettings: Codable {
 var name: String = "Oxy"
 var voice: String = "Aoede"
 var voiceOn: Bool = true
 var voiceEngine: String = "current"
 var autonomy: String = "Balanced"
 var proactiveBriefings: Bool = true
 var healthAlerts: Bool = true
 var locationReminders: Bool = true
}

```

```

var homeLatitude: Double?
var homeLongitude: Double?
var designTemplate: String = "compact"
var designPalette: String = "stone"
var designMotion: String = "calm"
var accentColor: String = "stone"
var appTheme: String = "dark"
var bubbleStyle: String = "comfort"
var preferredMapsApp: String = "apple"
var preferredTransportMode: String = "driving"
var reviewBeforeOpeningApps: Bool = false
var confirmSensitiveAppOpens: Bool = true

enum CodingKeys: String, CodingKey {
 case name, voice, voiceOn, voiceEngine, autonomy, proactiveBriefings, healthAlerts, locationReminders
 case homeLatitude, homeLongitude, designTemplate, designPalette, designMotion
 case accentColor, appTheme, bubbleStyle, preferredMapsApp, preferredTransportMode, reviewBeforeOpeningApps
 case confirmSensitiveAppOpens
}

init() {}

init(from decoder: Decoder) throws {
 let container = try decoder.container(keyedBy: CodingKeys.self)
 name = try container.decodeIfPresent(String.self, forKey: .name) ?? "Oxy"
 voice = try container.decodeIfPresent(String.self, forKey: .voice) ?? "Aoede"
 voiceOn = try container.decodeIfPresent(Bool.self, forKey: .voiceOn) ?? true
 voiceEngine = try container.decodeIfPresent(String.self, forKey: .voiceEngine) ?? "current"
 autonomy = Self.normalizedAutonomy(try container.decodeIfPresent(String.self, forKey: .autonomy) ?? "Balanced")
 proactiveBriefings = try container.decodeIfPresent(Bool.self, forKey: .proactiveBriefings) ?? true
 healthAlerts = try container.decodeIfPresent(Bool.self, forKey: .healthAlerts) ?? true
 locationReminders = try container.decodeIfPresent(Bool.self, forKey: .locationReminders) ?? true
 homeLatitude = try container.decodeIfPresent(Double.self, forKey: .homeLatitude)
 homeLongitude = try container.decodeIfPresent(Double.self, forKey: .homeLongitude)
 designTemplate = try container.decodeIfPresent(String.self, forKey: .designTemplate) ?? "compact"
 designPalette = try container.decodeIfPresent(String.self, forKey: .designPalette) ?? "stone"
 designMotion = try container.decodeIfPresent(String.self, forKey: .designMotion) ?? "calm"
 accentColor = try container.decodeIfPresent(String.self, forKey: .accentColor) ?? designPalette
 appTheme = Self.normalizedTheme(try container.decodeIfPresent(String.self, forKey: .appTheme) ?? "dark")
 bubbleStyle = try container.decodeIfPresent(String.self, forKey: .bubbleStyle) ?? "comfort"
 preferredMapsApp = try container.decodeIfPresent(String.self, forKey: .preferredMapsApp) ?? "apple"
 preferredTransportMode = try container.decodeIfPresent(String.self, forKey: .preferredTransportMode) ?? "driving"
 reviewBeforeOpeningApps = try container.decodeIfPresent(Bool.self, forKey: .reviewBeforeOpeningApps) ?? false
 confirmSensitiveAppOpens = try container.decodeIfPresent(Bool.self, forKey: .confirmSensitiveAppOpens) ?? true
}

struct VoiceOption: Identifiable {
 let value: String
 let label: String
 var id: String { value }
}

struct AccentOption: Identifiable {
 let value: String
 let label: String
 let color: Color
 var id: String { value }
}

static let voiceOptions: [VoiceOption] = [
 VoiceOption(value: "Aoede", label: "Breezy"),
 VoiceOption(value: "Algenib", label: "Gravel"),
 VoiceOption(value: "Autonoe", label: "Bright"),
 VoiceOption(value: "Fenrir", label: "Electric"),
 VoiceOption(value: "Gacrux", label: "Mature"),
 VoiceOption(value: "Iapetus", label: "Measured"),
 VoiceOption(value: "Puck", label: "Playful"),
 VoiceOption(value: "Sulafat", label: "Warm"),
 VoiceOption(value: "Vindemiatrix", label: "Gentle"),
 VoiceOption(value: "Zubenelgenubi", label: "Casual"),
]

static let designTemplates = ["compact", "glass", "dense"]
static let designPalettes = ["stone", "mint", "blue", "violet"]
static let designMotions = ["calm", "snappy", "none"]
static let autonomyLevels = ["Quiet", "Low", "Balanced", "Active", "Bold"]
static func normalizedTheme(_ theme: String) -> String {
 switch theme {
 case "light", "system":
 return theme
 default:
 return "dark"
 }
}

static func normalizedAutonomy(_ autonomy: String) -> String {
 switch autonomy {

```

```

 case "Quiet", "Low":
 return autonomy
 case "Medium", "Balanced":
 return "Balanced"
 case "Medium-High", "Active", "High":
 return "Active"
 case "Assertive", "Bold":
 return "Bold"
 default:
 return "Balanced"
 }
}

static let accentOptions = [
 AccentOption(value: "stone", label: "Stone", color: Color.oxyDefaultStone),
 AccentOption(value: "mint", label: "Mint", color: Color.oxyGreen),
 AccentOption(value: "blue", label: "Blue", color: Color(red: 92/255, green: 154/255, blue: 245/255)),
 AccentOption(value: "cyan", label: "Cyan", color: Color(red: 48/255, green: 184/255, blue: 210/255)),
 AccentOption(value: "amber", label: "Amber", color: Color(red: 236/255, green: 168/255, blue: 65/255)),
 AccentOption(value: "coral", label: "Coral", color: Color(red: 238/255, green: 112/255, blue: 92/255)),
 AccentOption(value: "rose", label: "Rose", color: Color(red: 230/255, green: 124/255, blue: 154/255)),
 AccentOption(value: "violet", label: "Violet", color: Color(red: 162/255, green: 132/255, blue: 245/255)),
 AccentOption(value: "indigo", label: "Indigo", color: Color(red: 105/255, green: 126/255, blue: 235/255))
]

// MARK: - Appearance Lab

private struct DesignChoiceGroup: View {
 let title: String
 @Binding var selection: String
 let options: [String]
 let onChange: () -> Void

 var body: some View {
 VStack(alignment: .leading, spacing: 10) {
 Text(title)
 .font(.system(size: 13, weight: .semibold))
 .foregroundColor(Color.oxyText)

 HStack(spacing: 8) {
 ForEach(options, id: \.self) { option in
 Button {
 withAnimation(animation) {
 selection = option
 onChange()
 }
 } label: {
 Text(option.capitalized)
 .font(.system(size: 12, weight: .semibold))
 .foregroundColor(selection == option ? Color.oxyOnAccent : Color.oxySub)
 .frame(maxWidth: .infinity)
 .padding(.vertical, 9)
 .background(selection == option ? Color.oxyStone : Color.oxySurface3)
 .clipShape(RoundedRectangle(cornerRadius: 10))
 .overlay(
 RoundedRectangle(cornerRadius: 10)
 .stroke(selection == option ? Color.clear : Color.oxyLine2, lineWidth: 1)
)
 }
 }
 }
 }
 }
}

private var animation: Animation? {
 selection == "none" ? nil : .easeInOut(duration: 0.18)
}

private struct DesignPreview: View {
 let template: String
 let palette: String
 let motion: String

 private var accent: Color {
 switch palette {
 case "mint": return Color.oxyGreen
 case "blue": return Color(red: 92/255, green: 154/255, blue: 245/255)
 case "violet": return Color(red: 162/255, green: 132/255, blue: 245/255)
 default: return Color.oxyStone
 }
 }

 private var cornerRadius: CGFloat {
 switch template {
 case "dense": return 8

```

```

 case "glass": return 18
 default: return 12
 }
}

private var rowSpacing: CGFloat {
 template == "dense" ? 8 : 12
}

var body: some View {
 VStack(alignment: .leading, spacing: rowSpacing) {
 HStack {
 Circle()
 .fill(accent.opacity(0.18))
 .frame(width: 34, height: 34)
 .overlay(
 Image(systemName: "sparkles")
 .font(.system(size: 14, weight: .semibold))
 .foregroundColor(accent)
)

 VStack(alignment: .leading, spacing: 2) {
 Text("Oxy")
 .font(.system(size: 15, weight: .semibold))
 .foregroundColor(Color.oxyText)
 Text("Online")
 .font(.system(size: 12, weight: .medium))
 .foregroundColor(Color.oxySub)
 }

 Spacer()

 Text(motion.capitalized)
 .font(.system(size: 11, weight: .semibold))
 .foregroundColor(accent)
 .padding(.horizontal, 9)
 .padding(.vertical, 5)
 .background(accent.opacity(0.12))
 .clipShape(Capsule())
 }

 Text("I found the nearest McDonald's and opened Uber. Confirm in Uber.")
 .font(.system(size: template == "dense" ? 13 : 14, weight: .medium))
 .foregroundColor(Color.oxyText)
 .padding(template == "dense" ? 10 : 12)
 .frame(maxWidth: .infinity, alignment: .leading)
 .background(template == "glass" ? Color.oxySurface3.opacity(0.72) : Color.oxySurface3)
 .clipShape(RoundedRectangle(cornerRadius: cornerRadius))

 HStack(spacing: 8) {
 previewChip("Connectors", selected: true)
 previewChip("Memory", selected: false)
 previewChip("Settings", selected: false)
 }
 }
 .padding(14)
 .background(
 RoundedRectangle(cornerRadius: cornerRadius + 4)
 .fill(template == "glass" ? Color.oxySurface2.opacity(0.72) : Color.oxySurface2)
)
 .overlay(
 RoundedRectangle(cornerRadius: cornerRadius + 4)
 .stroke(accent.opacity(template == "mono" ? 0.18 : 0.32), lineWidth: 1)
)
 .animation(motion == "none" ? nil : .easeInOut(duration: motion == "snappy" ? 0.12 : 0.28), value: template)
 .animation(motion == "none" ? nil : .easeInOut(duration: motion == "snappy" ? 0.12 : 0.28), value: palette)
}

private func previewChip(_ label: String, selected: Bool) -> some View {
 Text(label)
 .font(.system(size: 11, weight: .semibold))
 .foregroundColor(selected ? Color.oxyOnAccent : Color.oxySub)
 .padding(.horizontal, 10)
 .padding(.vertical, 7)
 .background(selected ? accent : Color.oxySurface3)
 .clipShape(Capsule())
}

// MARK: - PendantSettingsSection

private struct PendantSettingsSection: View {
 var pendant: PendantBLEManager
 @State private var showUnpairConfirm = false

 var body: some View {
 VStack(alignment: .leading, spacing: 12) {

```

```

Text("PENDANT")
 .font(.system(size: 13, weight: .semibold))
 .foregroundColor(Color.oxySub)
 .textCase(.uppercase)

 .tracking(0.5)

VStack(spacing: 0) {
 HStack {
 VStack(alignment: .leading, spacing: 2) {
 Text("Status")
 .font(.system(size: 14, weight: .medium))
 .foregroundColor(Color.oxyText)
 Text(statusDescription)
 .font(.system(size: 12))
 .foregroundColor(statusColor)
 }
 Spacer()
 statusIndicator
 }
 .padding(.vertical, 4)

 if let name = pendant.peripheralName, pendant.isConnected {
 Divider().overlay(Color.oxyLine)
 HStack {
 Text("Device")
 .font(.system(size: 14, weight: .medium))
 .foregroundColor(Color.oxyText)
 Spacer()
 Text(name)
 .font(.system(size: 13))
 .foregroundColor(Color.oxySub)
 }
 .padding(.vertical, 4)
 }

 if let error = pendant.lastError {
 Divider().overlay(Color.oxyLine)
 Text(error)
 .font(.system(size: 12))
 .foregroundColor(.red.opacity(0.8))
 .frame(maxWidth: .infinity, alignment: .leading)
 .padding(.vertical, 4)
 }

 Divider().overlay(Color.oxyLine)

 HStack(spacing: 12) {
 if pendant.isConnected {
 Button("Unpair") {
 showUnpairConfirm = true
 }
 .font(.system(size: 14, weight: .semibold))
 .foregroundColor(.red.opacity(0.85))
 } else if pendant.connectionState == .scanning || pendant.connectionState == .connecting {
 Button("Cancel") {
 pendant.stopScan()
 }
 .font(.system(size: 14, weight: .semibold))
 .foregroundColor(Color.oxySub)
 } else {
 Button("Scan for Pendant") {
 pendant.startScan()
 }
 .font(.system(size: 14, weight: .semibold))
 .foregroundColor(Color.oxyStone)
 }
 Spacer()
 }
 .padding(.vertical, 4)
}
.padding(16)
.background(Color.oxySurface2)
.clipShape(RoundedRectangle(cornerRadius: 16))
.overlay(
 RoundedRectangle(cornerRadius: 16)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
}
.alert("Unpair Pendant", isPresented: $showUnpairConfirm) {
 Button("Unpair", role: .destructive) { pendant.unpair() }
 Button("Cancel", role: .cancel) {}
} message: {
 Text("This will disconnect and forget the paired pendant. You can pair again later.")
}
}

```

```

private var statusDescription: String {
 switch pendant.connectionState {
 case .disconnected: return "Not connected"
 case .scanning: return "Scanning..."
 case .connecting: return "Connecting..."
 case .connected: return "Connected"
 case .error: return "Error"
 }
}

private var statusColor: Color {
 switch pendant.connectionState {
 case .connected: return Color.oxyGreen
 case .error: return .red.opacity(0.8)
 case .scanning, .connecting: return Color.oxyStone
 default: return Color.oxySub
 }
}

@ViewBuilder
private var statusIndicator: some View {
 switch pendant.connectionState {
 case .connected:
 Circle()
 .fill(Color.oxyGreen)
 .frame(width: 10, height: 10)
 case .scanning, .connecting:
 ProgressView()
 .controlSize(.small)
 .tint(Color.oxyStone)
 case .error:
 Image(systemName: "exclamationmark.triangle.fill")
 .font(.system(size: 14))
 .foregroundColor(.red.opacity(0.8))
 case .disconnected:
 Circle()
 .fill(Color.oxySub.opacity(0.4))
 .frame(width: 10, height: 10)
 }
}
}

#Preview {
 SettingsView()
 .environment(AppState())
}

```

## LoginView.swift — OxyApp/OxyApp/Views/Auth/LoginView.swift

import SwiftUI

```

struct LoginView: View {
 @Environment(AppState.self) private var appState
 @State private var page = 0
 @State private var userId = ""
 @State private var password = ""
 @State private var isRegistering = false
 @State private var isLoading = false
 @State private var errorMessage: String?

 private let authService = AuthService()

 var body: some View {
 ZStack {
 Color.oxyBg.ignoresSafeArea()

 VStack(spacing: 0) {
 TabView(selection: $page) {
 OnboardingSlide(
 icon: "mic.fill",
 subIcon: "waveform",
 title: "Ask Oxy",
 content: "Messages, bookings, reminders\n- just say it."
)
 .tag(0)

 OnboardingSlide(
 icon: "arrow.triangle.branch",
 subIcon: nil,
 title: "Connected",
 content: "Gmail, Telegram, Uber and more\n- one request, done."
)
 .tag(1)

 OnboardingSlide(
 icon: "brain.head.profile",

```

```

 subIcon: nil,
 title: "Remembers you",
 content: "Gets better\nthe more you use it."
)
 .tag(2)

 LoginFormPage(
 userId: $userId,
 password: $password,
 isRegistering: $isRegistering,
 isLoading: $isLoading,
 errorMessage: $errorMessage,
 onSubmit: submit
)
 .tag(3)
}
.tabViewStyle(.page(indexDisplayMode: page < 3 ? .always : .never))
.animation(.easeInOut, value: page)

if page < 3 {
 Button(action: { withAnimation { page = min(page + 1, 3) } }) {
 Text(page == 2 ? "Get Started" : "Next")
 .font(.system(size: 17, weight: .semibold))
 .frame(maxWidth: .infinity)
 .frame(height: 54)
 .background(Color.oxyStone)
 .foregroundColor(Color.oxyOnAccent)
 .clipShape(RoundedRectangle(cornerRadius: 16))
 }
 .padding(.horizontal, 28)
 .padding(.bottom, 48)
 .transition(.opacity)
}
}
}

private func submit() {
 guard !userId.isEmpty, !password.isEmpty else { return }
 isLoading = true
 errorMessage = nil

 Task {
 do {
 let response: AuthResponse
 if isRegistering {
 response = try await authService.register(userId: userId, password: password)
 } else {
 response = try await authService.login(userId: userId, password: password)
 }

 if let token = response.token, let returnedUserId = response.userId {
 await MainActor.run {
 appState.login(userId: returnedUserId, token: token)
 }
 } else {
 await MainActor.run {
 withAnimation { errorMessage = response.error ?? "Authentication failed" }
 isLoading = false
 }
 }
 } catch {
 await MainActor.run {
 withAnimation { errorMessage = error.localizedDescription }
 isLoading = false
 }
 }
 }
}

// MARK: - Onboarding Slide

private struct OnboardingSlide: View {
 let icon: String
 let subIcon: String?
 let title: String
 let content: String

 var body: some View {
 VStack(spacing: 36) {
 Spacer()

 ZStack {
 Circle()
 .fill(Color.oxyStone.opacity(0.1))
 .frame(width: 148, height: 148)

```

```

 VStack(spacing: 10) {
 Image(systemName: icon)
 .font(.system(size: 54, weight: .semibold))
 .foregroundColor(Color.oxyStone)

 if let subIcon {
 Image(systemName: subIcon)
 .font(.system(size: 20, weight: .medium))
 .foregroundColor(Color.oxyStone.opacity(0.55))
 }
 }
 }

 VStack(spacing: 14) {
 Text(title)
 .font(.system(size: 30, weight: .bold, design: .rounded))
 .foregroundColor(Color.oxyText)

 Text(content)
 .font(.system(size: 16))
 .foregroundColor(Color.oxySub)
 .multilineTextAlignment(.center)
 .lineSpacing(3)
 }

 Spacer()
 Spacer()
}
.padding(.horizontal, 36)
}
}

```

```
// MARK: - Login Form Page
```

```

private struct LoginFormPage: View {
 @Binding var userId: String
 @Binding var password: String
 @Binding var isRegistering: Bool
 @Binding var isLoading: Bool
 @Binding var errorMessage: String?
 let onSubmit: () -> Void

 var body: some View {
 ScrollView {
 VStack(spacing: 0) {
 Spacer().frame(height: 72)

 Text("Oxy")
 .font(.system(size: 34, weight: .bold, design: .rounded))
 .foregroundColor(Color.oxyText)
 .padding(.bottom, 48)

 VStack(spacing: 16) {
 VStack(spacing: 12) {
 HStack(spacing: 12) {
 Image(systemName: "person.fill")
 .font(.system(size: 15))
 .foregroundColor(Color.oxySub)
 .frame(width: 20)

 TextField("User ID", text: $userId)
 .textInputAutocapitalization(.never)
 .autocorrectionDisabled()
 .font(.system(size: 15))
 .foregroundColor(Color.oxyText)
 }
 .padding(.horizontal, 16)
 .frame(height: 52)
 .background(Color.oxySurface2)
 .clipShape(RoundedRectangle(cornerRadius: 14))
 .overlay(
 RoundedRectangle(cornerRadius: 14)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
 }

 HStack(spacing: 12) {
 Image(systemName: "lock.fill")
 .font(.system(size: 15))
 .foregroundColor(Color.oxySub)
 .frame(width: 20)

 SecureField("Password", text: $password)
 .font(.system(size: 15))
 .foregroundColor(Color.oxyText)
 }
 .padding(.horizontal, 16)
 .frame(height: 52)
 }
 }
 }
 }
}

```



```

 .background(Color.oxySurface2)
 .clipShape(RoundedRectangle(cornerRadius: 14))
 .overlay(
 RoundedRectangle(cornerRadius: 14)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
 }

 if let errorMessage {
 HStack(spacing: 8) {
 Image(systemName: "exclamationmark.triangle.fill")
 .font(.system(size: 12))
 .foregroundColor(Color.oxyRed)
 Text(errorMessage)
 .font(.system(size: 13))
 .foregroundColor(Color.oxyRed)
 }
 .frame(maxWidth: .infinity, alignment: .leading)
 .padding(.horizontal, 4)
 .transition(.opacity.combined(with: .move(edge: .top)))
 }

 Button(action: onSubmit) {
 HStack(spacing: 8) {
 if isLoading {
 ProgressView()
 .tint(.white)
 .scaleEffect(0.85)

 } else {
 Text(isRegistering ? "Create Account" : "Sign In")
 .font(.system(size: 16, weight: .semibold))
 Image(systemName: "arrow.right")
 .font(.system(size: 14, weight: .semibold))
 }
 }
 .frame(maxWidth: .infinity)
 .frame(height: 52)
 .background(
 LinearGradient(
 colors: [Color.oxyStone, Color.oxyStone.opacity(0.85)],
 startPoint: .leading,
 endPoint: .trailing
)
)
 .foregroundColor(Color.oxyOnAccent)
 .clipShape(RoundedRectangle(cornerRadius: 14))
 .shadow(color: Color.oxyStone.opacity(0.3), radius: 8, y: 4)
 }
 .disabled(isLoading || userId.isEmpty || password.isEmpty)
 .opacity(userId.isEmpty || password.isEmpty ? 0.6 : 1)
 .animation(.easeInOut(duration: 0.2), value: userId.isEmpty || password.isEmpty)

 Button(action: {
 withAnimation(.easeInOut(duration: 0.2)) {
 isRegistering.toggle()
 }
 }) {
 Text(isRegistering ? "Already have an account? **Sign in**" : "New to Oxy? **Create account**")
 .font(.system(size: 13))
 .foregroundColor(Color.oxySub)
 }
 .padding(.horizontal, 28)

 Spacer().frame(height: 60)
}

}
.scrollDismissesKeyboard(.interactively)
}

}

#Preview {
 LoginView()
 .environment(AppState())
}

```

## HistoryView.swift — OxyApp/OxyApp/Views/History/HistoryView.swift

```

import SwiftUI

struct HistoryView: View {
 @Environment(AppState.self) private var appState
 @State private var sessions: [ChatSessionSummary] = []
 @State private var isLoading = true

 var body: some View {

```

```

NavigationStack {
 ZStack {
 Color.oxyBg.ignoresSafeArea()

 if isLoading {
 ProgressView()
 .tint(Color.oxyStone)
 } else if sessions.isEmpty {
 VStack(spacing: 12) {
 Image(systemName: "bubble.left.and.bubble.right")
 .font(.system(size: 40))
 .foregroundColor(Color.oxyDim)
 Text("No conversations yet")
 .font(.system(size: 15))
 .foregroundColor(Color.oxySub)
 }
 } else {
 ScrollView {
 LazyVStack(spacing: 0) {
 ForEach(sessions) { session in
 Button {
 NotificationCenter.default.post(
 name: .oxyJumpToChat,
 object: nil,
 userInfo: [
 "sessionId": session.id,
 "lastAt": session.lastAt ?? ""
]
)
 } label: {
 SessionRow(session: session)
 }
 .buttonStyle(.plain)

 if session.id != sessions.last?.id {
 Divider()
 .overlay(Color.oxyLine)
 .padding(.horizontal, 16)
 }
 }
 }
 .padding(.vertical, 8)
 }
 }
 .navigationTitle("Chats")
 .navigationBarTitleDisplayMode(.large)
 .toolbarBackground(Color.oxySurface1, for: .navigationBar)
 .toolbarBackground(.visible, for: .navigationBar)
 .task {
 await loadSessions()
 }
 .refreshable {
 await loadSessions()
 }
 }
}

private func loadSessions() async {
 do {
 let data = try await APIClient.shared.request(
 path: "/history/\(AppState.userId)/sessions"
)
 let response = try JSONDecoder().decode(ChatSessionsResponse.self, from: data)
 sessions = response.sessions
 isLoading = false
 } catch {
 isLoading = false
 }
}

// MARK: - Session Row

private struct SessionRow: View {
 let session: ChatSessionSummary

 var body: some View {
 HStack(spacing: 0) {
 Text(session.title)
 .font(.system(size: 15, weight: .medium))
 .foregroundColor(Color.oxyText)
 .lineLimit(1)

 Spacer(minLength: 12)

 Text(session.relativeTime)
 }
 }
}

```

```

 .font(.system(size: 13))
 .foregroundColor(Color.oxyDim)
 }
 .padding(.horizontal, 16)
 .padding(.vertical, 14)
}
}

// MARK: - ChatSessionSummary extension

private extension ChatSessionSummary {
 var relativeTime: String {
 guard let date = Date.oxyParse(lastAt ?? startedAt) else { return "" }
 let diff = Date().timeIntervalSince(date)
 if diff < 60 { return "just now" }
 if diff < 3600 { return "\(Int(diff / 60))m ago" }
 if diff < 86400 { return "\(Int(diff / 3600))h ago" }
 if diff < 7 * 86400 { return "\(Int(diff / 86400))d ago" }
 let fmt = DateFormatter()
 fmt.dateFormat = "d MMM"
 return fmt.string(from: date)
 }
}

#Preview {

 HistoryView()
 .environment(AppState())
}

```

## MemoryView.swift — OxyApp/OxyApp/Views/Memory/MemoryView.swift

```

import SwiftUI

struct MemoryView: View {
 @Environment(AppState.self) private var appState
 @State private var summary = MemorySummary()
 @State private var isLoading = true
 @State private var isSaving = false
 @State private var draft = ""
 @State private var saveMessage: String?
 private let embedded: Bool

 init(embedded: Bool = false) {
 self.embedded = embedded
 }

 var body: some View {
 if embedded {
 memoryContent
 } else {
 NavigationStack {
 memoryContent
 }
 }
 }

 private var memoryContent: some View {
 ZStack {
 Color.oxyBg.ignoresSafeArea()

 ScrollView {
 VStack(alignment: .leading, spacing: 16) {
 MemoryDropBox(
 draft: $draft,
 isSaving: isSaving,
 message: saveMessage,
 onSave: { Task { await saveMemory() } }
)
 .scrollTransition(axis: .vertical) { content, phase in
 content
 .opacity(phase.isIdentity ? 1 : 0)
 .offset(y: phase.isIdentity ? 0 : 24)
 }

 if isLoading {
 HStack(spacing: 10) {
 OxySkeletonCard(height: 68, cornerRadius: 14)
 OxySkeletonCard(height: 68, cornerRadius: 14)
 OxySkeletonCard(height: 68, cornerRadius: 14)
 }
 .transition(.opacity.combined(with: .move(edge: .bottom)))
 } else {
 HStack(spacing: 10) {
 MemoryStat(title: "Saved", value: "\(summary.total)")
 MemoryStat(title: "Learned", value: "\(summary.learned)")
 MemoryStat(title: "Profile", value: summary.profile ? "On" : "Off")
 }
 }
 }
 }
 }
 }
}

```

```

 }
 .scrollTransition(axis: .vertical) { content, phase in
 content
 .opacity(phase.isIdentity ? 1 : 0)
 .offset(y: phase.isIdentity ? 0 : 22)
 }
 }

 if let lastUpdated = summary.lastUpdated {
 Text("Updated \${formattedDate(lastUpdated)}")
 .font(.system(size: 12, weight: .medium))
 .foregroundColor(Color.oxyDim)
 .frame(maxWidth: .infinity, alignment: .center)
 .padding(.top, 6)
 .scrollTransition(axis: .vertical) { content, phase in
 content
 .opacity(phase.isIdentity ? 1 : 0)
 .offset(y: phase.isIdentity ? 0 : 16)
 }
 }
 }
 .padding(16)
 .animation(.spring(response: 0.4, dampingFraction: 0.8), value: isLoading)
}

}
.navigationTitle("Memory")
.navigationBarTitleDisplayMode(.large)
.toolbarBackground(Color.oxySurface1, for: .navigationBar)
.toolbarBackground(.visible, for: .navigationBar)
.task { await loadMemory() }
.refreshable { await loadMemory() }
}

private func loadMemory() async {
 await MainActor.run {
 isLoading = true
 saveMessage = nil
 }
 do {
 let data = try await APIClient.shared.request(path: "/memory/\${appState.userId}")
 let response = try JSONDecoder().decode(MemoryResponse.self, from: data)
 await MainActor.run {
 summary = response.summary ?? MemorySummary()
 isLoading = false
 }
 } catch {
 await MainActor.run {
 summary = MemorySummary()
 isLoading = false
 }
 }
}

private func saveMemory() async {
 let content = draft.trimmingCharacters(in: .whitespacesAndNewlines)
 guard !content.isEmpty, !isSaving else { return }
 await MainActor.run {
 isSaving = true
 saveMessage = nil
 }
 do {
 _ = try await APIClient.shared.request(
 path: "/memory",
 method: "POST",
 body: [
 "userId": appState.userId,
 "content": content
]
)
 await MainActor.run {
 draft = ""
 saveMessage = "Saved."
 isSaving = false
 }
 await loadMemory()
 } catch {
 await MainActor.run {
 saveMessage = "Could not save that."
 isSaving = false
 }
 }
}

private func formattedDate(_ dateStr: String) -> String {
 let iso = ISO8601DateFormatter()
 guard let date = iso.date(from: dateStr) else { return dateStr }

```

```

 let fmt = DateFormatter()
 fmt.dateFormat = "HH:mm - d MMM"
 return fmt.string(from: date)
 }
}

private struct MemoryDropBox: View {
 @Binding var draft: String
 let isSaving: Bool
 let message: String?
 let onSave: () -> Void

 var body: some View {
 VStack(alignment: .leading, spacing: 14) {
 HStack(spacing: 12) {
 ZStack {
 RoundedRectangle(cornerRadius: 14)
 .fill(Color.oxyStone.opacity(0.14))
 .frame(width: 46, height: 46)
 Image(systemName: "lock.open.fill")
 .font(.system(size: 18, weight: .semibold))
 .foregroundColor(Color.oxyStone)
 }

 VStack(alignment: .leading, spacing: 3) {
 Text("Drop a memory")
 .font(.system(size: 20, weight: .semibold))
 .foregroundColor(Color.oxyText)
 Text("Add it once. Oxy keeps it for later.")
 .font(.system(size: 13))
 .foregroundColor(Color.oxySub)
 }
 Spacer()
 }

 TextField("Remember that...", text: $draft, axis: .vertical)
 .font(.system(size: 15))
 .foregroundColor(Color.oxyText)
 .lineLimit(2...4)
 .padding(12)
 .background(Color.oxySurface1)
 .clipShape(RoundedRectangle(cornerRadius: 14))
 .overlay(
 RoundedRectangle(cornerRadius: 14)
 .stroke(Color.oxyLine2, lineWidth: 1)
)

 HStack {
 if let message {
 Text(message)
 .font(.system(size: 12, weight: .medium))
 .foregroundColor(message == "Saved." ? Color.oxyGreen : Color.oxyRed)
 }
 Spacer()
 Button(action: onSave) {
 HStack(spacing: 6) {
 if isSaving {
 ProgressView()
 .scaleEffect(0.7)
 .tint(Color.oxyOnAccent)
 } else {
 Image(systemName: "arrow.down.to.line.compact")
 .font(.system(size: 12, weight: .bold))
 }
 Text(isSaving ? "Saving" : "Drop in")
 .font(.system(size: 13, weight: .semibold))
 }
 .foregroundColor(Color.oxyOnAccent)
 .padding(.horizontal, 13)
 .padding(.vertical, 9)
 .background(Color.oxyStone)
 .clipShape(Capsule())
 }
 .disabled(draft.trimmingCharacters(in: .whitespacesAndNewlines).isEmpty || isSaving)
 }
 }
 .padding(16)
 .background(Color.oxySurface2)
 .clipShape(RoundedRectangle(cornerRadius: 18))
 .overlay(
 RoundedRectangle(cornerRadius: 18)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
 }
}

private struct MemoryStat: View {

```

```

let title: String
let value: String

var body: some View {
 VStack(alignment: .leading, spacing: 3) {
 Text(value)
 .font(.system(size: 18, weight: .bold, design: .rounded))
 .foregroundColor(Color.oxyText)
 Text(title)
 .font(.system(size: 11, weight: .medium))
 .foregroundColor(Color.oxySub)
 }

 .frame(maxWidth: .infinity, alignment: .leading)
 .padding(12)
 .background(Color.oxySurface2)
 .clipShape(RoundedRectangle(cornerRadius: 14))
 .overlay(
 RoundedRectangle(cornerRadius: 14)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
}

struct MemorySummary: Codable {
 var total: Int = 0
 var learned: Int = 0
 var profile: Bool = false
 var lastUpdated: String?
}

struct MemoryResponse: Codable {
 let summary: MemorySummary?
}

#Preview {
 MemoryView()
 .environment(AppState())
}

```

## ConnectorsView.swift — OxyApp/OxyApp/Views/Connectors/ConnectorsView.swift

```

import SwiftUI
#if canImport(Uikit)
import UIKit
#endif

struct ConnectorsView: View {
 @Environment(AppState.self) private var appState
 @Environment(\.scenePhase) private var scenePhase
 @State private var connectors: [Connector] = []
 @State private var isLoading = true
 @State private var googleStatus: GoogleStatus = .idle
 @State private var errorMessage: String?
 @State private var cardsVisible = false

 enum GoogleStatus: String {
 case idle, connecting, connected, needsReconnect, error
 }

 var body: some View {
 NavigationStack {
 ZStack {
 Color.oxyBg.ignoresSafeArea()

 if isLoading {
 VStack(spacing: 12) {
 OxySkeletonCard(height: 92)
 OxySkeletonCard(height: 148)
 OxySkeletonCard(height: 148)
 }
 .padding(16)
 } else {
 ScrollView {
 VStack(spacing: 24) {
 if let errorMessage {
 ErrorBanner(message: errorMessage)
 }

 // Google section
 googleSection
 .opacity(cardsVisible ? 1 : 0)
 .offset(y: cardsVisible ? 0 : 18)
 .animation(.spring(response: 0.4, dampingFraction: 0.8).delay(0.02), value: cardsVisible)

 // Grouped connectors
 let nonGoogle = connectors.filter { $0.id != "google" && $0.implemented }

```

```

 let grouped = Dictionary(grouping: nonGoogle) { $0.category }

 ForEach(grouped.keys.sorted(), id: \.self) { category in
 if let items = grouped[category] {
 connectorSection(title: category, connectors: items)
 }
 }
 .padding(16)
 }
}

.navigationTitle("Connectors")
.navigationBarTitleDisplayMode(.large)
.toolbarBackground(Color.oxySurface1, for: .navigationBar)
.toolbarBackground(.visible, for: .navigationBar)
.task {
 await loadConnectors()
}
.refreshable {
 await loadConnectors()
}
.onChange(of: scenePhase) { _, phase in
 if phase == .active && googleStatus == .connecting {
 Task { await loadConnectors() }
 }
}
}

}

// MARK: - Google Section

private var googleConnector: Connector? {
 connectors.first(where: { $0.id == "google" })
}

private var googleSection: some View {
 VStack(alignment: .leading, spacing: 12) {
 Text("Google")
 .font(.system(size: 13, weight: .semibold))
 .foregroundColor(Color.oxySub)
 .textCase(.uppercase)
 .tracking(0.5)

 HStack(spacing: 14) {
 AppIconView(candidates: [googleConnector?.icon ?? "", "google"], fallbackSystemName: "envelope.fill")
 .frame(width: 44, height: 44)

 VStack(alignment: .leading, spacing: 2) {
 Text("Google")
 .font(.system(size: 16, weight: .semibold))
 .foregroundColor(Color.oxyText)

 Text(googleSubtitle)
 .font(.system(size: 13))
 .foregroundColor(Color.oxySub)
 }

 Spacer()

 Button(action: handleGoogleAction) {
 ConnectorPill(
 label: googleButtonLabel,
 tint: googleStatus == .connected ? Color.oxySub : Color.oxyStone,
 isBusy: googleStatus == .connecting
)
 }
 .disabled(googleStatus == .connecting)
 }
 .padding(14)
 .background(Color.oxySurface2)
 .clipShape(RoundedRectangle(cornerRadius: 16))
 .overlay(
 RoundedRectangle(cornerRadius: 16)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
 }
}

private var googleButtonLabel: String {
 switch googleStatus {
 case .idle: return "Connect"
 case .connecting: return "Connecting..."
 case .connected: return "Disable"
 case .needsReconnect: return "Reconnect"
 case .error: return "Retry"
 }
}

```

```

 }
}

private var googleSubtitle: String {
 switch googleStatus {
 case .connected:
 return "Gmail · Calendar · Connected"
 case .needsReconnect:
 return "Gmail · Calendar · Reconnect needed"
 default:
 return "Gmail · Calendar"
 }
}

// MARK: - Connector Section

private func connectorSection(title: String, connectors: [Connector]) -> some View {
 VStack(alignment: .leading, spacing: 12) {
 Text(title)
 .font(.system(size: 13, weight: .semibold))
 .foregroundColor(Color.oxySub)
 .textCase(.uppercase)
 .tracking(0.5)

 LazyVGrid(columns: [
 GridItem(.flexible()),
 GridItem(.flexible())
], spacing: 12) {
 ForEach(Array(connectors.enumerated()), id: \.element.id) { index, connector in
 ConnectorCard(
 connector: connector,
 onAction: { handleConnectorAction(connector) }
)
 .opacity(cardsVisible ? 1 : 0)
 .offset(y: cardsVisible ? 0 : 22)
 .animation(.spring(response: 0.4, dampingFraction: 0.8).delay(Double(index) * 0.08), value: cardsVisible)
 }
 }
 }
}

// MARK: - Actions

private func loadConnectors() async {
 do {
 let data = try await APIClient.shared.request(
 path: "/connectors/\(appState.userId)"
)
 let response = try JSONDecoder().decode(ConnectorsResponse.self, from: data)
 await MainActor.run {
 connectors = response.connectors
 if let google = connectors.first(where: { $0.id == "google" }) {
 googleStatus = GoogleStatus(connector: google)
 } else {
 googleStatus = .idle
 }
 errorMessage = nil
 isLoading = false
 cardsVisible = false
 DispatchQueue.main.async {
 cardsVisible = true
 }
 }
 } catch {
 await MainActor.run {
 errorMessage = error.localizedDescription
 if googleStatus == .connecting {
 googleStatus = .error
 }
 isLoading = false
 cardsVisible = true
 }
 }
}

private func connectGoogle() {
 googleStatus = .connecting
 Task {
 do {
 let data = try await APIClient.shared.request(
 path: "/auth/google/start",
 queryItems: [URLQueryItem(name: "userId", value: appState.userId)]
)
 if let json = try? JSONSerialization.jsonObject(with: data) as? [String: Any],
 let urlStr = json["url"] as? String,
 let url = URL(string: urlStr) {
 await MainActor.run {

```



```

 errorMessage = nil
 UIApplication.shared.open(url)
 }
} else {
 await MainActor.run {
 googleStatus = .error
 errorMessage = "Google connect failed."
 }
}
} catch {
 await MainActor.run {
 googleStatus = .error
 errorMessage = error.localizedDescription
 }
}
}
}

private func handleGoogleAction() {
 guard let connector = googleConnector else {
 connectGoogle()
 return
 }
 if connector.enabled && connector.connectionState == "connected" {
 updateConnector(connector, enabled: false)
 } else {
 connectGoogle()
 }
}

private func handleConnectorAction(_ connector: Connector) {
 guard connector.implemented else { return }
 if connector.id == "google" && (!connector.enabled || connector.connectionState == "needs_reconnect") {
 connectGoogle()
 return
 }
 updateConnector(connector, enabled: !connector.enabled)
}

private func updateConnector(_ connector: Connector, enabled: Bool) {
 Task {
 do {
 let _ = try await APIClient.shared.request(
 path: "/connectors",
 method: "POST",
 body: [
 "userId": appState.userId,
 "connectorId": connector.id,
 "enabled": enabled
]
)
 await MainActor.run {
 if let idx = connectors.firstIndex(where: { $0.id == connector.id }) {
 connectors[idx].enabled = enabled
 connectors[idx].connectionState = enabled ? "connected" : "available"
 connectors[idx].statusText = enabled ? "Connected" : "Available"
 if connector.id == "google" {
 googleStatus = enabled ? .connected : .idle
 }
 errorMessage = nil
 }
 }
 } catch {
 await MainActor.run {
 errorMessage = error.localizedDescription
 }
 }
 }
}

// MARK: - Connector Card

private struct ConnectorCard: View {
 let connector: Connector
 let onAction: () -> Void

 private var sfSymbol: String {
 switch connector.id {
 case "google": return "envelope.fill"
 case "imessage": return "message.fill"
 case "whatsapp": return "phone.fill"
 case "spotify": return "music.note"
 case "reminders": return "checklist"
 case "deliveroo": return "takeoutbag.and.cup.and.straw.fill"
 case "uber": return "car.fill"
 }
 }
}

```

```

 case "telegram": return "paperplane.fill"
 case "monzo": return "banknote.fill"
 case "homekit": return "house.fill"
 case "trainline": return "tram.fill"
 case "maps": return "map.fill"
 case "notion": return "doc.text.fill"
 case "betfair": return "chart.line.uptrend.xyaxis"
 case "netflix": return "play.tv.fill"
 default: return "puzzlepiece.fill"
 }
}

var body: some View {
 VStack(spacing: 12) {
 AppIconView(candidates: [connector.icon, connector.id], fallbackSystemName: sfSymbol)
 .frame(width: 44, height: 44)

 VStack(spacing: 2) {
 Text(connector.name)
 .font(.system(size: 13, weight: .medium))
 .foregroundColor(Color.oxyText)
 .lineLimit(1)

 Text(connector.statusText)
 .font(.system(size: 10, weight: .medium))
 .foregroundColor(connector.statusColor)
 }

 Button(action: onAction) {
 ConnectorPill(label: connector.actionLabel, tint: connector.actionTint, isBusy: false)
 }
 .disabled(!connector.implemented)
 }
 .overlay(alignment: .topTrailing) {
 if connector.enabled {
 ConnectorCheckmark()
 .padding(10)
 .transition(.opacity.combined(with: .move(edge: .bottom)))
 }
 }
 .frame(maxWidth: .infinity)
 .padding(.vertical, 14)
 .padding(.horizontal, 8)
 .background(Color.oxySurface2)
 .opacity(connector.implemented ? 1.0 : 0.45)
 .clipShape(RoundedRectangle(cornerRadius: 14))
 .overlay(
 RoundedRectangle(cornerRadius: 14)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
}

private struct ConnectorCheckmark: View {
 @State private var isDrawn = false

 var body: some View {
 Circle()
 .fill(Color.oxyGreen)
 .frame(width: 18, height: 18)
 .overlay {
 CheckmarkShape()
 .trim(from: 0, to: isDrawn ? 1 : 0)
 .stroke(Color.oxyOnAccent, style: StrokeStyle(lineWidth: 2, lineCap: .round, lineJoin: .round))
 .frame(width: 8, height: 7)
 }
 .scaleEffect(isDrawn ? 1 : 0.72)
 .onAppear { isDrawn = true }
 .animation(.spring(response: 0.4, dampingFraction: 0.8), value: isDrawn)
 }
}

private struct CheckmarkShape: Shape {
 func path(in rect: CGRect) -> Path {
 var path = Path()
 path.move(to: CGPoint(x: rect.minX, y: rect.midY))
 path.addLine(to: CGPoint(x: rect.midX * 0.82, y: rect.maxY))
 path.addLine(to: CGPoint(x: rect.maxX, y: rect.minY))
 return path
 }
}

private struct ConnectorPill: View {
 let label: String
 let tint: Color
 let isBusy: Bool

```

```

var body: some View {
 HStack(spacing: 6) {
 if isBusy {
 ProgressView()
 .scaleEffect(0.65)
 .tint(tint)
 }
 Text(label)
 .font(.system(size: 12, weight: .semibold))
 .lineLimit(1)
 }
 .foregroundColor(tint)
 .padding(.horizontal, 12)
 .padding(.vertical, 7)
 .background(tint.opacity(0.12))
 .clipShape(Capsule())
 .overlay(
 Capsule()
 .stroke(tint.opacity(0.28), lineWidth: 1)
)
}

private struct AppIconView: View {
 let candidates: [String]
 let fallbackSystemName: String

 var body: some View {
 Group {
 if let img = firstAssetImage(from: candidates) {
 Image(uiImage: img)
 .resizable()
 .scaledToFill()
 } else if let url = firstURL(from: candidates) {
 AsyncImage(url: url) { phase in
 if case .success(let image) = phase {
 image.resizable().scaledToFill()
 } else {
 brandFallback
 }
 }
 } else if let emoji = firstEmoji(from: candidates) {
 Text(emoji)
 .font(.system(size: 20))
 } else {
 brandFallback
 }
 }
 .frame(maxWidth: .infinity, maxHeight: .infinity)
 .clipShape(RoundedRectangle(cornerRadius: 10))
 .overlay(
 RoundedRectangle(cornerRadius: 10)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
 }

 private var brandFallback: some View {
 let brand = brandStyle
 return ZStack {
 RoundedRectangle(cornerRadius: 10)
 .fill(brand.background)
 if let text = brand.text {
 Text(text)
 .font(.system(size: brand.fontSize, weight: .bold))
 .foregroundColor(brand.foreground)
 .minimumScaleFactor(0.55)
 .lineLimit(1)
 .padding(.horizontal, 4)
 } else {
 Image(systemName: fallbackSystemName)
 .font(.system(size: 17, weight: .semibold))
 .foregroundColor(brand.foreground)
 }
 }
 }

 private var brandStyle: (text: String?, background: Color, foreground: Color, fontSize: CGFloat) {
 let names = candidates.map { $0.lowercased() }
 if names.contains("uber") { return ("Uber", .black, .white, 12) }
 if names.contains("ubereats") { return ("Eats", .black, Color(red: 6/255, green: 193/255, blue: 103/255), 11) }
 if names.contains("google") { return ("G", .white, Color(red: 66/255, green: 133/255, blue: 244/255), 21) }
 if names.contains("maps") { return (nil, Color(red: 66/255, green: 133/255, blue: 244/255), .white, 17) }
 if names.contains("telegram") { return (nil, Color(red: 42/255, green: 171/255, blue: 238/255), .white, 17) }
 if names.contains("trainline") { return ("TL", Color(red: 0/255, green: 169/255, blue: 126/255), .white, 15) }
 if names.contains("deliveroo") { return ("D", Color(red: 0/255, green: 204/255, blue: 188/255), .white, 19) }
 if names.contains("netflix") { return ("N", .black, Color(red: 229/255, green: 9/255, blue: 20/255), 21) }
 return (nil, Color.oxySurface3, Color.oxySub, 17)
 }
}

```

```

 }

 private func firstURL(from candidates: [String]) -> URL? {
 for name in candidates {
 if name.lowercased().hasPrefix("http"), let url = URL(string: name) {
 return url
 }
 }
 return nil
 }

 private func firstAssetImage(from candidates: [String]) -> UIImage? {
 #if canImport(Uikit)
 for name in candidates where !name.isEmpty {
 if let img = UIImage(named: name) { return img }
 }
 #endif
 return nil
 }

 private func firstEmoji(from candidates: [String]) -> String? {
 candidates.first { candidate in
 candidate.unicodeScalars.contains { $0.properties.isEmojiPresentation }
 }
 }
}

// MARK: - Models

struct Connector: Codable, Identifiable {
 let id: String
 let name: String
 let icon: String
 let category: String
 var enabled: Bool
 let implemented: Bool
 var connectionState: String
 var statusText: String

 enum CodingKeys: String, CodingKey {
 case id, name, icon, category, enabled, implemented, connectionState, statusText
 }

 init(from decoder: Decoder) throws {
 let c = try decoder.container(keyedBy: CodingKeys.self)
 id = try c.decode(String.self, forKey: .id)
 name = try c.decode(String.self, forKey: .name)
 icon = try c.decodeIfPresent(String.self, forKey: .icon) ?? "■"
 category = try c.decodeIfPresent(String.self, forKey: .category) ?? "Other"
 enabled = try c.decodeIfPresent(Bool.self, forKey: .enabled) ?? false
 implemented = try c.decodeIfPresent(Bool.self, forKey: .implemented) ?? false
 connectionState = try c.decodeIfPresent(String.self, forKey: .connectionState) ?? (enabled ? "connected" : "available")
 statusText = try c.decodeIfPresent(String.self, forKey: .statusText) ?? (enabled ? "Connected" : "Available")
 }

 var actionLabel: String {
 if connectionState == "needs_reconnect" { return "Reconnect" }
 if connectionState == "needs_setup" { return "Setup" }
 if connectionState == "degraded" { return enabled ? "Enabled" : "Enable" }
 return enabled ? "Disconnect" : "Connect"
 }

 var actionTint: Color {
 if !implemented { return Color.oxyDim }
 if connectionState == "needs_reconnect" || connectionState == "needs_setup" || connectionState == "degraded" { return Color.oxyStone }
 return enabled ? Color.oxySub : Color.oxyGreen
 }

 var statusColor: Color {
 switch connectionState {
 case "connected": return Color.oxyGreen
 case "needs_reconnect", "needs_setup", "degraded": return Color.oxyStone
 default: return Color.oxySub
 }
 }
}

private extension ConnectorsView.GoogleStatus {
 init(connector: Connector) {
 if connector.connectionState == "needs_reconnect" {
 self = .needsReconnect
 } else if connector.enabled {
 self = .connected
 } else {
 self = .idle
 }
 }
}

```

```

}

struct ConnectorsResponse: Codable {
 let connectors: [Connector]
}

#Preview {
 ConnectorsView()
 .environment(AppState())
}

```

## ProactiveView.swift — OxyApp/OxyApp/Views/Proactive/ProactiveView.swift

```

import SwiftUI

struct ProactiveView: View {
 @Environment(AppState.self) private var appState
 @State private var briefings: [Briefing] = []
 @State private var isLoading = false
 @State private var isChecking = false
 @State private var errorMessage: String?

 private let service = ChatService()
 private var visibleBriefings: [Briefing] {
 briefings.filter(\.isWorthShowing)
 }

 var body: some View {
 NavigationStack {
 ZStack {
 Color.oxyBg.ignoresSafeArea()

 ScrollView {
 LazyVStack(spacing: 12) {
 if let errorMessage {
 ErrorBanner(message: errorMessage)
 .padding(.horizontal, 16)
 }

 ProactiveHeader(
 isChecking: isChecking,
 onCheckNow: { Task { await checkNow() } }
)
 .padding(.horizontal, 16)
 .padding(.top, 12)

 if isLoading &&& briefings.isEmpty {
 ProgressView()
 .tint(Color.oxyStone)
 .padding(.top, 28)
 } else if visibleBriefings.isEmpty {
 EmptyProactiveState()
 .padding(.horizontal, 16)
 .padding(.top, 24)
 } else {
 ForEach(visibleBriefings) { briefing in
 BriefingCard(briefing: briefing) {
 Task { await markRead(briefing) }
 }
 }
 .padding(.horizontal, 16)
 }
 }
 }
 .padding(.bottom, 24)
 }
 .refreshable {
 await loadBriefings()
 }
 }
 .navigationTitle("Today")
 .navigationBarTitleDisplayMode(.inline)
 .toolbarBackground(Color.oxySurface1, for: .navigationBar)
 .toolbarBackground(.visible, for: .navigationBar)
 }
 .task {
 await loadBriefings()
 }
}

private func loadBriefings() async {
 isLoading = true
 errorMessage = nil
 do {
 briefings = try await service.loadBriefings(userId: appState.userId)
 } catch {
 errorMessage = error.localizedDescription
 }
}

```

```

 isLoading = false
 }

 private func checkNow() async {
 guard !isChecking else { return }
 isChecking = true
 errorMessage = nil
 await NativeIntegrationManager.shared.syncNativeContext(userId: appState.userId)
 do {
 try await service.runProactiveCheck(userId: appState.userId)
 briefings = try await service.loadBriefings(userId: appState.userId)
 } catch {
 errorMessage = error.localizedDescription
 }
 isChecking = false
 }

 private func markRead(_ briefing: Briefing) async {
 await service.markBriefingRead(userId: appState.userId, briefingId: briefing.id)
 if let index = briefings.firstIndex(where: { $0.id == briefing.id }) {
 briefings.remove(at: index)
 }
 }
}

private struct ProactiveHeader: View {
 let isChecking: Bool
 let onCheckNow: () -> Void

 var body: some View {
 HStack(spacing: 12) {
 Image(systemName: "sparkles")
 .font(.system(size: 18, weight: .semibold))
 .foregroundColor(Color.oxyStone)
 .frame(width: 34, height: 34)
 .background(Color.oxyStone.opacity(0.12))
 .clipShape(Circle())

 VStack(alignment: .leading, spacing: 3) {
 Text("Oxy's read on today")
 .font(.system(size: 16, weight: .semibold))
 .foregroundColor(Color.oxyText)
 Text("Only useful nudges. No noise.")
 .font(.system(size: 12))
 .foregroundColor(Color.oxySub)
 }

 Spacer()

 Button(action: onCheckNow) {
 if isChecking {
 ProgressView()
 .tint(Color.oxyOnAccent)
 } else {
 Image(systemName: "arrow.clockwise")
 .font(.system(size: 14, weight: .semibold))
 }
 }
 .frame(width: 38, height: 34)
 .foregroundColor(Color.oxyOnAccent)
 .background(Color.oxyStone)
 .clipShape(RoundedRectangle(cornerRadius: 10))
 .disabled(isChecking)
 }
 .padding(14)
 .background(Color.oxySurface2.opacity(0.7))
 .clipShape(RoundedRectangle(cornerRadius: 16))
 .overlay(
 RoundedRectangle(cornerRadius: 16)
 .stroke(Color.oxyLine, lineWidth: 1)
)
 }
}

private struct BriefingCard: View {
 let briefing: Briefing
 let onDismiss: () -> Void

 var body: some View {
 VStack(alignment: .leading, spacing: 10) {
 HStack(spacing: 8) {
 Image(systemName: iconName)
 .font(.system(size: 13, weight: .semibold))
 .foregroundColor(Color.oxyStone)
 Text(briefing.title ?? title)
 .font(.system(size: 14, weight: .semibold))
 .foregroundColor(Color.oxyText)
 }

```

```

 Spacer()
 Text(timeLabel)
 .font(.system(size: 11))
 .foregroundColor(Color.oxyDim)
 }

 Text(cleanBody)
 .font(.system(size: 14))
 .foregroundColor(Color.oxyText)
 .fixedSize(horizontal: false, vertical: true)

 HStack {
 Text(sourceLabel)
 .font(.system(size: 11, weight: .medium))
 .foregroundColor(Color.oxySub)
 Spacer()
 Button("Done", action: onDismiss)
 .font(.system(size: 12, weight: .semibold))
 .foregroundColor(Color.oxyStone)
 }
}
.padding(14)
.background(.ultraThinMaterial)
.clipShape(RoundedRectangle(cornerRadius: 14))
.overlay(
 RoundedRectangle(cornerRadius: 14)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
}

private var title: String {
 briefing.kind
 .replacingOccurrences(of: "_", with: " ")
 .capitalized
}

private var iconName: String {
 if briefing.kind.contains("health") { return "heart.fill" }
 if briefing.kind.contains("location") { return "location.fill" }
 if briefing.kind.contains("failed") { return "exclamationmark.arrow.triangle.2.circlepath" }
 return "sparkles"
}

private var cleanBody: String {
 briefing.body
 .replacingOccurrences(of: #"\[^\]]*(unknown|\.unknown|Maps error|couldn't find)\[^\]]*"#, with: "", options: .regularExpression)
 .replacingOccurrences(of: " ", with: " ")
 .trimmingCharacters(in: .whitespacesAndNewlines)
}

private var sourceLabel: String {
 switch briefing.source {
 case "healthkit": return "HealthKit"
 case "location": return "Location"
 case "action_log": return "Action follow-up"
 case "schedule": return "Scheduled"
 default: return "Proactive"
 }
}

private var timeLabel: String {
 guard let createdAt = briefing.createdAt,
 let date = ISO8601DateFormatter().date(from: createdAt) else {
 return ""
 }
 return DateFormatter.localizedString(from: date, dateStyle: .none, timeStyle: .short)
}

private struct EmptyProactiveState: View {
 var body: some View {
 VStack(spacing: 10) {
 Image(systemName: "moon.zzz.fill")
 .font(.system(size: 26))
 .foregroundColor(Color.oxyStone)
 Text("Nothing needs you right now.")
 .font(.system(size: 16, weight: .semibold))
 .foregroundColor(Color.oxyText)
 Text("Oxy will only interrupt when there's something actually useful.")
 .font(.system(size: 13))
 .foregroundColor(Color.oxySub)
 .multilineTextAlignment(.center)
 }
 .frame(maxWidth: .infinity)
 .padding(.vertical, 34)
 .padding(.horizontal, 18)
 }
}

```

```

 .background(Color.oxySurface2)
 .clipShape(RoundedRectangle(cornerRadius: 16))
 }
}

private extension Briefing {
 var isWorthShowing: Bool {
 if source == "action_log" { return false }
 let lowerKind = kind.lowercased()
 if lowerKind.contains("failed") || lowerKind.contains("cancel") { return false }
 let lower = body.lowercased()
 let noisyFragments = [
 "unknown",
 "maps error",
 "try a diff",
 "that hit a snag",
 "cancelled",
 "canceled",
 "was cancelled",
 "was canceled"
]
 if noisyFragments.contains(where: { lower.contains($0) }) {
 return false
 }
 return true
 }
}

#Preview {
 ProactiveView()
 .environment(AppState())
}

```

## Color+Oxy.swift — OxyApp/OxyApp/Extensions/Color+Oxy.swift

```

import SwiftUI
import UIKit

extension Color {
 /// Background. Dark stays true black; Light/System adapt to the user's appearance.
 static var oxyBg: Color {
 dynamicColor(light: UIColor(red: 247/255, green: 247/255, blue: 244/255, alpha: 1),
 dark: UIColor(red: 12/255, green: 12/255, blue: 12/255, alpha: 1))
 }
 /// Surface 1.
 static var oxySurface1: Color {
 dynamicColor(light: UIColor(red: 255/255, green: 255/255, blue: 252/255, alpha: 1),
 dark: UIColor(red: 21/255, green: 21/255, blue: 21/255, alpha: 1))
 }
 /// Surface 2.
 static var oxySurface2: Color {
 dynamicColor(light: UIColor(red: 239/255, green: 239/255, blue: 235/255, alpha: 1),
 dark: UIColor(red: 30/255, green: 30/255, blue: 30/255, alpha: 1))
 }
 /// Surface 3.
 static var oxySurface3: Color {
 dynamicColor(light: UIColor(red: 226/255, green: 226/255, blue: 221/255, alpha: 1),
 dark: UIColor(red: 40/255, green: 40/255, blue: 40/255, alpha: 1))
 }
 /// Surface 4.
 static var oxySurface4: Color {
 dynamicColor(light: UIColor(red: 211/255, green: 211/255, blue: 205/255, alpha: 1),
 dark: UIColor(red: 51/255, green: 51/255, blue: 51/255, alpha: 1))
 }
 static var oxyLine: Color {
 dynamicColor(light: UIColor.black.withAlphaComponent(0.08),
 dark: UIColor.white.withAlphaComponent(0.06))
 }
 static var oxyLine2: Color {
 dynamicColor(light: UIColor.black.withAlphaComponent(0.12),
 dark: UIColor.white.withAlphaComponent(0.10))
 }
 static var oxyText: Color {
 dynamicColor(light: UIColor(red: 22/255, green: 22/255, blue: 21/255, alpha: 1),
 dark: UIColor(red: 242/255, green: 242/255, blue: 242/255, alpha: 1))
 }
 static var oxySub: Color {
 dynamicColor(light: UIColor(red: 99/255, green: 99/255, blue: 94/255, alpha: 1),
 dark: UIColor(red: 136/255, green: 136/255, blue: 136/255, alpha: 1))
 }
 static var oxyDim: Color {
 dynamicColor(light: UIColor(red: 145/255, green: 145/255, blue: 138/255, alpha: 1),
 dark: UIColor(red: 80/255, green: 80/255, blue: 80/255, alpha: 1))
 }
 static var oxyOnAccent: Color {
 Color(red: 12/255, green: 12/255, blue: 12/255)
 }
}

```



```

/// Stone accent — #C8B89A
static let oxyDefaultStone = Color(red: 200/255, green: 184/255, blue: 154/255)
static var oxyStone: Color { oxyAccent }
static var oxyAccent: Color {
 guard let object = oxySettingsObject,
 let accent = object["accentColor"] as? String else {
 return oxyDefaultStone
 }
 switch accent {
 case "mint": return Color(red: 76/255, green: 175/255, blue: 130/255)
 case "blue": return Color(red: 92/255, green: 154/255, blue: 245/255)
 case "rose": return Color(red: 230/255, green: 124/255, blue: 154/255)
 case "violet": return Color(red: 162/255, green: 132/255, blue: 245/255)
 case "cyan": return Color(red: 48/255, green: 184/255, blue: 210/255)
 case "amber": return Color(red: 236/255, green: 168/255, blue: 65/255)
 case "coral": return Color(red: 238/255, green: 112/255, blue: 92/255)
 case "indigo": return Color(red: 105/255, green: 126/255, blue: 235/255)
 default: return oxyDefaultStone
 }
}
private static var oxySettingsObject: [String: Any]? {
 guard let data = UserDefaults.standard.data(forKey: "oxy_settings") else { return nil }
 return try? JSONSerialization.jsonObject(with: data) as? [String: Any]
}
private static func dynamicColor(light: UIColor, dark: UIColor) -> Color {
 Color(UIColor { traits in
 isLightMode(traits) ? light : dark
 })
}
private static func isLightMode(_ traits: UITraitCollection) -> Bool {
 let theme = UserDefaults.standard.string(forKey: "oxy_appTheme")
 ?? (oxySettingsObject?["appTheme"] as? String)
 ?? "dark"
 if theme == "light" { return true }
 if theme == "system" { return traits.userInterfaceStyle == .light }
 return false
}
/// Green — #4CAF82
static let oxyGreen = Color(red: 76/255, green: 175/255, blue: 130/255)
/// Red — #C0503E
static let oxyRed = Color(red: 192/255, green: 80/255, blue: 62/255)
}

```

## ErrorBanner.swift — OxyApp/OxyApp/Views/Components/ErrorBanner.swift

```

import SwiftUI

struct ErrorBanner: View {
 let message: String
 var onRetry: (() -> Void)?
 var onDismiss: (() -> Void)?

 var body: some View {
 HStack(spacing: 10) {
 Image(systemName: "exclamationmark.triangle.fill")
 .font(.system(size: 14))
 .foregroundColor(Color.oxyRed)

 Text(message)
 .font(.system(size: 12))
 .foregroundColor(Color.oxyText)
 .lineLimit(2)

 Spacer()

 if let onRetry {
 Button("Retry", action: onRetry)
 .font(.system(size: 12, weight: .semibold))
 .foregroundColor(Color.oxyStone)
 }

 if let onDismiss {
 Button(action: onDismiss) {
 Image(systemName: "xmark")
 .font(.system(size: 10, weight: .bold))
 .foregroundColor(Color.oxySub)
 }
 }
 }
 .padding(.horizontal, 14)
 .padding(.vertical, 10)
 .background(Color.oxySurface3)
 .clipShape(RoundedRectangle(cornerRadius: 10))
 .padding(.horizontal, 16)
 }
}

```

```
#Preview {
 ErrorBanner(message: "Network connection lost", onDismiss: {})
 .background(Color.oxyBg)
}
```

## LoadingIndicator.swift — OxyApp/OxyApp/Views/Components/LoadingIndicator.swift

```
import SwiftUI

struct LoadingIndicator: View {
 var label: String = "Loading..."
 var tint: Color = .oxyStone

 var body: some View {
 VStack(spacing: 12) {
 ProgressView()
 .tint(tint)
 Text(label)
 .font(.system(size: 12))
 .foregroundColor(Color.oxySub)
 }
 .frame(maxWidth: .infinity, maxHeight: .infinity)
 .background(Color.oxyBg)
 }
}

struct OxyThinkingIndicator: View {
 var label: String?
 var compact: Bool = false

 @State private var active = false

 var body: some View {
 HStack(spacing: compact ? 8 : 10) {
 ZStack {
 Circle()
 .fill(Color.oxyStone.opacity(active ? 0.18 : 0.08))
 .frame(width: compact ? 20 : 24, height: compact ? 20 : 24)
 .scaleEffect(active ? 1.06 : 0.92)
 Image(systemName: "sparkle")
 .font(.system(size: compact ? 9 : 11, weight: .bold))
 .foregroundColor(Color.oxyStone)
 .rotationEffect(.degrees(active ? 8 : -8))
 }
 .animation(.easeInOut(duration: 0.95).repeatForever(autoreverses: true), value: active)

 if let label, !label.isEmpty {
 Text(label)
 .font(.system(size: compact ? 12 : 13, weight: .semibold))
 .foregroundColor(Color.oxySub)
 .lineLimit(1)
 }
 }
 .accessibilityLabel(label ?? "Oxy is thinking")
 .onAppear { active = true }
 }
}

struct OxySkeletonCard: View {
 var height: CGFloat = 84
 var cornerRadius: CGFloat = 16

 @State private var shimmer = false

 var body: some View {
 RoundedRectangle(cornerRadius: cornerRadius)
 .fill(Color.oxySurface2)
 .frame(height: height)
 .overlay(
 LinearGradient(
 colors: [
 .clear,
 Color.white.opacity(0.10),
 .clear
],
 startPoint: .leading,
 endPoint: .trailing
)
 .rotationEffect(.degrees(8))
 .offset(x: shimmer ? 260 : -260)
)
 .clipShape(RoundedRectangle(cornerRadius: cornerRadius))
 .overlay(
 RoundedRectangle(cornerRadius: cornerRadius)
 .stroke(Color.oxyLine2, lineWidth: 1)
)
 .onAppear { shimmer = true }
 }
}
```

```
 .animation(.easeInOut(duration: 1.25).repeatForever(autoreverses: false), value: shimmer)
 .accessibilityHidden(true)
 }
}

#Preview {
 VStack(spacing: 16) {
 LoadingIndicator()
 OxyThinkingIndicator()
 OxyThinkingIndicator(label: "Thinking")
 }
}
```

# Firmware

## firmware/README.md — firmware/README.md

# Oxy Pendant Firmware

Arduino firmware for the Oxy pendant, targeting the **Seeed XIAO nRF52840 Sense** dev board.

## What it does

1. Advertises over BLE as **"OxyPendant"** using Nordic UART Service
2. Waits for the iOS app to connect
3. **Immediately starts streaming** audio from the onboard PDM microphone once connected
4. Streams 16-bit PCM @ 16 kHz mono over BLE UART in 20-byte chunks
5. LED stays on while streaming
6. Stops streaming when BLE disconnects

The iOS app handles voice activity detection (VAD) – it detects when you start/stop speaking and automatically transcribes your words into Oxy Cloud.

## Hardware requirements

- **Seeed XIAO nRF52840 Sense** (has built-in PDM microphone – no external mic needed)
- No button or touch sensor required for testing

## Setup – Arduino IDE

**IMPORTANT:** You must use the **Seeed nRF52 mbed-enabled Boards** package (NOT the Adafruit nRF52 BSP). The Adafruit BSP has a known PDM bug that causes audio to be lost.

1. **Add Seeed board package:**
  - Open Arduino IDE → Settings → Additional Board Manager URLs
  - Add: `https://files.seeedstudio.com/arduino/package_seeedstudio_boards_index.json`
  - Go to Tools → Board Manager → search **"Seeed nRF52 mbed"** → Install **"Seeed nRF52 mbed-enabled Boards"**
2. **Select board:**
  - Tools → Board → Seeed nRF52 mbed-enabled Boards → **Seeed XIAO BLE Sense - nRF52840**
3. **Open sketch:**
  - File → Open → navigate to `firmware/oxy-pendant/oxy-pendant.ino`
4. **Upload:**
  - Connect the XIAO board via USB-C
  - Tools → Port → select the board's serial port
  - Click Upload (→)
5. **Verify:**
  - Open Serial Monitor (115200 baud)
  - You should see `[Oxy] Pendant ready – will stream on BLE connect`

## Testing end-to-end

1. Flash the firmware to your XIAO nRF52840 Sense
2. Build the iOS app in Xcode and install on your iPhone
3. Open the app → Settings → Pendant → Scan for Pendant
4. Once connected, the pendant LED turns on and starts streaming
5. Speak near the pendant – the app detects your voice, transcribes it, and sends it to Oxy Cloud
6. Enable **Voice Replies** in the app's Settings for spoken responses

**Debugging:** Check the Xcode console for `[PendantBridge] Audio batch:` lines – these show the audio levels being received. If `max` and `avg` are both 0, the audio is not being received.

## Audio format

- **Sample rate:** 16,000 Hz
- **Bit depth:** 16-bit signed PCM (little-endian)
- **Channels:** 1 (mono)
- **BLE chunk size:** 20 bytes (10 samples per BLE packet)
- **PDM gain:** 40 (boosted from default 20, range 0-80)

## How voice detection works

The pendant streams audio continuously. The iOS app (`PendantAudioBridge`) handles:

- **Chunk accumulation:** Buffers BLE chunks (~0.6ms each) into ~256ms batches
- **Format conversion:** Converts Int16 PCM → Float32 with 8x gain amplification
- **Speech recognition:** Feeds batches into Apple's `SFSpeechRecognizer`
- **Apple's built-in VAD:** Detects speech start/end automatically
- **Automatic restart:** Ready for the next utterance immediately

## Pin mapping

| Function    | Pin         | Notes                   |
|-------------|-------------|-------------------------|
| PDM Mic CLK | P0.20       | Built-in on Sense board |
| PDM Mic DIN | P0.21       | Built-in on Sense board |
| LED         | LED_BUILTIN | On while streaming      |

## Adapting for production hardware

oxy-pendant.ino — firmware/oxy-pendant/oxy-pendant.ino

Page 245 · Confidential

Page 246 · Confidential

```

}

BLE.setLocalName(DEVICE_NAME);
BLE.setDeviceName(DEVICE_NAME);

nusService.addCharacteristic(txChar);
nusService.addCharacteristic(rxChar);
BLE.addService(nusService);
BLE.setAdvertisedService(nusService);
BLE.advertise();

Serial.print("[Oxy] BLE advertising as ");
Serial.println(DEVICE_NAME);
}

// =====
// Main loop
// =====
void loop() {
 BLE.poll();

 BLEDevice central = BLE.central();

 if (central) {
 Serial.print("[Oxy] Connected: ");
 Serial.println(central.address());

 for (int i = 0; i < 3; i++) {
 digitalWrite(LED_BUILTIN, LOW); delay(100);
 digitalWrite(LED_BUILTIN, HIGH); delay(100);
 }

 delay(500);
 startStreaming();

 while (central.connected()) {
 BLE.poll();
 handleRXCommand();
 updateLED();

 if (isStreaming && pdmReady) {
 pdmReady = false;
 int bytesToSend = pdmBytesAvail;

 static unsigned long lastLog = 0;
 if (millis() - lastLog > 5000) {
 lastLog = millis();
 Serial.print("[Oxy] Audio samples: ");
 int n = min(5, bytesToSend / 2);
 for (int i = 0; i < n; i++) {
 Serial.print(pdmBuffer[i]);
 Serial.print(" ");
 }
 Serial.println();
 }

 const uint8_t* data = (const uint8_t*)pdmBuffer;
 int remaining = bytesToSend;
 while (remaining > 0) {
 int chunk = min(remaining, 20);
 txChar.writeValue(data, chunk);
 data += chunk;
 remaining -= chunk;
 delayMicroseconds(500);
 }
 }
 }

 stopStreaming();
 Serial.println("[Oxy] Disconnected");
 BLE.advertise();
 }
}

```

# Documentation

## Onboarding Permission Matrix — docs/ONBOARDING\_PERMISSION\_MATRIX.md

# Onboarding And Permission Matrix

Oxy should ask for permissions when the user understands the value, not all at once.

```
Capability	When To Ask	User Value Copy	Denied-State Recovery
Location	First near-me, maps, Uber, travel, or proactive location feature	Find nearby places, routes, pickup/drop-off, and useful local nu	
Contacts	First message, call, FaceTime, or recipient lookup	Match names like "Josh" to the right phone/email.	Ask for the number/email o
Music	First native playback/add-to-library request	Play exact songs through Apple Music.	Open Apple Music search link and show enable MU
Calendar	First calendar creation/read request	Add or check events when asked.	Ask to enable Calendar or continue with a text draft.
Reminders	First reminder request	Create reminders directly on device.	Ask to enable Reminders.
Microphone	First voice message	Let Oxy hear the prompt.	Keep text input fully usable.
Notifications	First proactive briefing setup	Send useful briefings and follow-ups.	Keep Today tab available without pushes.
HealthKit	First health question/health alert enable	Answer health questions from local Health data.	Say Health access is unavailable and
Connectors	When user opens Connectors or requests connector action	Connect Google, Telegram, Trainline, Netflix, etc.	Show reconnect/per
```

## Required First-Run Flow

1. Account sign-in.
2. One-screen value explanation: Oxy is useful only with user-approved context.
3. Ask for no sensitive permission until a feature needs it.
4. Show Settings controls for memory, proactive behavior, permissions, and connectors.
5. Include support/privacy links before payment.

## Acceptance Tests

- Fresh install can chat without granting optional permissions.
- Every permission-denied path has useful copy and no dead-end "failed".
- Every connector failure says reconnect, permission, unavailable, or not configured.
- User can disable proactive/health/location nudges from Settings.

## Hardening Runbook — docs/OXY\_HARDENING\_RUNBOOK.md

# Oxy Hardening Runbook

This is the working checklist for making Oxy usable as a serious personal assistant, not just a demo.

## 1. Brain And Context Evals

Required baseline before every release:

- Run `npm run smoke`.
- Run `npm run brain:evals`.
- Add a regression test for every bad screenshot or user report before shipping the fix.
- Cover at least these categories: contextual references, corrections, memory, current facts, local actions, travel, messaging, calendar, media,

Acceptance:

- A vague follow-up is never routed by keyword alone.
- A current factual answer either uses search grounding or clearly says it cannot verify.
- A failed action gives a recovery path.

## 2. Deployment Verification

Backend proof:

- `GET /version` returns `gitCommit`, `gitBranch`, `cloudRunRevision`, `deployId`, `packageVersion`, `buildTime`, `nodeVersion`, and `environment`.
- `GET /health` returns `status`, `missingEnv`, and the same version payload.
- Every backend response includes `X-Oxy-Commit`.
- iOS Settings shows the backend commit under Diagnostics.

Manual check after each push:

```
```bash
curl -s https://oxy-151340634966.europe-west2.run.app/version
```
```

Acceptance:

- The deploy proof shown in Settings matches `gitCommit` when the deploy provides one; otherwise it matches the Cloud Run revision in `cloudRunR

## 3. Observability

Every chat request must leave enough logs to explain a bad answer:

- request start with short message preview
- search decision and reason
- context brain decision and resolved context
- deterministic router decision
- Gemini first token and completion
- action start/completion



- action failure recovery metadata
- final request duration

Do not log secrets, full auth tokens, connector tokens, or full private message bodies.

#### ## 4. Memory Quality

Memory rules:

- Save stable user facts only.
- Do not save transient actions, guesses, or current factual answers as memory.
- If the user asks “do you remember...”, answer from memory/conversation first.
- If memory is missing, say so plainly.
- If the user corrects memory, the newer correction wins.

Acceptance prompts:

- “remember my usual station is Birmingham New Street”
- “do you remember what my usual station is”
- “no my usual station is Solihull”
- “what is my usual station”

#### ## 5. Current Facts

Use search grounding for anything that could have changed:

- news, charts, rankings, weather, prices, schedules, company facts, product recommendations, public figures, availability.

Rules:

- Do not answer with old model memory for current facts.
- If challenged, re-check instead of defending the previous answer.
- If no source is available, say what is missing.

#### ## 6. Connector Failure UX

Every connector failure must map to one of:

- needs reconnect
- missing permission
- backend config missing
- temporary provider failure
- no result found
- native app handoff required

The UI card must say what the user can do next.

#### ## 7. Onboarding And Permissions

First-run onboarding must explain:

- Oxy needs network access for the brain.
- Location powers near-me, maps, Uber, travel.
- Contacts powers message/call/email recipients.
- Calendar/reminders power scheduling.
- Music powers native playback.
- HealthKit powers health questions/briefings.
- Notifications power proactive briefings.
- Connectors are optional and can be disconnected.

Acceptance:

- Denying any permission does not make Oxy look broken.
- Each denied permission has a clear recovery path in Settings.

#### ## 8. Privacy, Support, And Legal

Before public release:

- Publish privacy policy.
- Publish terms.
- Publish support contact.
- Publish data deletion path.
- Explain memory controls.
- Explain connector OAuth/token storage.
- Explain location, contacts, calendar, reminders, notifications, HealthKit, audio, and chat history usage.
- State subscription price cap: maximum £15/month unless changed intentionally.

#### ## 9. Device Test Matrix

Run on physical iPhone, not only simulator:

- fresh install
- existing user upgrade
- no network
- slow network

- denied location
- denied contacts
- denied music
- denied notifications
- signed-out session
- stale backend deploy
- Cloud Run cold start

## ## 10. Release Checklist

Before a release is allowed:

```
```bash
npm run release:check
xcodebuild -project OxyApp/OxyApp.xcodeproj -scheme OxyApp -configuration Release -sdk iphones build
```
```

Manual prompts:

- "what's the most popular song on the billboard hot 100 right now"
- "play it"
- "is that right"
- "get me an uber to the nearest Aldi"
- "do you remember what my usual station is"
- "what about tomorrow"
- "send it to Josh"

Acceptance:

- Backend commit visible in Settings.
- Cloud Run `/version` matches Git.
- No dead-end failures.
- No hallucinated current facts.
- No vague follow-up gets hijacked by the wrong native tool.

## Privacy Support Checklist — docs/PRIVACY\_SUPPORT\_CHECKLIST.md

### # Privacy And Support Checklist

This is not legal advice. It is the product checklist Oxy must satisfy before public launch.

### ## Public Pages

- Privacy policy URL
- Terms URL
- Support email
- Data deletion instructions
- Subscription/cancellation instructions

### ## Data Disclosures

Explain clearly:

- Chat messages are sent to Oxy backend and Gemini for responses.
- Search grounding may be used for current facts.
- Memories store user-provided stable facts.
- Conversation history is saved unless vanish/private mode is active.
- Connector OAuth tokens are stored server-side for connected services.
- Location is used for near-me, maps, Uber, travel, and local recommendations.
- Contacts are used only to resolve requested recipients.
- Calendar/reminders are used only for requested scheduling tasks.
- HealthKit is used only for health summaries/briefings enabled by the user.
- Audio is used for transcription/voice interactions.
- Notifications are used for proactive briefings and action follow-ups.

### ## Controls

Required user controls:

- Sign out
- Delete account/data path
- View/edit/delete memory
- Disconnect connectors
- Toggle proactive briefings
- Toggle health alerts
- Toggle location reminders
- Disable voice output

### ## Support Playbook

Support responses must collect:

- App build/version
- Backend commit from Settings
- Approximate time of issue
- Prompt used
- Screenshot if available

- Connector/permission involved

Never request:

- Passwords
- OAuth tokens
- Full private contact lists
- Full message history unless explicitly needed and user-approved

## ## Pricing

Current planning constraint:

- Subscription maximum: £15/month.
- Any plan or copy mentioning a higher monthly price must be changed before launch.